



C# İLE NESNE YÖNELİMLİ PROGRAMLAMA VE ENTITY FRAMEWORK



ÖNSÖZ

Ülkemizde her alanda olduğu gibi yazılım alanında da bir terminoloji karmaşasının yaşandığı aşikardır. Son dönemde yazılanlar adeta çeviri yapan programların bir gecede ürettiği çıktılar birleştirilerek hazırlanmaktadır. Birçok programlama dili kullanarak yazılım geliştiriyor olmama rağmen, yazılım alanında içeriği anlama konusunda oldukça zorlanıyorum.

Bu kitapla birlikte tarihsel gelişimle birlikte programlama mantığının yapısal programlamaya geçişi sürecini de içerecek şekilde yazılımım gelişme süreci anlatılmıştır.

Yapısal programlamanın en çok uygulandığı Pascal dilidir. Pascal diline yakın zamanda ortaya çıkan C dili, kısa sürede sistem dili olarak tarihte yerini almıştır. Günümüzde sistem programlama ve yüksek hız gerektiren gömülü sistemlerde vazgeçilmez bir dil olmuştur.

Daha sonra C dilinden etkilenecek olan Java, C# gibi birçok nesne yönelimli programlama dili ve yapısal programlama üzerine kurulmuş dillerdir. Birçok modern programlama diline esin kaynağı olan bu dilin özellikle bilgisayar mühendisleri tarafından bilinmesi gerekir.

C# programlama dili ise Java gibi bir ara dil oluşturarak çalıştırılabilir kodun her platformda derlenmeden çalıştırılmasını hedefleyen ve programcının kurumsal yazılımları hızlıca geliştirmek için istenilen bileşenleri hazır olarak sunan kütüphanelere sahip bir saf nesne yönelimli programlama dilidir. Yüksek performans isteyen kurumsal yazılım projelerde çokça kullanılmaktadır.

Nesne yönelimli programlama mantığını oluşturan kavramların programcıların kafasında aynı ışığı yakması için bu kitap hazırlanmıştır. Bu dilin eğitimini verecek tüm kişiler için serbestçe kullanılabilir.

Bu kitabın güncel sürümüne <https://github.com/HoydaBre/csharp> adresinden erişilebilir. Amacım Türkçe düşünen ve konuşan öğrencilerimize kavramları doğru bir şekilde aktarmaktır. Katkılarınızı bekler iyi çalışmalar dilerim.

İlhan ÖZKAN

ilhanozkan[at]outlook.com

İKİNCİ SÜRÜM

İlk sürümden farklı olarak;

- Sınıf ve Nesne bölümü sadeleştirildi.
- Temsilciler ve Olaylar kısmı yeni bölüm yapıldı.
- Varlık Çerçevesi (Entity Framework) bölümü eklendi.
- Katmanlı Mimari bölümü sadeleştirildi ve örneği eklendi.
- Aşırı yükleme kavramı fonksiyonlar altına alındı.

İÇİNDEKİLER

ÖNSÖZ	1
İKİNCİ SÜRÜM.....	2
İÇİNDEKİLER	3
TEMEL BİLGİSAYAR KAVRAMLARI.....	10
Mühendis ve Teknisyen.....	10
Bilgisayara Niçin İhtiyaç Duyulur?	10
Veri ve Bilgi.....	10
En Basit Bilgisayar	10
İşlemcinin Emri Alma, Çözme ve İcra Döngüsü.....	12
İşlemci Tasarlama Stratejileri.....	13
Program ve Programlama	13
Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri.....	14
TEMEL YAZILIM KAVRAMLARI	16
Tarihçe.....	16
Genel Amaçlı Yüksek Düzey Diller.....	16
Arapsaçı Kod.....	17
Değişken	18
Fonksiyon.....	19
Yapısal Programlama	20
Nesne Yönelimli Programlama İhtiyacı.....	21
Nesne Yönelimli Programlama.....	22
DERLEYİCİ VE ÇALIŞTIRMA ORTAMI	23
.NET Framework.....	23
Ortak Dil Tarifnamesi.....	24
Ortak Tip Sistemi.....	24
Derleme ve İcra Süreci.....	25
Ortak Ara Dil Kodu	26
.NET Framework Kurulumu	26
Entegre Geliştirme Ortamı.....	28
Çevrimiçi Derleyiciler	29
Derleme Zamanı	29
Hatalar.....	29
Yapısal Programlamanın Genel Çerçevesi.....	30
Nesne Yönelimli Programın Genel Çerçevesi.....	31
C# Projesi Oluşturma ve Çalıştırma.....	32
Visual Studio 2022 ile Proje Oluşturma	32
.NET Core CLI ile Proje Oluşturma.....	33
C# PROGRAMLAMA DİLİNE GİRİŞ	35
Söz Dizim Kuralları.....	35
Talimatlar ve Anahtar Kelimeler.....	35
Açıklamalar.....	36
Değişken Tanımlama	36
Değer Tipler	36
void Tipi.....	37
Referans Tipler	37
Değişken Bildirimi.....	37
Değişken Kimliklendirme Kuralları.....	38
Değişmezler	39
Yerel Sabitler.....	39
Anonim Değişkenler	40
Dinamik Değişkenler	41
Null Olabilen Değer Tipler.....	41

Numaralandırma Tipi	41
Yazdığımız Kod Nasıl İcra Edilir?	42
Temel İşleçler	43
Aritmetik İşleçler	43
Mantıksal İşleçler	44
Eşitlik ve İlişkisel İşleçler	44
Bit Düzeyi İşleçler	45
Atama İşleçleri	45
İşleç Öncelikleri	46
Kayan Noktalı Sayı Hesapları	47
BASİT GİRİŞ ÇIKIŞ İŞLEMLERİ	49
Modüler Programlama	49
Konsolda Veri Okuma ve Yazma	49
Dizgiler	50
Temel Dizgi İşlemleri	50
Tam Dizgiler	52
Dizgi Enterpolasyonu	52
Konsola Yazılan Metni Biçimlendirme	52
Tarih Biçimlendirme	52
Para Birimi Biçimlendirme	53
Sayı Biçimini Değiştirme	54
Metinden Değer Tiplere Dönüşüm	55
KONTROL İŞLEMLERİ	56
Ardışık İşlem ve Kontrol İşlemleri	56
Akış Diyagramları	56
Duruma Göre Seçimler	57
If Talimatı	57
If-Else Talimatı	59
Sarkan Else	62
Switch Talimatı	62
Üçlü Şart İşleci	63
null Kaynaştırma İşleci	64
İlişkisel Döngüler	64
Sayaç Kontrollü Döngüler	65
While Talimatı	65
Do-While Talimatı	66
For Talimatı	67
İç İçe Döngü Kodu	68
Döngülerde Gözcü Kontrolü	69
BigInteger Veri Tipi	70
Dallanmalar	70
Continue ve Break Talimatları	71
Return Talimatı	72
Goto Talimatı	73
DİZİLER	74
Dizi Nedir?	74
Dizi Tanımlama	74
Dizi Elemanlarını Değiştirme	75
Dizi Üzerinde Yinelemeler	75
Dizileri Kopyalama	78
Dizileri Karşılaştırma	78
Çok Boyutlu Diziler	78
Pürüzlü Diziler	82
Foreach Talimatı	82

SINIF VE NESNE	83
Sınıf ve Nesne Nedir?	83
Erişim Değiştiricileri	84
Yöntemler	85
Yöntem Tanımı	85
Yöntem Çağırma	86
Yöntemlere Parametre Geçirme	87
Yöntemlere Referans Olarak Parametre Geçirme	88
Yöntem Parametrelerini Çıktı Olarak Kullanma	89
Ön Tanımlı Argüman	89
Özyinelemeli Yöntemler	89
Parametre Olarak Diziler	91
Çağrı Kuralı	92
Nesne Yapıcısı	93
Soyut, Somut ve Bilgi Gizleme	94
Sarmalama	94
Kalıtım	97
Kalıtım Olmadan Sınıf Tasarımı	97
Kalıtım İçeren Sınıf Uygulaması	98
Miras Alınan Üyeleri Gizleme	101
Aşırı Yükleme	102
Yöntemlerin Aşırı Yüklmesi	102
Yapıcıların Aşırı Yüklmesi	103
Taban Sınıfın Yapıcısının Çağırılması	104
İşleçlerin Aşırı Yüklmesi	105
this Göstericisi	106
Statik Üyeler ve Statik Sınıf	107
Sabit Alanlar	107
readonly Kelimesi	108
Değiştirilemez Nesneler	109
Geçersiz Kılma	110
Ara Yüzler veya Roller	111
Nesne Yıkıcı	112
Çok Biçimlilik	113
Sınıf Çeşitleri	115
Nesnelerle İlgili İşleçler	117
Tip Kontrol İşleci	117
Güvenli Tip Dönüşüm İşleci	117
Eşitlik İşleci	117
TEMSİLCİLER VE OLAYLAR	119
Temsilciler	119
Lamda İfadeleri	119
Lamda İfadeleri	120
Lamda Talimatları	120
Temsilci Başlatmada Lamda İfadeleri	121
Anonim Değişkenlerle Lamda İfadeleri	121
Anonim Yöntemler	122
Olaylar	122
Olaylara Tepki Vermek İçin Lamda İfadeleri	125
Genişleme Yöntemleri	126
İsim Uzayları	127
SINIFLAR ARASI İLİŞKİLER VE SINIF TASARLAMA İLKELERİ	129
Unified Modeling Language	129
Sınıf Diyagramları	130

Sınıflar Arası İlişkiler.....	131
Genelleştirme İlişkisi.....	131
Bağımlılık İlişkisi.....	131
İş Birliği İlişkisi.....	132
Bütünleşme İlişkisi.....	133
Sınıf Tasarlama İlkeleri.....	134
Tek Sorumluluk İlkesi.....	135
Açık-Kapalı İlkesi.....	135
Liskov İkame İlkesi.....	135
Ara Yüzleri Ayırma İlkesi.....	135
Bağımlılıkları Ters Çevirme İlkesi.....	135
YAPILAR, BİRLİKLER VE NİTELİKLER.....	136
Yapı Tanımlaması.....	136
Yapı Değişkenleri.....	136
Yapılarda Yapıcılar.....	137
Yapılarda Ara Yüzü Gerçekleştirme.....	137
Boyutu Sabit Dizi Elemanına Sahip Yapılar.....	138
Birlikler.....	139
Nitelikler.....	140
İSTİSNA YÖNETİMİ.....	143
Yapısal Programlamada Hata Yönetimi.....	143
İstisna (Özel Durum) Yönetimi.....	143
İstisnaları İşleme.....	144
Ön Tanımlı İstisnalar.....	144
İstisnai Durumun Belirlenmesi.....	145
when Süzgeci.....	146
Kullanıcı Tanımlı İstisna Sınıfları.....	146
finally Bloğu.....	147
İstisnayı Tekrar Fırlatma.....	148
Özel Durum İşleme Kuralları.....	148
Kontrol Akışını İstisna Kodları ile Gerçekleştirmeyin.....	148
Zorunlu Olmadıkça İstisnayı Tekrar İmal Etmeyin.....	149
İz Kaydı Olmadan İstisnaları Görmezden Gelmeyin.....	149
İşleyemeyeceğiniz İstisnaları Yakalamayın.....	149
TİP DÖNÜŞÜMLERİ VE GÖSTERİCİLER.....	150
Tip Dönüşümleri.....	150
Örtük Dönüşümler.....	150
Açık Dönüşümler.....	151
Kullanıcı Tanımlı Dönüşümler.....	151
Yardımcı Sınıflarla Dönüşümler.....	152
Kutulama.....	153
Göstericiler.....	153
Gösterici Tipi Bildirme.....	153
Gösterici Aritmetiği.....	155
Referanstan Çıkarma İşleci Veri Tipinin Bir Parçasıdır.....	156
Gösterici Üye Erişim İşleci.....	156
KOLEKSİYONLAR.....	157
ArrayList.....	157
Jenerik.....	157
yield Anahtar Kelimesi.....	159
HashSet<T> Karma Kümesi.....	160
Dictionary<TKey, TValue> Sözlük.....	160
SortedSet<T> Sıralı Küme.....	161
List<T> Liste.....	161

Stack<T> Yığın	162
LinkedList<T> Bağlı Liste	162
Queue<T> Kuyruk	162
İndisleyiciler	162
LINQ SORGULARI	164
LINQ Nedir?	164
IEnumerable ve IEnumerator Arayüzleri	165
IQueryable Arayüzü	165
LINQ Genişleme Yöntemleri	165
Where	166
Select	166
OrderBy ve OrderByDescending	166
Group By	167
First ve Last	167
FirstOrDefault ve LastOrDefault	167
Single	168
SingleOrDefault	168
Sum, Min, Max ve Average	168
Any ve All	168
Count	169
FindAll	169
Distinct	169
Intersect, Except ve Union	170
OfType	171
SelectMany	172
Join	172
Skip ve Take	174
Range ve Repeat	174
Contains	174
Zip	174
Aggregate	175
Let Anahtar Kelimesi	175
DOSYALAR VE AKIŞLAR	176
Dosyalar ve Klasörler	176
Dosya Oluşturma	176
Dosya Taşıma	177
Dosya Silme	177
Bir Klasördeki Dosyaları Öğrenme	177
Metin Dosyasını Okuma	177
Büyük Metin Dosyaları	178
Akış Nedir?	178
Akışlara Veri Yazma ve Okuma	179
Akış Olmadan Metin Dosyaları Oluşturma	179
İkili Dosyaya Yazma ve Okuma	180
VERİ TABANI İŞLEMLERİ	181
Veri Tabanı Nedir?	181
Bağlantı Dizgisi	181
İlişkisel Veri Tabanı İşlemleri	182
Veri Tabanında Tablolar Oluşturma	182
Sorgu Parametreleri	183
Veri Tabanından Veri Okuma	183
İşlemler	184
Veri Kaynaklarına Erişim Yöntemleri	185
NoSQL Veri Tabanı	186

DESENLER	188
Desen Nedir?	188
Nesne İmalatı ile İlgili Tasarım Desenleri.....	189
Soyut Fabrika Deseni.....	189
Fabrika Yöntemi Deseni.....	191
Tekil Nesne Deseni.....	192
Kurucu Deseni.....	193
Prototip Deseni.....	195
Yapısal Tasarım Desenleri.....	196
Vitrin Deseni.....	196
Adaptör Deseni.....	198
Bileşik Deseni.....	199
Dekoratif Deseni.....	201
Sinekşiklet Deseni.....	203
Vekil Deseni.....	205
Köprü Deseni	206
Davranışla İlgili Tasarım Desenleri.....	208
Sorumluluk Zinciri Deseni	208
Komut Deseni.....	209
Yineleyici Deseni.....	211
Gözlemci Deseni.....	213
Strateji Deseni.....	215
Şablon Yöntem Deseni.....	217
Ziyaretçi Deseni.....	218
Hatıra Deseni.....	221
Arabulucu Deseni.....	222
Durum Deseni.....	224
Yorumlayıcı Deseni.....	226
Model-View-Controller Mimari Deseni	228
MVC Deseni Temel Yapısı	228
Gözlemci ile MVC Deseni İçin Örnek Proje.....	228
Projede MVC Ara Yüzlerinin Oluşturulması	229
Projeye Gözlemci Deseninin Uygulanması.....	230
MVC İskeletine Uygun Projenin Tamamlanması.....	231
KURUMSAL YAZILIMLAR.....	239
Kurumsal Yazılım İçin Gereklilikler	239
Yazılım Geliştirme Modelleri	240
Şelale Modeli	240
Çevik Model.....	241
Modern Yazılımları Özellikleri	243
Katmanlı Mimari	243
Yekpare Uygulamalar	243
İki Katmanlı İstemci-Sunucu Uygulamaları.....	244
Üç Katmanlı Uygulamalar	245
Çok Katmanlı Uygulamalar	247
Katmanların Tasarlanması.....	248
Katmanlar Arası İletişim	252
Bileşen Kullanımı	253
Dağıtık Programlama	254
Dağıtık Programlama Teknolojileri	254
Web Servisler.....	255
RESTFul Web API Örneği.....	256
Örnek Kurumsal Uygulama	259
VARLIK ÇERÇEVESİ	263

Nesne-İlişkili Eşleme.....	263
Önce Kod.....	264
Önce Kod Kuralları.....	268
Var Olan Bir Kuralı Geçersiz Kılma.....	268
Birincil Anahtar Kuralı.....	269
Veri Tipi Keşfi.....	269
Ondalık Kuralı.....	271
İlişki Kuralı.....	271
Yabancı Anahtar Kuralı.....	272
Önce Kod Veri Açıklamaları.....	273
[Column] Niteliği.....	273
[Required] Niteliği.....	273
[MaxLength] ve [MinLength] Nitelikleri.....	273
[ForeignKey(string)] Niteliği.....	273
[InverseProperty(string)] Niteliği.....	274
[ComplexType] Niteliği.....	275
[Range(min,max)] Niteliği.....	275
[NotMapped] Niteliği.....	276
[Table] Niteliği.....	276
[Index] Niteliği.....	277
[Key] Niteliği.....	277
[StringLength(int)] Niteliği.....	277
[DatabaseGenerated] Niteliği.....	278
Mevcut Veri Tabanı Tablolarıyla Çalışma.....	278
Veri Göçü.....	279
ŞEKİL LİSTESİ.....	288
TABLO LİSTESİ.....	290
DİZİN.....	291

TEMEL BİLGİSAYAR KAVRAMLARI

Mühendis ve Teknisyen

Mühendis (**engineer**), ihtiyaç duyulan ürünü; ekonomik (**cost**), güvenli (**safe**) ve kullanışlı (**functional**) olarak zamanında (**on time**) ortaya koyan kişilerdir. Bunu yapmak için; keşif (**invent**), tasarım (**design**), analiz (**analysis**), uygulama (**build**) ve sinama (**test**) yaparlar¹. Teknisyenler (**technician**) ise genellikle laboratuvarlarda teknik ekipmanlarla ilgilenen veya bu ekipmanlar ile pratik çalışmalar yapan kişilerdir. Genel olarak amacı ekipmanın bakım ve kullanımına ilişkindir.

Konumuz yazılım olduğundan, yazılım mühendisleri ise istenilen bilgisayar yazılımını ekonomik, güvenli ve kullanışlı olarak zamanında ortaya koyan kişilerdir. Programcılar ise teknisyenlere karşılık gelir. Yani geliştirme aşamasında bazı yazılım kısımlarının kodlanmasını veya geliştirilen yazılımın çalışır tutulmasını sağlarlar.

Bilgisayara Niçin İhtiyaç Duyulur?

Bir çiftçi niçin traktöre ihtiyaç duyar? Bir berber niçin elektrikli tıraş makinesi kullanır? Benzer şekilde bir mühendis niçin bilgisayara ihtiyaç duyar?

İşte bütün bu soruların cevabı yapılacak işleri daha kısa sürede yapmaktır. Yani bütün araç gereç ve makinalara olan ihtiyacımız hız (**speed**) ile ilgilidir. Bütün makinelere gibi bilgisayar da yapılacak işleri hızlandırmak için kullanılır. Bilgisayarlar; veriden hızlıca bilgi elde etmemizi ve tekrarlanan hesaplamaları hatasız yapmamızı hızlıca sağlayan makinelerdir.

Veri sorumlusu, veri hazırlama kontrol işletmeni gibi unvanları bu sebeple ortaya çıkmıştır. Bu unvanlara ihtiyaç duyulmasının sebebi verinin belli bir şekle uygun olarak bilgisayara girilmesi ve işlenen verinin saklanması ve raporlanması bilgisayar dünyasında önemli bir yer tutmasıdır.

Bilgisayarlar ilk zamanlarda daha çok cebirsel ifadeleri hatasız ve hızlı bir şekilde yapmak için kullanılmışlardır. Bunu daha sonradan ALGOL (**ALGO**ritmic **L**anguage) adını alacak IAL (**I**nternational **A**lgebraic **L**anguage) ya da FORTRAN (**FOR**mula **TRAN**slation) dilinden de anlayabiliriz.

Veri ve Bilgi

Veriler (**data**), genellikle bir ölçüm ya da sayım sonucu elde edilen ve hakkında az şey bilinen edilen varlıklardır. Örneğin ölçülen hava sıcaklığı, derse giren öğrenci sayısı, tartılan sebzenin ağırlığı gibi şeyler.

Bilgi (**information**) ise verinin işlenerek ortaya çıkan anlamlı verilerdir. Yani ortalama hava sıcaklığı, ölçülen yerde bir ölçüm yapmaya gerek kalmadan sıcaklığı yaklaşık olarak bilmeye yarayan bir bilgidir. Bir sınıftaki not ortalaması, o sınıftan rastgele seçilen bir öğrencinin notunu yaklaşık belirleyen bir bilgidir. Kısacası bilgi, işlenmiş veridir.

En Basit Bilgisayar

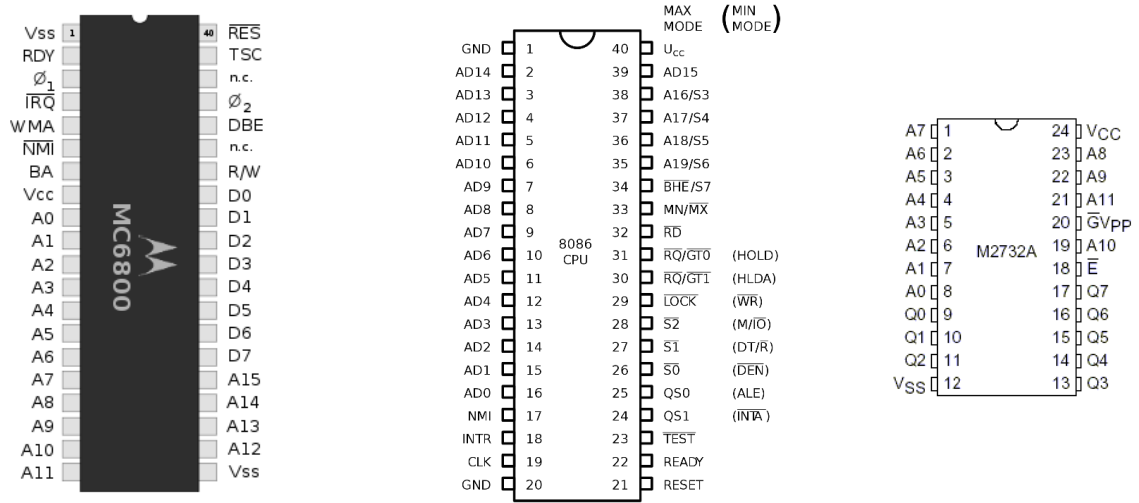
En basit bilgisayar;

- Merkezi işlem birimi (**Central Processing Unit-CPU**) kısaca işlemci,
- Bellek (**memory**) ve
- Bunları birbirine bağlayan elektrik yollarından (**bus**) oluşur.

İşlemci dışında bulunan bellek ise işlemcinin işleyeceği emirleri barındırır. Belleğin hem adres (**address**) hem de veri (**data**) bacakları bulunur. Belleğin bu bacakları elektrik yolları ile işlemcinin adres ve veri bacaklarına bağlıdır. Benzer şekilde işlemcinin de bellek ile iletişimini sağlayan adres ve veri bacakları bulunur. Bunların yanında okuma (**read**), yazma (**write**), kesme (**interrupt**) veya kristal bağlanan bacakları gibi kontrol bacakları (**control**) da bulunur.

¹ <https://www.bls.gov/>

Şekil 1'de görüldüğü üzere 6801 işlemcisinin 15 adet adres bacağı, 8 adet veri bacağı bulunmaktadır. 8086 işlemcisinin ise 20 adet adres bacağı bulunmakta olup bu bacakların 16 adedi aynı zamanda veri bacağı olarak kullanılmaktadır. 2732 belleğin ise 12 adet adres bacağı, Q ile gösterilen 8 adet veri bacağı bulunmaktadır. Bu bacaklar birbirlerine elektrik yolları (bus) ile bağlanır.



Şekil 1. Motorola 6802, Intel 8086 İşlemciler ile 2732 EPROM Belleği

İşlemci ile bellek arasında bağlantı sağlayan yollar (bus) adres veya veri taşır. Bu yolların veri taşıyanlarına veri yolu (data bus), adres taşıyanına ise adres yolu (address bus) adı verilir. Veri yolu; hem işlemci tarafından belleğe veri yazmak veya bellekten işlemciye taşınacak veriler için kullanıldığından çift yönlüdür. Adres yolu ise belleğin hangi bölgesindeki veriye ulaşılacağını belirleyen hatlar olup tek yönlüdür.

Bilgisayarlar elektrikle çalışan aletlerdir. İşlemci ile bellek arasında bir hatta elektrik varsa "1" yoksa "0" olarak anlamlandırılır. Bunlar ikili sayı sisteminin rakamlarıdır (Binary DigiT- bit). Dolayısıyla;

- 1 elektrik hattının taşıyacağı veri 0 ya da 1 olabilir.
- 2 elektrik hattının taşıyacağı veri 00, 01, 10 ve 11 olabilir. Yani 2^2 kadar farklı veri taşıyabilir.
- n adet elektrik hattının taşıyacağı veri 2^n farklı alternatifte bilgi olacaktır.

İşlemcinin veri yolunun (data bus) genişliği kelime uzunluğudur ve işlemcinin aynı anda işlem yaptığı bit sayısını da verir. Veri yolu, 8 bit ise 8 bitlik-BYTE işlemci, 16 bit ise 16 bitlik-WORD işlemci, 32 bit ise 32 bitlik-DWORD (Double WORD) işlemci, 64 bit ise 64 bitlik-QWORD (Quad WORD) işlemci olarak adlandırılır. Bu nedenle veri yolu genişliği ile işlemcinin akümülatör ve kaydedicilerin bit sayısı da aynı olur.

Adreslenebilir bellek miktarı ise işlemcinin adres hatlarının sayısı yani adres yolu (address bus) ile belirlenir. Adres yolu ne kadar geniş ise adreslenebilir bellek miktarı da o kadar artar.

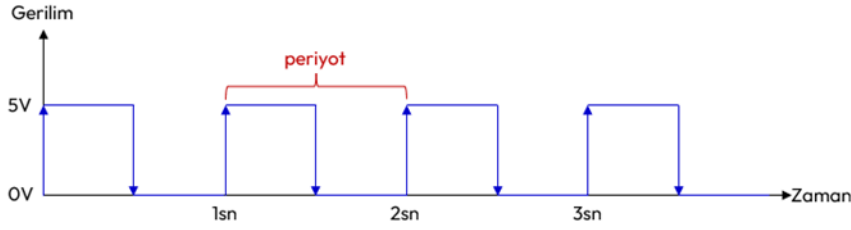
İşlemci	Veri Yolu	Kelime Uzunluğu	Adres Yolu	Adreslenebilir Bellek: bayt	Kilo bayt	Mega bayt	Giga bayt
Motorola 6802	8 Bit	BYTE	16 Bit	$2^{16}=65.536$	64		
Intel 8086	16 Bit	WORD	20 Bit	$2^{20}=1.048.576$	1024	1	
Intel 80286	16 Bit	WORD	24 Bit	$2^{24}=16.777.216$		16	
Intel Pentium	32 Bit	DWORD	32 Bit	$2^{32}=4.294.967.296$		4096	4
Intel i7	64 Bit	QWORD	36 Bit	$2^{36}=68.719.476.736$			64

Tablo 1. Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri

Bilgisayarlarda kablodaki gerilimin değeri genelde 5V, 3.3V, 2.8V, 2V, 1.5V veya 1V gibi değerler alır. Kullanıcılar için gerilimin ne olduğu değil varlığı önemlidir. Gerilim değerinin yüksek olması fazla akım çekme ve ısınma anlamına gelir. Bu nedenle zamanla daha düşük gerilimle çalışan işlemciler tasarlanmıştır.

İşlemciler kare dalga olarak oluşturulmuş elektrikselsel bir işareti saat olarak kullanır. Bu işaret işlemciye $\emptyset$ ya da CLK bacaklarından uygulanır. Saniyede 1 kez bir dalga üreten işaretin frekansı $1/1s_n = 1$ Hertz=

1 Hz. olarak hesaplanır. Bir mikro saniyede bir kez dalga üreten işaretin frekansı; $1/1\mu s = 1/10^{-6}s = 10^6$ Hertz = 1.000.000 Hertz = 1 MHz olarak hesaplanır.



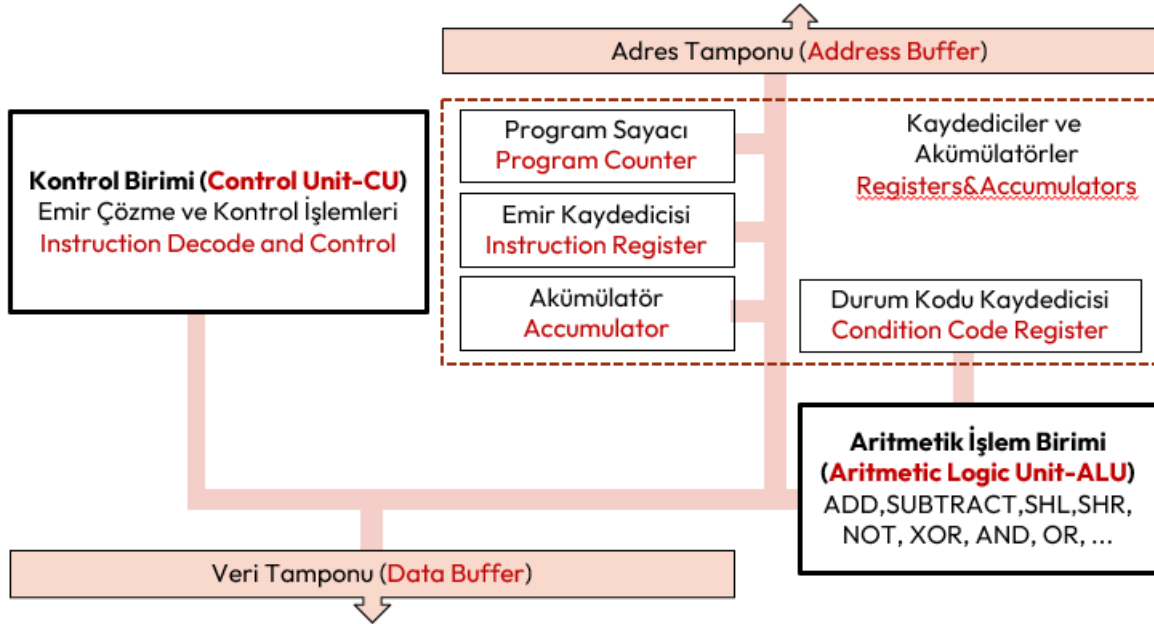
Şekil 2. Frekansı 1 olan Kare Dalga İşareti

İşlemciye uygulanan saatin frekansı ne kadar yüksek olursa işlemci o kadar hızlanır. Ancak işlemcilerin de tasarımından ötürü karşılayabileceği belli bir frekans değeri bulunmaktadır. Günümüzde 4 GHz frekansını karşılayan işlemciler bulunmaktadır.

İşlemcinin Emri Alma, Çözme ve İcra Döngüsü

Basit bilgisayarda;

- **Kaydediciler** (register), işlemci içerisinde adres ve veri saklayan mini belleklerdir.
- **Akümülatörler** ise bellekten alınan verileri saklamak için en sık kullanılan kaydedicilerdir.
- **Program sayacı** (program counter), işlemcinin icra edeceği **emri** (instruction) takip için kullanılır. Getirilecek bir sonraki emrin bellek adresini içerir.
- **Emir Kaydedicisi** (instruction register) işlemcinin o an icra ettiği emirdir.
- **Koşul kodu kaydedicisi** (condition code register), herhangi bir işlemin durumunu gösteren farklı bayraklar (flag) içerir.



Şekil 3. Basit bir İşlemcinin Yapısı

Bu bilgisayar tasarımı aynı zamanda **depolanmış program** (stored program) kavramı olarak adlandırılır ve *Von Neumann Mimarisi* olarak da bilinir². John Von Neumann tarafından 1946-1949 yılları arasında tasarlanan bu mimari günümüzde hala kullanılmakta ve modern bilgisayarlarda hala kullanılmaktadır.

Basit bir bilgisayara elektrik verildiğinde aşağıdaki üç adımlık çevrim elektrik kesilene kadar sürekli tekrarlanır;

1. **Emri Alma** (fetch): Program sayacının (program counter) tuttuğu adresteki emir (instruction) bellekten okunur.

² <https://www.youtube.com/watch?v=ByllwN8q2ss>

2. **Çözme (decode)**: İşlemci içerisindeki kontrol birimi (**control unit**) tarafından emrin ne anlama geldiği ve buna bağlı nelerin yapılacağını (örneğin ulaşılacak bellek adresi değiştirilir) belirler ve program sayacı bir sonraki emir için bir artırılır.
3. **İcra (execute)**: Çözülen emir koduna bağlı olarak gerekirse yine işlemci içinde bulunan aritmetik işlem birimini (**Aritmetic Logic Unit-ALU**) çalıştırılır ve emrin işlenmesi sonucunda ortaya çıkan değerler ise işlemci içinde bulunan **akümülatörlere (accumulator)** veya **kaydedicilerde (register)** saklanır.

Burada çözülecek **emirlerin (instruction)** her biri bir tamsayıya karşılık gelir. Bu sayılar **emir kodu (instruction code)** veya **makine kodu (opcode)** olarak da bilinirler. İşlemciler tüm sayıları emir kodu olarak anlamaz. Yani sınırlı sayıda sayı emri tanır. Yukarıda verilen Motorola 6802 işlemcisi 72 farklı emri, Intel 8086 işlemcisi ise 116 farklı emri tanır. Çözülebilecek tüm emirlerin oluşturduğu kümeye **emir seti (instruction set)** adı verilir.

İşlemci Tasarlama Stratejileri

İşlemciler, aşağıda verilen üç strateji ile tasarlanırlar;

1. **İndirgenmiş Emir Setli İşlemciler (Reduced Instruction Set Computing-RISC)**: Az sayıda emir alan yüksek hızlı işlemci tasarımı. RISC işlemciler az sayıda emre sahip olduğundan emrin çözülmesi ve icrası da kısa zaman almaktadır. Bu da çalıştırılacak kodu büyütür.
2. **Karmaşık Emir Setli İşlemciler (Complex Instruction Set Computing-CISC)**: Çok fazla emri anlayan karmaşık nispeten yavaş işlemci tasarımı. CISC işlemciler, çok sayıda emir tanıyabildiğinden emrin çözülmesi ve icra edilmesi nispeten daha uzun sürer. Ancak yüksek düzey dillerin daha kolay makine koduna çevrilmesini kolaylaştırır.
3. **Özel İşlemciler**: Bunlar grafik kartları ve yapay zekâ gibi belli amaçlar için tasarlanmış işlemcilerdir.

Genel olarak işlemcilerde “*fetch-decode-execute*” emir çevrimi 1 saat periyodunda tamamlanmaz. RISC işlemciler için bu çevrim, birçok emir için 1 saat periyodunda tamamlanır. CISC işlemciler için ise birden fazla saat periyodu gerekir. Çünkü CISC işlemcilerin emir seti daha geniş olduğundan emrin çözülmesi daha fazla vakit alır. Bu nedenle RISC işlemciler CISC işlemcilere göreceli olarak daha hızlıdır.

Günümüzde ticari olarak satılan işlemcilerin çoğu ortak emir seti kullanmaya başlamışlardır;

- Masaüstü bilgisayarların birçoğu Intel ve AMD Marka işlemciler kullanır ve bunlar IA-16, IA-32, x86-16, x86-64, AMD64, ... emir setleri ve eklentilerini desteklemektedir.
- Bilinen birçok telefon işlemcisi Acorn Risc Machine-ARM tabanlı işlemciler kullanır. Bu işlemcilerde ARMv7, ARMv8, ... emir setleri ve eklentileri kullanılmaktadır.

Program ve Programlama

Belli bir işlemi gerçekleştirmek üzere, işlemciye verilen bir dizi **emre (instruction)** **bilgisayar programı (computer program)** denir³. Bu bir dizi emrin, işlemcinin çalıştıracağı şekilde hazırlaması işlemine **programlama (programming)** denir.

Sembolik İsim (Mnemonic)	16 Tabanında Emir Kodu (Hexadecimal Opcode)	Tanım
ABA	1B	ADD A to B → A
INCA	4C	INCREMENT A
INCB	5C	INCREMENT B
SBA	10	SUBTRACT B from A → A
LDA	96	LOAD M to A, M: Memory
LDB	D6	LOAD M to B, M: Memory

Tablo 2. 6802 Emir Seti Örneği ve Sembolik İsim Listesi

³ "ISO/IEC 2382:2015". ISO. 2020-09-03. [Software includes] all or part of the programs, procedures, rules, and associated documentation of an information processing system.

Programlama **emir kodları** (**instruction code/opcode**) ile yapıldığından, programın dili, **makine dili** (**machine language**) olarak adlandırılır. Her işlemci üreticisi, her bir emre farklı makine kodu verdiği için programlama oldukça zorlu ve teknik bilgi gerektirmekteydi. Burada daha önce yazılmış bir programın ne yaptığını anlamak için her defasında bu listelere defalarca bakmak gerekiyordu.

Programın okunurluğunu artırmak için her bir emre **sembolik isimler** (**mnemonic**) verilmeye başlandı. Bu sembolik isimlerle yazılan program **montaj** (**assembly**) olarak adlandırılır. Montaj kodları bir metin dosyasına yazılarak bir dosyaya saklanır. Bu dosyalar montaj dosyaları olarak adlandırılır. Montaj dosyalarını makine diline çeviren programlar ilk **derleyicilerdir** (**compiler**). Yani derleyiciler, makine kodundan farklı olarak yazılmış program kodlarını makine koduna çeviren programlardır.

Aşağıda x86 emir seti kullanan Linux işletim sisteminde örnek **montaj** (**assembly**) örneği verilmiştir; Program genel amaçlı CL kaydedicisine koyulan sayıya kadar 2 ve 3'e bölünebilen sayıların toplamını bulup yine genel amaçlı DX kaydedicisine koyar⁴.

```
section .text                ; programı oluşturan emir kodları buradan başlar
global _start

_start:
    mov     dx, 0            ; dx 0 olsun. Toplamı tutacaktır.
    mov     cl, 99           ; 99 kez tekrarlanacak.

sum:
    mov     ax, cx           ; cx, ax e kopyalanır. Bölme işlemi için.
    mov     bl, 3            ; bl 3 olsun.
    div     bl               ; ax/bl.
    cmp     ah, 0            ; Kalan olup olmadığı kontrol ediliyor.
    jne     cont             ; Kalan yoksa, 'cont' etiketine git.
    mov     bl, 2            ; Sonra bl 2 olsun.
    div     bl               ; ax/bl
    cmp     ah, 0            ; Kalan olup olmadığı kontrol ediliyor.
    jne     cont             ; Kalan yoksa, 'cont' etiketine git.
    add     dx, c            ; Değilse, cx deki sayı 2 ve 3'e bölünür. Toplama ekle.

cont:
    loop    sum              ; Bir sonraki sayı için 'sum' etiketine dön.

exit:
    mov     eax, 1           ; Artık programdan çıkıyoruz.
    mov     ebx, 0           ; Return 0.
    int     80h              ; linux çekirdeğini çağır. (soft interrupt 80h!).
```

Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri

Genel amaçlı yani herkesin kendi amacına göre kullanacağı bilgisayarlar; Veri girişini sağlayan klavye, verileri görüntüleneceği monitör, işlenen verilerin elektrik kesildiğinde saklanacağı disk ve benzeri donanımlara sahip olmalıdır.

Genel amaçlı bilgisayarın kullanıcı isteklerine cevap verebilmesi için;

1. Çeşitli programlara sahiptirler. İşte bu programlar bütününe **işletim sistemi** (**operating system**) adı verilir. DOS, Windows, Unix, Linux, MacOS, ChromeOS, OS/2 bunların en çok bilinenleridir.
2. Elektrik verildiğinde klavye, monitör ve disk olup olmadığı kontrol eden Basic Input Output System-BIOS programı çalıştırılır. BIOS gerekli kontrollerden sonra bilgisayara işletim sistemini yükler. İşletim sistemi yüklendikten sonra, işletim sistemi kullanıcının vereceği komutları bekler.
3. Kullanıcı işletim sistemine vereceği konutlarla yeni bir program derleyebilir, derlenmiş bir programı çalıştırabilir.
4. Kullanıcının vereceği komutlar grafik olmayan işletim sistemlerinde (DOS, Unix, Linux gibi) komut yazılarak verilir. Komut yazılan bu ekrana **konsol** (**console**), terminal ya da komut satırı (**command prompt**) adı verilir. Sunucu işletim sistemlerinde hala tercih edilmektedir.

⁴ <https://github.com/armut/x86-Linux-Assembly-Examples>

5. Grafik ara yüzlerine sahip işletim sistemlerinde (Windows ve MacOS İşletim Sistemleri ile KDE, Mate, GNome, Cinnamon, LXQt, XFce ve Deepin gibi grafik ara yüzleri içeren Linux İşletim Sistemleri) fare tıklamasıyla çoğu komut verilebilmektedir.

İşletim sistemlerinin yaygınlaşması ile kullanıcının bilgisayarı donanım bilmeden yapabilmesi sağlanmıştır. Günümüzdeki işletim sistemlerinin temel programlama dili olarak C dili kullanılmaktadır.

TEMEL YAZILIM KAVRAMLARI

Tarihçe

1642 Yılında *Blaise Pascal* tarafından icat edilen Pascalline Hesap Makinesi, eldeli toplama, ödünç almalı çıkarma yapıyordu.

1671 Yılında *Gottfried Wilhelm Leibniz* tarafından icat edilen Leibniz Çarkı, toplama, ödünç almalı çıkarmanın yanında bölme, çarpma ve karekök işlemlerini yapıyordu.

1801 Yılında *Joseph Maria Jacquard* dokuma tezgahlarında desenleri oluşturmak için **delikli kartlar** (**punch card**) kullanmayı icat etmiştir. Sonradan bilgisayara veri girdisi sağlamak ve sonuçların çıktısını göstermek için kullanılacaktır.

1830 Yılında *Charles Babbage* fark makinesi olan analitik makineyi icat etmiştir. Buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Byron*, Babbage ile çalışmıştır. Byron, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi *Bernoulli* sayılarını üretmek için kullandı. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir. Bu sebeple *Byron*, ilk programcı olarak kabul görür.

1854 Yılında *George Boole* tarafından ikili sayı sistemini geliştirmiş bugün bilgisayarlarımızın kullandığı bool cebiri üzerine çalışmalar yapmıştır.

1890 Yılında *Herman Hollerith* tarafından delikli kartlarla bilgilerin yüklenebildiği ve bu bilgiler üzerinde toplama işlemlerinin yapılabilirdiği bir elektro mekanik araç geliştirdi. ABD'nin 1890 nüfus sayımında başarılı biçimde kullanıldı. *Hollerith* başarılı olunca bir şirket kurdu ve daha sonra üç firma işle birleşerek 1924 yılında adını IBM olarak değiştirdi.

1941 Yılında *Konrad Zuze Z3* isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

1944 Yılında *Howard Aitken*, IBM ile iş birliği yaparak MARK I'i yaptı. Bilgiler, MARK I'e delikli kartlarla veriliyor ve sonuçlar yine delikli kartlarla alınıyordu. Saniyede 5 işlem yapabiliyordu. Boyutları, 18 m uzunluğunda ve 2,5 m yüksekliğinde idi. İnsan müdahalesi olmadan sürekli olarak, hazırlanan programı yürüten ilk bilgisayar idi ve tam olarak elektronik bir bilgisayar değildi.

1943-1945 Yılları arasında *Konrad Zuze*, ilk yüksek düzey programlama dili olan Plankalkül'ü, Z1 bilgisayarı için geliştirmiştir. **Emir kodlarını** (**instruction code**) ve **sembolik isimleri** (**mnemonic**) içermeyen programlama dilleri **yüksek düzey programlama dili** (**high level programming language**) olarak adlandırılır. Yüksek düzey programlama dilleri çoğunlukla İngilizce sözcükleri kullanır.

1946 Yılında *John Von Neumann* önderliğinde Pensilvanya Üniversitesinde ENIAC (Elektronik Sayısal Hesaplayıcı ve Doğrulamalı) isimli sayısal elektronik bilgisayar tamamlandı. Askeri amaçla üretildi ve top mermilerinin menzillerini hesaplamak için kullanıldı. *Neumann* tarafından geliştirilmiş bu mimari (Stored-program computer and Universal Turing machine & Stored-program computer), günümüzdeki bilgisayarlarda hala kullanılmaktadır. 18,000 adet elektronik tüp kullanılan ENIAC; 150 kW gücünde idi ve 50 ton ağırlığıyla 167 m² yer kaplıyordu. Saniyede 5.000 toplama işlemi yapabiliyordu. Mark-I'den 1000 kat daha hızlıydı.

Bir önceki bölümde anlatıldığı üzere emir kodu veya sembolik isimler içeren diller **düşük düzey programlama dili** (**low level programming language**) olarak adlandırılır. Bu programlamanın yapılabilmesi için hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan matematikçiler bu programlama türünü kullanır.

Genel Amaçlı Yüksek Düzey Diller

Sembolik isimler yerine bildiğimiz konuşma diline yakın talimatlar olarak metin olarak yazılan programlara ilişkin diller, **yüksek düzey programlama dilleri** (**high level programming language**)

olarak adlandırılır. Yani **emir kodu** (instruction code) ve **sembolik isim** (mnemonic) kullanmayan programlama dilleri yüksek düzey programlama dili olarak adlandırılır.

Yüksek düzey dillerde **talimatlar** (statements) olarak yazılan programlar yine kendine has derleyiciler tarafından makine diline çevrilir. Yüksek seviyeli bir dilde yazılmış bir **talimat** (statement), bilgisayara belirtilen bir eylemi gerçekleştirmesini söyler. Talimatlar, yapılaması gerekeni kısa ve net olarak belirten **mantıksal cümlelerdir** (logical sequences). Her talimat, genellikle birden fazla makine kodu ya da **emirden** (instruction) oluşur.

1954 Yılında FORTRAN (FORmula TRANslation), *John Backus* liderliğinde IBM 704 için geliştirilirdi. 32 farklı **talimata** (statement) sahiptir. İlk genel amaçlı, yüksek düzey bir programlama dilidir.

Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN II programı verilmiştir⁵. Görüldüğü üzere talimatlar kısa öz ve anlaşılırdır.

```
C AREA OF A TRIANGLE - HERON'S FORMULA
C INPUT - CARD READER UNIT 5, INTEGER INPUT
C OUTPUT -
C INTEGER VARIABLES START WITH I,J,K,L,M OR N
  READ(5,501) IA,IB,IC
501 FORMAT(3I5)
  IF (IA) 701, 777, 701
701 IF (IB) 702, 777, 702
702 IF (IC) 703, 777, 703
777 STOP 1
703 S = (IA + IB + IC) / 2.0
  AREA = SQRT( S * (S - IA) * (S - IB) * (S - IC) )
  WRITE(6,801) IA,IB,IC,AREA
801 FORMAT(4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2,
  $13H SQUARE UNITS)
  STOP
  END
```

1956 Yılında ticari amaçlarla kullanılabilen ve seri halde üretimi yapılan ilk bilgisayar UNIVAC I oldu. UNIVAC, giriş-çıkış birimleri manyetik bant idi ve bir yazıcıya sahipti. UNIVAC 1951-1959 arasında üretilen bilgisayarlarda vakum tüpleri kullanıldı. Bu tüpler bir ampul büyüklüğünde, çok fazla enerji harcamakta ve çok fazla ısı yaymakta idiler. Veri ve programlar manyetik teyp ve tambur gibi bilgi saklama araçlarıyla saklandı. Veriler ve programlar bilgisayara delgi kartları ile yükleniyordu. 1959 yılı sonrasında üretilen bilgisayarlarda transistörler (yaklaşık 10 bin adet) kullanıldı. Artık transistor içeren ikinci kuşak bilgisayarlar hayatımıza girmiştir.

Arapsaçı Kod

1955-1959 Yılları arasında FLOW-MATIC B0(Business Language Version 0) Dili, *Grace Hopper* tarafından UNIVAC I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda B0 Dilinde örnek bir program verilmiştir⁶.

```
(0) INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1) COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2) TRANSFER A TO D .
(3) WRITE-ITEM D .
(4) JUMP TO OPERATION 8 .
(5) TRANSFER A TO C .
(6) MOVE UNIT-PRICE (B) TO UNIT-PRICE (C) .
(7) WRITE-ITEM C .
(8) READ-ITEM A ; IF END OF DATA GO TO OPERATION 14 .
(9) JUMP TO OPERATION 1 .
```

⁵ https://github.com/scivision/fortran-ii-examples/blob/main/herons_formula.f

⁶ <https://gist.github.com/marccg/f9f2da31a973ba319809>

```
(10) READ-ITEM B ; IF END OF DATA GO TO OPERATION 12 .
(11) JUMP TO OPERATION 1 .
(12) SET OPERATION 9 TO GO TO OPERATION 2 .
(13) JUMP TO OPERATION 2 .
(14) TEST PRODUCT-NO (B) AGAINST ZZZZZZZZZZ ; IF EQUAL GO TO OPERATION 16 ;
    OTHERWISE GO TO OPERATION 15 .
(15) REWIND B .
(16) CLOSE-OUT FILES C ; D .
(17) STOP . (END)
```

Görüldüğü üzere bu tarihlere kadar yazılan programlarda bir sürü GO TO ya da JUMP TO **talimatı** (**statement**) bulunmaktadır. Bu durumda programın ne yaptığı konusunda fikir yürütmeye çalıştığımızda okunaklılık oldukça azdır. İşte okunaklılığın az olduğu bu program kodlarına **arapsaçı kod** (**spaghetti code**) adı verilir. Günümüzde ne yaptığı anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

1958 Yılında LISP programlama dili, *John McCarthy* önderliğinde *Steve Russell* tarafından geliştirilmiş ikinci yüksek düzey programlama dilidir.

1959 Yılında Common Business-Oriented Language-COBOL programlama dili, *CODASYL* komitesi tarafından İngilizceye benzer ikinci yüksek düzey bir dil olarak geliştirilmiştir.

1958 Yılında daha sonra ALGOritmic Language-ALGOL adını alan International Algebraic Language-IAL Programlama Dili geliştirilmiştir. Bu dil daha sonra ortaya çıkan birçok dile önderlik etmiştir. Bu dil ile **kod blokları** (**code block**), **iç içe fonksiyon** (**nested function**), değişken **kapsamı** (**scope**) ve **dizi** (**array**) kavramları programcının hayatına girmiştir. ALGOL dili, barındırdığı özellikler sebebiyle, kendisinden sonra ortaya çıkan bütün programlama dilleri için bir esin kaynağı olmuştur.

İşin doğası gereği makine diline çevrilecek tüm metinler gözle okunabilmektedir. Yani bütün kaynak kodlar gözle okunabilir.

Değişken

Değişkenler (**variable**) aslında cebirden bildiğimiz değişkenlerdir. Cebir (**algebra**) işlemlerinde doğrudan problemde verilen sayıları değil, onları temsil eden değişkenleri kullanırız.

$$3x^2 + 2x + 10$$

$$c^2 = a^2 + b^2$$

$$c = 2\pi r$$

Yukarıdaki cebirsel ifadelerin ilkinde bir polinom, ikincisinde ise bir dik üçgenin kenar uzunlukları arasındaki ilişki verilmiştir. Bu ifadelerde **x**, **a**, **b** ve **r** bağımsız değişken, **c** ise bağımlı değişkendir. Aslında problemi çözerken bunlar sayılara karşılık gelir. Örneklerdeki **3,2,10,2** ve **π** ise **değişmez** (**literal**) sayılardır.

Bilgisayarda ise sayıları tutacak değişkenler, bir miktar bellek bölgesini işgal eder ve bu bellek bölgesinde çeşitli biçimlerde tutulurlar. Tamsayılar, **ikili** (**binary**) sayı olarak bellekte tutulur. Bu ikili sayının en anlamlı bitinin işaret biti olarak kullanılıp kullanılmayacağına göre bellekte biçimlendirilir;

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
8 Bit	BYTE	-127 ile +128 arasında veya 0 ile 255 arasında
16 bit	2 BYTE	-32.767 ile +32.768 arasında veya 0 ile 65.535 arasında
32 bit	4 BYTE	-2.147.483.648 ile +2.147.483.647 veya 0 ile 4.294.967.295 arasında
64 bit	8 BYTE	-9.223.372.036.854.775.808 ile + 9.223.372.036.854.775.807 arasında veya 0 ile 18,446,744,073,709,551,615 arasında

Tablo 3. Tamsayı Değişkenler ve Sayı Sınırları

Gerçek sayılar **kayan noktalı sayı** (**floating point number**) olarak biçimlendirilir; 1914 yılında *Leonardo Torres Quevedo* tarafından Charles Babage'ın analitik makinesine dayalı kayan nokta analizi yayınladı.

$$12345 = 1234,5 \times 10^1 = 123,45 \times 10^2 = 12,345 \times 10^3 = 1,2345 \times 10^4 = 0,12345 \times 10^5$$

Yukarıda ondalık tabanda verilen gerçek bir sayıya ilişkin kesir noktasının kaydırılması ile aynı sayı ifade edildiği bir örnek verilmiştir. Kesir noktası sola kaydırıldığında 10 ile çarpılmış, sağa kaydırıldığında ise 10 ile bölünmüş olur. Kayan noktalı sayının üs kısmı dışında kalan kısmı **taban (base)** veya **kesir (fraction)** olarak adlandırılır. Tabanın kesir noktası sağında kalan hanelerine ise **mantis (mantissa)** adı verilir.

1938 yılında *Konrad Zuse*, ilk **ikili (binary)** programlanabilir mekanik bilgisayar olan Z1'i yapmıştı. Bu bilgisayar, 7 bitlik işaretli üs, 17 bitlik anlamlı değer ve bir işaret biti içeren 24 bitlik ikili tabanda kayan noktalı sayı kullanmıştır. Bilgisayarlarda tamsayılar olduğu gibi kayan noktalı sayılar da ikili sayı tabanında tutulur⁷. Örneğin ondalık tabanda 50,25 sayısı ikili tabanda 1.1001001×2^5 olarak ifade edilir.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
32 bit	4 BYTE	$1,2 \times 10^{-38} \dots 3,4 \times 10^{+38}$ Tek hassasiyetli gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 8 bit üs ve geri kalan 23 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.
64 bit	8 BYTE	$2,3 \times 10^{-308} \text{ ila } 1,7 \times 10^{+308}$ Çift hassasiyetli gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 11 bit üs ve geri kalan 52 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.

Tablo 4. Kayan Noktalı Sayılar ve Sınırları

Program yazılırken tanımlanacak değişkenlere ilişkin bellek ihtiyacı programcı tarafından belirlenir. Programcı değişkene tahsis edilecek bellek miktarına ve tutacağı içeriğe göre **veri tipi (data type)** belirler. Programcı tarafından kullanılacağı amaca göre belirleyeceği veri tipinde istediği kadar değişken tanımlayabilir.

Fonksiyon

Matematikte **fonksiyon (function)**, bir veya daha fazla kümenin elemanını tek bir kümenin elemanına eşleyen **ilişkidir (relationship)**.

Girdi	İlişki	Çıktı
1	$\times 2$	2
2	$\times 2$	4
3	$\times 2$	6
4	$\times 2$	8
...		

Tablo 5. İki ile çarpma fonksiyonu.

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematiksel olarak aşağıdaki şekilde **ifade (expression)** edilir.

$$f(x) = 2x$$

Burada f, fonksiyon anlamında x ise girdi anlamında olup parametre olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x$$

Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$x \in \mathbb{R} \text{ olmak üzere } f(x) = 2x + 3$$

Bu fonksiyon, reel sayı kümesindeki her **x** elemanını, hesaplanan **f(x)** değerine eşleyen bir ifadedir. Yani **x=1.0** argümanını **f(1.0)=5.0** reel sayısına eşler. Kısaca fonksiyon **5.0** reel sayısını hesaplayıp **döndürür (return)**.

⁷ IEEE Computer Society (2019-07-22). IEEE Standard for Floating-Point Arithmetic. IEEE STD 754-2019. IEEE. pp. 1-84.

$a \in \mathbb{R}$ ve $b \in \mathbb{Z}$ olmak üzere $g(a, b) = 2a + b$

Bu fonksiyon ise reel sayı kümesindeki her a elemanı ile tamsayı kümesindeki b elemanını, hesaplanan $g(a, b)$ değerine eşleyen bir ifadedir. Yani $a=1.0$ ve $b=1$ argümanlarını $g(1.0, 1)$ fonksiyonu yine reel sayı kümesinden 3.0 reel sayısına eşler. Kısaca fonksiyon, 3.0 reel sayısını hesaplayıp döndürür (return). Buradaki x , a ve b parametre olarak adlandırılır. $f(1.0)$ ifadesindeki 1.0 ise parametrenin o an andığı değeri belirtir ve argüman (argument) veya gerçek parametre (actual parameter) ya da olarak adlandırılır. Bu fonksiyonlar, $f(3.0)+g(3.0, 2)+f(2.0)+g(1.0, 3)$ gibi tanımlandıktan sonra tekrar tekrar çağrılabilirler (call).

Yapısal Programlama

1958 Yılındaki FORTRAN-II diline çıktı üretmeyen fonksiyonlar için SUBROUTINE, değer üreten fonksiyonlar için FUNCTION, CALL ve RETURN özellikleri eklenmiştir.

1958 ile 1969 Yılları arasında ALGOL diline çıktı üretmeyen fonksiyonlar için PROCEDURE ve BEGIN-END arasında kod bloğu özellikleri eklenmiştir.

1966 Yılındaki FORTRAN-IV diline INTEGER, REAL, DOUBLE PRECISION, COMPLEX ve LOGICAL veri tipleri (data type) ile Mantıksal IF, aritmetik (üç yollu) IF ve DO döngüsü talimatları (statement) eklenmiştir.

Bu yıllarda her ne kadar çıktı üretmeyen fonksiyonlar için SUBROUTINE, birden fazla çağrı noktası olabilen COROUTINE ve FUNCTION kullanılıyor olmasına rağmen hala GO TO/JUMP TO ifadeleri kullanılmakta ve kafa karışıklığına devam etmekteydi. 1968 Yılında *Edsger W. Dijkstra* önderliğinde GO TO/JUMP TO ifadesini zararlı olarak ilan edilmiştir.

1970 Yılında *Niklaus Wirth* tarafından Pascal dili geliştirildi. Bu dil ilk geliştirilen yapısal programlama dilidir. Yapısal programlamada (structural programming) verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (if, for, do, repeat) birbirinden ayrılmıştır. Pascal dili, kendi kendini çağıran özyinelemeli (recursive) fonksiyonlar ile değişkenlerin adreslerini tutan göstericileri (pointer) desteklemektedir⁸.

Yapısal programlamanın genel çerçevesi;

1. Programın başladığı yeri belirten bir ana fonksiyon (main function) tanımlanır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Yani programlama genel olarak fonksiyonların birbirini çağırmasıyla yapılır.
2. Her fonksiyonda ilk önce veri yapıları (data structure) tanımlanır. Veri yapıları;
 - a. En basit veri yapısı int, float ve double gibi ilkel veri tiplerinden (data type) olabilen değişken (variable) olabileceği gibi,
 - b. İlkel veri tiplerinden meydana gelen dizi (array),
 - c. İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan yapılar (structure) olabilir.
 - d. Fonksiyonda tanımlanan veri yapılarının ardından bu yapıları işleyecek kontrol yapıları (control structure) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe aşağıdaki talimatlardan (statement) oluşur. if, if-else, switch, while, for, do-while, continue, break, goto ve return.

Pascal dilinde ana fonksiyon nokta ile biten BEGIN-END bloğudur. C ve C++ dilinde ana fonksiyon, main() fonksiyonudur. C# dilinde ise Main() fonksiyonudur. C# dilinin yazım kurallarını aldığı C Dilinden burada bahsetmek gerekir;

C Programlama Dili, *Dennis M. Ritchie* tarafından Bell Laboratuvarlarında UNIX işletim sistemini geliştirmek için geliştirilen genel amaçlı, üst düzey bir dildir. C, ilk olarak 1972'de DEC PDP-11 bilgisayarında çalıştırılmıştır. *Ken Thompson* tarafından geliştirilen B adlı daha eski bir dilden gelişmiştir. 1978 yılında *Brian Kernighan* ve *Dennis Ritchie*, günümüzde K&R standardı olarak bilinen C'nin ilk kamuya açık kılavuzunu hazırlamışlardır. C programlama dili; Öğrenmesi kolaydır, Yapısal (structural) programlama dilidir, Hızlı ve verimli (efficient) programlar üretir, Assembly dili desteği ile

⁸ Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

düşük düzey programlamayı (low level programming) da destekler, Standart başlık dosyaları sayesinde çeşitli işletim sistemlerine taşınarak (portable) derlenebilir.

C Dilinde yapısal programlama yapıldığından veri ile bunu işleyen kontrol yapıları birbirinden ayrıldığından arapsaçı programlamanın aksine okunaklılık oldukça yüksektir.

Nesne Yönelimli Programlama İhtiyacı

Yapısal programlamada talimatlar (statements) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya emreden programlama (imperative programming) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler. Emreden paradigmanın bir örneği olan yapısal programlamada;

- Kod ne kadar büyürse, modüllere ayırma imkânı olmasına rağmen, fonksiyonlar arasındaki bağımlılık da o kadar artar. Bir fonksiyonun parametrelerindeki değişiklik, onun kullanıldığı tüm yerlerde değişiklik gerektirir. Değişim yönetimi (change management) çok zordur ve uzun zaman alır.
- Hata durumunda yapılacak işlemlere ilişkin kod ile iş sürecini gerçekleştiren kod iç içedir. Aynı fonksiyonun bazı durumlarda hata, bazı durumlarda ise değer döndürmesi mümkündür. Çoğu durumda hataların izini sürmek (error handling) işi zorlaştırır.
- Yapısal programlamada, gösterici (pointer) kullanımında erişilmesi istenmeyen bellek bölgelerine erişilmesi halinde istenmeyen program davranışları ortaya çıkar. Kodun güvenli (safe code) olarak çalıştığının incelenmesi çok zordur.
- Sürekli olarak benzer projelerde aynı kodları tekrar yazmak (duplicate code) gereklidir.
- Emreden paradigmanın burada belirtilen sebepler başta olmak üzere çeşitli problemleri bulunmaktadır. Bu nedenle 1980'li yıllarda birçok yazılım projesi başarısız (fail) olmuştur. Bir yazılım; Kullanıcı ihtiyaçlarının karşılanamaması, Öngörülen bütçenin aşılması ve Zamanında teslim edilememesi durumlarında başarısız olur;

Yapısal programlamadaki bu sıkıntıları gören yazılımcıların imdadına Simula ve Smalltalk programlama dillindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi.

Smalltalk, 1972 yılında Xerox Park şirketinde Alan Kay önderliğinde Dan Ingalls, Adele Goldberg, Ted Kaehler, Diana Merry, Scott Wallace ve Peter Deutsch tarafından geliştirilmiş bir dildir. Smalltalk dilinde nesneler (object) temel yapıtaşlarıdır. Nesneler;

- Durum (state) ve davranışlara (behavior) sahiptir.
- Programlama, nesnelerin birbirlerine ileti göndermesiyle (message-passing) yapılır.
- Nesnelerin kendi ya da bir başka nesnenin davranış ve durumlarını öğrenebilme yeteneği yani yansıma (reflection) özelliği vardır.

Emreden paradigma (imperative programming) aksine nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılan programlamaya nesne yönelimli programlama (object oriented programming-OOP) paradigması adı verilir.

C++ programlama dili, C diline yapılan nesne yönelimli programlama (object oriented programming-OOP) eklentileri yapılarak geliştirilmiş bir programlama dilidir. 1979 senesinde bir Danimarkalı bilgisayar bilim adamı olan Bjarne Stroustrup, sonradan C++ olarak bilinecek olan "C with Classes" üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı. C++, nesne yönelimli programlama kavramlarına sahip C programlama dilinin bir uzantısıdır. Bu nedenle saf nesne yönelimli programlama dili değildir.

C# programlama dili, Anders Hejlsberg, Scott Wiltamuth ve Peter Golde tarafından Microsoft Şirketine geliştirilip ilk olarak Temmuz 2000'de dağıtılmıştır. C# dilindeki karma (#) karakteri, C++ diline de eklenti (++) yapılarak dört adet + karakterinin bir araya gelmesi anlamındadır.

2016 yılına kadar kapalı kaynak (closed source) olarak .Net platformunun bir parçası olan bu dil 2016 yılında açık kaynak (open source) haline gelmiştir. Şu an için Windows, Linux ve macOS işletim

sistemleri için ücretsiz ve açık kaynaklı bir bilgisayar yazılım çerçevesi (framework) olarak kullanıcılara sunulmuştur.

C# programlama dili birden fazla paradigmayı destekleyen genel amaçlı (general purpose), saf nesne yönelimli üst düzey programlama dilidir (high level programming language).

Nesne Yönelimli Programlama

Yapısal programlamada talimatlar (statement) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya emreden programlama paradigması (imperative programming) adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler

Emreden programlama (imperative programming) bir örneği olan yapısal programlamada talimatlar (statements) art arda koda yazılarak programlama yapılır. 1980'li yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür. Kodlar büyüdükçe sorunlar artmış ve *Nesne Yönelimli Programlama İhtiyacı* başlığı altında anlatılan problemler ortaya çıkmıştır. Bu yıllarda Simula ve Smalltalk yaklaşımı yazılımcıların imdadına yetişmiştir.

Bu dillerde nesneler programın (objects) temel yapıtaşlarıdır. Nesneler; durum (state) ve davranışlara (behavior) sahiptir. Programlama, nesnelerin birbirlerine ileti göndermesiyle (message-passing) yapılır. Emreden paradigmanın aksine nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılan programlamaya nesne yönelimli programlama (object oriented programming) paradigması adı verilir.

C# programlama dili, Anders Hejlsberg, Scott Wiltamuth ve Peter Golde tarafından Microsoft Şirketinde geliştirilip ilk olarak Temmuz 2000'de dağıtılmıştır. C++ ve C dilinden daha fazla kütüphaneye sahiptir. Birçok kavramı bir anda özümsemek gerektiğinden öğrenme eğrisi diktir.

DERLEYİCİ VE ÇALIŞTIRMA ORTAMI

Genel amaçlı bilgisayarların çok kullanılmasıyla birlikte metin editörleri de (Windows Not Defteri, Mac TextEdit, Notepad++, Epsilon, EMACS, nano, vim veya vi) işletim sistemlerinin bir parçası olmuştur. Metin editörlerinde yazılan programlar ise **derleyiciler** (**compiler**) tarafından makine koduna çevrilerek **icra edilebilir** (**executable**) dosya olarak kaydedilebilmektedir.

C# dilinde programlama öğrenmeye başlamak için ilk adım, programı yazabileceğiniz bir metin editörü kullanmak ve yazdığınız programa ilişkin kaynak kodları ***.cs** uzantısı ile dosya olarak saklamaktır. İkinci adım ise .Net Yazılım Çerçevesinin (Kısaca **.Net Framework**) uygun sürümünü **işletim sisteminizde** (**operating system**) kurmaktır. Üçüncü adım ise yazdığınız kodu derleyerek çalışabilen bir icra edilebilir dosyayı oluşturup çalıştırmaktır. Derleyici için girdi, kaynak kodu dosyasıdır. Kaynak dosyasında yazılan kaynak kodu, içerisinde C# **talimatları** (**statement**) olan, insan tarafından okunabilen metin dosyasıdır.

.NET Framework

.NET Framework, platformun çalışan uygulamalarına çeşitli hizmetler sağlayan bir **çalıştırma ortamıdır** (**run-time**). İki ana bileşenden oluşur:

1. **Base Class Library (BCL)**: Geliştiricilerin kendi uygulamalarından çağırabileceği, test edilmiş ve yeniden kullanılabilir bir kitaplık olan **.NET Framework sınıf kitaplığıdır** (**.Net Class Library**).
2. **Common Language Runtime (CLR)**: Uygulamaları çalıştıran ve yöneten motordur (engine) yani **ortak dil çalışma ortamıdır**. İş parçacığı (**thread**) yönetimi, **çöp toplama** (**garbage collection**), **tip güvenliği** (**type safety**), **istisna işleme** (**exception handling**) ve daha fazlası gibi hizmetler sağlar.

.NET Framework çalışan uygulamalar için sağlanan hizmetleri sunar:

1. Bellek yönetimi: Birçok programlama dilinde, programcılar bellek ayırmayı ve serbest bırakmayı ve nesne ömrünü işlemekten sorumludur. .NET Framework uygulamalarda, CLR bu hizmetleri uygulama adına sağlar.
2. Ortak tip sistemi (**Common Type System-CTS**): Geleneksel programlama dillerinde temel tipler derleyici tarafından tanımlanır, bu da başka dillerde yazılan uygulamaların ortak çalışabilirliği karmaşılaştırır. .NET Framework, temel tipleri ortak tip sistemi tarafından tanımlanır ve .NET Framework ile çalışan tüm dillerde ortaktır.
3. Kapsamlı bir sınıf kitaplığı: Programcılar, sık kullanılan alt düzey programlama işlemlerini işlemek için çok miktarda kod yazmak yerine, .NET Framework sınıf kitaplığından bir tür ve üyeleri olan kolay erişilebilir bir kitaplık kullanır.
4. Geliştirme çerçeveleri: .NET Framework, web uygulamaları için ASP.NET, veri erişimi için ADO.NET, hizmet odaklı uygulamalar için Windows Communication Foundation (WCF).
5. Dil birlikte çalışabilirliği: .NET Framework hedef olan dil derleyicileri, kaynak kodları **ortak ara dile** (**Common Intermediate Language-CIL**) derler ve CLR tarafından çalışma zamanında derlenir. Bu özellik ile, bir dilde yazılan yöntemler diğer diller tarafından erişilebilir olur. Böylece programcılar tercih ettiği dilde uygulama oluşturmaya odaklanmaktadır.
6. Sürüm uyumluluğu: Nadir özel durumlar dışında belirli bir .NET Framework sürümü kullanılarak geliştirilen uygulamalar, daha sonraki bir sürümde değişiklik yapılmadan çalışır.
7. Yan yana çalıştırma: .NET Framework, aynı bilgisayarda ortak dil çalışma ortamının birden çok sürümünün var olmasına izin vererek sürüm çakışmalarını çözmeye yardımcı olur. bu, birden çok uygulama sürümünün ve bir uygulamanın oluşturulduğu .NET Framework sürümünde çalışabileceği anlamına gelir.
8. Çoklu sürüm desteği: Geliştiriciler standart bir sürümü tarafından desteklenen birden çok .NET Framework platformda çalışan sınıf kitaplıkları oluşturur.

Ortak Dil Tarifnamesi

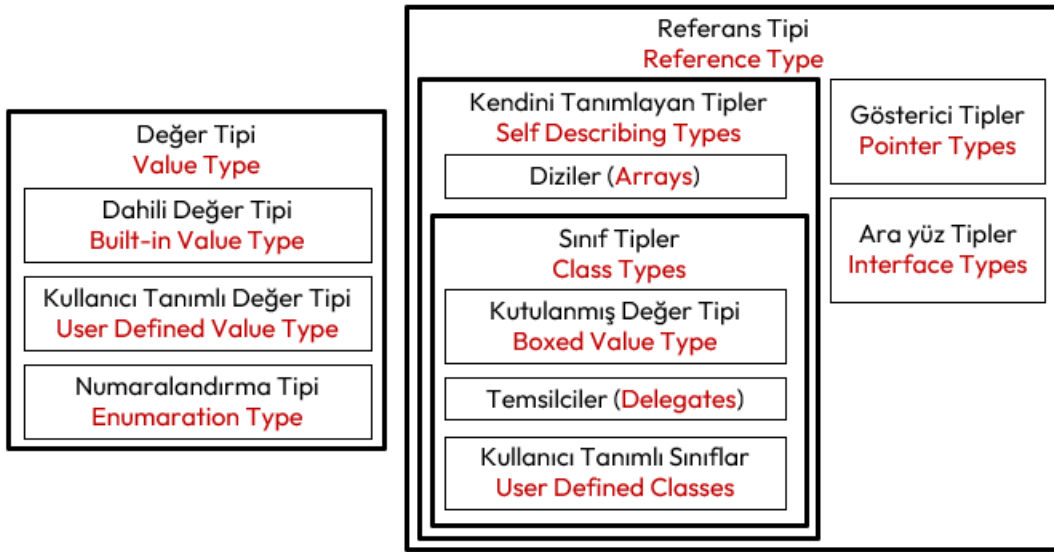
Ortak Dil Tarifnamesi (**Common Language Specification-CLS**) programlama dillerinin nesne modellemesinden kaynaklanan farklılıklarını bertaraf etmek için geliştirilmiş standartlar dizisidir. Bunlar aşağıda belirtilen başlıklarda toplanmaktadır;

- **Yapı** (**structure**) standardı.
- **Ortak ara dil** (**Common Intermediate Language-CIL**): Programlama dili ne olursa olsun, bu diller herkes tarafından bilinecek bir ara koda dönüştürülür ya da derlenir.
- **Veri tipi standardı** (**Common Type System-CTS**) aşağıda anlatılmıştır.
- **Olay** (**event**) standartları.
- Verilerin **bellekteki yerleşimine** (**persistence**) ilişkin standartlar: Aynı blok içinde tanımlanan **char**, **int**, **double** değişkenlerinin tanımlandığı örneği göz önüne alacak olursak; C dilinde, belleğe ilk önce **char**, sonra **int**, daha sonra **double** yerleşir (**row major**). Pascal dilinde ise ilk önce **double**, sonra **int**, son olarak da **char** yerleşir (**column major**).

İşte bu tip değişken yerleşimlerinden kaynaklanacak problemleri gidermek için bu standartlar oluşturulmuştur.

Ortak Tip Sistemi

Ortak Tip Sistemi (**CTS**), dillerdeki veri tipleri arasındaki farklılıkların bertaraf edilmesine yönelik standartlardır. Bu sistem ile bir dilde 4 bayt, diğer bir dilde 8 bayt olan gerçek sayı tanımlaması gibi karmaşıklıklar giderilmiştir.



Şekil 4. Ortak Tip Sistemi (CTS)

CTS, aşağıdaki veri tiplerini desteklemektedir:

1. **Değer Tipleri** (**value type**)
 - a) **Dâhili değer tipleri** (**built-in value type**): **bool**, **integer**, **byte**, **double**, **char** gibi veri tipleridir.
 - b) **Kullanıcı tanımlı değer tipleri** (**user-defined value type**): **struct** gibi kullanıcı tarafından, yerleşik değer tiplerinin çeşitli şekillerde bir araya getirilip tek bir isim altında toplanmasıyla oluşan tiplerdir.
 - c) **Numaralandırma** (**enumeration**): Bir kümeye ait elemanların her birine bir numara atanarak kod okunabilirliğini yükseltmek için tanımlanan **enum** tipleridir.
2. **Referans Tipler** (**reference type**)
 - a) **Gösterici tipler** (**pointer type**)
 - b) **Ara yüz Tipler** (**interface type**)
 - c) **Kendini tanımlayan tipler** (**self-description type**)
 - d) **Diziler** (**array**): Tek boyutlu, çok boyutlu ya da **dizilerin dizisi** (**jagged array**) gibi diziler.
 - e) **Sınıflar** (**class**): **object** ve **string** gibi ön tanımlı sınıflar.

- f) Kullanıcı tanımlı sınıflar (user-defined class)
- g) Doğrudan ulaşımı engellenmiş ya da kutulanmış değer tipler (boxed value types)
- h) Temsilciler (delegate): Yöntemleri referans gösteren tipler.

Derleme ve İcra Süreci

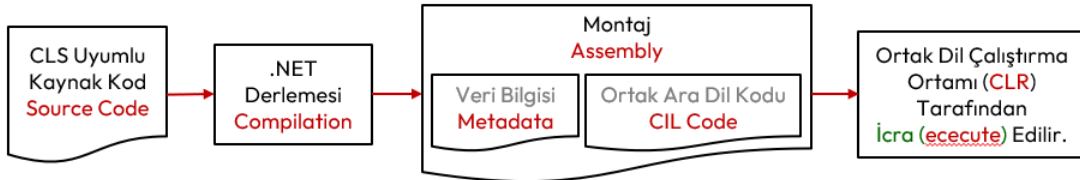
Klasik yazılım geliştirme araçlarıyla derlenen bir uygulama aşağıda gösterildiği gibi bir süreçten geçer. Yani kaynak kod derlenir ve doğrudan işletim sistemi tarafından işlemciye hedef gösterilerek çalıştırılacak şekilde emir kodlarından (instruction code) oluşan çalıştırılabilir bir uygulama elde edilir.



Şekil 5. Klasik Derleme Süreci

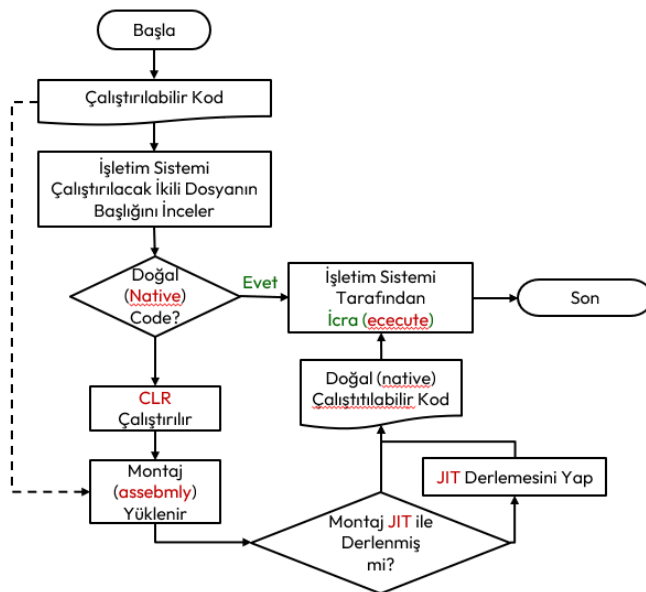
İşlemci doğal olarak ikili sayı sisteminde çalıştığından bu çalıştırılabilir koda ikili-çalıştırılabilir (binary-executable) veya doğal kod (native code) adı verilir.

.NET Framework altyapısında ise kaynak kod derlenir ve ortak dil çalışma ortamı (CLR) tarafından çalıştırılacak şekilde ortak ara dil (CIL) ve veri bilgisinden (metadata) oluşan bir uygulama elde edilir. Derlenmiş bu .Net uygulamasına montaj (assembly) adı verilir.



Şekil 6. .Net Framework Derleme Süreci

Burada derlenecek kaynak kod, ortak dil tarifnamesinde (CLS) belirtilen standartlara uymak zorundadır. CIL kodu ile oluşan bu montaj (assembly) uygulamanın işlemcinin anladığı emir kodları (instruction code) ile hiçbir ilgisi yoktur ve işletim sistemine bağımlı değildir. Bu şekilde yapılan .NET Framework derleme işlemine, yönetilmiş derleme (managed compilation) adı verilir. Burada CLS uyumlu olarak hazırlanmış kaynak kodlara yönetilmiş kod (managed code) adı verilir. CLS uyumlu hazırlanmayan kodlara ise yönetilmemiş kod (unmanaged code) adı verilir. Yönetilmiş kodlar CLR tarafından, yönetilmemiş kodlar ise işletim sistemi tarafından çalıştırılır.



Şekil 7. .Net Altyapısı Derleme ve Çalıştırma Süreci

İşletim sistemi, tarafından bir çalıştırılabilir (executable) kodun icrasını yukarıdaki şekilde yapar. Klasik derleme sonucunda oluşan doğal kod (native code), işletim sistemi tarafından doğrudan belleğe yüklenerek çalıştırılır. .NET Framework de ise, montaj (assembly) uygulamanın çalışması için, ortak dil

çalıştırma ortamının (CLR) işletim sisteminde çalışıyor olması gerekir. Çalışıyor olan CLR, ilgili montaj uygulamasını belleğe yükler ve **ani derleyiciler** (**Just-in-time compiler-JIT compiler**) tarafından derlenerek **doğal kod** (**native code**) elde edilir. Bu doğal kod, CLR kontrolünde, işletim sistemi tarafından çalıştırılır. Kısaca **ortak dil altyapısı** (CLI) standartlarına uyumlu olarak çalışan **ortak dil çalıştırma ortamı** (CLR), kurulu olduğu işletim sisteminin kaynaklarını kullanarak, oluşturulan CIL kodunun doğru bir şekilde çalıştırılmasını sağlayan altyapıdır.

Yukarıda anlatılan derlenmiş montaj, hangi işletim sistemine taşınırsa taşınsın, derlenmeden çalıştırılması sağlanmış olur. Bunun için tek gereklilik CLR altyapısının o işletim sisteminde çalışıyor olmasıdır. CLR altyapısı, bir işletim sisteminde çalışan **bileşenin** (**component**) başka bir işletim sistemine problemsiz olarak taşınarak çalışmasını sağlayan bir yapıdır. İşte bu işleme **karşılıklı çalışma** (**interoperability**) adı verilir. Yani uygulamanın işletim sistemine olan bağımlılığını ortadan kaldırmaktadır.

Microsoft tarafından Visual Basic, Visual Java, Visual C++ dillerinde küçük değişiklikler yapılarak CLS uyumlu hale getirilerek .NET dilleri geliştirilmiştir. Böylece yazılımcı takımında dil farklılıklarından doğan geliştirme problemlerine de çözüm bulunmuştur. Böylece yazılımcıların .Net mimarisine geçmek için harcayacağı eğitim ve zamandan tasarruf sağlamıştır.

Ortak dil çalıştırma ortamının montaj uygulamayı belleğe yükleyip, ani derleyicilerden geçirerek işlemcinin çalıştıracığı **doğal kod** (**native code**) elde edilir. Bazı durumlarda içinde bulunan işletim sistemine uygun olarak bu doğal kod önceden oluşturulabilir. Buna **vaktinden önce doğal derleme** (**native ahead-of-time- NAOT**) adı verilir⁹. Bu durum için .net derleyicileri optimize edilmemiştir. Bazı test senaryoları için kullanılabilir.

Ortak Ara Dil Kodu

Yapısal programlama ve fonksiyon kavramının bilgisayarlarda kullanılmasıyla birlikte işlemciler de de değişiklikler olmuştur. *İşlemcinin Emri Alma, Çözme ve İcra Döngüsü* başlığı altında belirtilen işlemci kaydedicilere yığın bellek adresini tutan (**stack register**) kaydediciler gibi yeni kaydediciler de eklenmiştir.

Ortak dil tarifnamesinde (CLS) belirtilen standartlara uymak zorunda olan birçok programlama dili bulunmaktadır; VB.NET, C#.NET, VC++.NET, J#.NET, F#.NET, Jscript.NET, WindowsPowerShell, Iron phyton, Iron Ruby. Bu dillerde yazılan kodlar derlenerek **ortak ara dile** (CIL) dönüştürülür.

Fiziki bir işlemci için derlenmiş **doğal kod** (**native code**) dosyaları gibi, .Net altyapısında da **ortak dil kodu** (CIL code), sanki sanal bir işlemci varmış gibi üretilir ve **çalıştırma ortamı** (CLR) tarafından içinde bulunduğu işletim sistemine ait işlemcinin emir kodlarına JIT ile derlenir.

Gerçek işlemcilerde olduğu gibi CIL koduna derlenen her bir uygulama, dört **bellek bölgesinden** (**segment**) oluşur;

1. **Kod bellek** ya da kaynak koda ithafla **metin bellek** (**code segment / text segment**) icra edilecek emirleri içeren kısım.
2. Programın işleyeceği verileri tutan **veri belleği** (**data segment**) kısmı.
3. Yöntemleri **çağırmadan** (**call**) önce mevcut işlemci durumu ve yerel değişkenler gibi bazı işlemleri depolamak için ve fonksiyondan **dönülünce** (**return**) işlemciyi ve çağrı ortamını eski hale getirmek için kullanılan **yığın bellek** (**stack segment**) kısmı. Bu bellek bu nedenle **son giren veri ilk çıkar** (**last in first out-LIFO**) mantığıyla çalışır.
4. Programın icrası sırasında dinamik olarak eklenip çıkarılan veriler için hurdalık gibi kullanılan **öbek bellek** (**heap segment**) kısmı.

.NET Framework Kurulumu

Çerçeve yapının son sürümü, <https://dotnet.microsoft.com/en-us/download> adresinden indirilerek kurulur. Kurulum tamamlandıktan sonra konsoldan aşağıdaki komut yazılarak kurulum teyit edilebilir;

⁹ <https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/?tabs=windows%2Cnet8>

```
C:\Users\ILHANOZKAN>dotnet

Usage: dotnet [options]
Usage: dotnet [path-to-application]

Options:
  -h|--help          Display help.
  --info             Display .NET information.
  --list-sdks        Display the installed SDKs.
  --list-runtimes    Display the installed runtimes.

path-to-application:
  The path to an application .dll file to execute.
C:\Users\ILHANOZKAN>
```

Sonrasında PATH değişkenine C# Derleyicisinin aşağıda belirtilen yolu eklenir.

```
C:\Windows\Microsoft.NET\Framework\v4.0.30319\
```

Yerel ortam değişkenlerini Windows bilgisayarda iki şekilde ayarlayabilirsiniz:

Birinci Seçenek;

1-Başlat'a tıklayın, "Hesabınızın ortam değişkenlerini düzenleyin" arayın ve sonuçlarda Denetim Masasını uygulamasını tıklayın.

2-Kullanıcı değişkenlerini değiştirebileceğiniz bir pencere açacaktır.

3-"... için kullanıcı değişkenleri" grubunda "Path" değişkenine çift tıklayarak yukarıda verilen dosya dolunu ekleyebilirsiniz.

İkinci Seçenek;

1-Başlat'a sağ tıklayın ve komut satırına tıklayın.

2-sysdm.cpl yazın ve Tamam'a tıklayın

3-"Gelişmiş" sekmesine tıklayın ve ardından alttaki "Ortam Değişkenleri..." düğmesine tıklayın

4-Bu seçenek, kullanıcı değişkenlerini ve sistem değişkenlerini değiştirebileceğiniz önceki seçenekteki pencereyi açacaktır.

5-"... için kullanıcı değişkenleri" grubunda "Path" değişkenine çift tıklayarak yukarıda verilen dosya dolunu ekleyebilirsiniz.

Böylece kurulum tamamlanmış olur. Konsolda **csc** komutu yazılarak derleyicinin düzgün kurulup kurulmadığı kontrol edilebilir.

```
C:\Users\ILHANOZKAN>csc.exe
Microsoft (R) Visual C# Compiler version 4.8.9232.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.

This compiler is provided as part of the Microsoft (R) .NET Framework, but only supports
language versions up to C# 5, which is no longer the latest version. For compilers that
support newer versions of the C# programming language, see
http://go.microsoft.com/fwlink/?LinkID=533240

warning CS2008: Hiçbir kaynak dosya belirtilmedi
error CS1562: Kaynağı olmayan çıkışlar için /out seçeneği belirtilmiş olmalıdır

C:\Users\ILHANOZKAN>
```

Daha sonra metin editörü ile ilk kaynak kodumuzu oluşturmak için komut satırında **notepad hello.c** komutu yazılır.

```
C:\Users\ILHANOZKAN>notepad hello.cs
```

Eğer böyle bir dosya yoksa editör bu dosyayı oluşturmak için onay ister. Açılan metin editörü penceresine ilk C programı yazılır ve **hello.cpp** olarak dosyaya kaydedilir.


```
using System;
using System.IO;

namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Aşağıdaki komut ile **hello.cpp** olarak hazırlanan kaynak kodumuz derleyici tarafından icra edilebilir **hello.exe** dosyası olarak derlenir.

```
C:\Users\ILHANOZKAN>csc hello.cs
```

Derlenen programı çalıştırmak için yine aynı komut satırında **hello.exe** yazarak icra edilebilir programımız işletim sistemi tarafında işlemciye hedef gösterilerek çalıştırılır.

```
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
```

```
C:\Users\ILHANOZKAN>
```

Burada programın **notepad** programında yazılması, **csc** programı derlenmesi ile ve derlenen **hello.exe** programının çalışması tarafımızdan yapılmıştır.

Mac OSX ve Linux işletim sistemlerinde de derleyici kurulumu tamamlandıktan sonra terminal yani konsol uygulaması açılarak aşağıdakine benzer şekilde derlemeler yapılabilir.

Entegre Geliştirme Ortamı

Şimdiye kadar anlatılan **kod yazma** (implementation), **derleme** (compile), **icra** (execute) ya da **koşma** (run) ve **izleme** (trace) gibi süreçlerin tamamını tek bir çatı altında yürütmemizi sağlayan programlar, **entegre geliştirme ortamı** (Integrated Development Environment-IDE) olarak adlandırılır. C# programlama dili için en çok kullanılan entegre geliştirme ortamları aşağıda verilmiştir;

- **Visual Studio** – Microsoft tarafından geliştirilen, C dilinde yazılmış, güçlü, yüksek performanslı uygulamalar oluşturmak için kullanılabilen bir geliştirme ortamıdır. Yalnızca Windows'ta çalışır. Visual Studio, **kod tamamlama** (intellisense), **kullanıcı ara yüzü** (user interfaceUI) hazırlama desteği ve **hata ayıklama** (debugger) ve birçok **eklentisi** (plug-in) olan muazzam özelliklere sahiptir. Bu geliştirme ortamının kurulumu dipnotta verilmiştir.
- **Visual Studio Code** – Microsoft tarafından geliştirilen açık kaynaklı bir geliştirme ortamıdır. Windows, macOS ve Linux gibi işletim sistemlerinde çalışır. Git **sürüm kontrol** (version control) sistemleriyle çok iyi çalışır. Ayrıca, akıllı kod tamamlamanın dikkate değer özellikleriyle birlikte gelir.
- **Xcode** – MacOSX işletim sisteminde yazılım geliştirmek için kullanılan bir entegre geliştirme ortamıdır.
- **JetBrains Rider** – Windows, Mac OS X ve Linux'ta .NET ve .NET Core uygulamalarını destekliyor. Rider'ın hızlı performansı, geniş platform ve çalışma zamanı desteği, sürüm kontrol sistemleriyle sorunsuz entegrasyonu ve kapsamlı **derleme çözümleme** (decompile) özellikleri öne çıkan özelliklerinden bazılarıdır.
- **MonoDevelop** – .NET dilleri için oluşturulmuş açık kaynaklı bir geliştirme ortamıdır. Yazılım geliştirme için Mono ve .NET framework'ünü kullanır. MonoDevelop ile MacOSX, Windows ve Linux dahil olmak üzere çeşitli işletim sistemleri için masaüstü ve web uygulamalarını verimli bir şekilde oluşturabilirsiniz.
- **.NET Core CLI** – .NET Core uygulamalarını geliştirmek, oluşturmak, çalıştırmak ve yayınlamak için **komut satırı arayüzüdür** (command line interface). Bu ara yüz adından da anlaşılacağı üzere grafik ara yüzünden yoksundur.

C# ile Nesne Yönelimli Programlama ve Entity Framework: <https://github.com/HoydaBre/csharp>

Entegre geliştirme ortamları metin dosyası olarak tutulan kaynak kodları programcıya renklendirerek gösterir. Bu da programcının kodu anlamasını daha da kolaylaştırır. Ancak kodu dosyaya yine sade metin olarak kaydeder.

Çevrimiçi Derleyiciler

Bütün bu entegre geliştirme ortamlarının yanında online olarak da derleme yapılan web uygulamaları da bulunmaktadır. Bunlardan birkaçı aşağıda verilmiştir;

- <https://www.tutorialspoint.com/csharp/index.htm>
- https://www.onlinegdb.com/online_csharp_compiler
- <https://onecompiler.com/csharp>

Derleme Zamanı

Derleyici programının derleme işlemine başlayıp bitirildiği sürece **derleme zamanı** (**compile time**) denir. Bu süreç başarısızlıkla da sonuçlanabilir ve eğer derleme işleminde hata meydana gelirse programcı hata mesajları ile uyarılır.

Bir derleyici program, kaynak dosyayı makine diline çevirme çabasında, kaynak dosyanın C# dilinin sözdizimi kurallarına uygunluğunu da denetler. Eğer dilin kurallarına uyulmamışsa, derleyici bu durumu bildiren bir ileti de vermek zorundadır.

```
using System;
using System.IO;
namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!")
        }
    }
}
```

Yukarıda hatalı yazılmış bir c programı derlendiğinde aşağıdaki hata alınacaktır.

```
C:\Users\ILHANOZKAN>csc.exe hello.cs
Microsoft (R) Visual C# Compiler version 4.8.9232.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.

This compiler is provided as part of the Microsoft (R) .NET Framework, but only supports
language versions up to C# 5, which is no longer the latest version. For compilers that
support newer versions of the C# programming language, see
http://go.microsoft.com/fwlink/?LinkID=533240

hello.cs(10,47): error CS1002: ; bekleniyor

C:\Users\ILHANOZKAN>
```

Hatalar

Programcı tarafından kodlama yapılırken genellikle aşağıdaki üç tip hata yapılır;

1. **Derleme zamanı hataları** (**compile time error**): Bu tip hatalar genelde kullanılan dilin **yazım kurallarına** (**syntax**) uyulmadığından, **talimatların** (**statement**) yanlış yazılmasından ya da programcının kod metninde uygun olmayan karakterlerin kullanılmasından kaynaklanır.
2. **Koşma zamanı hataları** (**run time error**): Kaynak kod, kurallara uygun olarak yazılmıştır ve herhangi bir yazım hatası bulunmaz. Bu tip hatalara en iyi örneklerden birisi sifıra bölme hatasıdır. Taşma hataları da bu hatalardandır. Daha sonra değinilecektir.

3. **Mantıksal hatalar** (logical error): Programcının çözüm için gerekli adımların oluşturulmasında, çözüm yönteminin yanlış olmasından ya da yanlış **işleçler** (operator) kullanmasından kaynaklanır. Örneğin bir büyüktür (>) işleci yerine küçüktür (<) işleci kullanıldığında ne bir yazım hatası ne de bir çalışma zamanı hatası ortaya çıkar. Fakat program kendisinden istenilen davranışları yerine getirmez ve uygun çıktıları üretmez. Faktöriyel hesaplanırken $0! = 1$ yerine $0! = 0$ kabul etmek buna örnek verilebilir.

Yapısal Programlamanın Genel Çerçevesi

Yapısal programlamanın genel çerçevesi aşağıda verilmiştir;

1. Programın başladığı yeri belirten bir **ana fonksiyon** (main function) tanımlanmak zorundadır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Yani iş sürecini sağlayacak programlama fonksiyonların birbirini çağırmasıyla yapılır.
2. Her fonksiyonda; İlk önce değişken, dizi ve yapı gibi **veri yapıları** (data structure) tanımlanır. Daha sonra bu veri yapılarını işleyecek **if, if-else, switch, while, for, do-while, continue, break, goto** ve **return** gibi **kontrol yapıları** (control structure) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe **talimatlardan** (statement) oluşabilir.

Yapısal programlamaya örnek olarak aşağıdaki C kodu örnek verilebilir;

```
//Koda Dahil Edilen Başlık
#include<stdio.h>

//Fonksiyon Prototipleri:
void diziOku(int pDizi[],int pUzunluk);
void diziYaz(int pDizi[],int pUzunluk);
float diziOrtalama(int pDizi[],int pUzunluk);

//Ana Fonksiyon
int main() {
    //Burada yapılmak istenen iş süreci fonksiyonlar ardımıyla yapılır.
    int dizi[5]; // Ana fonksiyonda da veri yapısı tanımlanabilir.
    float ortalama;
    diziOku(dizi,5);
    diziYaz(dizi,5);
    ortalama=diziOrtalama(dizi,5);
    printf("Dizi Ortalaması: %.2f",ortalama);
    return 0;
}

//Fonksiyonların (Prototipleri Yukarıda Verilmiş) Tanımlamaları:
void diziOku(int pDizi[],int pUzunluk){
    int sayac; // Veri yapısı tanımlandı
    printf("%d Elemanlı Dizi Okunacaktır:",pUzunluk);
    for (sayac=0; sayac < pUzunluk; sayac++) //Kontrol Yapısı
        scanf("%d",&pDizi[sayac]);
}

void diziYaz(int pDizi[],int pUzunluk){
    int sayac; // Veri yapısı
    printf("%d Elemanlı Dizi:\n",pUzunluk);
    for (sayac=0; sayac < pUzunluk; sayac++) // Kontrol Yapısı
        printf("%4d",pDizi[sayac]);
    printf("\n");
}

float diziOrtalama(int pDizi[],int pUzunluk){
    int sayac; // Veri yapıları
    float toplam=0;
    for (sayac=0; sayac < pUzunluk; sayac++) //Kontrol Yapısı
        toplam+=pDizi[sayac];
    return toplam/pUzunluk; //Kontrol Yapısı
```

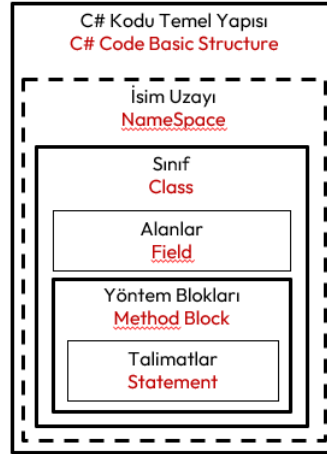
}

Nesne Yönelimli Programın Genel Çerçevesi

C# dili, Java dili gibi saf nesne yönelimli bir dildir. Nesne olmadan program yazılamaz. Nesne yönelimli programlamada;

- Nesneler, **durum** (state) ve **davranışlara** (behavior) sahiptir.
- Kodlaması yapılacak iş, nesnelerin birbirlerine **ileti göndermesiyle** (message-passing) yapılır.

Nesnelerin hangi durum ve davranışa sahip olacağı programcı tarafından kullanıcı tanımlı **sınıflarla** (class) belirlenir. Bu nedenle her c# programı en az bir sınıf içerir.



Şekil 8. C# Kodu Temel Yapısı

Kodun genel yapısı aşağıda verilmiştir;

```

1  using System; // System adındaki isim uzayındaki bileşenleri kullanacağız!.
2
3  namespace ÖrnekUygulama // ÖrnekUygulama adında bir isim Uzayı
4  { // Blok Başı
5      class Sınıf // "Sınıf" adında bir Sınıf.
6      {
7          // Alan Değişkenleri (fields):
8          int yaş = 5;
9          // Özellikler (properties):
10         public int Yaş
11         {
12             get { return yaş; }
13             set { yaş = value; }
14         }
15         public string Adi { get; set; }
16
17         // Metotlar (methods):
18         public void BilgileriYazdırYöntemi()
19         {
20             Console.WriteLine($"Yaş: {Yaş}, Adı: {Adi}"); //Talimat (statement)
21         }
22     }
23 } // Blok Sonu

```

Bir **sınıf** (class) her zaman bir **namespace** ile belirtilen bir isim uzayında olmak zorunda değildir. Sınıflar **alan değişkenleri** (field), **özellikler** (property) ve **yöntemler** (method) içerebilirler.

Yapısal programlamada ana fonksiyon ile program başlar. Nesne yönelimli programlamada ise programın başladığı yer için bir sınıfta tanımlanan **static void Main(string[] args)** yöntemi eklenir. Böylece derleme sonrasında **icra edilebilir** (executable) bir **uygulama** (application) ortaya çıkar. Bu yöntemin olmadığı kodlar **sınıf kütüphanesi** (class library) olarak adlandırılır.

Bir proje birden fazla kod dosyasından oluşabilir. Her bir sınıf ayrı bir dosyada yer alabilir. Tüm projeyi oluşturan bu kodlarının tamamına **kod tabanı** (codebase) adı verilir.

Aşağıda yukarıdaki sınıftan nesne imal edip kullanan bir konsol uygulaması verilmiştir;

```
using System; // System adındaki isim uzayındaki bileşenleri kullanacağız!.
namespace ÖrnekUygulama // ÖrnekUygulama adında bir isim Uzayı
{
    class Sınıf // "Sınıf" adında bir sınıf
    {
        // Alan Değişkenleri (fields):
        int yaş = 5;
        // Özellikler (properties):
        public int Yaş
        {
            get { return yaş; }
            set { yaş = value; }
        }
        public string Adi { get; set; }

        // Metotlar (methods):
        public void BilgileriYazdırYöntemi()
        {
            Console.WriteLine($"Yaş: {Yaş}, Adı: {Adi}");
        }
    }
    class Program // Programı başlatacak Main yöntemini içeren sınıf
    {
        static void Main(string[] args) // Programın başlangıç noktası
        {
            Sınıf nesne = new Sınıf(); // Sınıftan "nesne" adlı nesne imal edildi
            nesne.Yaş = 10; // nesnenin Yaş özelliğine değer atama
            nesne.Adi = "Ali"; // nesnenin Adi özelliğine değer atama
            nesne.BilgileriYazdır(); // nesnenin yöntemini çağırarak nesneye ileti-gönderme
        }
    }
}
```

C# Projesi Oluşturma ve Çalıştırma

Visual Studio 2022 ile Proje Oluşturma

Programda sırasıyla şu adımlar takip edilir: *Visual Studio* açılır;

1. Programı ilk defa çalıştırıyorsanız, karşılama sayfasından *Create a new project* seçiniz. Eğer program açık ise *File* menüsüne giderek *File → New Project* seçiniz.
2. Proje tipi olarak *Console Application* seçilir.
3. *Project Name*, *Location* ve *Solution Name* belirlenerek ilk proje oluşturulur.
4. Projeye oluşturma sihirbazının *Additional Information* sayfasında *Do not use top-level statements* kutucuğu ön tanımlı olarak **ŞEÇİLİ** değildir. Bu durumda projemiz yeni proje şablonuyla oluşur.
5. *Solution Explorer* üzerinde *Program.cs* dosyası açılır.

.NET 6'dan başlayarak, yeni proje şablonuyla oluşan C# konsol uygulamaları için **Program.cs** dosyası aşağıdaki şekilde oluşur;

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

Yeni proje şablonu ile oluşturulan projelerde **Program.cs** kodunun başında aşağıdaki açıklama her zaman bulunur;

```
// See https://aka.ms/new-console-template for more information
```

Eğer eski proje şablonuyla proje oluşturulmak istenirse yukarıda anlatılan dördüncü adımda projeye oluşturma sihirbazının *Additional Information* sayfasında *Do not use top-level statemets* kutucuğu **ŞEÇİLİ** bırakılır. Bu durumda **Program.cs** alışık olduğumuz eski şekilde oluşur;

```
namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Aslında ilk oluşan yeni tip **Program.cs** dosyasında yazılan kodlar, ikinci oluşan eski tip **Program.cs** dosyasındaki **static void Main(string[] args)** yönteminin içerisinde yazılıyor gibi kabul edebilirsiniz. Bu proje şablonuyla oluşturulan **Program.cs** dosyasına konsola bir şey yazmak ve konsoldan bir şey okumak için aşağıdaki yönergeler uygulamaya örtük olarak eklenir¹⁰:

```
- using System;
- using System.IO;
- using System.Collections.Generic;
- using System.Linq;
- using System.Net.Http;
- using System.Threading;
- using System.Threading.Tasks;
```

6. Menüde, Hata ayıklama için *Debug* → *Start Debugging* tıklanır veya F5 tuşuna basılarak hata ayıklama modunda program çalıştırılır. Ya da hata ayıklamadan derleme için *Start Without Debugging* tıklanarak veya Ctrl + F5 tuşlarına basarak program doğrudan çalıştırılabilir.

.NET Core CLI ile Proje Oluşturma

Konsol uygulamasında sırasıyla şu adımlar takip edilir:

1. Bil klasör oluşturulur (C:\Users\ilhan>mkdir hello_world)
2. Ardından oluşturulan klasöre girilir (C:\Users\ilhan>cd hello_world).
3. Yeni bir proje oluşturmak için **dotnet new console** komutu çalıştırılır.
4. İki adet dosya oluşacaktır;

Biri proje dosyası (hello_world.csproj);

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

Diğeri program **Program.cs** dosyası;

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

Program eski şekilde oluşturulmak istenirse **dotnet new console --use-program-main** komutu ile proje oluşturulur. Bu durumda **Program.cs** alışık olduğumuz şekilde oluşur.

5. Gerekli olan paketlerin projeye dahil edilmesi için **dotnet restore** komutu kullanılır.
6. Derleme **Hata Ayıklama** (debug) için yapılacak ise **dotnet build** veya

¹⁰ <https://aka.ms/new-console-template>

7. Derleme **Yayım** (**release**) için yapılacak **dotnet build -c Release** ile yapılır.
8. Derleme sonrası oluşan uygulamaya çalıştırılacak ise **dotnet run** komutu yazılarak yapılır.

```
C:\Users\ilhan>mkdir hello_world

C:\Users\ilhan>cd hello_world

C:\Users\ilhan\hello_world>dotnet new console
"Konsol Uygulaması" şablonu başarıyla oluşturuldu.

Oluşturma sonrası eylemleri işleniyor...
C:\Users\ilhan\hello_world\hello_world.csproj geri yükleniyor:
Geri yükleme başarılı oldu.

C:\Users\ilhan\hello_world>dotnet restore
Geri yükleme tamamlandı (0,3sn)

"0,6" sn'de başarılı oluşturun

C:\Users\ilhan\hello_world>dotnet run
Hello, World!

C:\Users\ilhan\hello_world>
```

C# PROGRAMLAMA DİLİNE GİRİŞ

Söz Dizim Kuralları

C# dili için oluşturulacak kaynak kod programcı tarafından metin dosyasına yazılırken belli **söz dizim kullarına** (**syntax**) uyar. C# dilinde de kaynak kodu oluşturan karakterler ön tanımlı olarak UTF-8 karakterleridir. Aşağıda belirtilen anahtar kelimelerle program yazılır.

Talimatlar ve Anahtar Kelimeler

Anahtar kelimeler (**keywords**), programlamada kullanılan ve C# derleyicisi tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimelerdir** (**reserved word**). Başka hiçbir amaç için veya değişkenler veya yöntem veya sınıf adları gibi yerlerde kimliklendirme için kullanılamazlar.

C# sözdiziminin bir parçası olan önceden tanımlanmış sözcüklerdir ve **sınıf** (**class**), **alan** (**field**) ve **yöntem** (**method**) blokları içinde olan **talimatların** (**statement**), bir parçasıdır. **Talimatlar** yapılıması gerekeni kısa ve net olarak belirten **mantıksal cümlelerdir** (**logical sequences**). C# programlama dilinde;

Anahtar kelimeler, küçük-büyük harf ayrımı yapıldığından her zaman küçük harflerle yazılırlar.

Yöntemler içinde yer alacak her talimat (**statement**), anahtar kelimeler içerecek şekilde yazılır ve noktalı virgül karakteri ile biter. Bu nedenle talimatlar **gelişigüzel** (**free format**) yazılabilir.

abstract	event	namespace	static
as	explicit	new	string
base	extern	null	struct
bool	false	object	switch
break	finally	operator	this
byte	fixed	out	throw
case	float	override	true
catch	for	params	try
char	foreach	private	typeof
checked	goto	protected	uint
class	if	public	ulong
const	implicit	readonly	unchecked
continue	in	ref	unsafe
decimal	int	return	ushort
default	interface	sbyte	using
delegate	internal	sealed	virtual
do	is	short	void
double	lock	sizeof	volatile
else	long	stackalloc	while
enum			

Tablo 6. Anahtar Kelime Listesi¹¹

C# dilinde anahtar kelime olmayıp **bağlam** (**context**) olarak kullanılan başka anahtar kelimeler de vardır;

add	field	not	select
allows	file	notnull	set
alias	from	nuint	unmanaged (function pointer calling convention)
and	get	on	unmanaged (generic type constraint)
ascending	global	or	value
args	group	orderby	var
async	init	partial (type)	when (filter condition)
await	into	partial (member)	where (generic type constraint)
by	join	record	where (query clause)

¹¹ <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/>

descending	let	remove	with
dynamic	nameof	required	yield
equals	nint	scoped	managed (function pointer calling convention)

Tablo 7. Bağlam Olarak Kullanılan Anahtar Kelime Listesi

Yüksek seviyeli bir dilde yazılmış bir **talimat** (**statement**), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, işlemciye gönderilen birden fazla CIL kodu ile gerçekleştirilir.

Açıklamalar

C# programlama dilinde tırnaklı parantez ile yazılan kodlar birbirinden ayrılır. Parantez açıldıktan sonra kapanana kadar yazılanlar **kod bloğu** (**code block**) olarak adlandırılır. İstenildiği kadar blok iç içe alınabilir Her **talimat** (**statement**), anahtar kelimeler içerecek şekilde yazılır ve noktalı virgül karakteri ile biter. Programcı kodu yazarken **açıklama** (**comment**) yapma gereği duyabilir. Bunu iki şekilde yapar;

1. Kod metninde bulunulan yerden sonra satırın sonuna kadar olan bir açıklama yapılmak istendiğinde, açıklama öncesinde iki adet bölü karakteri (//) kullanılır.
2. Eğer birden çok satırı içeren bir açıklama yapılacak ise açıklama /* ile */ karakterleri arasına alınır.

Aşağıda açıklama eklenmiş kod örneği bulunmaktadır.

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Merhaba!"); //Talimat ; ile biter.
/*
Bu program, açıklama (comment) içeren basit bir ana fonksiyonu olan C# programıdır.
İlhan ÖZKAN, Ocak 2025
*/
{ // Bi Kod Bloğu Başlangıcı
  { // İç Kod Bloğu Başlangıcı
    //iç bloğa ait girinti...
    Console.ReadLine(); //Konsoldan Satır Okuma Talimatı.
    //...
  } //İç Kod Bloğu Bitişi
} // Bir Kod Bloğu Bitişi
```

C# dilinde kodun bir başka programcı tarafından anlaşılması için bir kod bloğu içindeki açıklama ve talimatları bloktan itibaren girintili olarak yazarız. Kodu bu şekilde yazmamızın derleyici açısından hiçbir önemi yoktur. Derlemenin ilk aşamasında bütün bu açıklamalar ve beyaz boşluk karakterleri metinden çıkartılır.

Değişken Tanımlama

C# dilinde yöntemler, yapısal programlama dilindeki fonksiyonlar gibi tanımlanır. Yöntemlerde **veri yapıları** (**data structure**) ve **kontrol yapıları** (**control structure**) birbirinden ayrıdır. Yöntem kodlanırken önce veriyi taşıyan veri yapıları tanımlanmalı daha sonra veriyi işleyecek kontrol yapıları tanımlanmalıdır.

Değişkenler veri yapılarının temel öğeleridir. **Değişkenin** (**variable**) ne olduğu *Değişken* başlığı altında verilmiştir. Değişkenler, belleğin belli bir bölgesinde yer kaplar ve çeşitli biçimde verilere ilişkin değerleri tutarlar.

Değer Tipler

C# dilinde bellekte yer işgal eden ve değerleri tutan değişkenler **value tipler** (**value type**) olarak adlandırılır:

1. **Tamsayı değer tipleri** (**integral value type**):

Takma Adı	Aralık	Bellek Boyutu	.NET Veri Tipi
sbyte	-128 ile 127	Signed 8-bit integer	System.SByte

byte	0 ile 255	Unsigned 8-bit integer	System.Byte
short	-32,768 ile 32,767	Signed 16-bit integer	System.Int16
ushort	0 ile 65,535	Unsigned 16-bit integer	System.UInt16
int	-2,147,483,648 ile 2,147,483,647	Signed 32-bit integer	System.Int32
uint	0 ile 4,294,967,295	Unsigned 32-bit integer	System.UInt32
long	-9,223,372,036,854,775,808 ile 9,223,372,036,854,775,807	Signed 64-bit integer	System.Int64
ulong	0 ile 18,446,744,073,709,551,615	Unsigned 64-bit integer	System.UInt64

Tablo 8. Tamsayı Değer Tipleri¹²

2. Kayan noktalı değer tipler (floating point value type):

Takma Adı	Yaklaşık Değer Aralığı	İncelik	Bellek Boyutu	.NET Veri Tipi
float	$\pm 1.5 \times 10^{-45}$ ile $\pm 3.4 \times 10^{38}$	~6-9 hane	4 byte	System.Single
double	$\pm 5.0 \times 10^{-324}$ ile $\pm 1.7 \times 10^{308}$	~15-17 hane	8 byte	System.Double
decimal	$\pm 1.0 \times 10^{-28}$ ile $\pm 7.9228 \times 10^{28}$	28-29 hane	16 byte	System.Decimal

Tablo 9. Kayan Noktalı Değer Tipleri¹³

- Mantıksal olarak **true** ya da **false** tutabilen **bool** değer tipi.
- Unicode UTF-16 karakterini tutan 16 bitlik **char** değer tipi. `'\u0000'` ile `'\uFFFF'` arasında değer alır. `'\u0000'` veya kısaca `'\0'` karakteri **null** karakter olarak adlandırılır.
- Yukarıda verilen değer tipleri içeren **yapılar** (**struct**) ve **numaralandırma** (**enum**) tipleri de değer tipleridir.
- Ayrıca **null** olabilir değer türü T? Yukarıda anlatılan değer tiplerinden olan T'nin tüm değerlerini ve ek bir **null** değeri temsil eder. Değer tipleri bu şekilde tanımlanmadığı sürece **null** atama yapılamaz.
- Değer **tuple** bir değer tipidir, ancak basit bir tip değildir.

void Tipi

Hiçbir değeri olmayan ve bellekte yer kaplamayan **void** veri tipi vardır. Bir yöntemin (veya yerel bir fonksiyonun) dönüş tipi olarak **void**, metodun bir değer döndürmediğini belirtmek için kullanırsınız. Ayrıca, bilinmeyen bir tipe gösterici **bildiriminde** (**declaration**) bir referans tipi **void** olarak kullanabilirsiniz.

Referans Tipler

Referans tiplerinin değişkenleri bellekte nesne referanslarını tutar. Değer tiplerinin değişkenleri doğrudan değerleri içerir. Referans tipleriyle, iki değişken aynı nesneye referans verebilir; bu nedenle, bir değişken üzerindeki işlemler diğer değişken tarafından referans verilen nesneyi etkileyebilir. Değer tiplerinde, her değişkenin kendi veri kopyası vardır ve bir değişken üzerindeki işlemlerin diğerini etkilemesi mümkün değildir.

Referans tipleri **bildirebilmek** (**declaration**) için **class**, **interface**, **delegate**, **record** anahtar kelimeleri ile nesneler oluşturulabilir. Bunların yanında C# dilinde **dynamic**, **object** ve **string** referans tipleri ön tanımlıdır.

Değişken Bildirimi

Değişkenler (**variable**) bellekte yer tahsis edilen depolama konumlarını temsil eder. Nesne yönelimli programlamada;

- Sınıflarda tanımlanan **alanlar** (**field**) değişkendir. Alanlar, sınıflardan imal edilen nesnenin durumlarına ilişkin verileri tutarlar.
- Yöntemler** (**method**) de yapısal programlamadaki fonksiyonlar gibi kodlanır. Yani yöntemler içinde ilk önce **veri yapıları** (**data structure**) tanımlanır daha sonra bu değişkenleri işeyecek kontrol

¹²<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/integral-numeric-types>

¹³<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types>

yapıları tanımlanır. İşte yöntemler içerisinde ilk önce tanımlanacak veri yapılarının en basiti, değişkenlerdir. Bu değişkenler **yerel değişken** (**local variable**) olarak adlandırılır.

Her değişkenin, hangi değerleri depolanabileceğini belirten bir tipi vardır. Bu nedenle değişkenlerin veri tipine bağlı olarak kimlik verilerek **tanımlanması** (**definition**) gerekir. Yapılan bu tanımlama işlemine **bildirim** (**declaration**) adı da verilir. Bunun için;

1. Değişken tanımlaması yapılabilmesi için ilk önce **veri tipinin** (**data type**) belirlenmesi ve
2. Değişkenin kullanılacağı yerler göz önüne alınarak benzersiz bir **kimlik** (**identifier**) verilir.

```
// See https://aka.ms/new-console-template for more information
char yas;      // veri tipi "char" ve kimliği "yas" olan değişken bildirimi
int kat;       // veri tipi "int" ve kimliği "kat" olan değişken bildirimi
float kilo;    // veri tipi "float" ve kimliği "kilo" olan değişken bildirimi
decimal para; // veri tipi "decimal" ve kimliği "para" olan değişken bildirimi
```

Yukarıda tanımlaması yapılan **yas**, **kat**, **kilo** ve **para** değişkenleri ana fonksiyon olan **Program.cs** içinde tanımlandığından, tanımlanan değişkenler sadece program içinde geçerlidir. Tırnaklı parantez ile tanımlanan kod bloğu ya da program içinde benzer kimlik verilmiş bir başka değişkene izin verilemez.

Değişkenin kapsamı (**variable scope**); Bildirimi yapılan bir **değişkenin** (**variable declaration**), **erişilebileceği** (**access**), değişkenin üzerinde çalışılabileceği program kodu olarak tanımlanır. **Yerel değişkenler** (**local variable**) bulunduğu kod bloğu içerisinde bildirimi yapıp kullanılabilen değişkenlerdir.

Değişken Kimliklendirme Kuralları

C# programlama dilinde **değişkenlerin kimliklendirilmesinde** (**identifier definition**) aşağıda verilen kurallara uyulur;

1. Kimliklendirmede anahtar kelimeler kullanılamaz!
2. Kimliklendirme rakamla başlayamaz!
3. Kimliklendirmede yukarıdaki iki kurala uyacak şekilde UTF-8 kodlamasında herhangi bir karakter kullanılabilir.

```
// See https://aka.ms/new-console-template for more information
//Tamsayı Değişken Bildirimleri:
byte yaş;
int tamsayı;
long uzunTamsayı;
//Gerçek Sayı Değişken Bildirimleri:
float oran;
double gerceksayı;
/*Aynı veri tipinde birden fazla değişkene aralarına virgül koyarak kimlik verilebilir. */
int kat1, kat2, kat3;
float ölçek1, ölçek2;
decimal alınanÜcret, paraÜstü;
```

Bütün bu kuralların yanında değişkenlere kimlik verilirken yazılan kodun okunaklılarını artırmak için **Camel Case** olarak kimlik verilir. Bu kimliklendirme türünde ilk sözcük tamamıyla küçük, izleyen sözcüklerin ise baş harfi büyük yazılır. Bu kural zorunlu değildir ama koda bakım yapacak sonraki programcıların işini kolaylaştırmak için ahlaki olarak tercih edilir.

```
// See https://aka.ms/new-console-template for more information
int sayac;
int öğrenciBoy;
int asansörünOlduğuKat;
float asansörAğırlığı;
int öğrenciYaşı;
char ilkHarf;
float ortalamaNot;
```

Değişmezler

Programcı, kodlama yaparken bazı **değişmezleri** (**literal**) ister istemez kullanır. Bu değişmezler, derleme öncesinde de sonrasında da kaynak koddakinin kelimesi kelimesine aynıdır. Değişkenlere verilen **ilk değerler** (**initial value**) ile **katsayılar** (**coefficient**) en çok kullanılan değişmezlerdir. Değişkenlere ilk değer verme işlemine **başlatma** (**initialization**) adı verilir. Değişkenler tanımlanırken sahip olacağı ilk değerleri belirten değişmezler ile verilebilir;

```
// See https://aka.ms/new-console-template for more information
uint işaretSizTamsayı= 5U;
int tamsayı = -5;
sbyte işaretliBayt = -120;
decimal ondalık = 30.5M;
double çiftİncelikliKayanNoktalıSayı = 30.5D;
float kayanNoktalıSayı = 30.5F;
long uzunTamsayı = 5L;
ulong işaretSizUzunTamsayı = 5UL;
byte bayt = 127;
short kısaTamsayı = -12;
ushort işaretSizKısaTamsayı = 220;
bool evetHayır = true;
bool doğruYanlış = false;
char karakter1 = 'h'; // Tek tırnak içinde UTF-16 kodlamasına göre
char karakter2 = '\u006A';
char karakter3 = '\x0013';
char karakter4 = 'Ş';
/* Metin değişmezler çift tırnak arasında yazılır. Bu nedenle içinde çift tırnak kullanmak
   için \" olarak yazılır. Bir sonraki satıra geçmek için \ karakteri kullanılır. \
   karakteri kullanmak için \\ yazılır. */
string metin = "hello, this is a string literal";
/* Yukarıdaki metin değişmesinde birçok kural vardır. Onları gözardı etmek için @ karakteri
   ile başlayan * tam metin (verbatim string) değişmesi kullanılır. Sadece çift tırnak
   için iki adet "" kullanılır.*/
string tammetin= @"İlhan
    Özkan \ """;
```

Aşağıdaki kod örneğinde programcı, **3f**, **3.14f**, **3.14f** ve **2.0f** olmak üzere dört farklı değişmez kullanılmıştır. Bu değişmezlerin her biri derleyici tarafından farklı olarak değerlendirilir.

```
// See https://aka.ms/new-console-template for more information
float yariCap = 3f;
float daireninAlani = 3.14f * yariCap * yariCap;
float daireninCevresi = 2.0f * 3.14f * yariCap;
```

Yerel Sabitler

Değişmezlerin (**literal**) çokça kullanılması derleme sonrası üretilen icra edilebilir kodu da büyütür. Bunun yerine çok kullanılan değişmezler bellekte bir kez yer kaplasın diye **const** kelimesiyle **yerel sabitler** (**const**) tanımlanabilir. Bulunduğu kod bloğunda değişken gibi bildirim yapılmasına rağmen yerel sabitler değişken değildir ve değiştirilemezler. Sabit değişkenler etik olarak büyük harfle kimliklendirilir.

```
// See https://aka.ms/new-console-template for more information
const float PI=3.14f;
float yariCap = 3f;
float daireninAlani = PI * yariCap * yariCap;
float daireninCevresi = 2.0f * PI * yariCap;
```

Böylece aynı yerel sabiti **PI** birden çok yerde kullanılır ve her seferinde aynı bellek bölgesine erişildiğinden bellekten tasarruf edilmiş olunur. Bildirim sırasında const sıfatı kullanılan yerel sabite **ilk değer** (**initial value**), bir **değişmez** (**literal**) ile verilmelidir.

Bir sabit tanımlamanın üstünlükleri aşağıda sıralanmıştır;

- Gelişmiş Kod Okunabilirliği: Bir değişkene `const` sıfatı vermek, diğer programcılara değerinin değiştirilmemesi gerektiğini belirtir; bu da kodun anlaşılmasını ve sürdürülmesini kolaylaştırır.
- Gelişmiş Veri Tipi Güvenliği: `const` kullanılması, değerlerin yanlışlıkla değiştirilmemesini sağlayabilir ve kodumuzda hata ve eksiklik olma olasılığını azaltabilir.
- Gelişmiş Optimizasyon: Derleyiciler, değerlerinin program yürütme sırasında değişmeyeceğini bildikleri için `const` değişkenlerini daha etkili bir şekilde optimize edebilirler. Bu, daha hızlı ve daha verimli kodla sonuçlanabilir.
- Daha İyi Bellek Kullanımı: Değişkenlere `const` sıfatı vermek, değerlerinin bir kopyasını oluşturmayı engeller; bu da bellek kullanımını azaltabilir ve performansı artırabilir.
- Gelişmiş Uyumluluk: Değişkenlere `const` sıfatı vermek, kodunuzu `const` değişkenleri kullanan diğer kitaplıklar ve API'lerle daha uyumlu hale getirebilir.
- Gelişmiş Güvenilirlik: `const` kullanarak, değerlerin beklenmedik şekilde değiştirilmemesini sağlayarak kodunuzu daha güvenilir hale getirebilir ve kodunuzdaki hata ve eksiklik riskini azaltabilirsiniz.

Anonim Değişkenler

Değişkenler veri tipi verilmeden de tanımlanır. Bu durumda veri tipi değişkene verilen ilk değerden çıkarılır. Bu tip değişkenlere **anonim tip** (**anonymous type**) adı verilir. Anonim tipler **var** kelimesiyle tanımlanır ve statik tip denetimini korur yani tip denetimi derleme zamanı yapılır.

```
// See https://aka.ms/new-console-template for more information
var pi=3.14f;
var sayac=0;
var durum=false;
var gerçek=10.5;
var ondalıklı=100000.0m;

var anon = new { Value = 1 };
Console.WriteLine(anon.Value); //1
//Console.WriteLine(anon.Id); Derleme Hatası Olur!

var anon2 = new { x = 1, y = 2 };
// anon.x == 1
// anon.y == 2

int a = 10;
int b = 20;
var anon3 = new { a, b };
// anon3.x == 10
// anon3.y == 20

var anon4 = new { adı="ilhan", yaşı=50 };
// anon4.adı == "ilhan"
// anon4.yaşı == 50
```

Anonim tiplerin eşitliği de sorgulanabilir;

```
var anon = new { Foo = 1, Bar = 2 };
var anon2 = new { Foo = 1, Bar = 2 };
var anon3 = new { Foo = 5, Bar = 10 };
var anon3 = new { Foo = 5, Bar = 10 };
var anon4 = new { Bar = 2, Foo = 1 };
// anon.Equals(anon2) == true
// anon.Equals(anon3) == false
// anon.Equals(anon4) == false (anon ve anon4 farklı tiplerdir)
```

Anonim tiplerin içerdiği özellikler aynı ise aynı olarak kabul edilir;

```
var anon = new { Foo = 1, Bar = 2 };
```

```
var anon2 = new { Foo = 7, Bar = 1 };
var anon3 = new { Bar = 1, Foo = 3 };
var anon4 = new { Fa = 1, Bar = 2 };
// anon ve anon2 aynı tiptir
// anon ve anon3 farklı tiptir (Bar ve Foo farklı sırada olduğundan)
// anon ve anon4 farklı tiplerdir (özellik isimleri farklı olduğundan)
```

Dinamik Değişkenler

Dinamik değişkenler, anonim değişkenlerin aksine, değişkenlerin tipi **dinamik** (**dynamic**) olarak derleme zamanı yerine çalıştırma zamanında veri tipi kontrolü yapılan değişken tiptir.

```
// See https://aka.ms/new-console-template for more information
dynamic val = "Merhaba";
Console.WriteLine(val.Id); // Derlenir, ama çalıştırma anında istisna olur!
```

Null Olabilen Değer Tipler

Değer tiplerin alabileceği alt ve üst sınırlar bellidir. Ancak bir değer tipe, değeri yok anlamında null atayamazsınız. Bunu sağlamak için değişkeni **null** olabilecek şekilde aşağıdaki şekillerde tanımlama yapılır;

```
int? i = null;
//Ya da
Nullable<int> i = null;
//Ya da
var i = (int?)null;
```

null olmayan bir değeri bu değişkenlere aşağıdaki şekilde yapılabilir.

```
int? i = 0;
//Ya da
Nullable<int> i = 0;
```

null olabilen bir veri tipini değer tipe aşağıdaki şekilde atayabiliriz.

```
int? i = 10;
int j = i ?? 0; // null ise 0 - null coalescing operator
int j = i.GetValueOrDefault(0); // null ise 0
int j = i.HasValue ? i.Value : 0; // null ise 0 - conditional tenary operator
```

Numaralandırma Tipi

Numaralandırma (**enumeration**) olarak adlandırılan **enum**, bir grup tamsayı ile ilişkilendirilmiş (integral) **değişmezden** (**literal**) oluşan numaralandırılmış kullanıcı tanımlı bir değer veri tipidir.

Bir **enum** aşağıdaki türlerden herhangi birinden türetilebilir: **byte**, **sbyte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**. Varsayılan olarak **int** veri tipi altta yatar ve **enum** tanımında veri tipi belirterek değiştirilebilir;

```
// See https://aka.ms/new-console-template for more information
byte işeBaşlananGün=1; // Pazartesi
Günler işbaşı = Günler.Pazartesi;
Aylar ramazan = Aylar.Mart;
Renkler arabanınRengi = Renkler.Kırmızı;

enum MedeniHal { Belirtilmemiş, Evli, Bekar, Boşanmış, Dul }; //Ön tanımlı olarak int
enum Günler : byte { Pazartesi = 1, Salı = 2, Çarşamba = 3, Perşembe = 4, Cuma = 5 }
enum Aylar : int { Ocak = 1, Şubat = 2, Mart = 3, Nisan = 4, Mayıs = 5, Haziran = 6,
                  Temmuz = 7, Ağustos = 8, Eylül = 9, Ekim = 10, Kasım = 11, Aralık = 12 }
enum Renkler : uint { Kırmızı = 1, Mavi = 2, Yeşil = 3, Sarı = 4, Beyaz = 5, Siyah = 6 }
```

Numaralandırmalar bir tamsayının her bir bitini temsil edecek şekilde **bayrak** (**flag**) olarak kullanılabilir;

```
// See https://aka.ms/new-console-template for more information
```

```
var İkiBayrakHavada = Bayraklar.BeşinciDurum | Bayraklar.ÜçüncüDurum;
Console.WriteLine(İkiBayrakHavada); //ÜçüncüDurum, BeşinciDurum
var ÜçüncüBayrakHavada = Bayraklar.ÜçüncüDurum;
Console.WriteLine(ÜçüncüBayrakHavada); //ÜçüncüDurum
var BaşkaİkiBayrakHavada = Bayraklar.BirinciDurum | Bayraklar.İkinciDurum;
Console.WriteLine(BaşkaİkiBayrakHavada); //BirinciDurum, İkinciDurum
```

```
[Flags]
enum Bayraklar
{
    //None = 0
    BirinciDurum = 1, //1.Bit
    İkinciDurum = 2, //2.Bit
    ÜçüncüDurum = 4, //3.Bit
    DördüncüDurum = 8, //4.Bit
    BeşinciDurum = 16 //5.Bit
}

[Flags]
enum FlagsEnum
{
    None = 0,
    Option1 = 1,
    Option2 = 2,
    Option3 = 4,
    Default = Option1 | Option3,
    All = Option1 | Option2 | Option3,
}
```

Yazdığımız Kod Nasıl İcra Edilir?

Derleyici tarafından işlemcinin anlayacağı makine diline çevrilen programımız aslında yazdığımız **talimatları** (**statement**) sırasıyla çalıştırır. C dilinde olduğu gibi her talimat noktalı virgül ile biter.

	Boy	Kilo	bki	boyKare
1 float boy = 1.80f;	1.80	?	?	?
2 float kilo = 100.0f;	1.80	100.0	?	?
3 float bki;	1.80	100.0	?	?
4 float boyunKare;	1.80	100.0	?	?
5 boyunKare = boy * boy;	1.80	100.0	?	3.24
6 bki = kilo / boyunKare;	1.80	100.0	30.86419753	3.24
7 boy = 1.50f;	1.50	100.0	30.86419753	3.24

Tablo 10. Örnek bir C# Programının İcra Sırası

Yapısal programlamada program, ana fonksiyondan başlar. Bu nedenle icra, **main** fonksiyonu içindeki ilk talimatla başlar ve solda verilen sırayla icra edilir. Sağda ise her icra sonrası değişkenlerin değerleri gösterilmiştir. Her talimatın icrasında neler olduğu aşağıda verilmiştir;

1. **boy** değişkenine **1.80** değeri atanır.
2. **kilo** değişkenine **100.0** atanır.
3. **bki** değişkeni tanımlanır.
4. **boyKare** değişkeni tanımlanır. Daha sonraki birkaç satır açıklama olduğundan icra edilecek bir şey yoktur.
5. **boy** ile **boy** değişkeni çarpılır ve sonuç **boyKare** değişkenine atanır. Artık **boyKare** değişkeninin değeri bu noktadan sonra **3.24** olmuştur.
6. **kilo** değişkeni **boyKare** değişkenine bölünür ve sonuç **bki** değişkenine atanır. Artık **bki** değişkeninin değeri bu noktadan sonra **30.86419753** olmuştur.
7. **boy** değişkenine **1.50** atanmıştır. Bu atama sonrası **boyKare** ve **bki** değişkenlerinin değeri değişmemiştir. Çünkü bu değişkenleri değiştiren 5. ve 6. talimatlar icra edilmiştir.

Temel İşleçler

İşleçler (*operator*) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara *işlenen* (*operand*) adı verilir. Yüksek düzey dillerde de aritmetik işleçler ve bunun yanında birçok işleç de bulunur. En çok kullanılan işleç, atama (=) işlecidir. Atama işleci, sağdaki işleneni soldakine atar. Bu nedenle kodlama yapılırken $2+2=x$ veya $a+b=x$ yazılamaz!

Aritmetik İşleçler

C# programlama dilinde aritmetik işleçlerin (*arithmetic operator*) çalışma şekli *işlenenlerin* (*operand*) veri tipine göre değişir.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
+	Sol ve sağındaki yer alan işlenenleri toplar.
-	Soldaki işlenenden sağdakini çıkarır.
*	Sol ve sağındaki yer alan işlenenleri çarpar.
/	Soldaki işleneni sağdakine böler. Eğer işlenenlerin her iki de tamsayı ise sadece kalan kısmı atılır. Bu durumda son uç yine tamsayı olarak üretilir.
%	<i>Kalan</i> (<i>modulus</i>) işleci, soldaki işlenenin sağdakine bölümünden kalanı verir.
++	<i>Artırım</i> (<i>increment</i>) işleci, işlenenin sağında veya solunda kullanılabilir. İşlenen solda ise işlenenin değeri kullanılmadan önce 1 artırılacaktır (<i>pre-increment</i>). İşlenen sağda ise işlenenin değeri kullanıldıktan sonra 1 artırılacaktır (<i>post-increment</i>).
--	<i>Eksiltme</i> (<i>decrement</i>) işleci, işlenenin sağında veya solunda kullanılabilir. İşlenen solda ise işlenenin değeri kullanılmadan önce 1 eksiltilecektir (<i>pre-decrement</i>). İşlenen sağda ise işlenenin değeri kullanıldıktan sonra 1 eksiltilecektir (<i>post-decrement</i>).
-	Tekli eksi: Kendisinden sonra gelen işlenenin işaretini değiştirir. İşlenen pozitif ise negatif yapar, negatif ise pozitif yapar.
+	Tekli +: Kendisinden sonra gelen işlenenin işaretini pozitif yapar.

Tablo 11. Aritmetik İşleçler

Artırım (++) ve eksiltme (--) işleçleri hariç olmak üzere bu işleçler *int*, *uint*, *long* ve *ulong* tiplerini işlenen olarak alır. İşlenenler diğer tamsayı tipler olduğunda (*sbyte*, *byte*, *short*, *ushort* veya *char*), değerleri bir işlemin sonuç tipi olan *int* tipine dönüştürülür.

```
// See https://aka.ms/new-console-template for more information
int a = 9, b = 4, res;
float fa = 6.6f, fb = 2.2f, fres;
res = a + b; // işlem sonucu res=13
res = a - b; // işlem sonucu res=5
res = a * b; // işlem sonucu res=36
res = a / b; // işlem sonucu res=9/4=2 tam 1/4=2
/* Toplama işlecinin işlenenleri tamsayı olduğundan, bölme sonucundaki tam kısım
   bölme sonucudur.*/
res = (int)(a / 4.3);
/* İşlem sonucu double olacaktır. Bu durumda res değişkeni de int olduğundan
   sonucu int türüne çevirmemiz gerekir. res=9/4.3=9.0/4.3=2.0930= 2 tam 0.0930=2 */
res = a % b; // işlem sonucu res=9%4= 2 tam 1/4=1
fres = fa / fb; // işlem sonucu res=6.6/2.2=3.00
```

C# programlama dilinde tamsayı bölme işlemi işlemcinin en basit yetenekleriyle çözülür;

```
// See https://aka.ms/new-console-template for more information
int a = 19 / 2; /* a = 2*9+1 = 9 */
int b = 18 / 2; /* b = 9 */
int c = 255 / 4; /* c = 4*63+3 = 63 */
int d = 45 / 4; /* d = 4*11+1 = 11 */
```



```
double aa = 19 / 2.0; /* aa = 9.5 */
double bb = 18.0 / 2; /* bb = 9.0 */
double cc = 255 / 2.0; /* cc = 127.5 */
double dd = 45.0 / 4; /* dd = 11.25 */
double ee = 45 / 4; /* ee = 11 çünkü bölme tamsayılar arasında yapılmıştır */
```

Mantıksal İşleçler

Tekli işleçler (**unary operator**) sadece bir işlenen üzerinde işlem yapar.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
!	Mantıksal DEĞİL işleci işlenenden önce kullanılır. İşleneninin mantıksal durumunu tersine çevirmek için kullanılır. İşlenen true ise false , false ise true verir.
&	Mantıksal VE: İşlenenlerin her ikisi mantıksal olarak true ise true sonucunu verir.
	Mantıksal VEYA: İşlenenlerin en az biri mantıksal olarak true ise true sonucunu verir.
^	Mantıksal ÖZEL VEYA: İşlenenlerin mantıksal olarak aynı ise false , değilse true sonucunu verir.
&&	Şartlı VE: Eğer iki işlenen true ise true verir. Eğer soldaki işlenen false ise sağdakine bakılmaz.
	Şartlı VEYA: Eğer iki işlenenden biri true ise true verir. Eğer soldaki işlenen true ise sağdakine bakılmaz.

Tablo 12. Mantıksal İşleçler

Aşağıda tekli işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// See https://aka.ms/new-console-template for more information
int a = 5, b = 6;
bool res;
res = (a>0) && (b<0); // işlem sonucu res=true
res = (a>0) || (b<0); // işlem sonucu res=true
res = !(a>0); // işlem sonucu res=false
```

Eşitlik ve İlişkisel İşleçler

İlişkisel işleçler (**relational operator**), işlenenlerin arasındaki ilişkiyi verirler. Bu işleçler, karşılaştırma doğruysa **true**, değilse **false** değerini döndürür.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
==	Eşit mi? İşleci: İki işlenenin değeri birbirine eşit ise true, değilse false verir
!=	Farklı mı? İşleci: İki işlenenin değeri birbirinden farklı ise true, değilse false verir
>	Büyük mü? İşleci: soldaki işlenenin değeri soldakinden fazla ise true, değilse false verir
<	Küçük mü? İşleci: soldaki işlenenin değeri soldakinden az ise true, değilse false verir
>=	Büyük veya eşit mi? İşleci: soldaki işlenenin değeri sağdakinden fazla ya da iki işlenen eşit ise true, değilse false verir
<=	Küçük veya eşit mi? İşleci: soldaki işlenenin değeri sağdakinden az ya da iki işlenen eşit ise true, değilse false verir

Tablo 13. İlişkisel İşleçler

Aşağıda verilen ilişkisel işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// See https://aka.ms/new-console-template for more information
int a = 5, b = 6;
bool res;
res = a < b; // işlem sonucu res=true
res = a <= b; // işlem sonucu res=true
res = a > b; // işlem sonucu res=0
res = a >= b; // işlem sonucu res=0
res = a == b; // işlem sonucu res=0
res = a != b; // işlem sonucu res=true
```

Bit Düzeyi İşleçler

Bit düzeyi işleçler (**bitwise operators**) özellikle ikilik tabandaki tamsayılarda karşılıklı bitler üzerinde yapılan işlemlerdir.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
&	Bit düzeyi VE: Tamsayının karşılıklı bitleri VE işlemine tabi tutulur. Sonuç tamsayıda karşılaştırılan iki bit 1 ise sonuç bit 1, değilse 0 olacak şekilde işlem yapar.
	Bit düzeyi VEYA: Tamsayının karşılıklı bitleri VEYA işlemine tabi tutulur. Sonuç tamsayıda karşılaştırılan iki bitin herhangi biri 1 ise sonuç bit 1, değilse 0 olacak şekilde işlem yapar.
^	Bit düzeyi ÖZEL VEYA: Tamsayının karşılıklı bitleri ÖZEL VEYA işlemine tabi tutulur. Sonuç tamsayıda karşılaştırılan iki bitin her ikisi farklı ise sonuç bit 1, değilse 0 olacak şekilde işlem yapar.
~	Bit düzeyi NOT: Tekli işleç olup sonrasında gelecek sayının bitlerini 1 ise 0, 0 ise 1 olacak şekilde işlem yapar.
<<	Bit düzeyi sola kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sola kaydırır. En anlamlı bit taşar. Taşan bu bit 1 ise işlemcinin durum kaydedicisinde taşma bayrağı 1 olur.
>>	Bit düzeyi sağa kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sağa kaydırır. En anlamsız bit taşar. Taşan bu bit 1 ise işlemcinin durum kaydedicisinde taşma bayrağı 1 olur.
>>>	Bit düzeyi sağa kaydırma işleci ile aynı işlemi yapar ancak ilk işlenen işaretli bir tamsayıdır.

Tablo 14. Bit Düzeyi İşleçler

Bu işleçler **int**, **uint**, **long** ve **ulong** tiplerini işlenen olarak alır. İşlenenler diğer tamsayı tipler olduğunda (**sbyte**, **byte**, **short**, **ushort** veya **char**), değerleri bir işlemin sonuç tipi olan **int** tipine dönüştürülür. Aşağıda tamsayılar üzerinde yapılan bit düzeyi işlemler gösterilmiştir;

```
// See https://aka.ms/new-console-template for more information
int a = 6; // 0110
int b = 2; // 0010
int bitwise_and = a & b; // 0010
int bitwise_or = a | b; // 0110
int bitwise_xor = a ^ b; // 0100
int bitwise_not = ~b; // 1101
int left_shift = b << 2; // 0100
int right_shift = b >> 1; // 0001
int c = -1;
int unsigned_right_shift = c >>> 3;
```

Atama işleçleri

Atama işleçleri (**assignment operator**) en çok kullanılan işleçlerdir. Atama işleçleri her zaman sağdaki değeri sola atar! Dolayısıyla atama yapılacak uygun **veri tipinde** (**data type**) bir **değişken** (**variable**) olmak zorundadır.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
=	Sağdaki işleneni sola atar.
+=	Soldaki işleneni sağdaki işlenen üzerine toplar ve sonucu sola atar.
-=	Soldaki işlenenden sağdaki işlenenden çıkarır ve sonucu sola atar.
*=	Soldaki işleneni sağdaki işlenene çarpar ve sonucu sola atar.
/=	Soldaki işleneni sağdaki işlenene böler ve sonucu sola atar. Eğer işlenenler tamsayı ise kesirli kısım göz ardı edilir. Bu durumda sonuç tamsayıdır.
%=	Bu işleç tamsayılarla çalışır. Soldaki işleneni sağdaki işlenene böler ve kalanı sola atar.
<=>	İki işlenen tamsayının ikincisi kadar biti sola kaydırılır ve sonucu sola atar.

>>=	İki işlenen tamsayının ikincisi kadar biti sağa kaydırılır ve sonucu sola atar.
&=	İki işleneni mantıksal veya bit düzeyi VE işlemine tabi tutar ve sonucu sola atar.
=	İki işleneni mantıksal veya bit düzeyi VEYA işlemine tabi tutar ve sonucu sola atar.
^=	İki işleneni mantıksal veya bit düzeyi ÖZEL VEYA işlemine tabi tutar ve sonucu sola atar.

Tablo 15. Atama İşleçleri

Aşağıda verilen atama işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// See https://aka.ms/new-console-template for more information
int a = 10; /* a değişkenine 10 değışmezi = işleciyle atanıyor.*/
a += 10; /* a=a+10; ile eşdeğer: a değişkenine 10 ekleniyor. */
a -= 10; /* a=a-10; ile eşdeğer: a değişkeninden 10 çıkarılıyor. */
a *= 10; /* a=a*10; ile eşdeğer: a değişkeni 10 kat yapılıyor.*/
a /= 10; /* a=a/10; ile eşdeğer: a değişkeni 1/10 kat yapılıyor.*/
a %= 10; /* a=a%10; ile eşdeğer: a değişkeninin 10 a kalanı bulunup kendisine atanıyor. */
int b = 12;
//b-5=a; // HATA: Atama sağdan sola yapıldığından böyle bir talimat yazılamaz!
```

İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. C programlama dilinde yazılan ifadelerde de **işleç öncelikleri** (operator precedence) vardır.

Aşağıdaki tabloda çok kullanılan C# işleçleri en yüksek öncelikten en düşüğe doğru listelenmiştir. Her satırdaki operatörlerin önceliği aynıdır.

İşleçler	Kategori
x.y, f(x), a[i], x?.y, x?[y], x++, x--, x!, new, typeof, checked, unchecked, default, nameof, delegate, sizeof, stackalloc, x->y	Temel
+x, -x, !x, ~x, ++x, --x, ^x, (T)x, await, &x, *x, true, false	Tekli
x..y	Aralık
switch, with	switch ve with ifadeleri
x * y, x / y, x % y	Çarpım
x + y, x - y	Toplam
x << y, x >> y, x >>> y	Kaydırma
x < y, x > y, x <= y, x >= y, is, as	İlişkisel ve tip testi
x == y, x != y	Eşitlik
x & y	Mantıksal ve Bit Düzeyi VE
x ^ y	Mantıksal ve Bit Düzeyi ÖZEL VEYA
x y	Mantıksal ve Bit Düzeyi VEYA
x && y	Şartlı VE
x y	Şartlı VEYA
x ?? y	null kaynaştırma
c ? t : f	Şart işleci
x = y, x += y, x -= y, x *= y, x /= y, x %= y, x &= y, x = y, x ^= y, x <<= y, x >>= y, x >>>= y, x ??= y, =>	Atama

Tablo 16. İşleçlerin İşlem Öncelikleri¹⁴

Aşağıda işleçlerin önceliklerine ilişkin örnek programı lütfen inceleyiniz.

```
// See https://aka.ms/new-console-template for more information
int a = 10, b = 5;
```

¹⁴<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/#operator-precedence>

```

a = a - b + 2; // Öncelik Sırası +, -, = İşlem sonunda a=7
a = a / b + 2; // Öncelik Sırası /, +, = İşlem Sonunda a=7/5+2=3
a = 2 * 10 - 20 / 2 + 3 + (12 - 10); // Öncelik Sırası: (-) * / - + +
/*
İfadenin çözüm sırası:
> 2*10-20/2+3+2
> 20-20/2+3+2
> 20-10+3+2
> 10+3+2
> 13+2
> 15
*/

```

Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu *Değişken* başlığında anlatılmıştı. Kayan noktalı sayılar olarak değişkenlere verdiğimiz onluk tabanda değerler aslında arka planda ikilik tabanda tutulur ve bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```

float total = 0;
for(float a = 0; a != 2; a += 0.01f) {
    total += a;
}

```

Acemi programcı bunun 0, 0.01, 0.02, 0.03, gibi artarak, 1.97, 1.98, 1.99 aralığındaki her bir sayıyı toplayacağını ve 199 sonucunu, yani matematiksel olarak doğru cevabı vereceğini varsayar. Bu kodda doğru yürümeyen iki şey olur:

1. Yazıldığı haliyle program asla sonuçlanmaz. **a** asla 2'ye eşit olmaz ve döngü asla sonlanmaz.
2. Döngü mantığını **a < 2** şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam 199'dan farklı bir şey olur. IEEE754 uyumlu işlemlerde, genellikle bunun yerine yaklaşık 201'e ulaşır. Bunun olmasının nedeni, Kayan noktalı sayıların onluk tabanda kendilerine atanan değerlerin, ikilik tabanda yaklaşık değerlerini temsil etmesidir. Bu duruma aşağıdaki klasik örnek verilir;

```

double a = 0.1;
double b = 0.2;
double c = 0.3;
if(a + b == c)
    Console.WriteLine ("Bu satır İcra Edilmez!"); //IEEE754 standardında işlem yaparsak.
else
    Console.WriteLine ("Kayan Noktalı Sayılar İkili Sayı Sisteminde"+
        " Tutulduğundan Yaklaşık Olarak Hesaplanır.");

```

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır. 0.1, 0.2 ve 0.3, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki 1/3 sayısı gibi 0.33333333333333... şeklinde kesin olmayan biçimde depolanması gerektiğindendir);

```

/*64 bitlik kayan noktalı sayılar, tam sayı kısmı dahil olmak üzere
53 basamaklı hassasiyete sahiptir */
double a = 0.1;
//onluk sistemdeki 0.1 ikili sistemde kusurlu olarak saklanır
double b = 0.2;
//onluk sistemdeki 0.2 ikili sistemde kusurlu olarak saklanır
double c = 0.3;
//onluk sistemdeki 0.3 ikili sistemde kusurlu olarak saklanır
double f= a + b ;
//c değişkeni ile f nin onluk sistemde 0,3'e tam olarak eşit olmadığını unutmayın!

```

```
if (f==c)
    Console.WriteLine("Ok");
else
    Console.WriteLine("Garip Bir Durum Var"); // Konsola bu yazılır
```

BASİT GİRİŞ ÇIKIŞ İŞLEMLERİ

Modüler Programlama

C# programlama dili, aynı zamanda modüler bir programlama dilidir. Oluşturulan ve önceden test edilip doğrulanmış modüller olan **sınıf kütüphaneleri** (**class library**) yazacağımız kodlara **using** yönergesi ile dahil edilir.

```
using System; // System adındaki isim uzayındaki bileşenleri kullanacağız!.
//System isim uzayının içindeki Console sınıfını kullanabilmek için.
namespace ÖrnekUygulama // ÖrnekUygulama adında bir isim Uzayı
{
    class Program // Programı başlatacak Main yöntemini içeren sınıf
    {
        static void Main(string[] args) // Programın başlangıç noktası
        {
            Console.WriteLine("Merhaba"); // Konsola "Merhaba" yazdırır
        }
    }
}
```

Modüllere ayırmanın **soyutlama** (**abstraction**), **değişim yönetimi** (**change management**) ve **yeniden kullanım** (**reusing**) gibi üstünlükleri vardır. Modüllere ayrılan bir kodun bakımı da daha kolaydır. Ancak daha fazla fonksiyonun çağırılması, işlemciyi yorar. Çünkü her fonksiyon çağırısında işlemcinin mevcut durumu ve kaydediciler belleğe itilir ve fonksiyondan geri döndüğünde eski duruma dönmek için bu veriler tekrar geri çekilir. İşlemciye yüklenen bu **çağırma yükü** (**calling overhead**) günümüz bilgisayarlarında oldukça ihmal edilebilir seviyededir. Ancak milisaniyeler bazında yapılması gereken bir iş varsa böyle zaman kritik işler gerçekleştirmek için bu durum göz önüne alınmalıdır.

C# programlama dilinde en çok kullanacağımız modüllerden biri de giriş çıkış işlemlerinin (**input output-IO**) tanımlı olduğu **System** ve **System.IO** sınıf kütüphanesi ön tanımlı olarak koda dahil edilmiştir. Böylece konsola bir şey yazmak ve konsoldan bir şey okumak için **Console** sınıfını doğrudan kullanırız. Bunların yanında **Math**, **Random**, **string** gibi birçok sınıf kullanır hale geliriz^{15,16}.

Konsolda Veri Okuma ve Yazma

Konsoldan veri okuma ve yazma **Console** nesnesi üzerinden yapılır. Nesnenin **Write** yöntemi ile imlecin bulunduğu yerden itibaren parametre olarak girilen metin konsola yazılır. Nesnenin **WriteLine** yöntemi ile imlecin bulunduğu yerden itibaren parametre olarak girilen metin konsola yazılır ve alt satıra inilir. **Write** ve **WriteLine** yönteminin ilk parametresi metin olmak zorunda değildir.

```
// See https://aka.ms/new-console-template for more information
Console.Write("Merhaba ");
Console.WriteLine("İlhan ");
int yaş = 50;
Console.WriteLine(yaş);
float kilo = 99.5f;
Console.WriteLine(kilo);
decimal para = 545.5M;
Console.WriteLine(para);
/*
Merhaba İlhan
50
99,5
545,5
*/
```

¹⁵ <https://learn.microsoft.com/en-us/dotnet/api/system?view=net-9.0>

¹⁶ <https://learn.microsoft.com/en-us/dotnet/api/system.io?view=net-9.0>

Write ve **WriteLine** yönteminin ilk parametresi, biçimlendirme metni olarak kullanılabilir. İzleyen her bir parametrenin sıfırdan başlayan bir numarası vardır.

```
// See https://aka.ms/new-console-template for more information
int sayi1 = 20, sayi2 = 30;
Console.WriteLine("Sayılar:{0}, {1}",sayi1,sayi2);
int toplam = sayi1 + sayi2;
Console.Write("Sayıların Toplamı:{0}", toplam);

/*
Sayılar:20, 30
Sayıların Toplamı:50
*/
```

Dizgiler

Metinleri kod içerisinde işlemek için aslında karakter dizileri olan **dizgiler** (**string**) kullanılır. Ancak, daha yaygın uygulama olarak, bir dizgi değişkeni bildirmek için **string** anahtar sözcüğünü kullanılır. **string** anahtar sözcüğü, **System.String** sınıfı için bir takma addır. Aşağıdaki şekilde tanımlanabilirler;

```
// See https://aka.ms/new-console-template for more information
string adı, soyadı;
adı = "İlhan";
soyadı = "ÖZKAN";

char[] harfler = { 'M', 'e', 'r', 'h', 'a', 'b', 'a' }; //Karakter Dizisi
string[] metindizisi = { "Merhaba", "Dünya" }; //String Dizisi

string tamAdı = adı + " " + soyadı;
Console.WriteLine("Tam Adı: {0}", tamAdı);

string selamlama = new string(harfler);
Console.WriteLine("Selamlama: {0}", selamlama);

string mesaj = String.Join(" ", metindizisi);
Console.WriteLine("Message: {0}", mesaj);

DateTime zaman = new DateTime(2025, 3, 30, 16, 04, 0);
string metinOlarakZaman = String.Format("{0:t} on {0:D}", zaman);
Console.WriteLine("Zaman: {0}", metinOlarakZaman);

/*Program Çıktısı:
Tam Adı: İlhan ÖZKAN
Selamlama: Merhaba
Message: Merhaba Dünya
Zaman: 16:04 on 30 Mart 2025 Pazar
*/
```

Dizgilerin karakter uzunluğunu aşağıdaki gibi öğrenebiliriz;

```
// See https://aka.ms/new-console-template for more information
string str = "İlhan Özkan";
int length = str.Length;
```

Temel Dizgi İşlemleri

Dizgiler toplama işlemi ile birleştirilebilir;

```
// See https://aka.ms/new-console-template for more information
string s1 = "İlhan";
string s2 = "Özkan";
string s3 = s1 + " " + s2; //"İlhan Özkan"
```

```
string s4 = string.Concat(s1, " ", s2); //"İlhan Özkan"
string s5 = string.Join(" ", s1, s2); //"İlhan Özkan"
string s6 = string.Format("{0} {1}", s1, s2); //"İlhan Özkan"
string s7 = string.Concat(s1, " ", s2); //"İlhan Özkan"
string s8 = string.Join(" ", new string[] { s1, s2 }); //"İlhan Özkan"

string s9 = $"{s1} {s2}"; //"İlhan Özkan"
```

String.Format() yöntemini kullanarak dizgideki bir veya daha fazla öğeyi bir dizgide birleştirebiliriz;

```
String.Format("Yaşım: {0} ve Adım: {1}", 45, "İlhan") //Yaşım:45 ve Adım:İlhan
```

Burada ilk argüman biçimlendirme metnidir. Biçimlendirme metni içerisinde sıfırdan başlayarak izleyen argüman numaraları tırnaklı parantez içerisinde verilir.

Dizgiler içerisinde UTF-8 karakterleri 4 karakterli onaltılık sayı olarak da verilebilir;

```
string s = "\u0130lhan"; //"İlhan"
```

Dizgilere ilk değer çift tırnak karakterleri içerisinde verilir. Peki metin çift tırnak karakteri içeriyorsa ne yapılır? İşte burada çift tırnak karakteri gibi bazı karakterler **kaçış dizisi** (**escape sequence**) olarak adlandırılır;

Escape sequence	Karakter Adı	Unicode Kodu
\'	Tek Tırnak	0x0027
\"	Çift Tırnak	0x0022
\\	Ters Bölü	0x005C
\0	Boş (Null)	0x0000
\a	Uyarı Sesi (Alert)	0x0007
\b	Geri (Backspace)	0x0008
\e	Esc (Escape)	0x001B
\f	Yeni Sayfa (Form feed)	0x000C
\n	Yeni Satır(New line)	0x000A
\r	Satır Başı (Carriage return)	0x000D
\t	Yatay Sekme (Horizontal tab)	0x0009
\v	Dikey Sekme(Vertical tab)	0x000B
\u	Unicode escape sequence (UTF-16)	\uHHHH (0000 - FFFF) \u00E7 = "ç"
\U	Unicode escape sequence (UTF-32)	\U00HHHHHH (000000-10FFFF) \U0001F47D = "🐶"
\x	Unicode escape sequence "\u" ya benzer fakat onaltılık rakam sayısı değişir.	\xH[H][H][H] (0 - FFFF) \x00E7 = \x0E7 = \xE7 = "ç"

Tablo 17. Kaçış Dizisi Karakterleri

Metni belirlenen bir aralıkta sağa ya da sola **yaslama** (**padding**) da yapılabilir;

```
string s = "\u0130lhan \ '61\ '";
Console.WriteLine("\u0130lhan \ '61\ '"); //İlhan '61'
string paddedLeft = s.PadLeft(10, ' '); //Sola boşluk ekle
Console.WriteLine(paddedLeft); // İlhan
string paddedRight = s.PadRight(10, '+'); //Sağa + karakteri ekle
Console.WriteLine(paddedRight); //İlhan+++++
```

Bit tamsayıyı da metin olarak herhangi bir tabana çevirebiliriz;

```
Int32 Number = 252;
Console.WriteLine(Convert.ToString(Number, 2)); //11111000
Console.WriteLine(Convert.ToString(Number, 8)); //370
Console.WriteLine(Convert.ToString(Number, 10)); //252
Console.WriteLine(Convert.ToString(Number, 16)); //FC
```


Değer tipler `ToString()` yöntemi ile metne çevrilebilir;

```
int intValue = 31;
string zeroPaddedInteger = intValue.ToString("000"); //031
string customFormat = intValue.ToString("Değer 0 dır."); //Değer 31 dır.

double doubleValue = 10.456;
string roundedDouble = doubleValue.ToString("0.00"); //10,46
string integerPart = doubleValue.ToString("00"); //10
string customDoubleFormat = doubleValue.ToString("Değer 0.0\'dır."); //Değer 10.5 dır.

DateTime currentDate = DateTime.Now; // 14/05/2025 10:38:03
string dateTimeString = currentDate.ToString("dd-MM-yyyy HH:mm:ss"); // "14-05-2025 10:38:03"
string dateOnlyString = currentDate.ToString("dd-MM-yyyy"); // "14-05-2025"
string dateWithMonthInWords = currentDate.ToString("dd-MMMM-yyyy HH:mm:ss");
// "14-Mayıs-2025 10:38:03"
```

Tam Dizgiler

`Write` ve `WriteLine` yönteminin ilk parametresi, **tam dizgi** (**verbatim string**) olabilir. Bu tam metin, biçimlendirme amacıyla kullanılabilir.

```
// See https://aka.ms/new-console-template for more information
string filename1 = @"c:\documents\files\u0066.txt"; // Tam metin (verbatim string)
string filename2 = "c:\\documents\\files\\u0066.txt"; // Sıratan metin (string)
Console.WriteLine(filename1);
Console.WriteLine(filename2);
Console.WriteLine(@"İlhan
                  Özkan  ""  \  *");
/*
c:\documents\files\u0066.txt
c:\documents\files\u0066.txt
İlhan
                  Özkan  "  \  *
*/
```

Dizgi Enterpolasyonu

`Write` ve `WriteLine` yönteminin ilk parametresi, değişken kimliklerini içerebilen bir metin (**string interpolation**) olabilir. Bu durumda değişmez metnin önüne dolar karakteri konur. Dolayısıyla izleyen parametreleri kullanmak zorunda kalmayız. Basit aritmetik ifadeleri de yazabiliriz.

```
// See https://aka.ms/new-console-template for more information
string isim= "İlhan";
int yaş = 50;
float kilo = 99.5f;
decimal para = 545.5M;
Console.WriteLine($"Adınız: {isim}, Kilonuz: {kilo}, Paranız: {para}");
DateTime date = DateTime.Now;
Console.WriteLine($"Merhaba, Bugün {date.Date} {date.DayOfWeek}");
/*
Adınız: İlhan, Kilonuz: 99,5, Paranız: 545,5
Merhaba, Bugün 21.02.2025 00:00:00 Friday
*/
var StrWithMathExpression = $"1 + 2 = {1 + 2}"; // "1 + 2 = 3"
```

Konsola Yazılan Metni Biçimlendirme

Tarih Biçimlendirme

```
// See https://aka.ms/new-console-template for more information
DateTime date = new DateTime(1971, 09, 01, 11, 53, 24);
```

```

Console.WriteLine(String.Format("{0:dd}", date)); //Yalnızca Günü Yazar:01
//Yukarıdaki Kodun Kısa Hali:
Console.WriteLine($"{date:dd}");
Console.WriteLine("{0:dd}",date);

Console.WriteLine(String.Format("{0:dd/MM/yyyy}", date));
//Günü Ayı ve Yılı Yazar: 01.09.1971
//Yukarıdaki Kodun Kısa Hali:
Console.WriteLine($"{date:dd/MM/yyyy}");
Console.WriteLine("{0:dd/MM/yyyy}",date);

Console.WriteLine(String.Format("{0:dd/MM/yyyy HH:mm:ss}", date));
//Günü Ayı Yılı Saat Dakika ve Saniyeyi Yazar: 01.09.1971 11:53:24
//Yukarıdaki Kodun Kısa Hali:
Console.WriteLine($"{date:dd/MM/yyyy HH:mm:ss}");
Console.WriteLine("{0:dd/MM/yyyy HH:mm:ss}",date);

//Günü İsim Olarak Almanca Yazmak için:
Console.WriteLine(String.Format(new System.Globalization.CultureInfo("de-DE"),
                                "{0:dddd}",
                                date)); //Mittwoch

```

Tarih ve Zamanı aşağıdaki şekilde biçimlendirebiliriz;

```

// See https://aka.ms/new-console-template for more information
DateTime date = new DateTime(1971, 09, 01, 11, 53, 24);
Console.WriteLine($"{date:d}"); //Günü Yazar: 1.09.1971
Console.WriteLine($"{date:dd}"); //Günü Yazar: 01
Console.WriteLine($"{date:ddd}"); //Kısa Günü Yazar: Çar
Console.WriteLine($"{date:dddd}"); //Uzun Günü Yazar: Çarşamba
Console.WriteLine($"{date:D}"); //Uzun Tarih Yazar: 1 Eylül 1971 Çarşamba
Console.WriteLine($"{date:f}"); //Uzun Tarih ve Kısa Saati Yazar: 1 Eylül 1971 Çarşamba 11:53
Console.WriteLine($"{date:F}"); /* Uzun Tarih ve Uzun Saati Yazar:
                                1 Eylül 1971 Çarşamba 11:53:24 */
Console.WriteLine($"{date:g}"); //Kısa Tarih ve Kısa Saati Yazar: 1.09.1971 11:53
Console.WriteLine($"{date:hh}"); //Saatin 12 Saatlik Formatı: 11
Console.WriteLine($"{date:mm}"); //Dakika Yazar: 53
Console.WriteLine($"{date:MM}"); //Ayı Sayısal Olarak Yazar: 09
Console.WriteLine($"{date:MMM}"); //Ayı Kısa Olarak Yazar: Eyl
Console.WriteLine($"{date:MMMM}"); //Ayı Uzun Olarak Yazar: Eylül
Console.WriteLine($"{date:ss}"); //Saniye Yazar: 24
Console.WriteLine($"{date:r}"); //RFC1123 Formatında Yazar: Wed, 01 Sep 1971 11:53:24 GMT
Console.WriteLine($"{date:t}"); //Kısa Saati Yazar: 11:53
Console.WriteLine($"{date:T}"); //Uzun Saati Yazar: 11:53:24
Console.WriteLine($"{date:tt}"); //AM/PM Formatında Yazar: ÖÖ
Console.WriteLine($"{date:y}"); //Ay ve Yılı Yazar: Eylül 1971
Console.WriteLine($"{date:yy}"); //Yılı Kısa Olarak Yazar: 71
Console.WriteLine($"{date:yyyy}"); //Yılı Uzun Olarak Yazar: 1971

```

Para Birimi Biçimlendirme

Para birimi biçimlendiricisi olarak "c" biçim belirleyicisi kullanılır. Bir sayıyı para birimi miktarını temsil eden bir **dizgiye** (string) dönüştürür;

```

// See https://aka.ms/new-console-template for more information
using System.Globalization;
double para = 2510.752525;

//Sistem tarafından ön tanımlı olarak bozukluk iki hane yazılır.
//Programın çalıştığı bilgisayar türkçe ayarlarına sahip:
Console.WriteLine(string.Format("{0:c}", para)); // ?2.510,75
Console.WriteLine($"{para:C}"); // ?2.510,75

```

```
//Bozukluk hane sayısı aşağıdaki şekild belirtilir:
Console.WriteLine(string.Format("{0:c1}", para)); // ?2.510,8
Console.WriteLine(string.Format("{0:c2}", para)); // ?2.510,75
Console.WriteLine(string.Format("{0:c3}", para)); // ?2.510,753
Console.WriteLine(string.Format("{0:c4}", para)); // ?2.510,7525
Console.WriteLine();

NumberFormatInfo nfi = new CultureInfo("en-US", false).NumberFormat;
nfi.CurrencySymbol = "$";
nfi.NumberGroupSeparator = ",";
nfi.NumberDecimalSeparator = ".";
nfi.CurrencyPositivePattern = 0;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); // $2,510.75 - default
nfi.CurrencyPositivePattern = 1;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); // 2,510.75$
nfi.CurrencyPositivePattern = 2;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); // $ 2,510.75
nfi.CurrencyPositivePattern = 3;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); // 2,510.75 $
```

Sayı Biçimini Değiştirme

`NumberFormatInfo` hem tam sayı hem de gerçek sayıları biçimlendirmek için kullanılabilir.

```
// See https://aka.ms/new-console-template for more information
using System.Globalization;
double number = 1234567.89;

var invariantResult = string.Format(CultureInfo.InvariantCulture, "{0:#,###,##}", number);
Console.WriteLine(invariantResult); // 1,234,568
var customProvider = new NumberFormatInfo
{
    NumberDecimalSeparator = "_Ondalık_",
    NumberGroupSeparator = "_Bindelik_",
};

var customResult = string.Format(customProvider, "{0:#,###.##}", number);
Console.WriteLine(customResult); // 1_Bindelik_234_Bindelik_567_Ondalık_89
```

Sağa sola yaslama;

```
// See https://aka.ms/new-console-template for more information
int sayı = 12;
string metin = "alo";
string sol=string.Format("SOL: string: |{0,-5}| int:|{1,-5}|", metin, sayı);
Console.WriteLine(sol); //SOL: string: |alo | int:|12 |
string sağ=string.Format("SAĞ: string: |{0,5}| int:|{1,5}|", metin, sayı);
Console.WriteLine(sağ); //SAĞ: string: | alo| int:| 12|
```

Sayı sistemleri;

```
// See https://aka.ms/new-console-template for more information
int sayı = 1234;

string hexstring=string.Format("Hexadecimal: {0:x2}-{0:X4}", sayı); // Hexadecimal: 4d2-04D2
Console.WriteLine(hexstring);
Console.WriteLine($"{{sayı:x2}}-{{sayı:x4}}"); //4d2-04D2

// Tamsayılarda bindelik ayırıcı:
string bindlikstring=string.Format("Bindelik Ayırıcı: {0:#,##}", sayı);
Console.WriteLine(bindlikstring); //Bindelik Ayırıcı: 1.234
Console.WriteLine($"{{sayı:#,##}} |{{sayı,10:#,##}}|"); //1.234 | 1.234|

decimal ondalık = 1234.5678m;
```

```
string ondalıkstr=string.Format("Ondalık: {0:0.000}-Yüzde: {0:0.00%}",ondalık);  
Console.WriteLine(ondalıkstr); //Ondalık: 1234.568-Yüzde: 123456.78%  
Console.WriteLine($"{ondalık:0.000} |{ondalık,10:0.000%}|"); //1234.568 | 1234.568%|
```

Metinden Değer Tiplere Dönüşüm

`ReadLine` yöntemi ile konsoldan bir satır metin okunabilir. Sayı okumamız durumunda bu metnin istenilen değişken tipine dönüştürülmesi gerekir.

```
// See https://aka.ms/new-console-template for more information  
double sayi;  
Console.WriteLine("Karekökü Alınacak Sayıyı Giriniz");  
string okunan = Console.ReadLine();  
sayi = Convert.ToDouble(okunan);  
Console.WriteLine("Hesaplanan Karekök: {0}",Math.Sqrt(sayi));  
string str = "6161";  
int num = int.Parse(str);  
Console.WriteLine(num);  
string str2 = "6100";  
if (int.TryParse(str2, out int result)) {  
    Console.WriteLine(result);  
} else {  
    Console.WriteLine("Dönüştürme Başarısız Oldu.");  
}  
/*  
Adınız: İlhan, Kilonuz: 99,5, Paranız: 545,5  
Merhaba, Bugün 21.02.2025 00:00:00 Friday  
6161  
6100  
*/
```

KONTROL İŞLEMLERİ

Ardışık İşlem ve Kontrol İşlemleri

Nesne yönelimli programlama yöntemler, yapısal programlamadaki fonksiyonlar gibi kodlanır. Yani ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen kontrol yapılarına ilişkin talimatlar yazılır. Şu ana karar örneğini verdiğimiz kodlarda veri yapısı olarak sadece değişkenler kullanılmıştır. Sonrasında ise giriş çıkış işlemleri talimatlar içeren programlar yazılmıştır. Programın icrası ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte programın icrasını değiştirmeyen bu tür talimatlara **ardışık işlem** (**sequential operation**) adı verilir. Ardışık işlemler aşağıdaki üç tür **talimattan** (**statement**) oluşur.

1. Değişkenlerin kimliklendirildiği değişken **tanımlamaları** (**identifier definition**):

```
int yariCap=3;
const float PI=3.14f;
float daireninAlani,daireninCevresi;
```

2. **İfadeler** (**expression**); sabit, değişken ve **işleç** (**operator**) içeren talimatlardır:

```
daireninAlani=PI*yariCap*yariCap;
float daireninCevresi=2.0f*PI*yariCap;
```

3. Klavyeden veri okuma ve konsola veri yazma gibi **giriş çıkış işlemleri** (**input output operation**):

```
Console.WriteLine("Karekökü Alınacak Sayıyı Giriniz");
string okunan = Console.ReadLine();
```

Kontrol işlemleri (**control operation**) ise programın icra sırasını değiştiren kontrol yapıları olup bu **talimatlar** (**statement**) olup üç türü vardır;

1. **Duruma göre seçimler** (**conditional choice**): **if**, **if-else** ve **switch** talimatları.
2. **İlişkisel döngü** (**relational loop**): **while**, **do-while** ve **for** talimatları.
3. **Dallanmalar** (**jump**): **continue**, **break**, **goto** ve **return** talimatları.

Kontrol işlemlerinde; talimat olarak yazılmış **mantıksal satırlar** (**logical sequence**) ile **fiziksel olarak icra edilen satırlar** (**physical sequence**) birbirinden farklıdır.

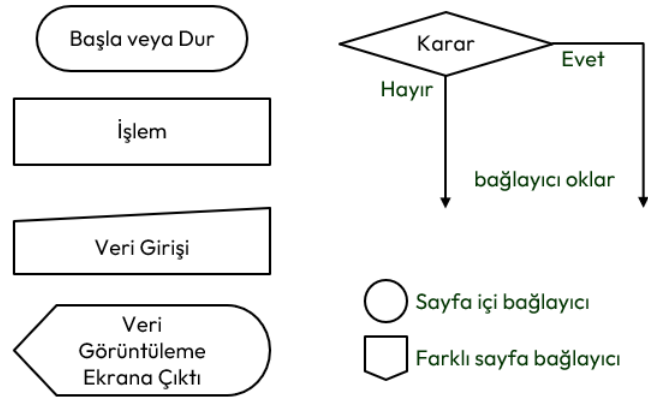
Akış Diyagramları

Kontrol işlemi içeren programın icrasını anlamak için akış diyagramlarını da anlamak gerekir. Akış diyagramı, adımların grafiksel gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde kullanılmak üzere genişletilmiştir.

Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış diyagramı, uygulanacak bir süreci veya projeyi tanımlamak için de kullanılabilir. Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

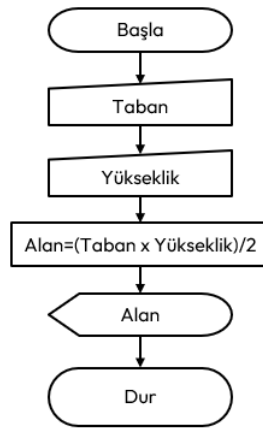
- Akış şemalarında tek bir başlangıç simgesi olmalıdır
- Bitiş simgesi birden çok olabilir.
- Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- Bağlaç simgesi sayfanın dolmasından ötürü parçalanan akış şemasının öğelerini birleştirmede kullanılır.
- Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

Aşağıda akış diyagramlarında kullanılan genel şekiller verilmiştir;



Şekil 9. Akış Diyagramları Genel Şekilleri

Aşağıda taban ve yüksekliği klavyeden girilen bir üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.



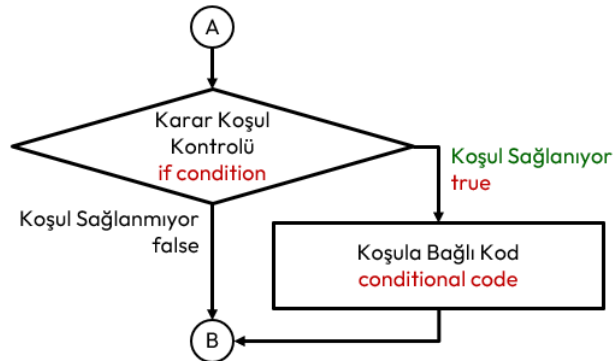
Şekil 10. Örnek Akış Diyagramı

Duruma Göre Seçimler

Duruma göre seçimler (conditional choice) programın akışını bir koşula göre değiştiren talimatlardır.

If Talimatı

If talimatı, karar vermeye (decision-making) ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. Koşul (condition) ifadesi **true** ise koşul kodu (conditional code) icra edilir değilse icra edilmez. Koşul ifadesi (expression) test edilir ve sıfırdan farklı ise **true**, aksi halde **false** kabul edilir.



Şekil 11. If Talimatı İcra Akışı

Sözde kodu aşağıda verilmiştir.

```
if (koşul)
    koşul-kodu;
```

Programın icrası, ilk talimatla başlar ve solda verilen sırayla icra edilir. Sağda ise her icra sonrası değişkenlerin değerleri gösterilmiştir.

		yas	cinsiyet
1	int yas = 18;	18	?
2	char cinsiyet = 'K';	18	'K'
3	if (yas < 30)	18	'K'
4	Console.WriteLine("Genç");	18	'K'
5	if (cinsiyet == 'E')	18	'K'
	Console.WriteLine("Erkek");	18	'K'

Tablo 18. If Talimatı İçeren Bir Programın İcra Sırası

Her talimatın icrasında neler olduğu aşağıda verilmiştir;

1. **yas** değişkenine **18** değeri atanır.
2. **cinsiyet** değişkenine **'K'** atanır.
3. **if** talimatındaki **koşul** (**condition**) test edilir. **yas<30** yani **18<30** testinde küçüktür işleci **true** verir. Bu nedenle izleyen **koşul kodu** (**conditional code**) icra edilir.
4. **Console.WriteLine("Genç");** koşul kodu olduğundan icra edilir.
5. **if** talimatındaki koşul test edilir. **cinsiyet=='E'** yani **'E'=='K'** testinde eşit mi işleci **false** verir. Dolayısıyla izleyen koşul kodu icra edilmez.

Bu durumda programın çalışması sonucu **Genç** çıktısı elde edilir. If talimatında **koşul kodu** (**conditional code**) her zaman **ardışık işlem** (**sequential operation**) olmaz. **if** gibi bir **kontrol işlemi** (**control operation**) de olabilir. Aşağıda **kademeli** (**compound/cascade**) if kullanımına ilişkin kod örneğinde ikinci **if**, birinci **if** talimatının koşul kodudur.

```
if (yas<30)
    if (yas<7)
        Console.WriteLine("Çocuk ");
```

Koşul kodu (**conditional code**) birden fazla talimattan oluşacak ise kod bloğu **{ }** içine alınır. Aşağıda verilen kod örneğinde ilk **if** talimatında **yas<30** ile test edilen koşul doğrulandığında kod bloğunun içindeki tüm talimatlar icra edilir.

```
if (yas<30) {
    if (yas<7)
        Console.WriteLine("Çocuk");
    if (yas<18)
        Console.WriteLine("Genç");
    if (yas>60)
        Console.WriteLine("Yaşlı");
}
```

Bahsedilen iki durum iç içe de olabilir. Buna ilişkin kod örneği de aşağıda verilmiştir.

```
if (cinsiyet=='E') {
    if (yas<7)
        Console.WriteLine("Erkek Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Delikanlı");
    if (yas>60)
        Console.WriteLine("Yaşlı Adam");
}
if (cinsiyet=='K') {
    if (yas<3)
        Console.WriteLine("Kız Bebek");
    if (yas<7)
        Console.WriteLine("Kız Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Kız");
    if (yas>60)
        Console.WriteLine("Yaşlı Kadın");
}
```


If talimatında **koşul** (condition) her zaman tek bir test ifadesinden oluşmayabilir. Bahsedilen duruma ilişkin kod örneği de aşağıda verilmiştir.

```
if ((cinsiyet=='E') && (yas<18))
    Console.WriteLine("Genç Delikanlı");
if ((cinsiyet=='K') && (yas>60))
    Console.WriteLine("Yaşlı Kadın");
// if (10>rakam>5) // HATA!. Böyle bir şart yazılamaz.
Console.WriteLine("rakam 10 ile 5 arasında ");
// if (rakam>10>rakam>5) // HATA!. Böyle bir şart yazılamaz.
Console.WriteLine("rakam 10 ile 5 arasında ");
```

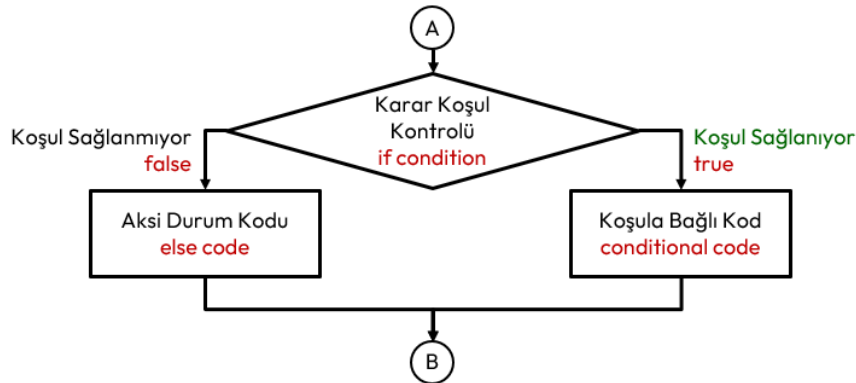
Bazen programcı tarafından aşağıdaki gibi **if** talimatları yazabilir.

```
if (true)
    Console.WriteLine("Bu metin konsola her zaman yazılır.");
if (false)
    Console.WriteLine("Bu metin konsola hiçbir zaman yazılmaz!");
```

If-Else Talimatı

If-Else Talimatı, bir başka **karar verme** (decision-making) talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatına benzer şekilde **koşul** (condition) ifadesi **true** ise **koşul kodu** (conditional code) icra edilir, değilse **aksi durum kodu** (else code) icra edilir.

```
if (koşul)
    koşul-kodu;
else
    aksi-durum-kodu;
```



Şekil 12. If-Else Talimatı Sözde Kodu ve İcra Akışı

Programın icrası, ilk talimatla başlar ve solda verilen sırayla icra edilir. Sağda ise her icra sonrası değişkenlerin değerleri gösterilmiştir

		yas	cinsiyet
1	int yas = 65;	65	?
2	char cinsiyet = 'K';	65	'K'
3	if (yas < 50)	65	'K'
	Console.WriteLine("Genç ");	65	'K'
4	else	65	'K'
5	Console.WriteLine("Yaşlı ");	65	'K'

Tablo 19. If-Else Talimatı İçeren Bir Programın İcra Sırası

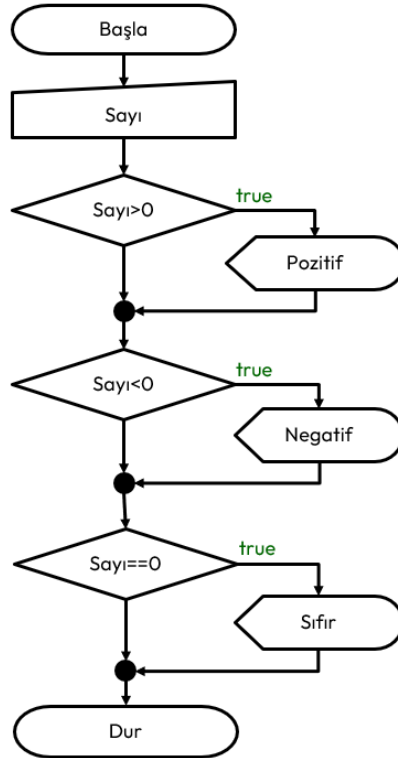
Her talimatın icrasında neler olduğu aşağıda verilmiştir;

1. **yas** değişkenine **65** değeri atanır.
2. **cinsiyet** değişkenine **'K'** atanır.
3. if talimatındaki **koşul** (condition) test edilir. **yas<50** yani **65<50** testinde küçüktür işleci **false** verir. Test sonucu **false** olduğundan aksi **durum kodu** (else code) icra edilir.
4. Programın icrası **else** ifadesinden devam eder.
5. **Console.WriteLine("Yaşlı ");** aksi durum kodu olduğundan icra edilir.

Bu durumda programın çalışması sonucu **Yaşlı** çıktısı elde edilir. If talimatında olduğu gibi bu talimatta da **koşul kodu** (conditional code) ya da **aksi durum kodu** (else code) birden fazla talimattan oluşacak ise kod bloğu { } içine alınır. Buna ilişkin kod örneği aşağıda verilmiştir.

```
if (cinsiyet=='E') {
    if (yas<7)
        Console.WriteLine("Erkek Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Delikanlı");
    if (yas>60)
        Console.WriteLine("Yaşlı Adam");
} else {
    if (yas<3)
        Console.WriteLine("Kız Bebek");
    if (yas<7)
        Console.WriteLine("Kız Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Kız");
    if (yas>60)
        Console.WriteLine("Yaşlı Kadın");
}
```

Örnek olarak klavyeden girilen bir sayının sıfırdan küçük mü? Büyük mü? ya da sıfıra eşit mi? Olduğunu bulan programı yazalım. Programın icra akışı aşağıdaki şekilde olacaktır.



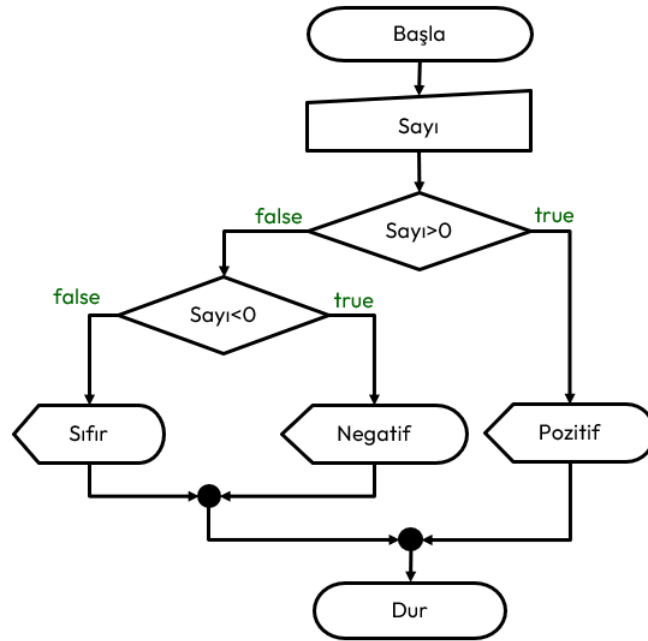
Şekil 13. Programın If Talimatlarıyla Yazılması Halinde İcra Akışı

Bu icra akışını sağlayan program aşağıda verilmiştir.

```
// See https://aka.ms/new-console-template for more information
int sayi;
Console.WriteLine("Sayı Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());
if (sayi > 0) // sayının pozitif olup olmadığı test ediliyor.
    Console.WriteLine("Pozitif"); // Burada sayının pozitif olduğu biliniyor.
if (sayi < 0) // sayının negatif olup olmadığı test ediliyor.
    Console.WriteLine("Negatif"); // Burada sayının negatif olduğu biliniyor.
if (sayi == 0) // sayının sıfır olup olmadığı test ediliyor.
    Console.WriteLine("Sıfır"); // Burada sayının sıfır olduğu biliniyor.
```

Bu akışta ilk if talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmaz. Benzer şekilde ilk iki test geçilirse üçüncü if talimatındaki teste gerek yoktur. Bu durumda aynı program if-else talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz.

```
// See https://aka.ms/new-console-template for more information
int sayi;
Console.WriteLine("Sayi Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());
if (sayi > 0)
{ // sayının pozitif olup olmadığı test ediliyor.
  Console.WriteLine("Pozitif"); // Burada sayının pozitif olduğu biliniyor.
}
else
{ // Burada sayının pozitif olmadığı biliniyor.
  if (sayi < 0)
  { // sayının negatif olup olmadığı test ediliyor.
    Console.WriteLine("Negatif"); // Burada sayının negatif olduğu biliniyor.
  }
  else
  { // Burada sayının pozitif ve negatif olmadığı test biliniyor.
    Console.WriteLine("Sıfır"); // Burada sayının sıfır olduğu biliniyor.
  }
}
}
```



Şekil 14. Programın If-else Talimatlarıyla Yazılması Halinde İcra Akışı

Program incelendiğinde sayının pozitif girilmesi halinde sadece ilk if talimatındaki test geçecek ve else sonrası aksi durum koduna ilişkin blok icra edilmeyecektir. Sayının negatif girilmesi halinde ilk if talimatının else bloğuna girilecek ve blok içindeki ilk if talimatındaki test geçecek ve bu if talimatının else kısmı çalıştırılmayacaktır. Sayı sıfır girilmiş ise blok içindeki else kodu çalıştırılacaktır. Bunların dışında blok içinde tek bir if-else talimatı bulunmaktadır. Dolayısıyla blok içine almaya gerek de yoktur. Bu durumda kodun nihai hali aşağıda verilmiştir.

```
if (sayi>0)
    Console.WriteLine("Pozitif");
else if (sayi<0)
    Console.WriteLine("Negatif");
else
    Console.WriteLine("Sıfır");
```

Sarkan Else

Aşağıdaki program örneği incelendiğinde else, hangi if talimatına aittir? Böyle bir kod programcı tarafından yazılabilir ve derleyici buna hiçbir sıkıntı çıkarmaz.

```
if (cinsiyet=='E') if (yas<18) Console.WriteLine("Delikanlı"); else Console.WriteLine("Adam");
```

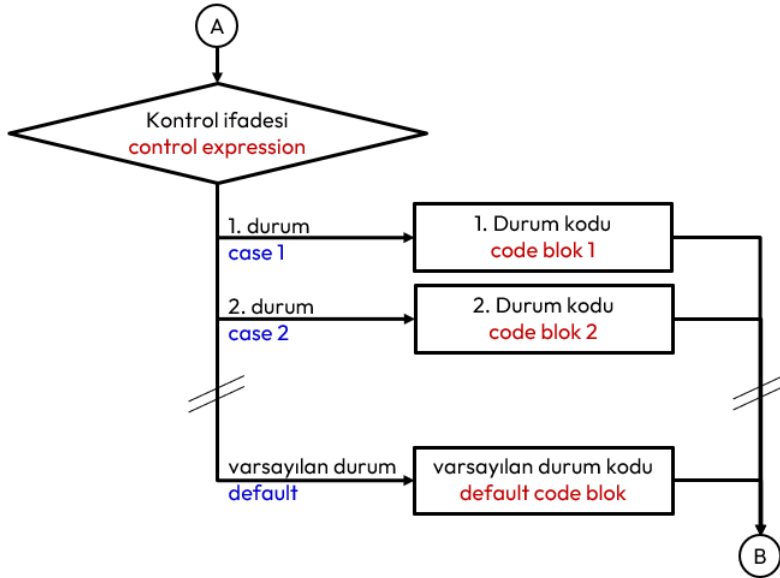
Bu durumda kodu aşağıdaki gibi okunaklı hale getirdiğimizde ilk **if** talimatının **koşul kodunun** (**conditional code**) ikinci if-else talimatı olduğu görülmektedir. Bu problem **sarkan else** (**dangling else**) problemi olarak bilinir. Kod bloğu kullanılmadan yazılan bu tip kodlarda else her zaman kendinden bir önceki if talimatına aittir.

```
if (cinsiyet=='E')
    if (yas<18)
        Console.WriteLine("Delikanlı");
    else
        Console.WriteLine("Adam");
```

Switch Talimatı

Switch talimatı, bir kontrol **ifadesi** (**expression**) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Sözde kodu aşağıda verilmiştir;

```
switch (kontrolifadesi) {
    case alternatif-1:
        alternatif-1-kodu;
        break;
    case alternatif-2:
        alternatif-2-kodu;
        break;
    // ...
    default:
        varsayılan-alternatif-kodu;
}
```



Şekil 15. Switch Talimatı Sözde Kodu ve İcra Akışı

Switch talimatına ilişkin kurallar;

1. **Kontrol ifadesi** (**control expression**) bir kez değerlendirilir.
2. Kod bloğu zorunludur. Yani bloğu olmayan bir **switch** talimatı yazılamaz!
3. Diğer birçok dilin aksine her bir alternatif case ifadesi, kontrol ifadesinin sonucunu bir **desen** (**pattern**) ile seçer ve iki nokta ile biter. Desenler, **ilişkisel desen** (**relational pattern**) ve **sabit desen** (**constant pattern**) olmak üzere iki türdür.

4. Blok için yazılan kod sonuna **break;** talimatı yazılarak **switch** bloğu dışına çıkılır. Zorunlu değildir, yazılmaz ise bir sonraki durum için yazılan kod icra edilir.
5. Alternatiflerin tümü yazılmayabilir.

Blok sonunda yer alacak şekilde üzerinde yer alan alternatifler dışında kalan tüm alternatifler için **default:** etiketli alternatif yazılabilir. Zorunlu değildir.

Aşağıda klavyeden girilen bir sayının 4 rakamına bölünmesinde kalanı gösteren bir program verilmiştir.

```
// See https://aka.ms/new-console-template for more information
int sayi;
Console.WriteLine("Bir Sayı Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());
switch (sayi % 4)
{ //konrol ifadesi bir işlem içerir
    case 3:
        Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 3 dir.");
        break;
    case 2:
    case 1:
        Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 2 veya 1 dir.");
        break;
    default: // Başka alternatif yoktur.
        Console.WriteLine("Sayı 4 Rakamına TAM Bölünür");
        break;
}
```

C ve C++ dillerinde case ifadelerinde sadece tamsayı değişmezler kullanılır. C# dilinde ise desenler kullanılabilir. Aşağıda gerçek sayı olarak yapılan bir ölçüm için alternatiflerin desen ile seçimi için yazılmış bir program örneği verilmiştir.

```
// See https://aka.ms/new-console-template for more information
double ölçüm;
Console.WriteLine("Ölçüm Giriniz:");
ölçüm = Convert.ToDouble(Console.ReadLine());
switch (ölçüm)
{
    case < 0.0:
        Console.WriteLine($"Ölçülen değer {ölçüm}, çok düşük.");
        break;
    case > 15.0:
        Console.WriteLine($"Ölçülen değer {ölçüm}, çok yüksek.");
        break;
    case double.NaN:
        Console.WriteLine("Yanlış Ölçüm.");
        break;
    default:
        Console.WriteLine($"Ölçülen değer: {ölçüm}.");
        break;
}
```

Üçlü Şart İşleci

Daha önce *Temel* İşleçler başlığı altında işlenenlere ek olarak, **ardışık işlemlerde** (**sequential operation**) if talimatlarına gerek kalmadan bir koşula göre **ifadeler** (**expression**) işlenecek ise **üçlü şart işleci** (**conditional tenary operator**) kullanılır. Aşağıda üçlü işlecin iki sözdizimi verilmiştir;

```
değişken = koşul ? kosulifadesi : aksidurumifadesi;
```

Aşağıda oy kullanma durumu bu işleçle işlenmiştir;

```
// See https://aka.ms/new-console-template for more information
int yas;
```

```
Console.WriteLine("Yaşınız?: ");
yas=Convert.ToInt32(Console.ReadLine());
string metin=(yas >= 18) ? "Oy Kullanabilirsin.": "Oy Kullanamazsın!";
Console.WriteLine(metin);
```

Aşağıda başka kullanımlarına ilişkin kod örneği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
int sayı;
bool tek, çift;
Console.WriteLine("Bir Sayı Giriniz: ");
sayı=Convert.ToInt32(Console.ReadLine());
tek = (sayı % 2==0) ? true : false;
çift =!tek;
int sonuç1 = tek ? sayı + 30 : sayı + 40;
int sonuç2 = çift ? sayı * 30 : sayı * 40;
Console.WriteLine($"Sonuç1: {sonuç1}, Sonuç2: {sonuç2}");
```

null Kaynaştırma İşleci

`null` **kaynaştırma işleci** (**null coalescing operator**) ile sol taraftaki işlenen `null` ise `null` olabilir bir tip için varsayılan bir değer belirtmenize olanak tanır.

```
string testString = null;
Console.WriteLine("Belirtilen Metin - " + (testString ?? "null'dür."));
```

Mantıksal olarak aşağıdaki koda eşdeğerdir;

```
string testString = null;
if (testString == null)
{
    Console.WriteLine("Belirtilen Metin - null'dür.");
}
else
{
    Console.WriteLine("Belirtilen Metin - " + testString);
}
```

`??=` işleci (**null coalescing assignment operator**) yalnızca değişken `null` ise bir değer atar;

```
int? number = null;
int result = number ?? 100; // 'number' null ise 100 atar
string message = null;
message ??= "Default Message"; // 'message' null ise atama yapar
```

İlişkisel Döngüler

İşlemciler iyi bir sayıcıdır. CPU içindeki **kaydediciler** (**registers**) sayma işlevini birçok matematiksel işlemi yapmak için de kullanılır. Program akışında belli problemlerin çözümünde, gerçekleştirilen belli adımların tekrarlanması sonucu gidilir. Bu tip problemler şimdiye kadar olan yöntemlerle yapılırsa, tekrar tekrar aynı kodu yazmak zorunda kalırız. Yazdığımız kodları tekrar tekrar icra edebilmek için **ilişkisel döngü** (**relational loop**) talimatlarını kullanırız.

Konsola 10 defa ismimizi yazdıran bir program örneğini ele alalım. Burada konsolda **satır başı** (**new line**) yapmak için konsola **Environment.NewLine** gönderilir.

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
```

```
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
```

Program incelendiğinde aynı `Console.WriteLine` talimatının tekrar tekrar yazıldığını görürüz. Bu istenmeyen bir durumdur. Bunu yapmamak için sayaç olarak kullanılan bir değişken kullanırız.

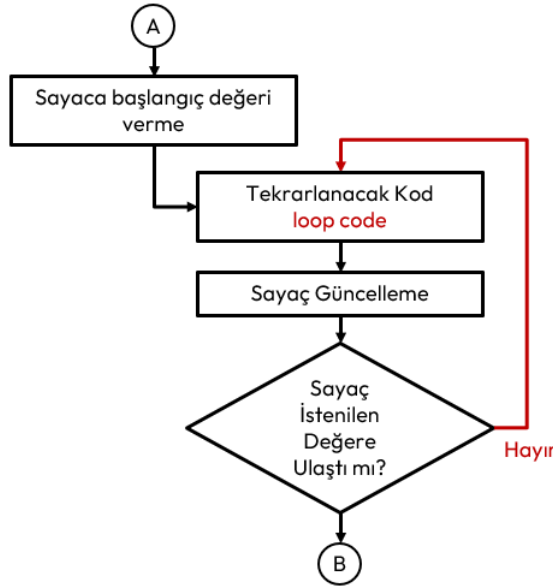
Sayaç Kontrollü Döngüler

Belirlenen bir **sayaç** (counter) değişkeninin istenilen bir değere ulaşip ulaşmadığı kontrol ederek kodun tekrar tekrar icra edilmesi sağlanır.

Yapısal programlama öncesinde bu döngü `goto` talimatıyla sağlanır. İcra sırasını etiketlenen bir yer olarak değiştirmek için `goto` talimatı kullanılır. Etiketlere kimlik verilirken yine **değişken kimliklendirme** (identifier definition) kuralları uygulanır.

```
// See https://aka.ms/new-console-template for more information
int sayaç=0;
Etiket:
Console.WriteLine("İLHAN");
sayaç++;
if(sayaç<10) goto Etiket;
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu üç adet talimat içeren **mantıksal satır** (logical sequence) olmasına karşın 30 adet **fiziksel satır** (physical sequence) icra edilmiştir.



Şekil 16. Sayaç Kontrollü Döngü İcra Akışı

C programlama dilinde ara seviye bir dil olduğundan bu talimat desteklenir. 1968 Yılında *Edsger W. Dijkstra* GOTO/JUMP TO ifadelerini zararlı olarak ilan edilmiştir¹⁷. GOTO kullanmamak için; while, do-while ve for **kontrol talimatları** yapısal programlamaya eklenmiştir.

While Talimatı

While döngüsü, **koşul** (condition) **true** olduğu sürece **tekrarlanacak kodu** (loop code) icra eder.

Tekrarlanacak kod, tek bir **talimat** (statement) olabileceği gibi bir kod bloğu da olabilir. Sözde kodu aşağıda verilmiştir;

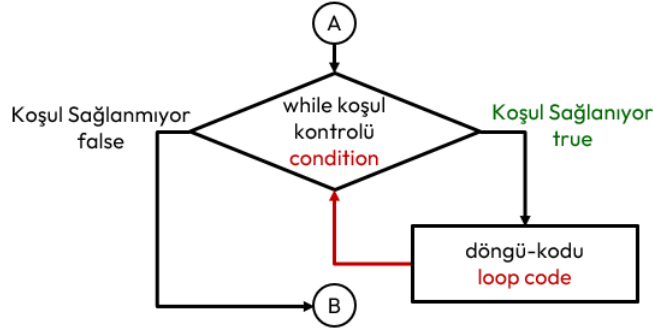
```
while (koşul)
    döngü-kodu;
```

Sayaç Kontrollü Döngüler başlığında verilen döngü, While talimatı ile aşağıdaki şekilde yazılabilir.

```
// See https://aka.ms/new-console-template for more information
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
```

¹⁷ <https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf>

```
while (sayaç<10) // while koşul kontrolü
{ // döngü bloğu
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
    sayaç=sayaç+1; //Sayaç Güncelleme
}
```



Şekil 17. While Talimatı İcra Akışı

Sayaç güncelleme ifadesi koşul ifadesine eklenerek program daha da kısaltılabilir. Her koşul kontrolü sonrası sayaç artırılacaktır.

```
// See https://aka.ms/new-console-template for more information
int sayaç=0; //sayaça İlk Değer 0 Olarak Verildi
while (sayaç++<10) // while koşul kontrolü ve sayaç güncelleme
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
```

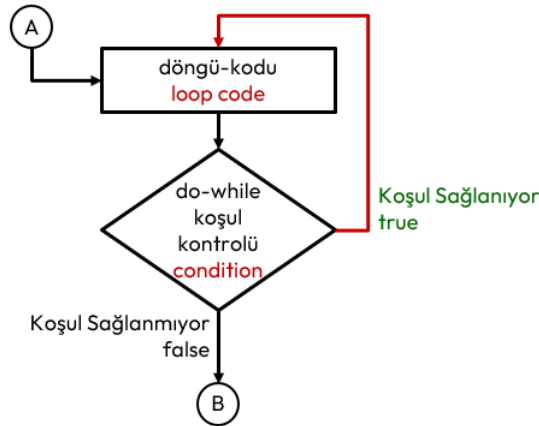
Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 10 adet fiziksel satır (physical sequence) icra edilmiştir.

Do-While Talimatı

Do-While döngüsü, **do** ile **while** saklı kelimeleri arasındaki **tekrarlanacak kodu** (loop code) **koşul** (condition) **true** olduğu sürece tekrarlayarak icra eder. Tekrarlanacak kod tek bir talimat (statement) olabileceği gibi bir kod bloğu da olabilir. While döngüsünde koşul kontrolü en başta yapılır, bu döngüde ise döngü bloğu en az bir kez çalıştırdıktan sonra koşul kontrolü yapılır.

Sözde kodu aşağıda verilmiştir;

```
do
    döngü-kodu;
while (koşul);
```



Şekil 18. Do-While Talimatı ve İcra Akışı

Sayaç Kontrollü Döngüler başlığında verilen döngü, Do-While talimatı ile aşağıdaki şekilde yazılabilir.

```
// See https://aka.ms/new-console-template for more information
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
do
```

```
{ // döngü bloğu
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
    sayaç=sayaç+1; //Sayaç Güncelleme
} while (sayaç<10); // Do-While koşul kontrolü
```

Sayaç güncelleme ifadesi koşul ifadesine eklenerek program daha da kısaltılabilir.

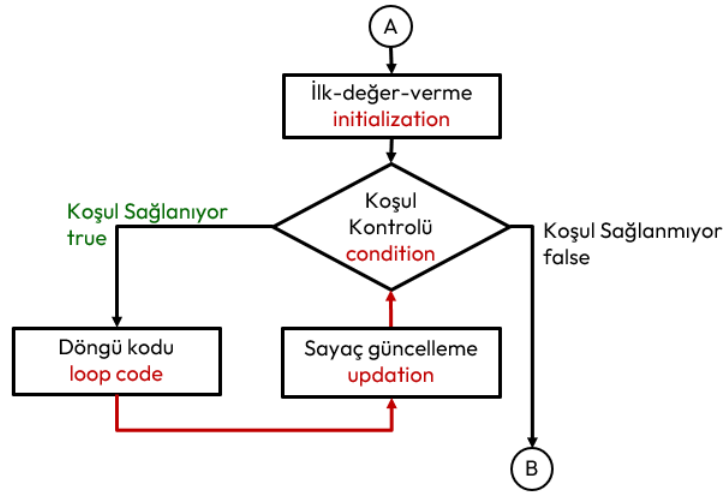
```
// See https://aka.ms/new-console-template for more information
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
do
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
while (sayaç++<10); // Do-While koşul kontrolü ve sayaç güncelleme
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu 1 adet talimat içeren **mantıksal satır** (logical sequence) içermesine rağmen 10 adet **fiziksel satır** (physical sequence) icra edilmiştir.

For Talimatı

For döngüsü özellikle tekrar edilen işlemlerin sayısı belli olduğunda kullanılan bir döngü yapısıdır. Sayaç kontrollü döngüler için en iyi seçimdir. Sözde kodu aşağıda verilmiştir.

```
for (ilk-değer-verme; koşul-kontrolü; sayaç-güncelleme)
    döngü-kodu;
```



Şekil 19. For Talimatı İcra Akışı

For talimatı aşağıdaki şekilde icra edilir;

1. İlk değer verme ifadesi bir kez icra edilir.
2. Sonra koşul kontrolü yapılır. Eğer test sonucu **true** ise döngü kodu icrasına geçilir. Aksi halde döngüden çıkılır.
3. Döngü kodu icra edilir.

Döngü kodu icrası bitince sayaç güncellemeleri yapılır ve tekrar ikinci adımdaki koşul kontrolüne dönülür. Sayaç Kontrollü Döngüler başlığında verilen döngü, For talimatı ile aşağıdaki şekilde yazılabilir.

```
// See https://aka.ms/new-console-template for more information
for (int sayaç=0; sayaç<10; sayaç++)
    Console.WriteLine("İLHAN");
```

Buradaki döngü mantıksal olarak iki satır talimattan oluşmaktadır. Ancak fiziksel olarak 10 satır icra edilmiştir. Aşağıda konsola yazdığımız her bir satırın başına satır numarası koyan bir program gösterilmektedir. Ayrıca sayaç sadece for bloğunda kullanılacak ise ilk değer verme ifadesinde tanımlanabilir.

```
// See https://aka.ms/new-console-template for more information
for (int sayaç=0; sayaç<10; sayaç++)
```



```
Console.WriteLine($"{sayaç}-İLHAN");
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren **mantıksal satır** (logical sequence) içermesine rağmen 10 adet **fiziksel satır** (physical sequence) icra edilmiştir.

For döngüsünde bir sayaç ile çalışılabileceği gibi birden fazla sayaç ile de çalışılabilir. Hem ilk değer verme hem de sayaç güncellemede birden fazla sayaca ilişkin işlem yapılabilir. Bunun için **virgül işleci** (comma operator) kullanılır.

```
// See https://aka.ms/new-console-template for more information
for (int artanSayaç=0, azalanSayaç=10;
     artanSayaç<10;
     artanSayaç++, azalanSayaç--)
    Console.WriteLine($"{artanSayaç}-İLHAN-{azalanSayaç}");
/* Program Çıktısı:
0-İLHAN-10
1-İLHAN-9
2-İLHAN-8
3-İLHAN-7
4-İLHAN-6
5-İLHAN-5
6-İLHAN-4
7-İLHAN-3
8-İLHAN-2
9-İLHAN-1
*/
```

Aşağıda klavyeden girilen iki sayı aralığında tek ve çift sayıların toplamını bulan ve ekrana yazan programı verilmiştir.

```
// See https://aka.ms/new-console-template for more information
int sayaç, sayı1,sayı2;
int tekToplam=0,çiftToplam=0;
Console.WriteLine("Birinci Sayıyı Giriniz: ");
sayı1=Convert.ToInt32(Console.ReadLine());
Console.WriteLine("İkinci Sayıyı Giriniz: ");
sayı2=Convert.ToInt32(Console.ReadLine());
if (sayı2<sayı1) { //sayı2, sayı1 den büyük olmalı.
    int temp=sayı1; //küçük ise yer değiştiriyoruz.
    sayı1=sayı2;
    sayı2=temp;
}
for (sayaç=sayı1; sayaç<=sayı2; sayaç=sayaç+1) {
    if (sayaç%2==1) //sayaç tek mi?
        tekToplam+=sayaç;
    else
        çiftToplam+=sayaç;
}
Console.WriteLine($"Tek Toplam:{tekToplam} Çift Toplam:{çiftToplam}");
/* Program Çıktısı:
Birinci Sayıyı Giriniz:
11
İkinci Sayıyı Giriniz:
1
Tek Toplam:36 Çift Toplam:30
*/
```

İç İçe Döngü Kodu

Döngüler içinde yer alan **döngü kodu** (loop code) bir başka döngüyü içerebilir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde iç içe döngü kullanırız. Aşağıda buna ilişkin bir örnek verilmiştir. İlk for döngüsünün tekrarlanan kodu ikinci for döngüsüdür.

```
// See https://aka.ms/new-console-template for more information
```

```
int satır,sütun;
for (satır=0; satır<5; satır++)
{
    for (sütun=0; sütun<4; sütun++)
        Console.Write("*");
    Console.WriteLine();
}
/* Program Çıktısı:
****
****
****
****
****
*/
```

Bir başka örnek olarak elemanları, satır ve sütun çarpımları olan 4x5 boyutlarındaki matrisi ekrana yazdıran program;

```
// See https://aka.ms/new-console-template for more information
int satır, sütun;
//BAŞLIK Yazdırılacak:
Console.Write("Sütunlar:");
for (sütun = 0; sütun < 4; sütun++)
    Console.Write($"{sütun,2} ");
Console.WriteLine();
//Matris Yazılacak:
for (satır = 0; satır < 5; satır++)
{
    Console.Write($"Satır {satır}:");
    for (sütun = 0; sütun < 4; sütun++)
        Console.Write("{0,2} ",satır*sütun);
    Console.WriteLine();
}/* Program Çıktısı:
Sütunlar: 0  1  2  3
Satır 0: 0  0  0  0
Satır 1: 0  1  2  3
Satır 2: 0  2  4  6
Satır 3: 0  3  6  9
Satır 4: 0  4  8  12
*/
```

Döngülerde Gözcü Kontrolü

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (**sentinel value**) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir.

Buna örnek olarak klavyeden 0 girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
// See https://aka.ms/new-console-template for more information
int sayı; /* okunan tamsayı */
int toplam = 0; /* girilen sayıların toplamı. başlangıçta 0*/
do
{
    Console.WriteLine("Bir sayı giriniz (Bitirmek için 0): ");
    sayı=Convert.ToInt32(Console.ReadLine());
    toplam += sayı; // sayı 0 girilse bile toplam etkilenmez
} while (sayı != 0); //sentinel value 0
Console.WriteLine($"Girilen Sayıların Toplamı: {toplam}");
/*Program Çıktısı:
Bir sayı giriniz (Bitirmek için 0):
10
Bir sayı giriniz (Bitirmek için 0):
20
```

```

Bir sayı giriniz (Bitirmek için 0):
30
Bir sayı giriniz (Bitirmek için 0):
5
Bir sayı giriniz (Bitirmek için 0):
0
Girilen Sayıların Toplamı: 65
*/

```

Aşağıda pozitif sayı girilmesini zorlayan bir kod örneği verilmiştir.

```

// See https://aka.ms/new-console-template for more information
int sayı, pozitifsayı = 0;
do
{ /* Pozitif girilmesini zorluyoruz */
    Console.WriteLine("Lütfen pozitif sayı giriniz: ");
    sayı=Convert.ToInt32(Console.ReadLine());
    if (sayı <= 0)
        Console.WriteLine("HATA:Pozitif Sayı GİRMEDİNİZ!");
} while (sayı <= 0);
pozitifsayı = sayı;
Console.WriteLine($"Girilen pozitif tamsayı {pozitifsayı} ");

```

Aynı örnek bir başka şekilde aşağıdaki gibi kodlanabilir.

```

// See https://aka.ms/new-console-template for more information
int sayı, pozitifsayı = 0;
Console.WriteLine("Lütfen pozitif sayı giriniz: ");
sayı = Convert.ToInt32(Console.ReadLine());
while (sayı <= 0)
{
    Console.WriteLine("HATA:Pozitif Sayı GİRMEDİNİZ!");
    Console.WriteLine("Lütfen pozitif sayı giriniz: ");
    sayı = Convert.ToInt32(Console.ReadLine());
}
pozitifsayı = sayı;
Console.WriteLine($"Girilen pozitif tamsayı {pozitifsayı} ");

```

BigInteger Veri Tipi

Fibonacci dizisi, her sayının kendisinden önceki iki sayının toplamı olduğu bir dizidir. Bu basit algoritma, en az 1000 ondalık basamağa ulaşana kadar Fibonacci sayılarını yineler, sonra da yazdırır. Bu değer, bir **ulong** veri tipinin tutabileceğinden bile önemli ölçüde daha büyüktür. Teorik olarak, **BigInteger** sınıfındaki tek sınır, uygulamanızın tüketebileceği RAM miktarıdır;

```

// See https://aka.ms/new-console-template for more information
using System.Numerics;
BigInteger l1 = 1;
BigInteger l2 = 1;
BigInteger current = l1 + l2;
while (current.ToString().Length < 1000)
{
    l2 = l1;
    l1 = current;
    current = l1 + l2;
}
Console.WriteLine(current);

```

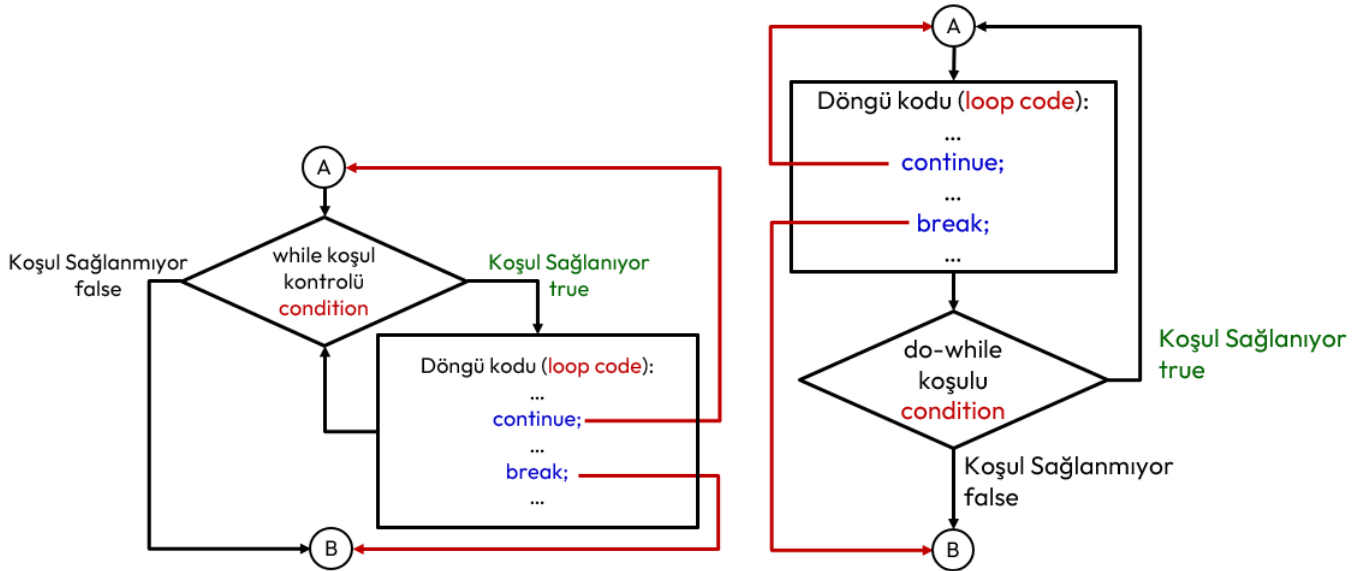
Dallanmalar

Dallanmalar (**jump**) programın akışını bir başka talimata yönlendiren talimatlardır.

Continue ve Break Talimatları

Break talimatı ile Switch talimatında ivedilikle switch bloğu dışına çıkıldığı anlatılmıştı. Benzer şekilde **do**, **while** ve **for** talimatlarında da ivedilikle yinelemeyi bitirip döngü bloğu dışına çıkılmasını sağlar. Continue talimatı yalnızca geçerli **yinelemeyi** (iteration) sonlandırır ve sonraki yinelemelerle devam eder. Dolayısıyla bu talimatlar sadece **döngü kodları** (loop code) içinde kullanılabilir.

break Talimatı döngüyü sonlandırır ve program icrası döngü sonrası ilk talimattan devam eder. Bu talimat, aynı şekilde While, Do-While ve For döngüleri ile kullanılabilir. **continue** Talimatı ise bir sonraki yinelemeyi yapmak için kullanılır. Bu talimatlarının döngü kodlarında kullanılması halinde icra akışı aşağıdaki şekilde verilmiştir.

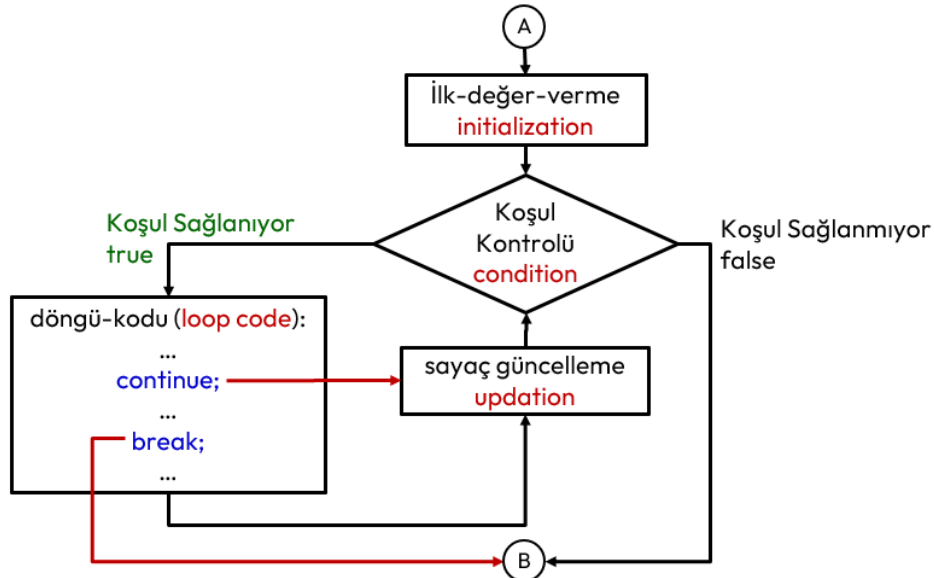


Şekil 20. While ve Do-While Döngü Kodunda break ve continue Talimatları İcra Akışı

While döngüsünde, döngü bloğunda **continue** kullanmadan önce koşulu değiştirecek bir talimat vermeliyiz. Aksi halde sonsuz döngüye girmiş oluruz. Do-While döngüsünde eğer **continue** talimatı bir şarta bağlanmalıdır. Yoksa sonsuz döngüye girilebilir.

Benzer durum for döngüsü için de geçerlidir. Ancak for döngüsünde **continue** talimatı sonrasında bir sonraki yineleme için önce sayaç güncelleme ifadeleri icra edilir ve sonra koşul testi yapılır. Bu durum göz önüne alınarak bu talimat kullanılmalıdır.

Bu talimatlar döngü kodu içinde amacımıza uygun olarak birden çok kullanabiliriz.



Şekil 21. For Döngü Kodunda break ve continue Talimatları İcra Akışı

Aşağıda 0 ile 15 arasındaki sayıların beşe bölünenleri ve 10 dışındaki sayılar ekrana yazdıran bir program örneği verilmiştir.

```
// See https://aka.ms/new-console-template for more information
for (int sayaç = 0; sayaç <= 15; sayaç = sayaç + 1)
{
    if (sayaç < 7) //sayaç'ın değeri 7 den küçük ise
        continue; //bir sonraki yinelemeye geç
    if (sayaç == 10) //sayaç değeri 10 ise
        continue; //bir sonraki yinelemeye geç
    if (sayaç % 5 == 0) // ssayaç değeri 5 e bölünüyorsa
        continue; //bir sonraki yinelemeye geç
    Console.WriteLine($"{sayaç}");
}
/*Program Çıktısı:
7
8
9
11
12
13
14
*/
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu üç adet talimat içeren mantıksal satır (logical sequence) olmasına karşın 42 adet fiziksel satır (physical sequence) icra edilmiştir. Bazı continue talimatları Console.WriteLine talimatının icrasını engellemiştir.

Aşağıda pozitif sayı girilmesini zorlayan sonsuz döngü içeren program verilmiştir. Programda döngü, pozitif sayı girildiğinde break talimatı ile kırılmaktadır.

```
// See https://aka.ms/new-console-template for more information
int sayı;
int pozitifsayı = 0;
while (true) /* Sonsuz döngü */
{
    Console.WriteLine("Lütfen pozitif sayı giriniz: ");
    sayı=Convert.ToInt32(Console.ReadLine());
    if (sayı <= 0)
        Console.WriteLine("HATA:Pozitif Sayı GİRMEDİNİZ!");
    else
        break; //döngüden çık
}
pozitifsayı = sayı;
Console.WriteLine("Girilen pozitif tamsayı: {pozitifsayı} ");
```

Return Talimatı

Yöntemlerde çokça kullanacağımız return talimatı yapısal programlamadaki bir fonksiyonun gibidir. Yöntemler de fonksiyonlar gibi kodlanır. Üreteceği değerleri return talimatı ile geri döndürür.

```
namespace ConsoleApp1
{
    internal class Program
    {
        static int KonsoldanBirTamsayıOku()
        {
            Console.WriteLine("Bir Tamsayı Giriniz:");
            int okunan = Convert.ToInt32(Console.ReadLine());
            return okunan;
        }
        static double KonsoldanBirGerçekSayıOku()
        {
            Console.WriteLine("Bir Gerçek Sayı Giriniz:");
```

```
        double okunan = Convert.ToDouble(Console.ReadLine());
        return okunan;
    }
    static void TamsayıyıYaz(int tamsayı)
    {
        Console.WriteLine($"{tamsayı}");
    }
    static int İşaretiniDeğiştir(int tamsayı)
    {
        return -tamsayı;
    }
    static int İkiTamsayıyıTopla(int tamsayı1,int tamsayı2)
    {
        return tamsayı1+tamsayı2;
    }
    static void Main(string[] args)
    {
        int sayı = KonsoldanBirTamsayıOku();
        TamsayıyıYaz(sayı);
        sayı=İşaretiniDeğiştir(sayı);
        TamsayıyıYaz(sayı);
        int toplam = İkiTamsayıyıTopla(sayı, 100);
        TamsayıyıYaz(toplam);
    }
}
```

Goto Talimatı

Tanımlı bir etikete program akışını yönlendiren **goto** talimatıdır. *Sayaç Kontrollü Döngüler* başlığında örneği verilmiştir.

DİZİLER

Şu ana kadar en basit **veri yapıları** (**data structure**) olan değişkenlerle karşılaştık. Programlamada en çok kullanılan bir başka veri yapısı da **dizilerdir** (**array**). Diziler dinamik veri yapıları olan **koleksiyonlardan** (**collection**) biridir. Dinamik veri yapılarına çalıştırma anında bellekte yer **new** anahtar kelimesiyle ayrılır!

Dizi Nedir?

Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (**array size**) denir. Elemanların sırası **indis** (**index**) olarak adlandırılır. Kümenin aksine aynı elemanlar dizide farklı konumlarda birkaç kez bulunabilir. Elemanlara terim adı da verilir.

Örneğin Elemanları 1'den 10'a kadar sayıların karesinden oluşan bir dizi; $(a_k)_{k=1}^{10}, a_k = k^2$ veya $(a_1, a_2, a_3, \dots, a_{10})$ veya $(1,4,9,\dots,100)$ olarak gösterilir. Örneğin;

- (K', I', T', A', P') Dizisi ile (P', A', T', I', K') dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları farklıdır.
- $(1,3,5,1,2,1,7)$ Dizisinde birden farklı indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.
- Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler yani **uzunluğu** (**size**) belirli diziler kullanılır.

Dizi Tanımlama

Üç tip dizi tanımlanabilir. Tek boyutlu diziler, her bir elemanı bir başka dizi olabilen çok boyutlu diziler veya her bir elemanı değişik boyutlu dizilerden oluşan **pürüzlü diziler** (**jagged array**). C# dilinde diziler aşağıdaki gibi tanımlanır;

```
veri-tipi[] dizi-kimliği; // Tek boyutlu dizi
veri-tipi[,] dizi-kimliği; // Çok boyutlu dizi
veri-tipi[][] dizi-kimliği; // Pürüzlü dizi
```

Dizilerin;

1. İçinde, benzer **veri tipindeki** (**data type**) veri öğelerini bulundurur ve bu öğeler bitişik bellek bölgesini paylaşırlar.
2. Elemanları, **ilkel veri tipleri** (**int, float, char**) veya daha sonra göreceğimiz kullanıcı tanımlı tip olan **yapı** (**struct**) veya **gösterici** (**pointer**) olabilir.
3. İçinde aynı değer birkaç farklı yerde bulunabilir.
4. Veri tipi, dizideki elemanlarının veri tipini belirler. Bir başka deyişle elemanların veri tipi dizinin veri tipiyle aynı olmalıdır.
5. Dizinin **indisi** (**index**) her zaman sıfırdan başlar! İndisler sıra belirttiğinden her zaman tamsayıdır!

```
// 5 elemanlı tek boyutlu bir tamsayı dizisi:
int[] ilkdeğeriolmayandizi = new int[5];
ilkdeğeriolmayandizi = null; // Üst satırda ayrılan bellek serbest bırakılıyor..
```

```
//eleman değerleri bilinen 6 elemanlı tamsayı dizisi
int[] dizi1 = new int[6] { 1, 2, 3, 4, 5, 6 };
int[] dizi2 = { 1, 2, 3, 4, 5, 6 };
```

```
int[] dizi3; //dizi tanımlandıktan sonra elemanlara ilk değer verilemez!
// dizi3 = { 24, 2, 13, 47, 45 }; //derleme hatası olur!
```

```
//Üstü Kapalı olarak dizi tanımlama
var arr = new [] { 1, 2, 3 }; // int[] ile aynıdır
var arr = new [] { "one", "two", "three" }; // string[] ile aynıdır
var arr = new [] { 1.0, 2.0, 3.0 }; // double[] ile aynıdır
```

Dizi Elemanlarını Değiştirme

Bir dizi tanımlandıktan sonra tamsayı indislerle elemanlara erişilebilir ve erişilen eleman değeri değiştirilebilir;

```
int[] dizi = new int[] { 0, 10, 20, 30};
Console.WriteLine(arr[2]); // dizinin 3. Elemanı yazdırılıyor: 20
arr[2] = 100; //dizinin 3.elemanına 100 değeri atandı.
Console.WriteLine(arr[2]); // güncellenmiş elemana yazdırılıyor: 100
```

Dizi Üzerinde Yinelemeler

Dizi üzerindeki elemanlarının tümüne **yineleme** (**iteration**) ile erişilebilir. Yineleme indis değişkeni içeren **for** döngüleriyle yapılır. Tek boyutlu dizilere örnek olarak klavyeden girilen 5 adet tamsayıyı, giriş sırasının tersinden ekrana yazan programını verebiliriz;

```
// See https://aka.ms/new-console-template for more information
int[] dizi = new int[5]; // 5 elemanlı tek boyutlu bir tamsayı dizisi
for (int indis = 0; indis < dizi.Length; indis++)
/* dizi elemanlarını gezecek ve indis olarak kullanılacak tamsayı bir değişken tanımlandı */
{
    Console.Write($"{indis}.Sayıyı Girin:");
    dizi[indis] = Convert.ToInt32(Console.ReadLine());
}
for (int indis = dizi.Length-1; indis >= 0; indis--)
/* dizi elemanlarını gezecek ve indis olarak kullanılacak tamsayı bir değişken tanımlandı */
{
    Console.WriteLine($"{indis}.Sayıyı:{dizi[indis]}");
}
/*
0.Sayıyı Girin:10
1.Sayıyı Girin:11
2.Sayıyı Girin:13
3.Sayıyı Girin:45
4.Sayıyı Girin:55
4.Sayıyı:55
3.Sayıyı:45
2.Sayıyı:13
1.Sayıyı:11
0.Sayıyı:10
*/
```

Bir başka örnek olarak konsoldan girilen 10 adet sınav notundan, ortalamanın üstünde olanları ekrana yazan programı;

```
// See https://aka.ms/new-console-template for more information
int[] dizi = new int[10]; // 10 elemanlı tek boyutlu bir tamsayı dizisi
for (int indis = 0; indis < dizi.Length; indis++)
/* dizi elemanlarını gezecek ve indis olarak kullanılacak tamsayı bir değişken tanımlandı */
{
    Console.Write($"{indis}.Notu Girin:");
    dizi[indis] = Convert.ToInt32(Console.ReadLine());
}
Console.WriteLine("{0} üzerindeki notlar:", dizi.Average());
for (int indis = 0; indis < dizi.Length; indis++)
{
    if ((double)dizi[indis] > dizi.Average())
        Console.WriteLine($"{indis}.Not:{dizi[indis]}");
}
/*Program Çıktısı:
0.Notu Girin:10
1.Notu Girin:15
2.Notu Girin:25
```



```

3.Notu Girin:35
4.Notu Girin:40
5.Notu Girin:60
6.Notu Girin:15
7.Notu Girin:20
8.Notu Girin:12
9.Notu Girin:16
24,8 üzerindeki notlar:
2.Not:25
3.Not:35
4.Not:40
5.Not:60
*/

```

Bir başka örnek olarak değeri belli 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani **sapma** (**deviation**) ve karesi alınmış sapmalar yani **varyans** (**variance**) ile standart sapmayı yazdıran program verilebilir.

```

// See https://aka.ms/new-console-template for more information
// 10 elemanlı tek boyutlu bir tamsayı dizisi:
decimal[] satislar = { 100.0m, 850.0m, 500.0m, 600.0m, 750.0m,
                      650.0m, 450.0m, 800.0m, 900.0m, 110.0m };
decimal ortalama = satislar.Average();
Console.WriteLine($"Ortalama:{ortalama}");
double varyansToplam = 0;
for (int indis = 0; indis < satislar.Length; indis++)
{
    decimal sapma = satislar[indis] - ortalama;
    decimal varyans = sapma * sapma;
    Console.WriteLine($"Sapma:{sapma}, varyans:{varyans}");
    varyansToplam += (double) varyans;
}
double standartSapma = Math.Sqrt(varyansToplam / satislar.Length);
Console.WriteLine($"Standat Sapma:{standartSapma}");
/*
Ortalama:571,0
Sapma:-471,0, varyans:221841,00
Sapma:279,0, varyans:77841,00
Sapma:-71,0, varyans:5041,00
Sapma:29,0, varyans:841,00
Sapma:179,0, varyans:32041,00
Sapma:79,0, varyans:6241,00
Sapma:-121,0, varyans:14641,00
Sapma:229,0, varyans:52441,00
Sapma:329,0, varyans:108241,00
Sapma:-461,0, varyans:212521,00
Standat Sapma:270,49768945408755
*/

```

Ortalamaya olan uzaklıklar yani sapma verilerin ortalamaya ne kadar yakın olduğunu gösteren dağılımdır. Sapmaların toplamı sıfır olabileceğinden karelerinin toplamı yani varyans hesaplanır. Varyansın eleman sayısına bağlı karekökü de standart sapmayı verir. Standart sapmanın büyük olması verilerin ortalamadan daha uzak yayıldıklarını; küçük bir standart sapma ise verilerin ortalama etrafında daha çok yakın gruplaştıklarını gösterir.

Bir başka örnek olarak 5 elemanlı diziye girilen elemanlardan **tekil** (**unique**) olanlarını bulan program verilebilir.

```

// See https://aka.ms/new-console-template for more information
int[] dizi = new int[5]; // 5 elemanlı tek boyutlu bir tamsayı dizisi
for (int indis = 0; indis < dizi.Length; indis++)
/* dizi elemanlarını gezecek ve indis olarak kullanılacak
   tamsayı bir değişken tanımlandı */

```

```

{
    Console.Write($"{indis}.Sayıyı Girin:");
    dizi[indis] = Convert.ToInt32(Console.ReadLine());
}
Console.WriteLine("Dizi İçindeki Tekil Elemanlar:");
for (int indis = 0; indis < dizi.Length; indis++)
{
    int indis2, sayac = 0; // Her elemanda sayacı sıfırla
    for (indis2 = 0; indis2 < dizi.Length; indis2++)
        if (indis != indis2) //elemanın kendisini kontrol etmiyoruz
            if (dizi[indis] == dizi[indis2])
                sayac++;
    if (sayac == 0)
        Console.WriteLine($"dizi[{indis}]:{dizi[indis]} tekil olarak dizide bulundu.");
}
/*
0.Sayıyı Girin:10
1.Sayıyı Girin:15
2.Sayıyı Girin:12
3.Sayıyı Girin:10
4.Sayıyı Girin:12
Dizi İçindeki Tekil Elemanlar:
dizi[1]:15 tekil olarak dizide bulundu.
*/

```

Bir başka örnek olarak 5 elemanlı belli olan dizide en büyük elemana en küçük elemanını ekleyen program verilebilir;

```

// See https://aka.ms/new-console-template for more information
int[] dizi = { 10, 12, 3, 4, 6, 12, 4, 7, 8, 9 };
Console.WriteLine("Dizinin İLK Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
Console.WriteLine();
int enKucuk = dizi[0], enBuyuk = dizi[0];
/* İlk elemanlar hem en küçük hem de en büyük olsun. */
int enKucukIndex = 0, enBuyukIndex = 0;
/* En küçük ve en büyük elemanların indisi 0 olsun. */
for (int indis = 0; indis < dizi.Length; indis++)
{ /* En küçük ve en büyük eleman ile
 * indislerini bulan kısım. */
    if (dizi[indis] < enKucuk)
    {
        enKucuk = dizi[indis];
        enKucukIndex = indis;
    }
    if (dizi[indis] > enBuyuk)
    {
        enBuyuk = dizi[indis];
        enBuyukIndex = indis;
    }
}
Console.WriteLine($"En Büyük Eleman İndisi: {enBuyukIndex} ve Eleman {enBuyuk}");
Console.WriteLine($"En Küçük Eleman İndisi: {enKucukIndex} ve Eleman {enKucuk}");
//En büyük elemana en küçüğünü ekleme
dizi[enBuyukIndex] += dizi[enKucukIndex];
Console.WriteLine("Dizinin SON Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
/*
Dizinin İLK Hali:
10 12 3 4 6 12 4 7 8 9
En Büyük Eleman İndisi: 1 ve Eleman 12
*/

```

```
En Küçük Eleman İndisi: 2 ve Eleman 3
Dizinin SON Hali:
10 15 3 4 6 12 4 7 8 9
*/
```

Aynı program dizi nesnesinin yöntemleriyle aşağıdaki gibi yazılabilir;

```
// See https://aka.ms/new-console-template for more information
int[] dizi = { 10, 12, 3, 4, 6, 12, 4, 7, 8, 9 };
Console.WriteLine("Dizinin İLK Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
Console.WriteLine();
int enKucuk = dizi.Min();
int enBuyuk = dizi.Max();
/* İlk elemanlar hem en küçük hem de en büyük olsun. */
int enKucukIndex = dizi.Single(enKucuk);
int enBuyukIndex = dizi.Single(enBuyuk);
Console.WriteLine($"En Büyük Eleman İndisi: {enBuyukIndex} ve Eleman {enBuyuk}");
Console.WriteLine($"En Küçük Eleman İndisi: {enKucukIndex} ve Eleman {enKucuk}");
//En büyük elemana en küçükünü ekleme
dizi[enBuyukIndex] += dizi[enKucukIndex];
Console.WriteLine("Dizinin SON Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
```

Dizileri Kopyalama

Dizinin bir kısmı bir başka diziye kopyalanabilir;

```
var kaynak1 = new int[] { 11, 12, 3, 5, 2, 9, 28, 17 };
var hedef1 = new int[3];
Array.Copy(kaynak1, hedef1, 3); // 11, 12, 3
```

Kaynak dizinin bir kısmı hedef dizinin bir bölümüne kopyalanabilir;

```
var kaynak2 = new int[] { 11, 12, 7 };
var hedef2 = new int[6];
kaynak2.CopyTo(hedef2, 2); // 0, 0, 11, 12, 7 , 0
```

Bir dizinin **özdeşi** (**clone**) yapılabilir;

```
var kaynak3 = new int[] { 11, 12, 7 };
var hedef3 = kaynak3.Clone(); // 11,12,17
```

Dizileri Karşılaştırma

Diziler, dizi nesnesinin **SequenceEqual** yöntemi ile karşılıklı elemanlar karşılaştırılabilir.

```
int[] dizi1 = { 3, 5, 7 };
int[] dizi2 = { 3, 5, 7 };
bool sonuç = dizi1.SequenceEqual(dizi2);
Console.WriteLine("İki Dizi Eşit Mi? {0}", sonuç); //true
```

Çok Boyutlu Diziler

Şu ana kadar gördüğümüz tek boyutlu diziler veya iki boyutlu diziler olan matrislerdir. Çok boyutlu dizilere örnek olarak En-Boy-Yükseklik içeren 3 boyutlu diziler verilebilir. Üç boyutlu dizilerde dizinin her bir elemanı bir matristir.

```
// 2 adet 3 boyutlu tamsayı dizisinden oluşan iki boyutlu dizi
int[,] çokboyutludizi = new int[2, 3];
/* Yukarıdaki gibi 2 satır 3 sütundan oluşan iki boyutlu dizi:
   Fakat eleman değerleri tanımlanırken biliniyor. */
int[,] çokboyutludizi1 = new int[2, 3] { { 1, 2, 3 }, { 4, 5, 6 } };
```

```
int[,] çokboyutludizi2 = { { 1, 2, 3 }, { 4, 5, 6 } };
```

Şu ana kadar gördüğümüz tek boyutlu dizilerdi. İki boyutlu diziler matematikten de bildiğimiz üzere **matris** (**matrix**) olarak adlandırılır. İki boyutlu dediğimizde en-boy ve satır-sütun gibi kavramlar akla gelmelidir.

Örnek olarak 3x4'lük matris elemanlarını klavyeden girip, tablo halinde ekrana yazdıran programı verebiliriz;

```
// See https://aka.ms/new-console-template for more information
int[,] matris = new int[3, 4]; // 3 satır 4 sütun
int satır, sütun; // indis değişkenleri
/*
for (sütun=0; sütun<matris.GetLength(1); sütun++) //Birinci Satır Okuyalım
    matris[0,sütun] = Convert.ToInt32(Console.ReadLine());
for (sütun=0; sütun<matris.GetLength(1); sütun++) //Birinci Satır Okuyalım
    matris[1,sütun] = Convert.ToInt32(Console.ReadLine());
for (sütun=0; sütun<matris.GetLength(1); sütun++) //Birinci Satır Okuyalım
    matris[2,sütun] = Convert.ToInt32(Console.ReadLine());
//Bunun yerine iç içe iki for tanımlanır;
*/
for (satır = 0; satır < matris.GetLength(0); satır++)
    for (sütun = 0; sütun < matris.GetLength(1); sütun++)
        matris[satır, sütun] = Convert.ToInt32(Console.ReadLine());
Console.WriteLine("Okunan Matris:");
for (satır = 0; satır < matris.GetLength(0); satır++)
{
    for (sütun = 0; sütun < matris.GetLength(1); sütun++) //Birinci Boyut İçin
        Console.Write($"{matris[satır, sütun],4}");
    Console.WriteLine();
}/*
11
12
13
14
21
22
23
24
31
32
33
34
Okunan Matris:
 11 12 13 14
 21 22 23 24
 31 32 33 34
*/
```

Bir başka örnek olarak elemanları belirli 5x4'lük matrisin satır ve sütun toplamalarını ekrana yazan programı verilebilir;

```
// See https://aka.ms/new-console-template for more information
int[,] matris = { {10,20,30},
                  {40,50,60},
                  {70,80,90},
                  {15,25,35},
                  { 5,15,25} };
int satır, sütun;
Console.WriteLine("Matris:");
for (satır = 0; satır < matris.GetLength(0); satır++)
{
    for (sütun = 0; sütun < matris.GetLength(1); sütun++)
        Console.Write($"{matris[satır, sütun],4}");
}
```

```

        Console.WriteLine();
    }
    Console.WriteLine("Satır Toplamları:");
    for (satır = 0; satır < matris.GetLength(0); satır++)
    {
        int satırToplam = 0;
        for (sütun = 0; sütun < matris.GetLength(1); sütun++)
            satırToplam += matris[satır, sütun];
        Console.WriteLine($" {satırToplam,4}");
    }
    Console.WriteLine();
    Console.WriteLine("Sütun Toplamları:");
    for (sütun = 0; sütun < matris.GetLength(1); sütun++)
    {
        int sütunToplam = 0;
        for (satır = 0; satır < matris.GetLength(0); satır++)
            sütunToplam += matris[satır, sütun];
        Console.WriteLine($" {sütunToplam,4}");
    }
}
/*
Matris:
10  20  30
40  50  60
70  80  90
15  25  35
5   15  25
Satır Toplamları:   60   150   240   75   45
Sütun Toplamları:  140   190   240
*/

```

Çok boyutlu dizi örneği olarak şöyle bir örnek verilebilir; 3 farklı ders alan 10 öğrenci, her bir dersten 4 değişik not almaktadır. Bu notların ağırlıkları %10, %30, %20 ve %40'tır. Notlar rastgele 0 ile 100 arasında verilecektir. Her bir sınavın ortalaması ile her bir öğrencinin ağırlıklı not ortalamalarını bulan programı aşağıda verilmiştir;

```

// See https://aka.ms/new-console-template for more information
int[, ] notlar = new int[3, 4, 10]; //DERSSAYISI, SINAVSAYISI, ÖĞRENCİSAYISI
int[] agirlik = { 10, 30, 20, 40 }; //0devler:%10, Vize:%30, Hızlı Sınav:%20, Final:%40
Random rastgete = new Random();
int i, j, k; //indis değişkenleri
for (k = 0; k < notlar.GetLength(0); k++)
    for (i = 0; i < notlar.GetLength(2); i++)
        for (j = 0; j < notlar.GetLength(1); j++)
            notlar[k, j, i] = rastgete.Next(0, 100);
//Her bir sınavın Ortalaması hesaplanacak:
Console.WriteLine("Sınav Ortalamaları:");
for (k = 0; k < notlar.GetLength(0); k++)
{
    Console.WriteLine($"{k}. Ders İçin ");
    for (j = 0; j < notlar.GetLength(1); j++)
    {
        float sınavToplam = 0.0f;
        for (i = 0; i < notlar.GetLength(2); i++)
        {
            sınavToplam += notlar[k, j, i];
        }
        sınavToplam /= notlar.GetLength(2);
        Console.WriteLine($"{j}. Sınav Ortalaması: {sınavToplam}");
    }
}
//Her bir Öğrencinin Ağırlıklı Notu:
Console.WriteLine("Her bir Öğrencinin Ağırlıklı Ortalaması:");

```

```
for (k = 0; k < notlar.GetLength(0); k++)
{
    Console.WriteLine($"{k}. Ders İçin;");
    for (i = 0; i < notlar.GetLength(2); i++)
    {
        float agirlikliNot = 0.0f;
        for (j = 0; j < notlar.GetLength(1); j++)
        {
            agirlikliNot += notlar[k, j, i] * agirlik[j] / 100.0f;
        }
        Console.WriteLine($"{i}. Ortalaması: {agirlikliNot}");
    }
}
/*Program Çıktısı:
Sınav Ortalamaları:
0. Ders İçin ;
0. Sınav Ortalaması: 62,7
1. Sınav Ortalaması: 30,8
2. Sınav Ortalaması: 56,3
3. Sınav Ortalaması: 56,7
1. Ders İçin ;
0. Sınav Ortalaması: 57,7
1. Sınav Ortalaması: 45,8
2. Sınav Ortalaması: 49,1
3. Sınav Ortalaması: 53,2
2. Ders İçin ;
0. Sınav Ortalaması: 61,2
1. Sınav Ortalaması: 61,1
2. Sınav Ortalaması: 24,2
3. Sınav Ortalaması: 51,7
Her bir Öğrencinin Ağırlıklı Ortalaması:
0. Ders İçin;
0. Ortalaması: 38,3
1. Ortalaması: 45,7
2. Ortalaması: 41,1
3. Ortalaması: 68,4
4. Ortalaması: 42,5
5. Ortalaması: 50,399998
6. Ortalaması: 53,9
7. Ortalaması: 59,4
8. Ortalaması: 52,600002
9. Ortalaması: 42,199997
1. Ders İçin;
0. Ortalaması: 40,199997
1. Ortalaması: 42,799995
2. Ortalaması: 67,9
3. Ortalaması: 15,5
4. Ortalaması: 68,1
5. Ortalaması: 56,4
6. Ortalaması: 55,2
7. Ortalaması: 40,4
8. Ortalaması: 81,7
9. Ortalaması: 37,9
2. Ders İçin;
0. Ortalaması: 25,9
1. Ortalaması: 56,2
2. Ortalaması: 51
3. Ortalaması: 26,500002
4. Ortalaması: 71
5. Ortalaması: 68,5
6. Ortalaması: 65,3
7. Ortalaması: 51,9
```

```
8. Ortalaması: 46,7
9. Ortalaması: 36,699997
*/
```

Pürüzlü Diziler

Pürüzlü diziler (**jagged array**) dizilerin dizisi olarak adlandırılabilir. Buradaki her dizi farklı boyutta olabilir.

```
// See https://aka.ms/new-console-template for more information
// Pürüzlü (jagged) Dizi Tanımı
int[][] pürüzlüDizi = new int[5][]; // 5 adet dizinin dizisi
pürüzlüDizi[0] = new int[3] { 1, 2, 3 }; //birinci dizi 3
pürüzlüDizi[1] = new int[] { }; // ikinci dizi 0 elemanlı
pürüzlüDizi[2] = new int[2] { 4, 5 }; // üçüncü dizi 2 elemanlı
pürüzlüDizi[3] = new int[1] { 6 }; // dördüncü dizi 1 elemanlı
pürüzlüDizi[4] = new int[5] { 7, 8, 9, 10, 11 }; // beşinci dizi 5 elemanlı
Console.WriteLine("Pürüzlü Dizi:");
for (int i = 0; i < pürüzlüDizi.Length; i++)
{
    for (int j=0; j < pürüzlüDizi[i].Length; j++)
        Console.Write("{0,4}",pürüzlüDizi[i][j]);
    Console.WriteLine();
}
```

Foreach Talimatı

Yalnızca bir koleksiyondaki öğeler arasında döngü oluşturmak için kullanılan **foreach** döngüsü (**foreach loop**) **aralık tabanlı döngü** (**range based loop**) olarak da bilinir. Bu döngü ileride göreceğimiz bütün koleksiyonlarda kullanılır. Aşağıdaki gibi yazılır;

```
foreach (veritipi dizi-elemanını-temsıl-eden-değişken in dizi-kimliği)
    döngü-kodu;
```

Buna örnek olarak aşağıda bir dizi programı örneği verilebilir;

```
// See https://aka.ms/new-console-template for more information
int[] dizi = { 1, 2, 3, 4, 5 };
float toplam = 0.0f;
foreach (int eleman in dizi)
    toplam += eleman;
float ortalama = toplam / dizi.Length;
Console.WriteLine("Dizi Ortalaması:{0}", ortalama);
```

SINIF VE NESNE

Sınıf ve Nesne Nedir?

Nasıl ki dünyamızda evler bir **mimari plan** (**blueprint**) üzerinden inşa ediliyor, **nesneler** (**object**) de bir plan üzerinden inşa edilir. Nesnelere ilişkin verilerin tutulduğu **durumların** (**state**) ve nesnelerin göstereceği **davranışlarının** (**behavior**) tanımlandığı bu plana **sınıf** (**class**) adı verilir. **Durumlara** (**states**) ilişkin veriler, **alanlarda** (**fields**) tutulur.

Nesne yönelimli programlamada, altyapının bize sunduğu sınıfları veya nesneleri kullanarak programı geliştiremeyiz. Mutlaka programcı tarafından ihtiyaçlara göre veri içeren ve davranış gösteren nesnelere ihtiyaç duyarız. İşte bu nesneler, **kullanıcı tanımlı sınıflar** (**user defined class**) ile tanımlanır.

Bir mimari plandan birçok ev yapılabileceği gibi, bir sınıftan da birçok nesne imal edilebilir. İmal edilen her **nesne** (**object**) sınıfın bir **örneğidir** (**instance**). Kullanıcı tanımlı sınıflar, en basit şekilde aşağıdaki gibi tanımlanır;

```
[erişim-değiştiricisi] class sınıf-kimliği {
    [erişim-değiştiricisi] veri-tipi alanKimliği;
    [erişim-değiştiricisi] veri-tipi davranışKimliği() { };
}
```

Sözde kodu verilen sınıf tanımındaki kimlikler için değişken kimliklendirme kuralları geçerlidir. Tanımlanan bir sınıftan nesneler **new** anahtar kelimesiyle dinamik olarak imal edilir. Yani nesne **çalıştırma anında** (**run time**) bellekte oluşur.

Şimdi ortak davranış ve özelliklerin tanımlandığı bir **Öğrenci** sınıfı ve bu sınıftan nesne imal eden **Program.cs** sınıfını kodlayalım.

```
// See https://aka.ms/new-console-template for more information

Öğrenci ali = new Öğrenci(); /* Öğrenci sınıfından ali nesnesi new kelimesiyle imal edildi.*/
ali.yaş = 40; /* "ali" nesnesinin "yas" alanı (field) 40 olarak değiştirildi.*/
ali.yaşınıSöyle(); /* "ali" nesnesine yaşınıSöyle mesajı verildi: Yaşım:40 */
ali.cinsiyetiniSöyle(); /* "ali" nesnesine cinsiyetiniSöyle mesajı verildi:
                        Cinsiyetimi Söylemek İstemiyorum! */

class Öğrenci // Öğrenci sınıfı
{
    public uint yaş; // yaş durumuna ilişkin verileri tutacak alan (field)
    public char cinsiyet; // cinsiyet durumuna ilişkin verileri tutacak alan (field)
    public void yaşınıSöyle() // yaşınıSöyle davranışını gösterecek yöntem
    {
        Console.WriteLine($"Yaşım:{yaş}");
    }
    public void cinsiyetiniSöyle() // cinsiyetiniSöyle davranışını gösterecek yöntem
    {
        if (cinsiyet == 'E' || cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (cinsiyet == 'K' || cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
}
```

Programda **class Öğrenci** kodu ile bir Öğrenci sınıfı tanımlanmıştır. Sonrasında **Öğrenci** sınıfından imal edilecek nesnelerin cinsiyet ve yaş bilgilerini tutacak **durumlar** (**state**) için **cinsiyet** ve **yaş** alanı tanımlanmıştır. Bir sınıftan imal edilecek nesnelerin umuma açık davranış ve özellikleri **public** erişimiyle tanımlanır.

Daha sonra Program sınıfı Öğrenci sınıfından nesne imal edip kullanması için değiştirilecektir. Bir sınıftan nesne imal etme **new** anahtar kelimesiyle olur. Main yönteminde imal edilen **ali** nesnesinin **yaş** alanı **public** oluşu için değiştirilebilmiştir. Ayrıca **ali** nesnesine **yaşınıSöyle** ve **cinsiyetiniSöyle** davranışlarını göstermesi için **ileti gönderilmiştir** (**message-passing**). Burada şuna dikkat etmek gerekir nesnenin alanları değiştiğinde yani **durumları** (**state**) değiştiğinde davranışların da bu durumdan etkilendiği gözden kaçırılmamalıdır.

Sınıftan nesne imal etme işlemine **örnekleme** (**instantiation**) adı verilir. Her bir nesne sınıfın bir **örneğidir** (**instance**). Örnek üyelerine yukarıda görüldüğü üzere **nokta işleci** (**dot operator**) ile erişilir.

Nesnelerin her biri;

1. **Öbek** (**heap**) bellek yer kaplar.
2. Bir **kimliğe** (**identity**) sahiptir.
3. Verileri tutan **durumlara** (**state**) sahiptir.
4. **Özelliklere** (**property**) sahiptir. Nesne, durumlarına değer atamayı veya nesne verilerini dışarı aktarmayı kontrol eder. Yani verileri kendi mülküdür.
5. **Davranışlara** (**behavior**) sahiptir. Nesnenin verilerine göre farklı yöntemlere davranış gösterirler.
6. **Roller** (**role**) veya bir başka adıyla **ara yüzlere** (**interface**) sahiptir. Nesneler kullanıldıkları yerlerdeki rollerine göre çalışır.
7. **Olaylardan** (**event**) etkilenirler.

Erişim Değiştiricileri

Sınıftan imal edilen nesnelerin üyelerine başka nesne ve programların nasıl erişeceği **erişim değiştiricileri** (**access modifiers**) ile belirlenir. **Görünürlük** (**visibility**) sıfatları olarak da adlandırılan bu değiştiriciler kısaca sınıf üyelerinin sınıf içinden ve dışından erişimi belirleyen sıfatlardır. Bunlar;

- **public**: Erişimi umuma açık olarak belirlenen durum ve davranışlara diğer bütün bileşenlerden erişilir.
- **private**: Erişimi mahrem/şahsi/kişisel olarak belirlenen durum ve davranışlara sadece içinde bulunulan sınıf içerisinden erişilebilir.
- **protected**: Erişimi korumalı olarak belirlenen durum ve davranışlara sadece içinde bulunulan sınıf ve türemiş sınıflar içerisinden erişilebilir. Bu belirleyici ileride *Kalıtım* başlığında incelenecektir.
- **internal**: Erişimi dahili olarak belirlenen durum ve davranışlara sadece içinde bulunulan **montaj** (**assembly**) içinden erişilebilir.
- **protected internal**: Erişimi dahili korumalı olarak belirlenen durum ve davranışlara içinde bulunulan sınıf ve türemiş sınıflar veya içinde bulunulan **montaj** (**assembly**) içinden erişilebilir.
- **private protected**: Erişimi mahrem korumalı olarak belirlenen durum ve davranışlara içinde bulunulan sınıf veya sınıftan türemiş sınıflar veya içinde bulunulan **montaj** (**assembly**) içinden erişilebilir.
- **file**: Erişimi dosya olarak belirlenen durum ve davranışlara içinde bulunulan kaynak kod dosyası içinden erişilebilir.

```
// See https://aka.ms/new-console-template for more information
Kişi ahmet = new Kişi(); /* Kişi sınıfından ahmet nesnesi new kelimesiyle imal edildi.*/
ahmet.adı = "Ahmet"; /* "ahmet" nesnesinin "adı" alanı değiştirilebildi.
                        Çünkü bu alan umuma açık (public) tanımlı. */
ahmet.adınıSöyle(); /* "ahmet" nesnesine adını söyle davranışını
                    göstermesi için mesaj verildi (message-passing):
                    Adım:Ahmet */
//ali.sırDeğiştir(); /* Bu yöntem çalışmaz! Çünkü mahrem (private) olarak tanımlanmış. */

class Kişi // Kişi sınıfı
{
    public string adı; /* "Adı" kimlikli alan(field),
                       imal edilecek nesnelerinin adına ait
                       durumlara(state) ilişkin verileri tutar. */
    private string sır; /* "sır" kimlikli alan(field),
                       imal edilecek Kişi nesnelerinin
```

```

        sırrına ait durumlara(state)
        ilişkin verileri tutar. */
    public void sırDeğiştir() /* sırDeğiştir() yöntemi(method),
        imal edilecek Kişi nesnelerinin
        sırrını depğiştirme
        davranışını(behavior) tanımlıyor */
    {
        sır = "Gizli bir şey öğrendim!";
    }
    public void adınıSöyle() /* adınıSöyle() yöntemi(method),
        imal edilecek Kişi nesnelerinin
        adını söyleme davranışını(behavior) tanımlıyor */
    {
        Console.WriteLine($"Adım:{adı}");
    }
}

```

Yukarıda erişim belirleyicilerin uygulandığı basit bir **Kişi** sınıfı örnek verilmiştir. Bu sınıftan imal edilen nesnelerin **adı** özelliği ve **adınıSöyle** davranışına diğer tüm nesneler tarafından erişilip değiştirilebilir, ancak **sır** özelliğine ve **sırDeğiştir** davranışına ise hiçbir nesne tarafından erişilemez.

Yöntemler

Sınıf tanımında görüldüğü üzere yöntemler imal edilecek nesnelerin davranışlarıdır. Yöntemler yapısal programlamadaki fonksiyonlar gibi kodlanır; *Yapısal Programlama* mantığında fonksiyonlar içinde ilk önce değişkenler tanımlanır ve daha sonra bu değişkenlerde tutulan veriler kontrol işlemleri ile işlenir.

Yöntem Tanımı

C# dilinde yöntemler aşağıdaki gibi tanımlanır;

```

Erişim-Değiştirici Seçimlik-Erişim-Değiştirici Dönüş-tipi Yöntem-kimliği(Parametre-Listesi)
{ //Yöntem Blok Başlangıcı
    /*
        Yöntem Gövdesi
        [return dönüş-tipinde-değer;]
    */
} //Yöntem Bloğu Bitişi

```

Yöntemin tanımı; girdileri yani parametreleri, yaptığı işlem ve döndüreceği değeri içerecek şekilde başlık ve gövdesinin kodlanmasından oluşur.

- Yöntemler, **sınıf (class)** ve ileride göreceğimiz **yapı (struct)** ve **ara yüz (interface)** içinde **erişim değiştiriciler (access modifier)** ile tanımlanır.
- Seçimlik erişim değiştiricileri **statik**, **override**, **virtual**, **abstract**, **new** ve **sealed** olabilir. İleride anlatılacaktır.
- Yöntem adı, fonksiyonun benzersiz kimliğidir ve kimliklendirmede, değişken kimliklendirme kuralları geçerlidir.
- Bağımsız değişkenler, yöntemlere giren parametrelerdir. Parametre listesi, bir işlevin parametrelerinin veri tipini, sırasını ve sayısını belirtir. Parametreler isteğe bağlıdır; yani bir fonksiyon hiçbir parametre içermeyebilir.
- Parametre listesi değişken bildirimlerine benzer ve virgül işlecisi ile birbirlerinden ayrılırlar; **[veritipi parametre1][,veritipi parametre2][,veritipi parametre3]...**
- Yöntemin gövdesi, yöntemin ne yaptığını tanımlayan talimatları içerir. Yapısal programlamadaki gibi ilk önce değişkenler tanımlanmalı daha sonra bu değişkenleri işleyecek talimatlar yazılmalıdır. Yöntemde istenilen sonuca ulaşmak için kullanacağı adımlara karar verilir yani algoritma belirlenir.
- Yöntemler bir değer döndürebilir. Dönüş tipi, yöntemin döndürdüğü değerin veri tipidir. Dönüş tipine uygun olarak bir değer, **return** talimatı ile geri döndürülmelidir. **void** veri tipi hiçbir değer döndürmeyen yöntemlerin geri dönüş tipidir. Bu durumda yöntemden geri dönmek için **return** kelimesi tek başına kullanılabilir.

- **return** talimatı, bir fonksiyonun yürütülmesini sonlandırır ve kontrolü çağırdığı yere geri verir. Yani programın icrasına devam etmek için çağırdığı yere dönlür.
- Yöntemin çağrıldığı yerde, geri döndürülen değerin tipi ile eşleşen bir değişkene dönüş değeri atanabilir.

Yöntem Çağırma

Statik Üyeler ve Statik Sınıf başlığında detayı verilmiştir. Şu ana kadar çok kullandığımız **statik** bir yöntemler aşağıdaki gibi çağrılır.

```
System.Console.WriteLine("Merhaba");// Tek argüman
string adı = "Kullanıcı";
System.Console.WriteLine("Merhaba, {0}!", adı); // Çoklu Argüman
string input = System.Console.ReadLine(); // Argümansız string geri döndüren.
```

İmal edilmiş nesnelerde yani **örnek** (*instance*) yöntemi nokta işleci ile aşağıdaki gibi çağrılır;

```
Kişi ahmet = new Kişi();
ahmet.adınıSöyle();
Hesap h = new Hesap();
h.örnek();
```

Aşağıda **topla** yöntemine sahip bir **Hesap** sınıfı ve bu yöntemin kullanılması örnek verilmiştir.

```
// See https://aka.ms/new-console-template for more information
Hesap h = new Hesap(); // Hesap sınıfından h adında bir nesne oluşturuldu
Console.WriteLine(h.topla(20, 30)); /* h nesnesine topla mesajı verildi.
                                     topla yönteminin geri döndürdüğü değer konsola
yazdırıldı. */
h.örnekhesap(); /* h nesnesine örnekhesap mesajı verildi.
                 örnekhesap yönteminin geri döndürdüğü değer konsola yazdırıldı. */

class Hesap // Hesap Sınıfı
{
    public int topla(int p1, int p2)
    {
        return p1 + p2;
    }
    public void örnekhesap()
    {
        topla(1, 2);
        /* topla fonksiyonu 1 ve 2 argümanlarıyla çağrılıyor.
         * Ama geri döndürdüğü değer kullanılmıyor */
        Console.WriteLine(topla(3, 4));
        /* 3 ve 4 argümanlarıyla çağrılan topla
         * yönteminin geri döndürdüğü değer
         * konsola yazdırıldı */
        int toplam;
        toplam = topla(5, 6);
        Console.WriteLine(toplam);
        /* topla yönteminin geri döndürdüğü değer
         * toplam yerel değişkenine atandı */
        Console.WriteLine(topla(7, 8) + topla(9, 10));
        /* topla yönteminin geri döndürdüğü değer ile
         * bir başka topla yönteminin geri döndürdüğü değer
         * toplanarak konsola yazdırıldı */
        Console.WriteLine(topla(10, topla(11, 12)));
        /* topla yönteminin argüman olarak
         * bir başka topla yönteminin geri döndürdüğü değer
         * geçirildi ve konsola yazdırıldı */
    }
}
```

Yöntemlere Parametre Geçirme

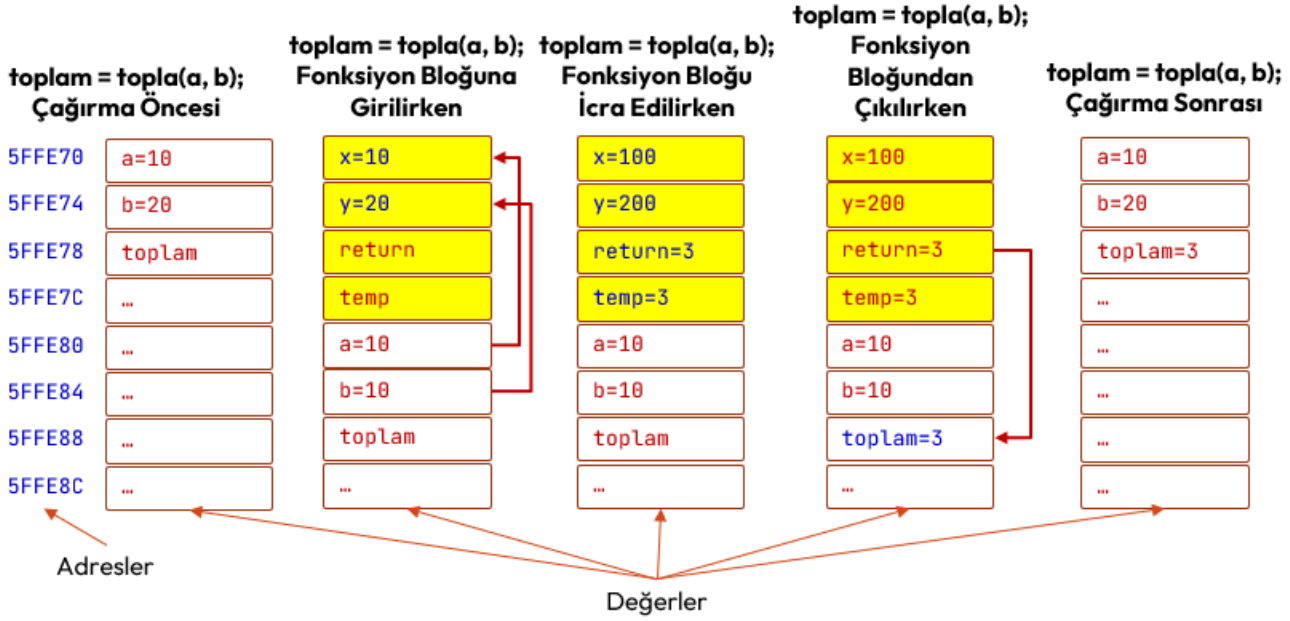
Bir yönteme değer tip parametre olarak geçirildiğinde **çağrı** (call/invoke) sırasında bellekteki değişim aşağıdaki programdan anlaşılabilir;

```
// See https://aka.ms/new-console-template for more information
int a = 10;
int b = 20;
int toplam = 0;
Hesap hesap = new Hesap(); //Hesap sınıfından bir hesap nesnesi imal ediliyor.
Console.WriteLine("çağrı öncesi a: {0}, b:{1}", a, b);
//çağrı öncesi a: 10, b:20
toplam = hesap.topla(a, b);
Console.WriteLine("çağrı sonrası a: {0}, b:{1}, toplam:{2}", a, b, toplam);
//çağrı sonrası a: 10, b: 20, toplam: 30

public class Hesap
{
    public int topla(int x, int y)
    { /* iki tamsayıyı toplayan fonksiyon tanımı*/
        /* Topla fonksiyonu yerel değişkenler */
        int temp = x + y;
        x = 100; y = 200;
        /*
            Burada değiştirilenler parametre değişkenleridir.
            main() içindeki yerel değişkenler değildir.
        */
        Console.WriteLine("çağrı içinde x: {0}, y:{1}", x, y);
        //çağrı içinde x: 100, y:200
        return temp;
    }
}
```

Yukarıdaki kod incelendiğinde;

- İcra sırası **topla(a,b)** fonksiyonuna geldiğinde fonksiyon çağrılmadan önce main fonksiyonu yerel değişkenleri a, b ve toplam değişkenleri yığın (stack) belleğe itilir (push).
- **topla(a,b)** fonksiyonu çağrıldığı anda fonksiyon bloğuna başlamadan parametrelere sırasıyla a ve b değişkenlerinin değerleri kopyalanarak fonksiyona geçirilir.
- **topla(a,b)** fonksiyonu icra edilirken fonksiyon yerelindeki **temp** ve parametre değişkenleri **x**, ve **y** istenildiği gibi değiştirilebilir. Geri dönüş değeri de belirlenir.
- **toplam=topla(a,b)** fonksiyon bloğundan çıkılırken geri dönüş değeri çağrı ortamındaki toplam değişkenine atanır ve fonksiyon için yığın (stack) belleğe itilen değerler geri çekilir.



Şekil 22. Fonksiyon çağırma sürecinde yığın bellek

Yukarıda örneği verilen programda fonksiyon çağırma sürecindeki **yığın bellek** (stack segment) değişimi gösterilmektedir. Bu örnekte olduğu gibi değer tiplerin parametre olarak bir fonksiyona geçirildiğinde yığın bellekte bu değerlerin bir kopyası oluşur ve yöntem içinde bu kopyalar kullanılır. Bu nedenle, yöntem içinde değer tip olarak geçirilen parametrede yapılan değişikliklerin argüman üzerinde hiçbir etkisi yoktur.

Bir yöntemde **değer tipi parametre olarak geçirirsek** (pass by value);

- Değer parametreleri değişkenlerin bir kopyasını oluşturur, bu nedenle yöntem içindeki herhangi bir değişiklik orijinal değeri değiştirmez. Ayrıca **değişmezleri** (literal) de yöntem parametre olarak geçirme imkânımız olur. Bu, orijinal verilerin değişmeden kalması gerektiğinde yararlıdır.
- Değer parametreleri orijinal değışkende değişiklik yapılmasını engeller. Orijinal değerleri değişmeden tutmak istediğinizde değer parametrelerini kullanabilirsiniz.
- Bellekte az yer kaplayan veri tipleri için değer parametreleri kullanabilirsiniz çünkü bunların kopyalanması kolaydır ve performansı etkilemez.

Yöntemlere Referans Olarak Parametre Geçirme

Bir referans değişken, bir değişkenin değerine değil bellek konumuna bir referanstır. Bir yöntemde **referans veri tipleri** ya da **bir referansı parametre olarak geçirdiğinizde** (pass by reference), değer parametrelerinin aksine, bu parametrelerin değerleri için yeni bir depolama konumu oluşturulmaz. Referans parametreleri, yöntem sağlanan gerçek parametrelerle aynı bellek konumunu temsil eder. Böylece parametre değerleri üzerinde değişiklik yapma imkânımız olur. Ancak değer tipleri için durum farklıdır. Değer tipindeki parametreleri yöntem içerisinde değiştirmek istiyorsak önünde **ref** anahtar kelimesi kullanmalıyız.

```
// See https://aka.ms/new-console-template for more information
Sayıİşlemleri işlemci = new Sayıİşlemleri(); // DeğerDeğiştir metodu için bir nesne oluşturduk
int a = 10;
int b = -10;
Console.WriteLine("DeğerDeğiştir, öncesi a:{0}, b:{1}", a, b); // a:10 b:-10
işlemci.DeğerDeğiştir(ref a, ref b);
Console.WriteLine("DeğerDeğiştir sonrası a:{0}, b:{1}", a, b); // a:-10 b:10

class Sayıİşlemleri
{
    public void DeğerDeğiştir(ref int x, ref int y)
    {
        int temp;
        temp = x;
```

```

        x = y;
        y = temp;
    }
}

```

Bir yönteme referans veri tipleri ya da **bir referansı parametre olarak geçirdiğinizde** (pass by reference);

- Orijinal değişkenleri değiştirme imkânımız olur.
- Hafızada çok yer kaplayan parametreler için çok kullanışlıdır.

Yöntem Parametrelerini Çıktı Olarak Kullanma

Bunların dışında yöntemdeki **return** ifadesi, bir fonksiyondan yalnızca bir değer döndürmek için kullanılabilir. Ancak, **out** parametrelerini kullanarak bir fonksiyondan birden fazla değer döndürebilirsiniz. **out** Parametreleri, referans parametrelerine benzerdir, ancak veriyi yönteme aktarmak yerine yöntemden dışarı aktarırlar.

```

// See https://aka.ms/new-console-template for more information
Sayıİşlemleri işlem = new Sayıİşlemleri();
int okunan, okunanınKaresi;
okunan = işlem.birSayı0ku(out okunanınKaresi);
Console.WriteLine($"Okunan: {okunan}, karesi:{okunanınKaresi}");

class Sayıİşlemleri
{
    public int birSayı0ku(out int kare)
    {
        int temp;
        Console.WriteLine("Bir sayı giriniz: ");
        temp = Convert.ToInt32(Console.ReadLine());
        kare = temp * temp;
        return temp;
    }
}

```

Ön Tanımlı Argüman

Argümanlar fonksiyonun çağrıldığı anda fonksiyona geçirilen gerçek parametrelerdir. Yöntemlere de fonksiyonlar gibi parametre geçirilebilir. Yukarıda verilen **topla** yönteminde iki farklı **int** parametresi aldığı görülmektedir. İstenirse bu parametrelere ön tanımlı bir değer atanabilir yani yöntemler **ön tanımlı argümanlar** (default argument) ile tanımlanabilir. Bu şekilde tanımlanan yöntem aşağıda örneği verildiği üzere **topla(10)**; şekilde çağrılabilir,

```

// See https://aka.ms/new-console-template for more information
Sayıİşlemleri işlem = new Sayıİşlemleri();
int sonuç= işlem.topla(10); // 10 + 1 = 11
Console.WriteLine($"Sonuç: {sonuç}"); //11

class Sayıİşlemleri
{
    public int topla(int p1, int p2 = 1) //İkinci parametre öntanımlı olarak 1
    {
        return p1 + p2;
    }
}

```

Özyinelemeli Yöntemler

Özyineleme (recursion), bir yöntemin kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, **yineleme** (iteration), **döngüler** (loop) veya tekrar yerine geçebilecek çok güçlü bir tekniktir. **Özyinelemeli** (recursive)

algoritmalarla, tekrarlar yöntemin kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur.

Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır.

1. Cevabı kesin olarak bildiğimiz **temel durum** (base case)
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n 'nin faktöriyeli, $n!$ ile gösterilir ve n 'den küçük veya ona eşit olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 3 \cdot 2 \cdot 1$$

Negatif olmayan bir tam sayı olan n 'nin faktöriyeli aynı zamanda n 'nin bir sonraki daha küçük faktöriyelle çarpımına eşittir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \cdot (n - 1)!$$

Örneğin;

$$5! = 5 \cdot 4! = 5 \cdot 4 \cdot 3! = 5 \cdot 4 \cdot 3 \cdot 2! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$$

Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir yöntem yazılmıştır.

```
// See https://aka.ms/new-console-template for more information
Faktöriyel f = new Faktöriyel();
Console.WriteLine(f.faktöriyelHesapla(5)); // 120
class Faktöriyel
{
    public uint faktöriyelHesapla(uint n)
    {
        uint sonuc = 1, sayaç;
        for (sayaç = n; sayaç > 0; sayaç--)
            sonuc = sonuc * sayaç;
        return sonuc;
    }
}
```

Sıfır sayısı da pozitif bir sayıdır ama sonucum sıfır çıkmaması için 0 sayısının faktöriyeli 1 kabul edilir. Bu aslında faktöriyel problemi için **temel durumdur** (base case) ve **boş ürün** (empty product) kuralı olarak bilinir. Kısaca faktöriyel fonksiyonu aşağıdaki şekilde gösterilir;

$$n \in \mathbb{N} \text{ olmak üzere } f(n) = n! = \begin{cases} 1, & n = 0 \\ n \cdot f(n - 1), & n > 0 \end{cases}$$

Özyinelemeli (recursive) olarak ise bu yöntem aşağıdaki şekilde kodlanabilir;

```
public uint faktöriyelHesapla(uint n)
{
    return n == 0 ? 1 : n * Factorial(n - 1);
}
```

Özyinelemeli olarak Fibonacci sayısı aşağıdaki gibi hesaplanabilir;

```
public int fibonacci(int i) {
    if(i <= 2) return 1;
    return fibonacci(i - 2) + fib(i - 1);
}
```

Özyinelemeli olarak bir tamsayının kuvveti aşağıdaki gibi hesaplanabilir;

```
public int kuvvet(int taban, int üs) {
    if (taban == 0) return 0;
    if (üs == 0) return 1;
    return taban * kuvvet(taban, üs - 1);
}
```


Parametre Olarak Diziler

Yöntemlere parametre olarak gönderilen dizinin boyutu belirten köşeli parantez açılır ve kapatılır. Bu durumda tek boyutlu diziler parametre olacak ise dizinin boyutu yazılmayabilir.

Aşağıda dizileri parametre olarak alan fonksiyon bildirim örnekleri bulunmaktadır;

```
void diziIsleyenFonksiyon1(int[] tekBoyutluDizi);
void diziIsleyenFonksiyon2(int[,] matris);
void diziIsleyenFonksiyon3(int[, ,] üçBoyutluDizi);
```

Aynı fonksiyonları aşağıdaki şekilde çağırabiliriz;

```
int[] notlar=new int[10];
int[,] dersNotlari=new int[2,10]; //2 dersi alan 10 öğrenci için notlar;
int[, ,] bolumDersNotlari=new int[4,2,10]; //4 bölümde 2 dersi alan 10 öğrenci için notlar;
//...
diziIsleyenFonksiyon1(notlar);
diziIsleyenFonksiyon2(dersNotlari);
diziIsleyenFonksiyon3(bolumDersNotlari);
//...
```

Aşağıda bir diziyi okuyan, ekrana yazan ve eleman ortalamalarını hesaplayan fonksiyonlara sahip bir program verilmiştir;

```
// See https://aka.ms/new-console-template for more information
int[] dizi = { 1, 2, 3, 4, 5 };
Diziİşlemleri işlemler = new Diziİşlemleri();
Console.WriteLine("Dizi:");
işlemler.diziyaz(dizi);
// 1 2 3 4 5
Console.WriteLine();
Console.WriteLine("Ortalama: {0}", işlemler.ortalama(dizi));
// Ortalaması: 3
int[,] matris = new int[4, 4]{ { 1, 2, 3, 4},
                              { 5, 6, 7, 8},
                              { 9, 10, 11, 12},
                              {13, 14, 15, 16} };

Console.WriteLine("Matris Önce:");
işlemler.matrisyaz(matris);
/*
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
*/
işlemler.transpose(matris);
Console.WriteLine("Matris Sonra:");
işlemler.matrisyaz(matris);
/*
1 5 9 13
2 6 10 14
3 7 11 15
4 8 12 16
*/

class Diziİşlemleri
{
    public float ortalama(int[] dizi)
    {
        return (float)dizi.Average();
    }
    public void diziyaz(int[] dizi)
    {
        for (int i = 0; i < dizi.Length; i++)
```



```

    {
        Console.Write($"{dizi[i],4}");
    }
}
public void transpose(int[,] dizi)
{
    for (int i = 0; i < dizi.GetLength(0); i++)
    {
        for (int j = i; j < dizi.GetLength(1); j++)
        {
            int temp = dizi[i, j];
            dizi[i, j] = dizi[j, i];
            dizi[j, i] = temp;
        }
    }
}
public void matrisyaz(int[,] dizi)
{
    for (int i = 0; i < dizi.GetLength(0); i++)
    {
        for (int j = 0; j < dizi.GetLength(1); j++)
        {
            Console.Write($"{dizi[i, j],4}");
            Console.WriteLine();
        }
    }
}
}

```

Çağrı Kuralı

Çağrı kuralı (**calling convention**), bir fonksiyon çağrısıyla karşılaşıldığında argümanların fonksiyona hangi sıraya göre iletileceğini belirtir. Birincisi C# dilinde kullanılır ve argümanlar soldan başlanarak sağa doğru fonksiyona geçirilir. İkincisi ise C/C++ dilinde kullanılır ve argümanlar sağdan başlanarak sola doğru fonksiyona geçirilir.

```

// See https://aka.ms/new-console-template for more information
Sayıİşlemleri işlem = new Sayıİşlemleri();
int sonuç = işlem.topla(10,20,30); //p1:10,p2:20,p3:30
Console.WriteLine($"Sonuç: {sonuç}"); // Sonuç: 60
int a = 1;
int toplam = işlem.topla(a, ++a, a++);
/* Beklenen Sonuç: 1+2+2=5
 * Parametreler: p1:1,p2:2,p3:2 */
Console.WriteLine($"Sonuç: {toplam}"); // Sonuç: 5

class Sayıİşlemleri
{
    public int topla(int p1, int p2, int p3)
    {
        Console.WriteLine($"p1:{p1},p2:{p2},p3:{p3}"); // p1: 1
        return p1 + p2+ p3;
    }
}

```

Yukarıdaki örnek incelendiğinde **sonuç1** ve **sonuc2** argümanlarının hangi sırada fonksiyona geçirildiğinin bir önemi yoktur. Ancak aşağıda verilen C++ kodu örneği farklı çıktı üretecektir.

```

#include <iostream>
using namespace std;
int topla(int, int, int);
int main () {
    int a = 1,toplam;
    toplam = topla(a, ++a, a++);
    cout << "toplam:" << toplam << endl;
}

```

```

}
int topla(int x, int y, int z) {
    cout << "x:"<< x << " y:" << y << " z:" << z<< endl;
    return x+y+z;
}
/* Program çalıştırıldığında:
x:3 y:3 z:1
toplam:7
*/

```

Bu kodun çıktısı **1 2 3** olarak beklenir ancak çağrı kuralından ötürü çıktı **3 3 1** olur. Bunun nedeni C++ dilinin çağrı kuralının sağdan sola olmasıdır. Bunu şöyle açıklayabiliriz;

1. Fonksiyona sağdan ilk argüman olarak a değişkeni değeri 1 geçirilir.
2. Ardından a++ ifadesi ile a değişkeni 2'ye yükseltilir.
3. Sonra ++a ifadesi ile a değişkeni 3'e yükseltilir.
4. Fonksiyona ikinci argüman olarak 3 geçirilir.
5. Fonksiyona son olarak a'nın son değeri olan 3 geçirilir.

Çağrı kuralı C++ dillerinde yazılmış bir kütüphanedeki fonksiyonları projemizde kullanmak istediğimizde önem kazanır¹⁸.

Nesne Yapıcısı

Bir mimari plandan birçok ev yapılabileceği gibi, bir sınıftan birçok nesne imal edilebilir. İmal edilen her nesne (object) sınıfın bir örneğidir (instance). İmal edilecek nesnelerin ön tanımlı (default) olarak belirlenen durumlarla (state) imal edilmesi gerekiyorsa nesne yapıcısı (constructor) kullanılır.

Yapıcıların görevi bir sınıftan nesneleri, varsayılan durumlara sahip olarak imal etmektir. Nesnelere ait veriler, alanlarda (field) tutulurlar ve nesnenin durumu (state) bu veriler belirler. Çünkü durumlar nesnenin davranışını etkiler.

Nesne yapıcıları sınıf kimliğiyle aynı olup yöntemlerin aksine geri değer döndürmezler. Amacı kısaca imal edilecek nesnelerin durumlarına ilk değer vermektir.

```

// See https://aka.ms/new-console-template for more information
Öğrenci ilhan = new Öğrenci(); /* Öğrenci sınıfından "ilhan" nesnesi;
                                Öğrenci() yapıcısı kullanılarak
                                yaş durumu 18 ve
                                cinsiyet durumu E olarak imal edildi. */

ilhan.yaşınıSöyle(); // Yaşım:18
ilhan.cinsiyetiniSöyle(); // Cinsiyetim: ERKEK
ilhan.yaş = 50; /* "ilhan" nesnesinin umuma açık "yaş" durumu değiştirildi. */
ilhan.yaşınıSöyle(); // Yaşım:50
/* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message passing). */

class Öğrenci // Öğrenci sınıfı
{
    public Öğrenci() // Öğrenci sınıfından imal edilecek ön tanımlı yapıcı
    {
        /* Yapıcı içerisinde imal edilecek nesnelerde, alanlara (field) ilk
        * değer verilir. */
        yaş = 18;
        cinsiyet = 'E';
    }
    public uint yaş;
    /* "yaş" kimlikli alan(field), öğrencinin yaşına ait durumlara(state)
    * ait verileri tutar. */
    public char cinsiyet;
    /* "cinsiyet" kimlikli alan(field), öğrencinin yaşına ait durumlara(state)
    * ait verileri tutar. */
}

```

¹⁸ <https://learn.microsoft.com/en-us/dotnet/standard/native-interop/calling-conventions>

```

public void yaşınıSöyle()
{
    /* yasiniSoyle() yöntemi(method), öğrencinin yaşını söyleme
    * davranışını(behavior) tanımlıyor */
    Console.WriteLine($"Yaşım:{yaş}");
}
public void cinsiyetiniSöyle()
{
    /* cinsiyetSoyle() yöntemi(method), öğrencinin cinsiyetini söyleme
    * davranışını(behavior) tanımlıyor */
    if (cinsiyet == 'E' || cinsiyet == 'e')
        Console.WriteLine("Cinsiyetim: ERKEK");
    else if (cinsiyet == 'K' || cinsiyet == 'k')
        Console.WriteLine("Cinsiyetim: KADIN");
    else
        Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
}
}

```

Örnekte **Öğrenci** sınıfından imal edilecek her nesne ön tanımlı olarak yaşı 12, ve cinsiyeti 'E' olarak üretilecektir. Hiçbir parametresi olmayan bu yapıcıya **ön tanımlı yapıcı** (default constructor) adı verilir.

Soyut, Somut ve Bilgi Gizleme

Soyut (abstract) kavramı kelime itibarıyla detay içermeyen, özet anlamına gelir. **Somut** (concrete) ise en ince detayına kadar bilinen, elle tutulur anlamına gelir. Bir şey hakkında soyutlama yapıldığında, o şeyin detayıyla ilgili olan tüm **bilgiler gizlenir** (information hiding).

Soyut sınıflar (abstract class) da tanımlanabilir ancak bu sınıflardan nesne imal edilemez. Yani bu sınıfın örneği (instance) olamaz. Daha çok kavramsal olarak kullanılır; Soyut bir sınıfın en az bir **soyut yöntemi** (abstract method) olur. Soyut sınıflar hangi davranışı göstereceğini söyler ama nasıl davranış göstereceğini anlatmaz. Bu nedenle abstract yöntem için gövde tanımlanmaz. **Somut sınıf** (concrete class) ile nesne imal edilebilir. **Yani örneği** (instance) vardır. Aşağıda soyut bir sınıf tanımı verilmiştir;

```

abstract class Soyut
{
    public abstract void Yöntem(); //Soyut Yöntemler için gövde tanımlanmaz!
}

```

Somut sınıflar ise ileride anlatılacaktır.

Bir sınıftan imal edilen nesneler tüm **durum** (state) veya **davranışlarını** (behavior) diğer nesnelere şeffaf olarak gösterip diğer nesnelerin değiştirmesine izin verilmemesi gerekir. Bu durumda nesnenin durum ve davranışları tutarsız hale gelir. Bu nedenle bir sınıftan imal edilen nesnelerin durum ve davranışlarının diğer nesnelere veya dış dünyaya kontrollü bir şekilde açılması gerekir.

Soyutlama (abstraction), kullanıcıya gerekli bilgileri gösterme ve kullanıcıya göstermek istemediği veya belirli bir kullanıcıyla ilgisi olmayan ayrıntıları gizleme işlemidir. Bir sınıfta soyutlama iki türlü yapılır;

- **Veri soyutlaması** (data abstraction): nesnenin durumları üzerinden yapılan soyutlama.
- **Kontrol soyutlaması** (control abstraction): nesnenin davranışları üzerinde yapılan soyutlama.

Sarmalama

Yukarıda verilen **Öğrenci** örneğinden devam edecek olursak imal edilecek nesnenin **yaş** durumuna bir başka nesne veya program **-1** gibi bir değer atayabilir. Bu durumda **yaşınıSöyle()** davranışında istenmeyen bir çıktı verilmesine sebep olur. Bunu gidermek için veri soyutlaması yapılması gerekir. Bu durumda yas alanına bir değer koymayı ve yas alanından bir değer almayı kontrol altına almalıyız. Benzer durum **cinsiyet** alanı için de geçerlidir.

Veri soyutlamasını yapmak için **yaş** ve **cinsiyet** alanlarını **mahrem** (**private**) olarak belirleyip bu alanlara değer atama ve değer okuma işlevlerini yerine getirecek **özellikler** (**property**) olan **Yaş** ve **Cinsiyet** tanımlanmıştır. Özellikler, nesnenin durumlarına kontrollü değer atayan ve değerleri kontrollü okuyan yapılardır. Özelliklerin veri tipi, kontrol edeceği **alanın** (**field**) veri tipiyle aynıdır. Bunun için **get** ve **set** kelimeleri kullanılır. Set yöntemindeki **value** atanmak istenen değeri gösteren değerdir;

```
// See https://aka.ms/new-console-template for more information
Öğrenci ilhan = new Öğrenci();
ilhan.Yaş = 50; // "ilhan" nesnesinin "yas" özelliği değiştirildi.
ilhan.yaşınıSöyle(); // Yaşım:50
ilhan.Yaş = 5; /* ilhan nesnesinin Yaş özelliği değişmez:
                Öğrenci 6 Yaşından Küçük Olamaz.*/
ilhan.Cinsiyet = 'H'; /* ilhan nesnesinin Cinsiyet özelliği değişmez:
                Cinsiyet E,e,K,k,B,b dışında bir değer olamaz. */

class Öğrenci
{
    public Öğrenci() // Öğrenci yapıcısı
    {
        yaş = 18;
        cinsiyet = 'E';
    }
    private uint yaş;
    public uint Yaş //Yaş Özelliği: Özelliğin get yada set yöntemi olur.
    {
        get { return yaş; }
        set {
            if (value < 6)
                Console.WriteLine("Öğrenci 6 Yaşından Küçük Olamaz.");
            else
                yaş = value;
        }
    }
    private char cinsiyet;
    public char Cinsiyet //Cinsiyet Özelliği: Özelliğin get yada set yöntemi olur.
    {
        get { return cinsiyet; }
        set {
            if (value == 'E' |
                value == 'e' |
                value == 'K' |
                value == 'k' |
                value == 'B' |
                value == 'b')
                cinsiyet = value;
            else
                Console.WriteLine("Cinsiyet E,e,K,k,B,b Dışında " +
                                   " Bir Değer Olamaz.");
        }
    }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (cinsiyet == 'E' || cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (cinsiyet == 'K' || cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
}
```

```
}
}
```

İşte bir **durumun** (**state**) veri soyutlaması yapılmış haline **özellik** (**property**) adı veririz. Çünkü artık duruma dışarıdan doğrudan müdahale edilemez. Nesnenin durum üzerinde kontrol hakları vardır.

Benzer şekilde **davranışlar** (**behavior**) da nesne dışına kontrollü olarak açılabilir. Bu durumda da **kontrol soyutlaması** (**control abstraction**) yapılır. Aşağıda faktöriyel hesaplayan bir hesap makinesinin ekranı yenileme mahrem davranışı olan **ekraniYenile()** buna örnek verilebilir;

```
// See https://aka.ms/new-console-template for more information
HesapMakinesi hesaplayici = new HesapMakinesi(); // Hesaplanan: 0
hesaplayici.Hesaplanan = 3;
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 6
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 720 (6! = 720)
hesaplayici.makineyiSıfırla();
hesaplayici.Hesaplanan = 8;
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 40320 (8! = 40320)

class HesapMakinesi
{
    private int hesaplanan;
    private void ekranıYenile()
    {
        Console.WriteLine($"Hesaplanan: {hesaplanan}");
    }
    public HesapMakinesi()
    { //Yapıcı
        hesaplanan = 0;
        ekranıYenile();
    }
    public int Hesaplanan
    {
        set
        {
            hesaplanan = value;
            ekranıYenile();
        }
        get
        {
            return hesaplanan;
        }
    }
    public void makineyiSıfırla()
    {
        hesaplanan = 0;
        ekranıYenile();
    }
    public void FaktöriyelHesapla()
    {
        int temp = 1;
        for (int sayac = hesaplanan; sayac > 0; sayac--)
            temp = temp * sayac;
        hesaplanan = temp;
        ekranıYenile();
    }
}
```

Bir sınıf üzerinde **veri soyutlaması** (**data abstraction**) ile veriler üzerine kılıf çekilir, hem de **kontrol soyutlaması** (**control abstraction**) ile davranışların üzerine bir kılıf çekilir. Bir sınıf üzerinde her iki işlemin birlikte yapılmasına **sarmalama** (**encapsulation**) adı verilir. Soyutlamaların gerekli olup olmadığına programcı karar verir.

Sarmalama işlemi örneklerde görüldüğü üzere **erişim değiştiricilerle** (**access modifier**) yapılır. Böylece imal edilen nesnelerin verileri ve davranışları güvenli bir şekilde dış dünyaya açılmış olur. Böylece, sınıf ve nesne gibi bileşenlerin içeriğini bilmeden **soyut** (**abstract**) olarak kullanmak mümkün olmaktadır. Programcı tasarladığı sınıfın soyut olarak katlanılacağını düşünerek sınıfları tasarlamalıdır.

Özetle **sarmalama** (**encapsulation**) tekniğini uygulamak, kullanımı kolay **sınıf** (**class**), **bileşen** (**component**), **kütüphane** (**library**) ve **çerçeve yapılar** (**framework**) elde etmemizi sağlar;

- n adet **veri** (**data**) ve n adet **davranış** (**behavior**) sarmalama işlemine tabi tutulursa **sınıf**,
- n adet sınıf sarmalama işlemine tabi tutulursa **bileşen**,
- n adet bileşen sarmalama işlemine tabi tutulursa **kütüphane**,
- n adet kütüphane sarmalama işlemine tabi tutulursa **çerçeve yapılar** oluşur.

Böylece bize projemizde proje süresini ve maliyetini kısaltma üstünlüğünü verir.

Kalıtım

Öğrenci, öğretmen ve müdürün olduğu bir sistemi örnek olarak ele alalım.

Kalıtım Olmadan Sınıf Tasarımı

Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak gerekir.

```
// See https://aka.ms/new-console-template for more information
public class Öğrenci
{
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public int ÖğrenciNo { get; set; }
    public void dersÇalış()
    {
        Console.WriteLine("Ders Çalışıyorum.");
    }
}
```

Burada **Yaş özelliğinde** (**property**) hiçbir durum kontrolünün yapılmaması **{ get; set; }** olarak kodlanır. Benzer şekilde Öğretmen için ayrı bir sınıf;

```
// See https://aka.ms/new-console-template for more information
public class Öğretmen
{
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
    }
}
```

```

        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public int VediğiDersKodu { get; set; }
    public void dersVer()
    {
        Console.WriteLine("Ders Veriyorum.");
    }
}

```

Müdür için ayrı bir sınıf;

```

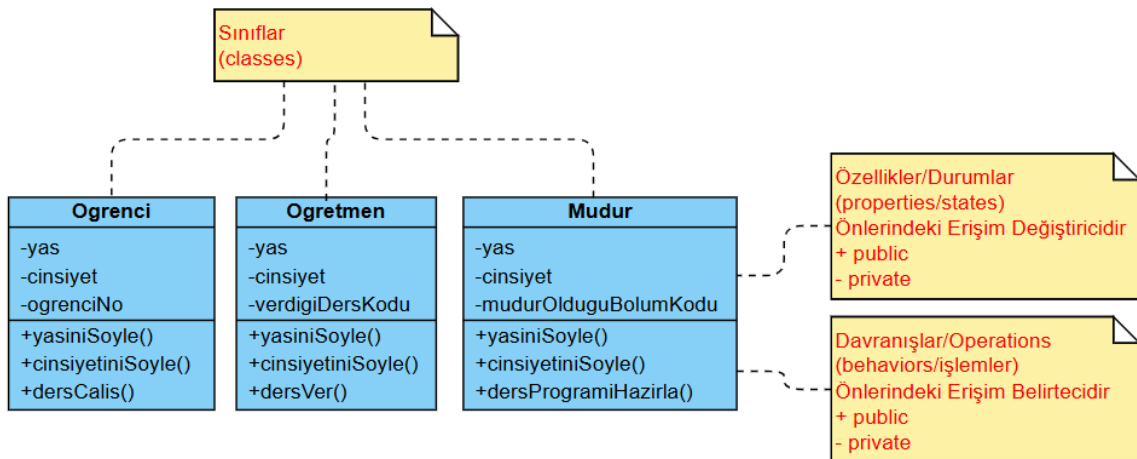
// See https://aka.ms/new-console-template for more information
public class Müdür
{
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public int MüdürOlduğuBölümKodu { get; set; }
    public void dersProgramıHazırla()
    {
        Console.WriteLine("Ders Programı Hazırlıyorum.");
    }
}

```

Bu sınıfların her birinde **Yaş** ve **Cinsiyet** gibi ortak **özellikler** (property) veya **yaşınıSöyle()** ve **cinsiyetiniSöyle()** gibi ortak **davranışlar** (behavior) bulunur. Her sınıf için bunlar tekrar tekrar tanımlanırlar. Hâlbuki bu gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanabilir.

Kalıtım İçeren Sınıf Uygulaması

Bütün bu ortak özellik ve davranışlar aşağıdaki diyagramdaki gibi tek bir sınıfta toplandığı zaman daha yönetilebilir bir kod elde edilir.



Şekil 23. Öğrenci, Öğretmen ve Müdür UML Sınıf Diyagramları

Yazılım analizi yapılırken takım içerisinde iletişimi kolaylaştırmak için aşağıdaki sınıf diyagramları (class diagram) kullanılır. Bunlar Unified Modelling Language-UML dilinin bir parçasıdır.

Ortak özellik ve davranışların bir sınıfta toplanarak bu sınıftan miras alınmasına kalıtım (inheritance) adı verilir. Yukarıdaki örnekte ortak özellik ve davranışlar **Kişi** sınıfına toplanarak aşağıdaki şekilde kod yeniden düzenlenebilir;

Aşağıda verilen ve Kişi olarak tanımlanan genel sınıf (general class), taban sınıf (base class) veya ebeveyn sınıf (parent class) olarak adlandırılır. Daha sonra tanımlanacak Öğrenci, Öğretmen ve Müdür sınıflarının ortak özellik ve davranışlarını barındırır.

```
// See https://aka.ms/new-console-template for more information
class Kişi
{
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
}
```

Aşağıda taban sınıftan türemiş (derived) Öğrenci, Öğretmen ve Müdür sınıfları verilmiştir. Bu sınıflar genel sınıf olan Kişi sınıfının yanında kendine özel özellik ve davranışlara da sahiptir. Özel sınıf (private class), türemiş sınıf (derived class) veya çocuk sınıf (child class) olarak adlandırılan bu sınıflar, mirasçı olup, kendilerinden imal edilmiş nesneler; hem taban sınıfın özellik ve davranışlarını, hem de kendi özellik ve davranışlarını sergilerler.

```
// See https://aka.ms/new-console-template for more information
class Öğrenci : Kişi
{
    public int ÖğrenciNo { get; set; }
    public void dersÇalış()
    {
        Console.WriteLine("Ders Çalışıyorum.");
    }
}
```



```

class Öğretmen: Kişi
{
    public int VediğiDersKodu { get; set; }
    public void dersVer()
    {
        Console.WriteLine("Ders Veriyorum.");
    }
}
class Müdür: Kişi
{
    public int MüdürOlduğuBölümKodu { get; set; }
    public void dersProgramıHazırla()
    {
        Console.WriteLine("Ders Programı Hazırlıyorum.");
    }
}

```

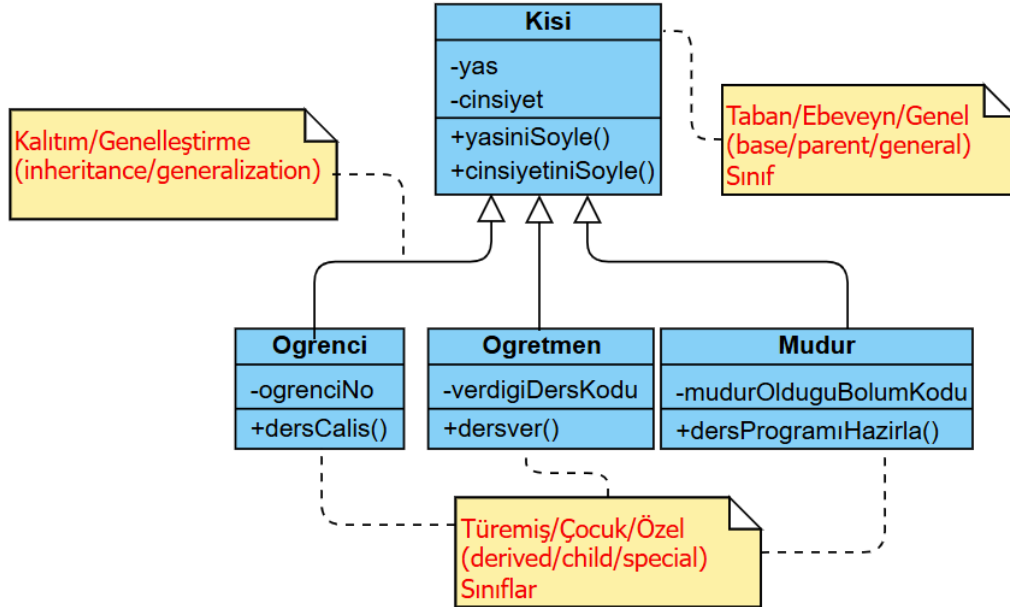
Programlamanın yapıldığı ana fonksiyonda bu üç sınıftan da nesne imal edilmiş ve çeşitli davranışlar göstermesi için kendilerine **ileti gönderilmiştir** (message-passing).

```

// See https://aka.ms/new-console-template for more information
Öğrenci ilhan = new Öğrenci();
Öğretmen mehmet = new Öğretmen();
Müdür abduallah = new Müdür();
ilhan.yaşınıSöyle();
ilhan.cinsiyetiniSöyle();
ilhan.dersÇalış();
mehmet.yaşınıSöyle();
mehmet.cinsiyetiniSöyle();
mehmet.dersVer();
abduallah.yaşınıSöyle();
abduallah.cinsiyetiniSöyle();
abduallah.dersProgramıHazırla();

```

Yeni haliyle sınıf diyagramları aşağıda verilmiştir;



Şekil 24. Kişi Taban Sınıfı Eklenmiş Kalıtım UML Sınıf Diyagramı

Değişim yönetimini kolayca yapma üstünlüğünü veren **kalıtımın** (inheritance) iki önemli özelliği vardır;

1. Paylaşılan bir **ortak kod** (shared-code) vardır.
2. Paylaşılan kodda yapılan **değişiklik anında alt sınıflara yansır** (snap-change).

Örneğimizden gidecek olursak, Kişi sınıfında yapılacak bir özellik veya davranış eklemesi anında bu sınıftan türemiş sınıflarda yapılmış olur.

Korumalı (**protected**) olarak taban sınıftan alınan durum ve davranışlara ait bir örnek aşağıda verilmiştir;

```
// See https://aka.ms/new-console-template for more information
Öğrenci damla = new Öğrenci();
damla.yaşınıSöyle(); // Kişi sınıfından miras alındı
damla.cinsiyetiniSöyle(); // Kişi sınıfından miras alındı
damla.dersÇalış(); // Öğrenci sınıfında tanımlandı
class Kişi
{
    protected uint Yaş { get; set; }
    protected char Cinsiyet { get; set; }
    private int TCKimlikNo { get; set; }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
    public void kimliğiniSöyle()
    {
        Console.WriteLine($"Kimlik Numaram:{TCKimlikNo}");
    }
}
class Öğrenci : Kişi
{
    public int ÖğrenciNo { get; set; }
    public void dersÇalış()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
        Console.WriteLine($"Cinsiyetim:{Cinsiyet}");
        //Console.WriteLine($"Kimlik Numaram: {TCKimlikNo}");
        /* Ebeveyn sınıfta TCKimlikNo private tanımlandığından erişilemez! */
        Console.WriteLine("Ders Çalışıyorum.");
    }
}
```

Türetilmiş bir sınıf, taban sınıfın yapıcıları, **yıkıcıları** (**destructor**) ve kopya yapıcıları dışında tüm taban sınıf yöntemlerini miras alır.

Nesne Yönelimli Programlamada sınıflar, birden fazla taban sınıftan miras alamayacak şekilde tasarlanır. Mantık olarak bir ev, birden fazla mimari plandan hazırlanmayacağından, **çoklu kalıtım** (**multiple inheritance**) sınıf tasarımında kullanılmaz! Bu nedenle saf nesne yönelimli diller olan Java ve .Net dillerinde **ara yüzler** (**interface**) ayrı bir anahtar kelimeyle tanımlandığından doğal olarak birden çok sınıftan miras alma engellenmiştir.

C# dilinde **kullanıcı tanımlı sınıflar** (**classes**) üstü kapalı olarak **object** sınıfından miras alır!

Miras Alınan Üyeleri Gizleme

Miras alınan bir **durum** (**state**), **özellik** (**property**) ve **davranışı** (**behavior**) gizlemek için **new** anahtar kelimesi kullanılır. Miras alınan bir üyeyi **new** ile gizlediğinizde, yeni üye taban sınıftaki yerini alır. Taban sınıfta tanımlanmış bir üye türemiş sınıfta görünür olduğu varsayılır. Çünkü taban sınıfta **private** veya bazı durumlarda **internal** olarak erişimi belirlenmiş üyeler, türemiş sınıfta zaten gizli olurdu. **new** değiştiricisini kullanmadan **public** veya **protected** üyeleri gizleyebilmenize rağmen bir derleyici uyarısı alırsınız. Bir üyeyi açıkça gizlemek için **new** kullanırsanız, bu uyarıyı bastırır.

```
// See https://aka.ms/new-console-template for more information
Taban taban = new Taban(10);
Türemiş türemiş = new Türemiş(30);
taban.Yöntem(); // Taban Sınıfıta 10
türemiş.Yöntem(); /* Türemiş Sınıfıta 0: Çünkü;
                    türemiş sınıfıta tanımlanan alan değişkeni
                    başlangıç değeri sıfırdır ve taban sınıfıdaki
                    x gizlenmiştir. */

public class Taban
{
    public Taban(int x)
    {
        this.x = x;
    }
    public int x;
    public void Yöntem()
    {
        Console.WriteLine($"Taban Sınıfıta {x}");
    }
}

public class Türemiş : Taban
{
    public Türemiş(int x) : base(x) { }
    new public int x;
    new public void Yöntem()
    {
        Console.WriteLine($"Türemiş Sınıfıta {x}");
    }
}
```

Aşırı Yükleme

Aşırı yükleme (**overloading**); aynı davranışın farklı **yöntemlerle** (**method**) yeniden tanımlanmasıdır. Yöntemler, yapıcılar ve işleçlerin bazıları aşırı yüklenebilir;

Yöntemlerin Aşırı Yüklmesi

Kısaca yöntemlerin aşırı yüklemesi, farklı parametrelerle yöntemin yeniden tanımlanmasıdır. Aşırı yüklemede yöntemin kimliği değişmez. Farklı parametrelerle dönüş tipi de farklı olarak tanımlanabilir. Aşağıda bir fonksiyonun aşırı yüklemesine bir örnek verilmiştir.

```
// See https://aka.ms/new-console-template for more information
Hesap h = new Hesap();
Console.WriteLine(h.topla(10, 20)); // 30
Console.WriteLine(h.topla(1, 2.0)); // 3
Console.WriteLine(h.topla(3.0, 1)); // 4
Console.WriteLine(h.topla(3.0, 3.0)); // 6

class Hesap
{
    public int topla(int p1, int p2)
    // "topla" kimlikli yöntem
    {
        return p1 + p2;
    }
    public double topla(double p1, int p2)
    // aşırı yüklenmiş "topla" kimlikli yöntem
    {
        return p1 + p2;
    }
    public double topla(int p1, double p2)
    // aşırı yüklenmiş "topla" kimlikli yöntem
    {
```

```

        return p1 + p2;
    }
    public double toplar(double p1, double p2)
    // aşırı yüklenmiş "topla" kimlikli yöntem
    {
        return p1 + p2;
    }
}

```

Farklı parametrelerle dönüş tipi de farklı olarak tanımlanabilir;

```

public int toplar(int p1, int p2);
public string toplar(int p1, int p2); //Derlenmez! Parametreler farklı olmalı!

```

Yöntemler varsayılan argümanlar ile tanımlandığında aşırı yükleme tanımlanmasında varsayılan argümanlar ile çakışmamasına çok dikkat edilmelidir. Aşağıdaki örnekte varsayılan argüman olan toplar değil diğeri ile işlem yapılır!

```

// See https://aka.ms/new-console-template for more information
Hesap h = new Hesap();
Console.WriteLine(h.toplar(10)); //13

class Hesap
{
    public int toplar(int p1, int p2=1) // "toplar" kimlikli yöntem
    {
        return p1 + p2;
    }
    public int toplar(int p1) // "toplar" kimlikli aşırı yüklenmiş yöntem
    {
        return p1 + 3;
    }
}

```

Yapıcıların Aşırı Yüklenmesi

Yapıcıların görevi kısaca imal edilecek nesnelerin durumlarına (state) ilk değer vermektir. Nesneleri de dışarıdan verilen parametrelerle farklı durumlarda imal etmek için aşırı yüklenmiş yapıcıları kullanabiliriz. Yani farklı parametrelerle yapıcıları yeniden tanımlayabiliriz. Yapıcıların kimliği sınıf kimliğiyle aynıdır. Aşırı yüklemede tanımlanacak yapıcının kimliği de sınıf kimliği ile aynı olur.

Aşağıda aşırı yüklenmiş yapıcı, parametrelili (parameterized constructor),

```

// See https://aka.ms/new-console-template for more information
Kompleks c1 = new Kompleks();
c1.Yaz();// 0 + 0i
Kompleks c2 = new Kompleks(5, 2);
c2.Yaz();// 5 + 2i
Kompleks c3 = new Kompleks(8);
c3.Yaz();// 8 + 0i
Kompleks c4 = c2; // c4 için Kopya Yapıcı Çalışır
c4.Yaz();// 5 + 2i

public class Kompleks
{
    private int gercek;
    private int sanal;
    public Kompleks() // Ön tanımlı (default) yapıcı
    {
        sanal = 0;
        gercek = 0;
    }
    public Kompleks(int r = 0, int i = 0) // Aşırı yüklenmiş parametrelili yapıcı
    {

```

```

        gercek = r;
        sanal = i;
    }
    public Kompleks(int r = 0) // Aşırı yüklenmiş bir başka parametrelili yapıcı
    {
        gercek = r;
        sanal = 0;
    }
    public Kompleks(Kompleks p) //Kopya Yapıcı: Aşırı yüklenmiş bir başka parametrelili yapıcı
    {
        gercek = p.gercek;
        sanal = p.sanal;
    }
    public void Yaz()
    {
        Console.WriteLine($"{gercek} + {sanal}i");
    }
}

```

Yukarıdaki örnekte;

- Aynı sınıftan bir nesneyi bir başka imal edilmiş nesnenin durumlarına sahip olarak imal edebiliriz. Bir nesneyi bir yapıcıya argüman olarak geçirmek ve yeni nesneye argümanın durumlarını vermek gerekir. **Kompleks(Kompleks p)** şeklinde kodlanmış bu yapıcıya **kopya yapıcı** (**copy constructor**) adı verilir.
- Yukarıdaki örnekte **Kompleks(int r = 0, int i = 0)** ve **Kompleks(int r = 0)** yapıcıları **parametrelili yapıcı** (**parameterized constructor**) olarak adlandırılır.

Taban Sınıfın Yapıcısının Çağırılması

Kalıtım (**inheritance**) durumunda ise taban sınıfın yapıcısı çağrılır. Bu duruma aşağıdaki örnek verilebilir;

```

// See https://aka.ms/new-console-template for more information
Öğrenci damla = new Öğrenci(1234, 25, 'K');
damla.yaşınıSöyle(); //Yaşım: 25
damla.cinsiyetiniSöyle(); //Cinsiyetim: KADIN
damla.kimliğiniSöyle(); //Kimlik Numaram: 0
damla.dersÇalış(); //Ders Çalışıyorum.
class Kişi
{
    private uint Yaş { get; set; }
    private char Cinsiyet { get; set; }
    private int TCKimlikNo { get; set; }
    public Kişi(uint yaş, char cinsiyet, int tckimlikno)
    {
        this.Yaş = yaş;
        this.Cinsiyet = cinsiyet;
        this.TCKimlikNo = tckimlikno;
    }
    public void yaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void cinsiyetiniSöyle()
    {
        if (Cinsiyet == 'E' || Cinsiyet == 'e')
            Console.WriteLine("Cinsiyetim: ERKEK");
        else if (Cinsiyet == 'K' || Cinsiyet == 'k')
            Console.WriteLine("Cinsiyetim: KADIN");
        else
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
    }
}

```

```

    public void kimliğiniSöyle()
    {
        Console.WriteLine($"Kimlik Numaram:{TCKimlikNo}");
    }
}
class Öğrenci : Kişi
{
    public Öğrenci(int öğrencino,
        uint yaş = 18,
        char cinsiyet = 'E',
        int tckimlikno = 0) : base(yaş, cinsiyet, tckimlikno)
    {
        this.ÖğrenciNo = öğrencino;
    }
    public int ÖğrenciNo { get; set; }
    public void dersÇalış()
    {
        Console.WriteLine("Ders Çalışıyorum.");
    }
}

```

Yukarıdaki örnekte olduğu gibi **ön tanımlı argümanlar** (default argument) da yapıcılarda kullanılabilir. Yapıcıların aşırı yüklenmesinin çeşitli üstünlükleri vardır;

- Nesne imal etmede esneklik: Aynı sınıftan farklı durumlara sahip nesneler imal edilebilir.
- Gelişmiş kod bakımı ile daha temiz ve okunabilir kod oluşur.
- Nesne imal etme mantığı diğer nesnelerden/programlardan gizlenir.
- Kopya yapıcılar ile nesne klonlama basitleşir.

İşleçlerin Aşırı Yüklenmesi

Hali hazırda tanımlı olan tekli (unary), aritmetik, eşitlik ve karşılaştırma **işleçlere** (operator) de bizim tanımladığımız sınıflardan imal edilen nesneler üzerinde işlem yapmak için aşırı yükleme yapılabilir.

İşleçler	Açıklama
<code>+x, -x, !x, ~x, ++, --, true, false</code>	true ve false işleçleri birlikte aşırı yüklenmelidir
<code>x + y, x - y, x * y, x / y, x % y, x & y, x y, x ^ y, x << y, x >> y, x >>> y</code>	
<code>x == y, x != y, x < y, x > y, x <= y, x >= y</code>	== ile != ve < ile > ve <= ile >= birlikte aşırı yüklenmelidir.

Tablo 20. Aşırı Yüklenebilecek İşleçler

Aşağıda işleç aşırı yüklemesini göstermek amacıyla yeni bir **Kompleks** sınıfı tanımlanmış ve üzerinde toplama işleci aşırı yüklenmiştir;

```

// See https://aka.ms/new-console-template for more information
Kompleks c1 = new Kompleks(3, 7);
c1.Yaz();// 3 + 7i
Kompleks c2 = new Kompleks(5, 2);
c2.Yaz();// 5 + 2i
Kompleks c3 = c1 + c2;
c3.Yaz();// 8 + 9i

public class Kompleks
{
    private int gercek;
    private int sanal;
    public Kompleks(int r = 0, int i = 0)
    {
        gercek = r;
    }
}

```

```

        sanal = i;
    }
    public static Kompleks operator +(Kompleks c1, Kompleks c2)
    {
        Kompleks temp = new Kompleks();
        temp.gercek = c1.gercek + c2.gercek;
        temp.sanal = c1.sanal + c2.sanal;
        return temp;
    }
    public void Yaz()
    {
        Console.WriteLine($"{gercek} + {sanal}i");
    }
}

```

this Göstericisi

İmlal edilen her nesne, **this** adı verilen önemli bir gösterici aracılığıyla kendi adresine erişebilir. **this göstericisi** (**this pointer**) tüm üye yöntemler için örtük bir parametredir. Aşağıda karmaşık sayıları örneği verilmiştir;

```

// See https://aka.ms/new-console-template for more information
Kompleks c1 = new Kompleks();
c1.Yaz();// 0 + 0i
Kompleks c2 = new Kompleks(5, 2);
c2.Yaz();// 5 + 2i
Kompleks c3 = new Kompleks(8);
c3.Yaz();// 8 + 0i
Kompleks c4 = c2; // c4 için Kopya Yapıcı Çalışır
c4.Yaz();// 5 + 2i

public class Kompleks
{
    private int gercek;
    private int sanal;
    public Kompleks()
    // Ön tanımlı (default) yapıcı
    {
        sanal = 0;
        gercek = 0;
    }
    public Kompleks(int r = 0, int i = 0) // Aşırı yüklenmiş parametrelili yapıcı
    {
        this.gercek = r;
        this.sanal = i;
    }
    public Kompleks(int r = 0) // Aşırı yüklenmiş bir başka parametrelili yapıcı
    {
        this.gercek = r;
        this.sanal = 0;
    }
    public Kompleks(Kompleks p) //Kopya Yapıcı:Aşırı yüklenmiş bir başka parametrelili yapıcı
    {
        this.gercek = p.gercek;
        this.sanal = p.sanal;
    }
    public void Yaz()
    {
        Console.WriteLine($"{gercek} + {sanal}i");
    }
}

```

Statik Üyeler ve Statik Sınıf

Programda çalışmaya başladığı anda bazı değişkenler ve yöntemlere henüz bellekte tahsis edilmemiş veya istenilen nesneler henüz imal edilmemiştir. Program daha çalışmadan bu tip değişken, yöntem ve nesnelerin bellekte hazır olması istenir. Uygulamamızın daha belleğe yüklenirken ve yaşamı boyunca bellekte bulunacak değişken, yöntem ve nesneler **static** olarak tanımlanır.

Programlarda sıkça kodladığımız **static void Main(string[] args)** fonksiyonu da bu nedenle **static** olarak tanımlanır. **Main** yöntemi dışında da bazı nesnelerin imal edilmesi veya değişken ve yöntemlerin belleğe program belleğe yüklenirken yüklenmesi gerekir. İşte bu değişkenler ve yöntemler de statik olarak tanımlanır. Statik sınıflar;

- Doğası gereği nesne yapıcısı barındırmazlar!
- Bu sınıflar türetilemez!
- Sınıfın tüm üyeleri statik olmak zorundadır!

Statik olan değişken veya yöntemler yani **statik üyeler** (**static member**), **örnek üyeleri** (**instance member**) gibi nesne ile imal edilip nesne ile ölmediklerinden, nesnelere ait değil sınıfa ait olarak nitelendirilir. Yani program bellekte kaldığı sürece statik sınıf ve üyeleri hayattadır.

```
// See https://aka.ms/new-console-template for more information
string bağlantı = Yardımcı1.GetConnection();
Yardımcı1.GenelSayaç++;
ulong f = Yardımcı1.Faktöriyel(5);

static class Yardımcı1 /* Statik sınıf */
{
    public static ulong GenelSayaç = 0;
    public static string GetConnection()
    {
        return "Data Source=(LocalDB)\\MSSQLLocalDB;" +
            "Integrated Security=True;Connect Timeout=30;Encrypt=True";
    }
    public static ulong Faktöriyel(uint n)
    {
        if (n > 1) return n * Faktöriyel(n - 1);
        else return 1;
    }
}
```

Sabit Alanlar

Alanların (**field**) nesnelere ilişkin **durumları** (**state**) yani verileri tuttuğu anlatılmıştı. Sabitler, derleme zamanında bilinen ve programın ömrü boyunca değişmeyen değişmez değerlerdir. Sabit alan ve yerel değişkenler **const** sıfatıyla tanımlanır.

Metinler (**string**) dışındaki referans tipleri sabitlere yalnızca **null** olarak ilk değer verilebilir! **Sınıflar** (**class**), **yapılar** (**struct**) ve **diziler** (**array**) dahil olmak üzere kullanıcı tanımlı tipler **const** tanımlanamaz! Çalışma zamanında yapıcı ile bir kez imal edilen ve bundan sonra değiştirilemeyen bir nesne, yapı veya dizi oluşturmak için **readonly** sıfatı kullanılır.

```
// See https://aka.ms/new-console-template for more information
const float PI = 3.1415f;
Takvim t = new Takvim();
t.Yaz();
/*
Ay Sayısı:12
Saat Sayısı:8786 */
Console.WriteLine(Takvim.TakvimAdı);
//const alanlar static olarak algılanır ve Sınıf İsmi ile erişilir.
Console.WriteLine(t.günler[0]); //Pazartesi
//readonly alanlar ise static olarak algılanmaz ve nesne üzerinden erişilir.
```



```
class Takvim
{
    public const int AySayısı = 12;
    public const int GünSayısı = 365;
    public const int HaftaSayısı = 52;
    public const int SaatSayısı = GünSayısı * 24 + 6;
    public const string TakvimAdı = "Takvim";
    public readonly string[] günler = { "Pazartesi", "Salı", "Çarşamba",
                                         "Perşembe", "Cuma", "Cumartesi",
                                         "Pazar" };

    public void Yaz()
    {
        Console.WriteLine($"Ay Sayısı:{AySayısı}");
        Console.WriteLine($"Saat Sayısı:{SaatSayısı}");
    }
}
```

readonly Kelimesi

readonly anahtar kelimesi aşağıdaki bağlamda kullanılabilen bir sıfattır.

1. Salt okunur bir **alan** (field) tanımlanmasında; alana atama yalnızca bildirimde veya aynı sınıftaki bir yapıcıda gerçekleşir.

```
class Takvim
{
    public readonly int AySayısı=12;
    public readonly int HaftaSayısı;
    public Takvim()
    {
        HaftaSayısı = 365 / 7;
        /* Sınıf içinde bile olsa Başka yerde bu alanlar değiştirilemez. */
    }
}
```

2. Salt okunur bir **yapı** (struct) tipi tanımında; yapı tipinin değiştirilemez olduğunu belirtir.

```
// See https://aka.ms/new-console-template for more information
var p1 = new Koordinat(0, 0);
Console.WriteLine(p1); // (0, 0)
var p2 = p1 with { X = 3 };
Console.WriteLine(p2); // (3, 0)
var p3 = p1 with { X = 1, Y = 4 };
Console.WriteLine(p3); // (1, 4)

public readonly struct Koordinat
{
    public Koordinat(double x, double y)
    {
        X = x;
        Y = y;
    }
    public double X { get; init; }
    public double Y { get; init; }
    public override string ToString()
    {
        return $"({X}, {Y})";
    }
}
```

3. **Örnekleme** (instantiation) yapılarak yani nesne imal edilerek salt okunur bir üye nesne tanımlanıyorsa; bu örnek üyesinin durumunu değiştirmediğini belirtir.

```
// See https://aka.ms/new-console-template for more information
Kişi ali = new Kişi("Ali", "YILMAZ");
ali.Yaz();

class Kitap
{
    public string Adı { get; set; }
    public Kitap(string adı)
    {
        this.Adı = adı;
    }
}

class Kişi
{
    public string Adı { get; set; }
    public string Soyadı { get; set; }
    public readonly Kitap matematikKitabı = new Kitap("Calculus I");
    public readonly Kitap kitap;
    public Kişi(string adı, string soyadı)
    {
        this.Adı = adı;
        this.Soyadı = soyadı;
        kitap = new Kitap("C# Programlama");
    }
    public void Yaz()
    {
        Console.WriteLine($"Adı:{Adı}, Soyadı:{Soyadı}");
        Console.WriteLine($"Matematik: {matematikKitabı.Adı}");
        Console.WriteLine($"Kitap: {kitap.Adı}");
    }
}
```

4. Bir `ref readonly` metodu dönüşünde, `readonly` kelimesi metodun bir referans döndürdüğünü ve bu referansa yazmalara izin verilmediğini belirtir.

Değiştirilemez Nesneler

Nesne imal edildikten sonra durumları değiştirilebilen nesneler **değiştirilebilir** (*mutable*) nesnelerdir. Bir nesnenin durumlarının değiştirilebilir olması, dahili verilerinin değiştirilebileceği ve bu da nesnenin bütünlüğünün korunmasında olası yan etkilere ve zorluklara yol açabileceği anlamına gelir. Değiştirilebilir sınıfların yaygın örnekleri arasında, elemanlarının eklenebildiği, kaldırılabilirdiği veya değiştirilebildiği birçok koleksiyon türü bulunur.

Değiştirilebilir nesneler yerine değiştirilemez nesneler kullanmak istediğimiz durumlar vardır. Değiştirilemez nesneler **iş parçacığı güvenli** (*thread safe*), ancak diğer yandan birçok durumda performans sorunlarına neden olabilirler. Her geliştiricinin günlük olarak kullandığı en popüler değiştirilemez nesne **dizgilerdir** (*string*). Metinlerle yapılan her işlem sonrasında yeni bir metin ortaya çıkar.

Değiştirilemez bir nesne elde etmek için sınıflardaki alanlar `readonly` tanımlanır. Bu sınıfın yapıcısında ilke değerler verilir ve nesne imalatı olduktan sonra bu alanlar değiştirilemez.

```
// See https://aka.ms/new-console-template for more information
Takvim t = new Takvim();
t.Yaz();
Console.WriteLine($"Saat Sayısı:{t.SaatSayısı}");
Console.WriteLine(t.TakvimAdı.ToUpper());
/* TakvimAdı immutable string sınıfındandır. Burada değeri "Takvim" aynen kalır ve
   Büyük harfe çevrilmiş olan yeni bir string imal edilir. */

class Takvim
{

```

```

public readonly int AySayısı;
public readonly int GünSayısı;
public readonly int HaftaSayısı;
public readonly int SaatSayısı;
public readonly string TakvimAdı;
public Takvim()
{
    AySayısı = 12;
    GünSayısı = 365;
    HaftaSayısı = GünSayısı / 7;
    SaatSayısı = GünSayısı * 24 + 6;
    TakvimAdı = "Takvim";
}
public void Yaz()
{
    Console.WriteLine($"Hafta Sayısı:{HaftaSayısı}");
    Console.WriteLine($"Gün Sayısı:{GünSayısı}");
}
}

```

Geçersiz Kılma

Yöntemi **geçersiz kılma** (**overriding**), nesne yönelimli programlamanın, taban sınıfta önceden tanımlanmış bir yöntemi türemiş bir sınıfta yeniden tanımlamasına olanak tanıyan bir kavramdır. Yani taban sınıfın davranışının türemiş sınıfta değiştirilmesidir. Geçersiz kılınacak yöntem, taban sınıfta ya **sanal** (**virtual**) ya da **soyut** (**abstract**) olarak tanımlanmalıdır.

```

class Taban-Sınıf-Kimliği {
    // geçersiz kılınacak yöntem:
    erişim-belirleyici virtual|abstract geri-dönüş-veri-tipi yöntem-kimliği() {
    }
};

class Türemiş-Sınıf-Kimliği: Taban-Sınıf-Kimliği {
    // geçersiz kılan yöntem:
    erişim-belirleyici override geri-dönüş-veri-tipi yöntem-kimliği() {
    }
};

```

Her iki sınıf ta da yöntemin kimliği aynıdır. Taban sınıftan imal edilen nesne taban sınıfın yöntemini, türemiş sınıftan imal edilen nesne ise türemiş sınıfın yöntemini icra eder.

```

// See https://aka.ms/new-console-template for more information
Baba baba = new Baba();
Çocuk çocuk = new Çocuk();
baba.yap(); //Baba bu şekilde yapar ...
çocuk.yap(); //Çocuk başka şekilde yapar..

class Baba
{
    public virtual void yap()
    {
        Console.WriteLine("Baba bu şekilde yapar ...");
    }
}
class Çocuk : Baba
{
    public override void yap()
    {
        Console.WriteLine("Çocuk başka şekilde yapar...");
    }
}

```

Ara Yüzler veya Roller

Ara yüzler (interface) nesnelerin sahip oldukları rollerdir (role). Ancak saf nesne yönelimli dillerde ara yüzler interface saklı kelimesiyle tanımlanırlar.

Örnek olarak bir öğretmen; Evde baba rolünde olup babanın sahip olduğu çocuklara bakma, yemek yapma gibi davranışları gösterir. İşte öğretmen rolündedir ve öğrenmenin sahip olduğu; öğretme, sınav/değerlendirme yapma, karne hazırlama, ders notu hazırlama gibi davranışları vardır. Bu öğretmen müzisyen rolünde ise gitar çalma, akort yapma gibi davranışları da vardır. İşte bunların hepsi ayrı bir roldür ve her bir rolde o role ilişkin davranışları sergiler.

Ara yüzler de referans tiplerdir. Bir sınıfa, bir veya birden fazla ara yüz uygulanabilir (interface implementation). Buna ara yüz gerçekleştirme (interface realization) adı da verilir. Ara yüzler içinde yöntemlere ilişkin desenler (signature) yazılır. Yani sadece yöntem başlığı yazılır. Gövdesi yazılmaz. Ayrıca yöntemler için erişim belirleyici kullanılmaz.

```
// See https://aka.ms/new-console-template for more information
Kişi ilhan = new Kişi("İlhan", "ÖZKAN");
//imal edilen nesne tüm arayüzdeki davranışları gösterebilir:
ilhan.AdınıSöyle(); // İlhan ÖZKAN
ilhan.Öğret(); // Öğretiyorum...
ilhan.GitarÇal(); // Gitar Çalıyorum...
ilhan.HanımaYardımEt(); // Hanıma Yardım Ediyorum...

//imal edilen nesne sadece bir roldeki davranışları da gösterebilir:
IBaba baba = ilhan;
baba.ÇocuğaBak(); // Çocuğa Bakıyorum...
baba.HanımaYardımEt(); // Hanıma Yardım Ediyorum...
İMüzisyen müzisyen = ilhan;
müzisyen.GitarÇal(); // Gitar Çalıyorum...
müzisyen.PiyanoÇal(); // Piyano Çalıyorum...
İÖğretmen öğretmen = ilhan;
öğretmen.DersNotuHazırla(); // Ders notu hazırlıyorum...
öğretmen.Öğret(); // Öğretiyorum...

interface IBaba //Baba Rolündeki Davranışlar:
{
    void ÇocuğaBak(); // yöntemin gövdesi olmaz!
    void HanımaYardımEt();
}
interface İMüzisyen //Müzisyen Rolündeki Davranışlar:
{
    void GitarÇal();
    void PiyanoÇal();
}
interface İÖğretmen//Öğretmen Davranışları:
{
    void Öğret();
    void DersNotuHazırla();
}
class Kişi : IBaba, İMüzisyen, İÖğretmen
{
    /*
     * Arayüz Gerçekleştirme/Uygulama(interface realization/implementation):
     * Gerçekleştirilen IBaba, İMüzisyen ve İÖğretmen arayüzlerinde bulunan yöntemler
     * Kişi sınıfında hepsi gövdeleriyle tanımlanmalıdır!
     */
    public string Adı { get; set; }
    public string Soyadı { get; set; }
    public Kişi(string adı, string soyadı)
    {
        this.Adı = adı;
    }
}
```

```

        this.Soyadı = soyadı;
    }
    public void AdınıSöyle()
    {
        Console.WriteLine($"{Adı} {Soyadı}");
    }
    public void ÇocuğaBak()
    {
        Console.WriteLine("Çocuğa Bakıyorum..");
    }
    public void HanımaYardımEt()
    {
        Console.WriteLine("Hanıma Yardım Ediyorum..");
    }
    public void PiyanoÇal()
    {
        Console.WriteLine("Piyano Çalıyorum..");
    }
    public void GitarÇal()
    {
        Console.WriteLine("Gitar Çalıyorum..");
    }
    public void Öğret()
    {
        Console.WriteLine("Öğretiyorum..");
    }
    public void DersNotuHazırla()
    {
        Console.WriteLine("Desr Notu Hazırlıyorum..");
    }
}

```

Nesne Yıkıcı

Sonlandırıcılar (**finalizer**) olarak adlandırılan **yıkıcı** (**destructor**), bir nesne yok edileceği zaman otomatik olarak çağrılan yapıcı gibi bir yöntemdir. Bir sınıf **örneği** (**instance**) **çöp toplayıcı** (**garbage collector-GC**) tarafından toplanırken gerekli olan son temizliği gerçekleştirmek için kullanılır. Bir sınıf içinde yalnızca bir yıkıcı tanımlanabilir. Yıkıcı, bir nesne çöp toplayıcı tarafından bellekten silinmeden hemen önce çalışan özel bir yöntemdir. Aşağıdaki gibi sınıf kimliği önünde yer alan **dalga** (**tilda**) karakteri ile tanımlanır. Parametre alamaz ve **erişim belirleyici** (**access modifier**) kullanılamaz. Manuel olarak çağrılmaz, sadece çöp toplayıcı tarafından otomatik çağrılır ve **static** olamaz.

```

~sınıf-kimliği() {
    // ...
}

```

Nesnelerin üstü kapalı olarak **object** sınıfından miras aldığı söylenmişti. Yıkıcı, nesnenin taban sınıfında **Finalize** yöntemini dolaylı olarak çağırır. Bu nedenle, bir sonlandırıcıya yapılan çağrı dolaylı olarak aşağıdaki koda çevrilir:

```

protected override void Finalize()
{
    try
    {
        // Cleanup statements...
    }
    finally
    {
        base.Finalize();
    }
}

```

Mutlaka yıkıcı bir yöntem çağırmak gerekiyorsa kaynak yönetimi için en iyi yöntem **IDisposable** ara yüzü gerçekleştirmek ve bu ara yüzün sunduğu **Dispose()** metodunu kullanmaktır.

```
class MyClass : IDisposable
{
    private bool disposed = false;

    // Dispose metodu
    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this); // Sonlandırıcının tekrar çalışmasını engeller
    }

    protected virtual void Dispose(bool disposing)
    {
        if (!disposed)
        {
            if (disposing)
            {
                // Yönetilen kaynakları temizle
            }
            // Yönetilmeyen kaynakları temizle
            disposed = true;
        }
    }

    // Destructor (finalizer): Sonlandırıcı
    ~MyClass()
    {
        Dispose(false);
    }
}
```

Bu yapı, hem **Dispose()** çağrıldığında, hem de çağrılmazsa ve çöp toplayıcı devreye girerse temizliği garanti altına alır. **GC.SuppressFinalize(this)** ifadesi, eğer **Dispose()** çağrıldıysa yıkıcının çalışmasını engeller, çünkü temizlik zaten yapılmıştır.

Çok Biçimlilik

Çok biçimlilik (**polymorphism**), birden çok nesnenin olduğu bir ortamda, verilen **aynı iletiye** (**message**) karşı, nesnelerin **farklı davranış** (**behavior**) göstermeleridir (**same message-different behavior**).

Çok biçimliliğin uygulanabilmesi (**implementation**) için üç şart vardır;

- **Kalıtım** (**inheritance**): Ortak davranışın tanımlandığı taban sınıf ve bu davranışın değiştiği türeyen sınıflar.
- Nesnelere aynı iletiyi verebilmek için ortak davranışı tanımlayan **Sanal Yöntem** (**virtual method**).
- **Geçersiz Kılan Yöntem** (**override method**): Devralınan davranışı kendine uyarlayan ve devralınan yöntemi geçersiz kılan yeni bir yöntem.

```
// See https://aka.ms/new-console-template for more information

SoyutKişi[] kişiler = new Kişi[3];
kişiler[0] = new Müdür("İlhan");
kişiler[1] = new Öğrenci("Mehmet");
kişiler[2] = new Öğretmen("Abdullah");
foreach (Kişi kişi in kişiler)
    kişi.raporYaz();
/*
Müdür İlhan Rapor Yazıyor..
Öğrenci Mehmet Rapor Yazıyor..
Öğretmen Abdullah Rapor Yazıyor..
*/
```

ÇOKBİÇİMLİLİK=AYNI MESAJ KARŞISINDA FARKLI DAVRANIŞ

(PolyMorphism: Same Message -> Different Behaviour):

Bütün nesnelere aynı ileti gönderiliyor ama
hepsi farklı davranış gösteriyor.

```

*/

abstract class SoyutKişi //Soyut Kisi Sınıfı
{
    public abstract void raporYaz();
    public string Adı { get; set; }
    public SoyutKişi(string adı)
    {
        Adı = adı;
    }
}

class Kişi : SoyutKişi
{
    public Kişi(string adı) : base(adı) { }
    public override void raporYaz()
    {
        Console.WriteLine($"Kişi {Adı} Rapor Yazıyor..");
    }
}

class Öğrenci : Kişi
{
    public Öğrenci(string adı) : base(adı) { }
    public override void raporYaz()
    {
        Console.WriteLine($"Öğrenci {Adı} Rapor Yazıyor..");
    }
}

class Öğretmen : Kişi
{
    public Öğretmen(string adı) : base(adı) { }
    public override void raporYaz()
    {
        Console.WriteLine($"Öğretmen {Adı} Rapor Yazıyor..");
    }
}

class Müdür : Kişi
{
    public Müdür(string adı) : base(adı) { }
    public override void raporYaz()
    {
        Console.WriteLine($"Müdür {Adı} Rapor Yazıyor..");
    }
}

```

Çok biçimlilik bize **çalıştırma anında** (run time) üstünlük sağlar. Bu nedenle nesneler de **çalıştırma anında** imal edilmiştir. Program çalıştırıldığında her **kişi** nesnesine aynı ileti verilmiş ama bu nesneler farklı davranış göstermiştir.

Yazılımcı için çok biçimlilik, farklı tipte nesnelerin aynı isimli **yöntemleri** (method) çağrıldığında her bir nesnenin kendine özgü davranışı gerçekleştirme yeteneği olup **çalıştırma anında** üstünlük sağlar. Yani yazılımcı, nesnenin tipine bakmaksızın, nesnelerden aynı davranışı göstermesini ister. Burada gösterilecek davranışın, taban sınıfın davranışı mı olduğu veya türeyen sınıfın davranışı mı olduğuna **çalıştırma anında** (run time) karar verilir. İşte bu karar verme işlemine **geç bağlama** (late binding) veya **dinamik bağlama** (dynamic binding) adı verilir.

Esnekliğin istenmediği bazı durumlarda ise karar verme işlemi derleme anında yapılır. Buna ise **statik bağlama** (static binding) adı verilir. İkisi arasındaki seçim programcı tarafından yapılır.

Yazılım yapılacak sistemin başlangıç koşullarına bağlı tasarım kararlarının tümü bilinemez. Ayrıca sistemin genişlemesi için gerekli tüm kodlamanın bitirilmesi beklenemez. İşte bunu gerçekleştiren dinamik bağlama, yazılım mimarisinin daha esnek (mevcut bileşenin sistemde yeniden yapılandırmasının kolayca yapılması) ve genişleyebilir (yeni bileşenlerin kolayca sisteme eklenmesi) olmasını sağlar.

Sınıf Çeşitleri

Soyut Sınıf

Soyut sınıfların (**abstract class**) bir **örneği** (**instance**) olamaz. Yani bu sınıftan nesne imal edilemez. Bu sınıfın en az bir **soyut yöntemi** (**abstract method**) olur. Soyut sınıflar hangi davranışın gösterileceğini belirler ama bu davranışların nasıl gösterileceğini anlatmaz. Bu nedenle soyut yöntem için gövde tanımlanmaz.

```
public abstract class Soyut
/* soyut (abstract) sınıf hangi davranışlara ve özelliklere sahip olacağını söyler. */
{
    public abstract void Yöntem();
    //Soyut Yöntemler için gövde tanımlanmaz!
}
```

Somut Sınıf

Somut sınıflardan (**concrete class**) nesne imal edilebilir yani **örneği** (**instance**) olabilir. Bu sınıflarda davranış ve durumlar eksiksiz tanımlıdır. Bir sınıf tanımlarken soyut bir sınıfa genel davranış ve durumları toplamak ve ondan miras alarak somut sınıfları tanımlamak her zaman iyi bir seçimdir.

```
public class Somut : Soyut
/* Somut (concrete) sınıf, her zaman soyut sınıftan miras almak zorunda değildir. */
{
    public override void Yöntem()
    /* soyut yöntem, soyut olan ve miras alınan Yöntem" metodunu * geçersiz kılar (override)
       Ve mutlaka ete kemiğe bürünen bir davranışı tanımlamak zorundadır. */
    {
        /* soyut olarak gösterilecek davranış burada kodlanır. */
    }
}
```

Taban Sınıf

Taban sınıf (**base class**), miras alınan sınıf veya genel özellik ve davranışların toplandığı yani genelleştirmenin yapıldığı genel sınıf (**general class**) veya ebeveyn (**parent**) sınıftır.

```
public class TabanSınıf
{
    public virtual void Yöntem()
    {
    }
}
```

Türetilmiş Sınıf

Türetilmiş sınıf (**derived class**), miras alan sınıf veya davranış ve durumlar hakkında uzmanlaşmış yani uzman (**special class**) veya çocuk (**child class**) sınıftır. Türetilmiş ya da miras almış sınıf.

```
public class TüretilenSınıf : TabanSınıf
{
    public override void Yöntem()
    {
    }
}
```


Kök Sınıf

Kök sınıf (**root class**), babası olmayan ya da yetim sınıf olup aynı zamanda baba sınıftır.

Yaprak Sınıf

Yaprak sınıf (**leaf class**), kendisinden henüz miras alan bir sınıf olmayan bir sınıftır. Kısaca türememiş sınıftır.

Kısır Sınıf

Kısır sınıf (**sealed class**) kendisinden türeyen bir sınıf tanımlayamayan bir sınıftır. Kısır yöntem (**sealed method**) ise türeyen sınıfta **geçersiz kılınamayan** (**override**) yöntemidir.

```
public class Sınıf
{
    public virtual sealed void Yöntem()
    { // Kısır bir Yöntem tanımlanıyor
    }
}
public class Çocuk : Sınıf
{
    public override void Yöntem() /* Derleme Sırasında Hata:
    Kısır yöntem geçersiz kılınamaz (override edilemez) */
    {
    }
}
public sealed class Kısır //Kısır (sealed) sınıf: miras bırakmaz.
{
    public Kısır() { }
    public void Yöntem()
    {
    }
}
public class Türemiş : Kısır
/* Derleme Sırasında Hata: Kısır sınıftan miras alınamaz. */
{
}
```

Tekil Sınıf

Tekil sınıftan (**singleton class**) yalnızca bir nesne imal edilebilir. Yani yalnızca bir örneği vardır. Bunun olabilmesi için **mahrem** (**private**) yapıcısı olması gerekir. *Tekil Nesne* başlığında incelenecektir.

Yardımcı Sınıf

Yardımcı sınıf (**utility class**), üyeleri statik olan sınıflardır. Ayrıca evrensel **yapılandırma ayarlarını** (**configuration settings**) veya **değişmezleri** (**literal**) depolamak için kullanılabilir. Bir kaynak havuzunu (önbellek, veri tabanı bağlantı havuzu, vb.) yönetmek ve örnekler arasında paylaşılan bir günlük sistemi uygulamak için yararlıdır. Bunların dışında, **izleme yöntemi** (**trace method**) çağrılar için de kullanılabilir. Yardımcı sınıflar **statik** (**static**) sınıflardır ve bu sınıflardan nesne imal edilemez. Yani Yarıcıları bulunmaz.

Kısmi Sınıf

Kısmi sınıf (**partial class**), bir sınıfın farklı yerlerde parçalar halinde tanımlanmasıdır. Kısmi sınıf bir dosya içinde tanımlanabileceği gibi kod tabanında herhangi bir yerde tanımlanabilir.

```
// See https://aka.ms/new-console-template for more information
Kişi kişi = new Kişi();
kişi.AdınıSöyle();
kişi.SoyadınıSöyle();
kişi.AdınıTamSöyle();

partial class Kişi
```

```
{
    string Adı { get; set; }
    public void AdınıSöyle()
    {
        Console.WriteLine($"Adım:{Adı}");
    }
}

partial class Kişi
{
    public Kişi()
    {
        Adı = "İlhan";
        Soyadı = "Özkan";
    }
    string Soyadı { get; set; }
    public void SoyadınıSöyle()
    {
        Console.WriteLine($"Soyadı:{Soyadı}");
    }
    public void AdınıTamSöyle()
    {
        Console.WriteLine($"Adı ve Soyadı:{Adı} {Soyadı}");
    }
}
```

Nesnelerle İlgili İşleçler

Tip Kontrol İşleci

is işleci bir nesnenin bir sınıfa ait olup olmadığını kontrol eder.

```
object obj = "Hello, World!";
if (obj is string)
{
    Console.WriteLine("obj nesnesinin veri tipi stringdir."); // Bunu konsola yazar
}
```

Güvenli Tip Dönüşüm İşleci

as işleci, istisna bir duruma düşmeden güvenli tip dönüşümünü (safe type casting) gerçekleştirir.

```
object obj = "Merhaba";
string str = obj as string;
if (str != null)
{
    Console.WriteLine("Tip dönüşümü başarılı: " + str); // Bunu konsola yazar
}
```

Eşitlik İşleci

C# dilinde iki farklı eşitlik türü vardır: referans eşitliği ve değer eşitliği. Değer eşitliği, eşitliğin yaygın olarak anlaşılan anlamıdır: iki nesnenin aynı değerleri içermesi anlamına gelir.

```
object a = new object();
object b = a;
System.Object.ReferenceEquals(a, b); //true

Console.WriteLine((2 + 2) == 4); // değer eşitliği: true
object s = 1;
object t = 1;
Console.WriteLine(s == t); //referans eşitliği, farklı nesneler: false

string a = "hello"; //const
```

```
string b = String.Copy(a); //instance
string c = "hello"; //const
Console.WriteLine(a == b); // metin değerlerini karşılaştırma: true

Console.WriteLine((object)a == (object)b); //metin referanslarını karşılaştırma: false
Console.WriteLine((object)a == (object)c); //metinlerin ikisi de sabit: true
```

TEMSİLCİLER VE OLAYLAR

Temsilciler

Temsilci (**delegate**), yöntemleri dolaylı olarak çağırmanın bir yoludur. Saf nesne yönelimli programlama dillerinde (Java ve C#), yarattığı karmaşadan ötürü **gösterici** (**pointer**) kullanımı tercih edilmez. Bu sebeple **işlev göstericisi** (**function pointer**) yerine, ondan daha yetenekli olan temsilciler kullanılır. Temsilciler, birden fazla **yöntemi** (**method**) referans gösterebilirler.

```
// See https://aka.ms/new-console-template for more information
public delegate void Yap(string iş); // Temsilci tanımı
/*
    Temsilci tanımı:
    public delegate void Yap(string iş);
    - Yap: Temsilci adı
    - string: Temsilciye gönderilecek parametre türü
    - void: Temsilcinin dönüş türü
    Temsilci, bir veya daha fazla yöntemi temsil eden bir türdür.
*/
public class Program
{
    public static void Main(string[] args)
    {
        YöntemliSınıf nesne = new YöntemliSınıf();

        Yap işyap = nesne.MusluğuAç; // tek bir yöntemi dolaylı çağırma

        // İstenirse birden fazla yöntemi temsil edebilir:
        işyap += nesne.MusluğuKapat;
        işyap += YöntemliSınıf.BulaşıkYık;

        işyap("İlhan Özkan"); // "Yap" temsilcisi çağrıldı (invoke the delegate)
        /*
            İlhan Özkan Musluğu Açtı.
            İlhan Özkan Musluğu Kapattı.
            İlhan Özkan Bulaşık Yıkadı.
        */
    }
}

public class YöntemliSınıf
{
    public void MusluğuAç(string açmessage) // Temsilci tanımındaki imzaya uygun yöntem
    {
        Console.WriteLine("{0} Musluğu Açtı.", açmessage);
    }
    public void MusluğuKapat(string kapatmessage) // Temsilci tanımındaki imzaya uygun yöntem
    {
        Console.WriteLine("{0} Musluğu Kapattı.", kapatmessage);
    }
    public static void BulaşıkYık(string yıkamessage)
    // Temsilci tanımındaki imzaya uygun yöntem ama static tanımlanmış.
    {
        Console.WriteLine("{0} Bulaşık Yıkadı.", yıkamessage);
    }
}
```

Lamda İfadeleri

Bir **lamda ifadesi** (**lambda expression**), yöntemleri dolaylı oluşturmak için özlü bir yol sağlar. ‘Lamda’ adı, *Alonzo Church* tarafından 1930'larda mantık ve hesaplanabilme hakkındaki soruları araştırmak

için icat edilen matematiksel bir form olan lamda hesabından gelir. Lamda hesabı, fonksiyonel bir programlama dili olan LISP'in temelini oluşturmuştur.

C# dilindeki Lambda ifadeleri **anonim yöntemler** (**anonymous method**) gibi kullanılır. Lamda ifadelerinde girdi değerlerin veri tipini belirtmenize gerek yoktur. Bu da kullanımı daha esnek hale getirir. Lamda ifadesi iki bölüme ayrılır, sol taraf girdidir, sonrasında lamda işleci (**=>**) bulunur. Sağ taraf ise **ifadedir** (**expression**).

Talimatların (**statement**), yapılması gerekeni kısa ve net olarak belirten **mantıksal cümlelerdir** (**logical sequences**) olduğu önceden anlatılmıştı. Talimatlar **anahtar kelimeler** (**keyword**) içerirler ve noktalı virgül karakteri ile biter. Bazı talimatlar, **ifade** (**expression**) olarak adlandırılır. Yalnızca sabit, değişken ve **işleç** (**operator**) içeren talimatlara ifade adı verilir. Bu tanımlardan hareketle lamda ifadesi ve lamda talimatı olmak üzere iki tür lamda vardır.

Lamda İfadeleri

Lamda ifadesi (**lambda expression**) aşağıdaki gibi yazılır;

```
girdi => ifade;
```

Aşağıda buna ilişkin bir örnek verilmiştir;

```
delegate int ikiIntParametreliTmsilci(int input1, int input2);
ikiIntParametreliTmsilci ikiTamsayiyiTopla = (x,y) => x + y;
ikiIntParametreliTmsilci ikiTamsayiyiÇarp = (x,y) => x * y;
```

delegate kelimesini kullanmadan geri değer döndürmeyen **Action** ve geri değer döndüren **Func** kelimeleri ile lamda ifadeleri tanımlayabiliriz. Aşağıda **delegate** olmadan kullanılan lamda ifadelerine yer verilmiştir;

```
// See https://aka.ms/new-console-template for more information
Action line = () => Console.WriteLine("-----"); //Parametresiz
line();
//Action delegate'leri, geriye değer döndürmezler!

Func<double, double> cube = x => x * x * x; //Tek Parametrelili
Console.WriteLine(cube(3));
//Func delegate'leri, geriye değer döndürürler!
/* Yukarıdaki kodun açılımı
double cube(double x) {
    return x * x * x;
}
*/

Func<int, double, double> kat = (x,y) => x * y; //İki Parametrelili
Console.WriteLine(kat(3,6.0));
/* Yukarıdaki kodun açılımı
double kat(int x, double y) {
    return x * y;
}
*/

//Veri tipi içeren parametrelili
Func<int, string, bool> MetinKarakterSayısıVerilendenBüyükMü = (int x, string s)
    => s.Length > x;
Console.WriteLine(MetinKarakterSayısıVerilendenBüyükMü(3, "Merhaba"));

//Parametreler Göz Ardı Edildi
Func<int, int, int> constant = (_, _) => 61;
Console.WriteLine(constant(3, 5)); // Output: 61
```

Lamda Talimatları

Lamda talimatı (**lambda tatement**) aşağıdaki gibi tanımlanır;

```
girdi => { talimatlar };
```

Aşağıda örneği vermiştir;

```
// See https://aka.ms/new-console-template for more information
Func<int, string> doubleThenAddElevenThenQuote = i => {
    var doubled = 2 * i;
    var addedEleven = 11 + doubled;
    return $"{addedEleven}";
}
doubleThenAddElevenThenQuote(5); // "'21'"
```

Temsilci Başlatmada Lamda İfadeleri

Temsilciler (*delegate*) yöntemleri dolaylı çağırmanın yoludur. C dilindeki fonksiyon göstericileri gibidir ancak ondan daha yeteneklidirler. Temsilcilere ilk değer verme yani başlatmada lamda ifadeleri kullanılır;

```
// See https://aka.ms/new-console-template for more information
public delegate int intGirdiAlanveİntDönenBirTemsilci(int input);
class Program
{
    static void Main(string[] args)
    {
        intGirdiAlanveİntDönenBirTemsilci ikiİleÇarp = x => x * 2;
        intGirdiAlanveİntDönenBirTemsilci karesiniAl = x => x * x;
        intGirdiAlanveİntDönenBirTemsilci küpünüAl = x => x * x * x;
        int sayi = 10;
        Console.WriteLine("10*2 = " + ikiİleÇarp(10)); // 20
        Console.WriteLine("sayi^2 = " + karesiniAl(sayi)); //100
        Console.WriteLine("sayi^3 = " + küpünüAl(sayi)); //1000
    }
}
```

Anonim Değişkenlerle Lamda İfadeleri

Bir lamda ifadesini tanımlarken **var** kelimesi ile temsilci tipi otomatik olarak çıkarılır. Aslında genel olarak bir ifadenin sonucunda ortaya çıkan veri tipinin çıkarımı otomatik olması isteniyorsa anonim değişkenler kullanılır.

```
var IncrementBy = (int source, int increment = 1) => source + increment;
Console.WriteLine(IncrementBy(5)); // 6
Console.WriteLine(IncrementBy(5, 2)); // 7
```

Ayrıca, açıkça yazılmış bir parametre listesindeki son parametre olarak, parametre dizileri veya koleksiyonları olan lambda ifadelerini de tanımlanabilir;

```
// See https://aka.ms/new-console-template for more information
var sum = (params IEnumerable<int> values) =>
{
    int sum = 0;
    foreach (var value in values)
        sum += value;
    return sum;
};
var empty = sum();
Console.WriteLine(empty); // 0
var sequence = new[] { 1, 2, 3, 4, 5 };
var total = sum(sequence);
Console.WriteLine(total); // 15
```

Anonim Yöntemler

Anonim yöntemler (*anonymous method*) bir kod bloğunu temsilci (*delegate*) parametresi olarak geçirmek için bir teknik sağlar. Bunlar bir gövdeye sahip ancak adı olmayan yöntemlerdir ve Lamda ifadeleri ile tanımlanırlar.

```
// See https://aka.ms/new-console-template for more information
delegate int IntOp(int lhs, int rhs);
class Program
{
    static void Main(string[] args)
    {
        // C# 2.0 definition
        IntOp add = delegate(int lhs, int rhs)
        {
            return lhs + rhs;
        };
        // C# 3.0 definition: Uzun
        IntOp mul = (lhs, rhs) =>
        {
            return lhs * rhs;
        };
        // C# 3.0 definition: Kısa
        IntOp sub = (lhs, rhs) => lhs - rhs;
        // Her bir anonim yöntemi çağıralım:
        Console.WriteLine("2 + 3 = " + add(2, 3));
        Console.WriteLine("2 * 3 = " + mul(2, 3));
        Console.WriteLine("2 - 3 = " + sub(2, 3));
    }
}
```

Olaylar

Nesneler, olaylardan (*event*) etkilenirler ve olaylar karşısında tepki verir ya da davranış (*behavior*) gösterirler. Olayı yayan (*publish*) nesneye diğer tüm nesneler tepki vermezler. Bu nedenle tepki verecek nesneler, davranış göstereceği olaya önceden abone (*subscribe*) olurlar. Abone olunan olay gerçekleştiği zaman tepki (*handler*) verirler.

Aşağıda bir saat nesnesinin tık sesi olayına tepki veren dinleyici nesnesi örneğine yer verilmiştir;

```
// See https://aka.ms/new-console-template for more information
/* "Kişi" sınıfında bir olay tanımlanacak.
 * "yaşDeğişti" olayına nasıl yöntemlerle abone olunacağına
 * "OlayOluncaAboneOlunacakYöntem" delegate ile karar verilir. */
public delegate void OlayOluncaAboneOlunacakYöntem(Kişi s);
public class Kişi
{
    /* "Kişi" sınıfında bir olay tanımlanır.
     * "yaşDeğişti" olayı, "yaş" değiştiğinde tetiklenecek olan bir olaydır. */
    public event OlayOluncaAboneOlunacakYöntem yaşDeğişti;
    public string İsim { get; set; }
    private int yaş;
    public int Yaş
    {
        get { return yaş; }
        set
        {
            if (yaş != value)
            {
                yaş = value;
                if (yaşDeğişti != null)

```

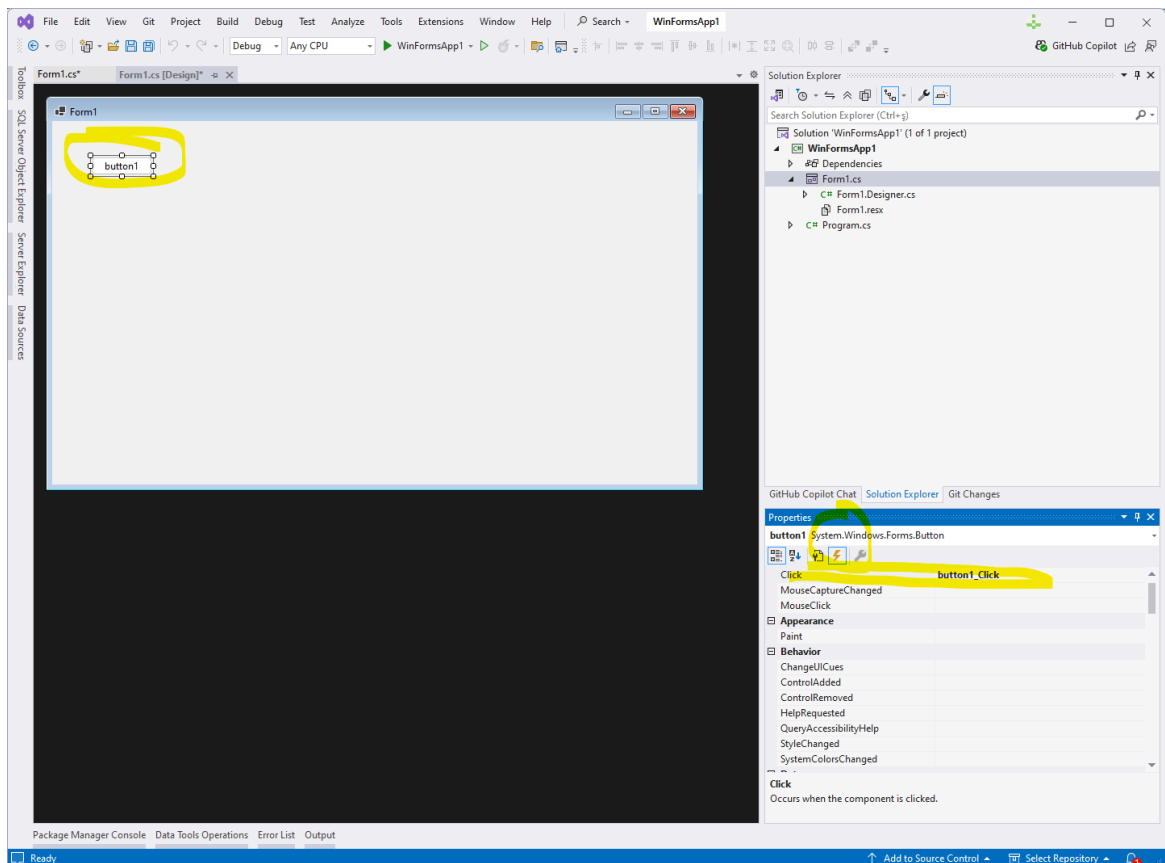
```

        {
            yaşDeğiřti(this); //Olayı tetikle!
        }
    }
}

public class Dinleyici
{
    /* "Dinleyici" sınıfından imal edilen nesneler,
    * "Kiři" sınıfından imal edilen nesnelere abone olabilir. */
    public void AboneOl(Kiři k)
    {
        k.yaşDeğiřti += this.kiřininYaşıDeğiřinceNeYapılacaksaYap;
    }
    /*Kiřide olay gerekleřtiğinde vereceėi tepkiye iliřkin davranıř tanımlanmalıdır. */
    private void kiřininYaşıDeğiřinceNeYapılacaksaYap(Kiři s)
    {
        Console.WriteLine("Kiřinin Yaşı Deėiřti!");
    }
}

internal class Program
{
    static void Main(string[] args)
    {
        Kiři kiři = new Kiři();
        Dinleyici dinleyici = new Dinleyici();
        dinleyici.AboneOl(kiři);
        kiři.Yaş = 20;
        kiři.Yaş = 30;
    }
}

```



řekil 25. Örnek Bir Windows Forms Uygulaması

C# dilinde ön tanımlı olarak olaylar aşağıdaki gibi tanımlıdır;

```
public event EventHandler<EventArgs> EventName;
```

<EventArgs> için jenerik başlığını inceleyebilirsiniz. Olaylara tepki verecek yöntemler ve abonelik aşağıdaki gibi tanımlanır;

```
public void HandlerName(object sender, EventArgsT args) { /* Handler logic */ }
EventName += HandlerName; // Olaya Abone Olma
```

Eğer Windows Forms uygulaması geliştiriyorsanız form tasarımında aşağıdaki kod verilebilir. Şekil 25'te verilen formda **Click** olayına çift tıklandığında IDE tarafından otomatik olarak bir tepki yöntemi hazırlanır ve bu yöntem bu olarak abone edilir.

```
namespace WinFormsApp1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        private void button1_Click(object sender, EventArgs e)
        {
        }
        private void Form1_Load(object sender, EventArgs e)
        {
        }
    }
}
```

Eğer bir olay olduğunda parametreler ile çalışacak isek EventArgs sınıfından türeyen bir sınıf tanımlamalıyız. Yukarıda verilen Kişi sınıfını C# için öntanımlı olan **EventArgs** ile aşağıdaki gibi yazabiliriz;

```
// See https://aka.ms/new-console-template for more information
public class YaşDeğişmeOlayıArgs : EventArgs
{
    public YaşDeğişmeOlayıArgs(int mevcutyaş, int yeniyaş)
    {
        this.MevcuyYaş = mevcutyaş;
        this.YeniYaş = yeniyaş;
    }
    public int MevcuyYaş { get; private set; }
    public int YeniYaş { get; private set; }
}

public delegate void YaşDeğişinceAboneOlunacakYöntem(object sender,
                                                    YaşDeğişmeOlayıArgs args);

public class Kişi
{
    public event YaşDeğişinceAboneOlunacakYöntem yaşDeğişt;
    public string İsim { get; set; }
    private int yaş;
    public int Yaş
    {
        get { return yaş; }
        set
        {
            if (yaş != value)
            {
```

```

        if (yaşDeğiştirdi != null)
        {
            YaşDeğişmeOlayıArgs args = new YaşDeğişmeOlayıArgs(yaş, value);
            yaşDeğiştirdi(this, args); //Olayı Tetikle!
        }
        yaş = value;
    }
}

public class Dinleyici
{
    /* "Dinleyici" sınıfından imal edilen nesneler,
    * "Kişi" sınıfından imal edilen nesnelere abone olabilir. */
    public void AboneOl(Kişi k)
    {
        k.yaşDeğiştirdi += this.kişininYaşıDeğişinceYap;
    }
    public void kişininYaşıDeğişinceYap(object sender, YaşDeğişmeOlayıArgs args)
    {
        Console.WriteLine($"{args.MevcutYaş} olan Kişinin Yaşı " +
            $"{args.YeniYaş} olarak Değişti!");
    }
}

internal class Program
{
    static void Main(string[] args)
    {
        Kişi kişi = new Kişi();
        Dinleyici dinleyici = new Dinleyici();
        dinleyici.AboneOl(kişi);
        kişi.Yaş = 20;
        kişi.Yaş = 30;
    }
}

```

Olaylara Tepki Vermek İçin Lamda İfadeleri

Lambda ifadeleri **olaylara** (event) **tepki** (handler) vermek için kullanılabilir. Bu durumda tepki gösterilecek olaya **abone** (subscribe) olduktan sonra abonelikten ayrılma söz konusu olmaz.

```
// See https://aka.ms/new-console-template for more information
public class YaşDeğişmeOlayıArgs : EventArgs
{
    public YaşDeğişmeOlayıArgs(int mevcutYaş, int yeniYaş)
    {
        this.MevcutYaş = mevcutYaş;
        this.YeniYaş = yeniYaş;
    }
    public int MevcutYaş { get; private set; }
    public int YeniYaş { get; private set; }
}

public delegate void YaşDeğişinceAboneOlunacakYöntem(object sender, YaşDeğişmeOlayıArgs args);

public class Kişi
{
    public Kişi()
    {
        yaş = 0;
        İsim = "İsimsiz";
        yaşDeğiştirdi = null;
    }
    public event YaşDeğişinceAboneOlunacakYöntem yaşDeğiştirdi;
```

```

public string İsim { get; set; }
private int yaş;
public int Yaş
{
    get { return yaş; }
    set
    {
        if (yaş != value)
        {
            if (yaşDeğişti != null)
            {
                YaşDeğişmeOlayıArgs args = new YaşDeğişmeOlayıArgs(yaş, value);
                yaşDeğişti(this, args);
            }
            yaş = value;
        }
    }
}
}
public class Dinleyici
{
    /* "Dinleyici" sınıfından imal edilen nesneler,
    * "Kişi" sınıfından imal edilen nesnelere abone olabilir. */
    public void AboneOl(Kişi k)
    {
        k.yaşDeğişti += this.kişininYaşıDeğişinceYap;
    }
    public void kişininYaşıDeğişinceYap(object sender, YaşDeğişmeOlayıArgs args)
    {
        Console.WriteLine($"{args.MevcuyYaş} olan Kişinin Yaşı "+
            $"{args.YeniYaş} olarak Değişti!");
    }
}
internal class Program
{
    static void Main(string[] args)
    {
        Kişi kişi = new Kişi();
        Dinleyici dinleyici = new Dinleyici();
        dinleyici.AboneOl(kişi);
        kişi.Yaş = 20;
        kişi.Yaş = 30;
        Console.WriteLine("----- ");
        YaşDeğişinceAboneOlunacakYöntem tepki = (sender, args)
            => Console.WriteLine("Kişinin Yaşı Değişemez!!");
        kişi.yaşDeğişti += tepki;
        kişi.yaşDeğişti += (sender, args)
            => Console.WriteLine("Kişinin Yaşı Savcılık Emriyle Değişebilir!");
        kişi.Yaş = 40;
    }
}

```

Burada lamda ifadesiyle tepki olarak kodlanan yöntem `kişi.yaşDeğişti -= tepki;` talimatı yazılarak abonelikten çıkılabilir. Ancak onun altındaki anonim yöntem için abonelikten vazgeçilemez!

Genişleme Yöntemleri

Genişleme yöntemleri (*extension method*), C# 3.0 ile hayatımıza girmiştir. Genişletme yöntemleri, yeni bir türetilmiş tip oluşturmadan, yeniden derlemeden veya orijinal türü başka şekilde değiştirmeden mevcut türlere davranışı genişletir ve ekler. Özellikle geliştirmek istediğiniz bir tipin kaynağını değiştiremediğinizde faydalıdırlar. Genişletme yöntemleri `int`, `char`, `string` gibi sistem tipleri, üçüncü

tarafarla kütüphanelerde tanımlanan tipler ve **class** gibi kendiniz tanımladığınız tipler için oluşturulabilir. Genişletme yöntemi orijinal tipin bir üye yöntemiymiş gibi çağrılabilir.

Bir genişletme yönteminin ilk parametresi her zaman **this** anahtar sözcüğünden önce gelmeli, ardından genişlettiğiniz nesnenin "geçerli" örneğine başvurmak için tanımlayıcı gelmelidir;

```
// See https://aka.ms/new-console-template for more information
static class StringExtensions
{
    public static string Shorten(this string text, int length)
    {
        return text.Substring(0, length);
    }
}
class Program
{
    static void Main()
    {
        // aşağıdaki atama String.ToUpper() yöntemini çağırır.
        var myString = "Hello World!".ToUpper();
        /* aşağıdaki yöntem Extensions sınıfının StringExtensions.Shorten()
        * yöntemini çağırır */
        var newString = myString.Shorten(5);
        // Aşağıda ise genişletilmiş yöntem çağırılmıştır.
        var newString2 = StringExtensions.Shorten(myString, 5);
    }
}
```

Bir sınıfın kaynak kodu mevcut değilse veya mevcut sınıfta değişiklik yapma izniniz yoksa, genişleme yöntemlerinin mevcut sınıfa yeni yöntemler ekleyerek bir sınıfın işlevselliğini genişletmek için bir yaklaşım olarak kullanılabileceğini söyleyebiliriz. Hatırlamanız gereken en önemli nokta, genişleme yöntemlerinin statik bir sınıfın statik yöntemleri olduğu, ancak genişletilmiş tipteki örnek yöntemlermiş gibi çağrılacaklarıdır.

İsim Uzayları

İsim uzayları (**namespace**) kodun düzenli, okunabilir ve yönetilebilir olmasını sağlamak için kullanılan bir yapıdır. Özellikle büyük projelerde isim çakışmalarını önlemek ve sınıfları mantıksal olarak gruplamak için kullanılır.

İsim uzayları, sınıf, ara yüz, yapı (**struct**) ve numaralandırma (**enum**) gibi kod bileşenlerini mantıksal gruplara ayırmak için kullanılır. Temel amacı isim çakışmalarını önlemek ve kodu düzenlemektir.

namespace anahtar kelimesi, bir kod içerisinde yer alan bileşenlerin nasıl düzenlendiğini anlamamıza yardımcı olur. Fiziksel bir klasöre benzer şekilde bileşenleri tasnif etmemizi sağlar. Bir isim uzayı iç içe istenildiği kadar isim uzayı barındırabilir.

İsim uzayının adını belirtmeden içindeki bileşenleri kullanabilmek için **using** anahtar kelimesi kullanılır. Aksi durumda sınıflarda olduğu gibi üyelere nokta işlecisi erişilebilir.

```
// See https://aka.ms/new-console-template for more information
using İsimUzayı;
Kullanıcı kullanıcı1 = new Kullanıcı
{
    Ad = "Ahmet",
    Soyad = "Yılmaz",
    Tip = KullanıcıTipi.ADMIN
};
Kullanıcı kullanıcı2 = new Kullanıcı
{
    Ad = "Mehmet",
    Soyad = "Demir",
    Tip = KullanıcıTipi.RAPORTÖR
}
```

```
};
İsimUzayı2.Kullanıcı kullanıcı3 = new İsimUzayı2.Kullanıcı
{
    Ad = "Ayşe",
    Soyad = "Kara"
};

namespace İsimUzayı
{
    enum KullanıcıTipi { ADMIN, RAPORTÖR, İZLEYİCİ }
    class Kullanıcı
    {
        public string Ad { get; set; }
        public string Soyad { get; set; }
        public KullanıcıTipi Tip { get; set; }
    }
}

namespace İsimUzayı2
{
    class Kullanıcı
    {
        public string Ad { get; set; }
        public string Soyad { get; set; }
    }
}
```

İsim uzayları nokta ile ayrılan fiziksel klasörlere benzer;

```
// See https://aka.ms/new-console-template for more information
İsimUzayı.KullanıcıArayüzü.Kullanıcı.Kullanıcı kullanıcı1 =
    new İsimUzayı.KullanıcıArayüzü.Kullanıcı.Kullanıcı
    {
        Ad = "Ahmet",
        Soyad = "Yılmaz",
        Tip = İsimUzayı.KullanıcıArayüzü.Kullanıcı.KullanıcıTipi.ADMIN
    };
kullanıcı1.Ad = "Ali"; // Değişiklik yapıldı

namespace İsimUzayı.KullanıcıArayüzü.Kullanıcı
{
    enum KullanıcıTipi { ADMIN, RAPORTÖR, İZLEYİCİ }
    class Kullanıcı
    {
        public string Ad { get; set; }
        public string Soyad { get; set; }
        public KullanıcıTipi Tip { get; set; }
    }
}
```

İsim uzaylarını **using** kelimesi ile referans göstererek içerdiği bileşenleri kullanabiliriz;

```
// See https://aka.ms/new-console-template for more information
using System.Text;

var sb = new StringBuilder(); //<- var sb = new System.Text.StringBuilder(); yerine
```

İsim uzaylarına takma ad vererek de kullanabiliriz;

```
using st = System.Text;

var sb = new st.StringBuilder(); //<- var sb = new System.Text.StringBuilder(); yerine
```

SINIFLAR ARASI İLİŞKİLER VE SINIF TASARLAMA İLKELERİ

Unified Modeling Language

Unified Modeling Language (UML) nesneye dayalı yazılım geliştirme ile gelişen bir modelleme aracıdır. Bu nedenle Nesne Yönelimli Programlama başlığında açıklanan nesne yönelimli birçok kavram, izleyen sayfalarda daha da pekişecektir. Adından da anlaşılacağı üzere nesneye dayalı problem çözmenin kalbi, model oluşturmaktır.

Model, gerçek dünyadaki karmaşık problemin esaslarını soyut olarak önümüze koyar. Buradan hareketle UML'i, basit bir dil, gösterim ya da sözdizimi (syntax) gibi algılamalıyız. UML'in, yazılımı nasıl geliştireceğimizi kesin olarak belirtmediği göz önüne alınmalıdır.

UML, bir grafik modelleme dili olup yazılım sistemlerini oluşturan ana elemanları (Ki bunlar İngilizce "artifact" olarak adlandırılır ve el yapımı anlamına gelmektedir.) çeşitli şekillerden oluşacak şekilde tanımlamak için oluşturulmuş bir kurallar bütünüdür. Örnek verecek olursak, bir mimari projeye ilişkin maket ne ise yazılım için UML de aynı şeyi ifade eder. Yani UML, söz konusu yazılımın neyi yapacağını anlatan çeşitli şekillerin anlamlı bir şekilde bir araya getirilmesini sağlayan bir standarttır.

Yazılım mühendisliğiyle (software engineering) birlikte 1989 ve 1994 yılları arasında elliden fazla modelleme dili kullanılmaya başlanmıştır. Bu nedenle bu dönem yöntem savaşlarının yaşandığı bir dönem olarak adlandırılır. Bu yöntemlerin çoğu kendi sözdizimini oluşturmuş olmakla birlikte kullandıkları dil elemanları açısından birbirleriyle benzerlik göstermektedirler.

Doksanlı yılların ortalarında üç yöntem, daha güçlü ve yaygın hale gelmiştir. Bu yöntemlerin her biri, diğerlerinin içerdiği elemanları da içeriyordu ancak her birinin diğerine karşı çeşitli güçlü özellikleri bulunuyordu. Bu yöntemler:

- *Booch* yönteminin **tasarım (design)** ve **kodlama (implementation)** yönleri oldukça iyi idi. *Grady Booch* Ada diliyle oldukça çalışmıştı ve nesne yönelimli tekniklerin Ada diline uygulanması konusunda yaptığı çalışmalarla bu konuda önemli bir oyuncu olmuştur. Booch metodu güçlü olmasına rağmen model, birçok bulut şeklinden oluşmasından dolayı gösterim şekli oldukça sevimsizdi.
- Object Modeling Technique (OMT) yöntemi, *Jim Rumbaugh* tarafından geliştirilmiş olup, analiz ve veri merkezli bilgi sistemlerinin modellenmesinde oldukça iyi bir yöntemdir.
- Object Oriented Software Engineering (OOSE) modeli use-case olarak bilinen bir model olup *Ivar Jacobson* tarafından geliştirilmiştir. Use-case modeli, yazılımı yapılacak sistemin davranışlarını anlamaya yönelik gelişmiş güçlü bir tekniktir.

1994 yılında *Jim Rumbaugh* ve General Electric firmasından ayrılan *Grady Booch* birlikte Rational şirketini kurarak bir araya geldiler. Bu birlikteliğin amacı, kendi yöntemlerini bir araya getirecek yeni ve tek bir yöntem olan "Unified Method"u oluşturmak idi.

1995 yılında *Ivar Jacobson* da Rational şirketine katıldı. *Ivar Jacobson*'ın use-case'ler konusundaki fikirleriyle birlikte Rational şirketi, bugün Unified Modeling Language-UML olarak adlandırılan, yeni bir "Unified Method" geliştirdi. Üç kişiden oluşan bu grup "Three Amigos" olarak bilinir.

Yöntem savaşlarının ardından güçlü yazılım üreticileri bir araya gelerek OMG adında bir şirketler birliği kurdular¹⁹. Bu şirketler birliği 1997 yılında UML'e sahip çıkarak herkes tarafından kullanılan bir dil olmasını ve bir standart haline gelmesini sağlamıştır.

UML sadece, gerçek dünyadaki süreçlerin modelini ortaya koyan bir **gösterim (notation)** sistemidir. Süreçlerin standartlaştırılmasına yönelik bir dil değildir. UML ile ortaya çıkan modeller, süreçleri **somut**

¹⁹ OMG (Object Management Group), www.omg.org

(**abstract**) olarak tanımlayacaktır, ancak süreçlerin doğruluğu hakkında bir bilgi vermeyecektir. Yani, A şirketi için çalışan bir süreç, B şirketine uygulandığında felaketle sonuçlanabilir.

Model, yukarıda da anlatıldığı üzere çözülecek probleme ilişkin bir **soyutlamadır** (**abstraction**). Etki alanı (domain) ise problemin yaşandığı gerçek dünyayı ifade eder.

Model, birbirlerine **ileti** (**message**) gönderen nesnelerden (object) oluşur. Bu modelde nesneler **canlı** (**alive**) olarak düşünülmelidir. Nesneler bir şeyler bilir (**know**) ve bildikleriyle bir şeyler yaparlar (**do**).

Modelde nesneler, nesne tanımında verilen **özellik** (**property**) ve **alanları** (**field**) gösteren **niteliklere** (**attribute**) sahiptirler. Nesnelerin gösterdikleri **davranış** (**behavior**) ya da geliştirdiği **yöntemler** (**method**) ise işlem (operation) olarak adlandırılır. Nesnenin özelliklerine ait o anki değerler ise nesnenin **durumunu** (**state**) belirler.

UML, sekiz tür diyagrama sahiptir ve bu kitapta sınıf diyagramları anlatılacaktır²⁰.

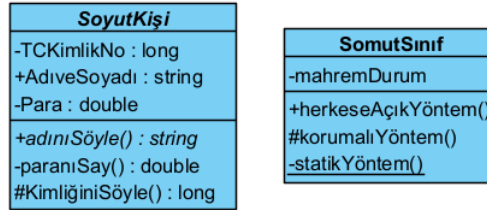
Sınıf Diyagramları

UML diyagramlarından biri olan **sınıf diyagramları** (**class diagram**), yazılımı geliştirilecek olan sisteme ait **sınıfları** (**class**) ve bu sınıflar arasındaki **ilişkiyi** (**relationship**) gösterir. Sınıf diyagramları durağandır (**static**). Yani sadece sınıfların birbiriyle etkileşimini (**what interacts**) gösterir, bu etkileşimler sonucunda ne olduğunu (**what happens**) göstermez.

Sınıf diyagramlarında sınıflar, bir dikdörtgen ile temsil edilir. Bu dikdörtgen üç parçaya ayrılır. En üstteki birincisine sınıf ismi yazılır. Ortadaki ikincisinde **nitelikler** (**attribute**), en alttaki ve sonuncusunda ise **işlemler** (**operation**) yer alır.

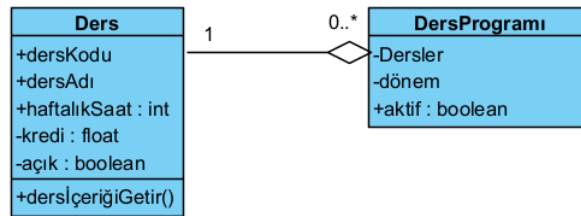
Sınıf isminin italik yazılması, sınıfın **soyut** (**abstract class**) sınıf olduğunu gösterir. Benzer şekilde işlemlerin yer aldığı kısımda, yöntemlerin italik yazılması halinde, ilgili yöntemin **soyut yöntem** (**abstract method**) olduğunu gösterir.

Sınıflarda **erişim belirleyiciler** (**access modifier**), özel karakterlerle birbirinden ayrılır. Burada “-” karakteri **mahrem** (**private**), “+” karakteri **umuma açık** (**public**), “#” karakteri ise **korunmalı** (**protected**) olduğunu anlatmaktadır. **Statik üyeler** (**static member**), altı çizili olarak gösterilir.



Şekil 26. Örnek Bir Sınıf Diyagramı

Çokluk (**multiplicity**), ilişkide bir sınıfın **örnek** (**instance**) sayısını gösterir. İlişkiyi gösteren çizginin sonunda bir numara olarak gösterilir.



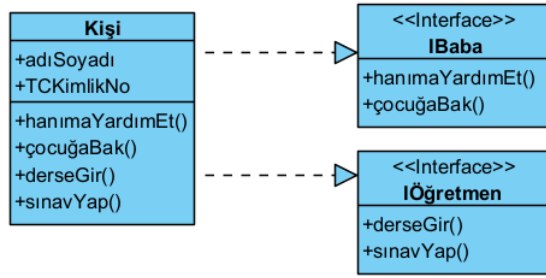
Şekil 27. Sınıf Diyagramlarında Sınırlama ve Çokluk Örneği

Bu numara, mümkün olabilecek örnek sayısıdır. Çokluk bir sonraki başlıkta verilen, **iş birliği** (**association**), **bileşim** (**composition**) ve **bütünleşmeye** (**aggregation**) uygulanır. Çokluk örnekleri:

- **0..1** Sıfır ya da 1 örnek.
- **0..*** Veya ***** Örnek sayısı sınırsızdır.

²⁰ www.togethersoft.com/services/practical_guides/umlonlinecourse/index.html

- 1 Yalnızca 1 örnek
- 1.. En az bir örnek



Şekil 28. Sınıf Diyagramlarında Ara yüz Gerçekleştirme

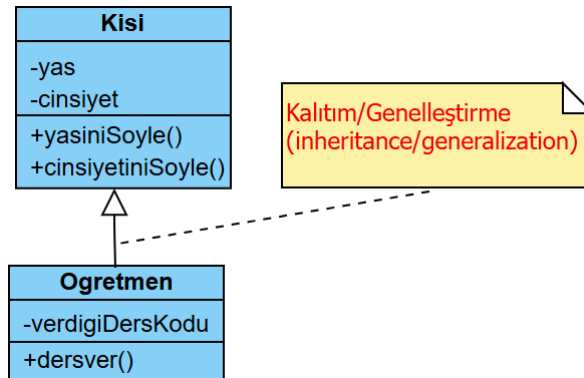
Ara yüz gerçekleştirme (interface realization) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (interface) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar **lollipop diyagramı** olarak da adlandırılır.

Sınıflar Arası İlişkiler

Nesneleri imal ettiğimiz sınıflar arasında ya ilişki yoktur ya da aşağıda sıralanan ilişkiler vardır;

Genelleştirme İlişkisi

Genelleştirme (generalization), taban sınıfla (base class) ile türemiş sınıf (derived class) arasındaki kalıtım (inheritance) ilişkisidir. Örneği *Kalıtım* başlığında verilmiştir. Aşağıda Kişi sınıfı ile bu sınıftan miras alan Öğretmen sınıfının UML diyagramı verilmiştir. Ara yüzler (interface) veya roller de benzer şekilde gösterilir.

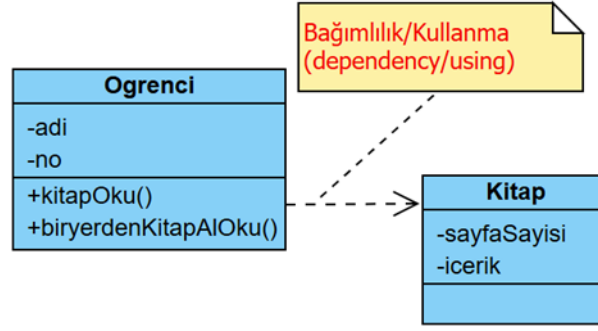


Şekil 29. Genelleştirme İlişkisi UML Diyagramı

Bağımlılık İlişkisi

Bağımlılık (dependency) ya da bir başka deyişle kullanma (using), bir sınıf ile diğer arasındaki geçici ilişkidir. Burada kullanan sınıf istemci (client) diğer sınıf ise sağlayıcı (supplier) olarak adlandırılır. İki türlü bağımlılık vardır;

- İstemci sınıf, sağlayıcıyı sınıfı parametre olarak kullanır (dependency as a parameter).
- İstemci sınıf, sağlayıcı sınıftan örnekleme yapar (dependency as an instantiation).



Şekil 30. Bağımlılık ilişkisi UML Diyagramı

```

// See https://aka.ms/new-console-template for more information
Öğrenci ali = new Öğrenci();
Kitap birkitap = new Kitap();
ali.kitapOku();
ali.biryerdenKitapAlOku(birkitap);

public class Kitap
{
    public int SayfaSayısı { get; set; }
    public string İçerik { get; set; }
}
public class Öğrenci
{
    public string Adı { get; set; }
    public int No { get; set; }
    public void kitapOku()
    {
        Kitap kitap = new Kitap(); // dependency as an instantiation
        Console.WriteLine($"{kitap.İçerik}");
    }
    public void biryerdenKitapAlOku(Kitap kitap) // dependency as a parameter
    {
        if (kitap.SayfaSayısı > 100)
        {
            /* Yaratılacak Öğrenci Nesneleri bir başka yerde
             * imal edilmiş "kitap" nesnesini parametre olarak
             * alarak kullanıyor. */
            Console.WriteLine("Uzun Kitap...");
        }
    }
}

```

Örnekteki Öğrenci ve Kitap sınıfı arasındaki bağımlılık ilişkisi aşağıdaki UML diyagramında verilmiştir.

İş Birliği İlişkisi

Ortaklık olarak da adlandırılan **iş birliği** (**association**), iki farklı sınıf arasındaki sürekli olan **bütün-parça** (**whole-part**) ilişkisidir. Bu ilişkide, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. İlişkinin sürekli olabilmesi için kullanılan sınıftan bir değişken **alan** (**field**) olarak tanımlanır. İki türü vardır;

- **Açık iş birliği** (**public association**): Kullanılan sınıfa ilişkin değişken **public** erişimi ile tanımlanır.
- **Gizli iş birliği** (**private association**), **içerme** (**containment**) ya da **bileşim** (**composition**): Kullanılan sınıfa ilişkin değişken **private** erişimi ile tanımlanır ve dışarıdan erişime izin verilmez.

```

// See https://aka.ms/new-console-template for more information
Öğretmen ilhan = new Öğretmen();
ilhan.kitapOku(); // ilhanın kitabına erişilemez!
Öğrenci ali = new Öğrenci();
ali.kitapOku(); // alinin kitabına dışarıdan erişilebilir!
Kitap alininkitabı = ali.kitap;

```

```

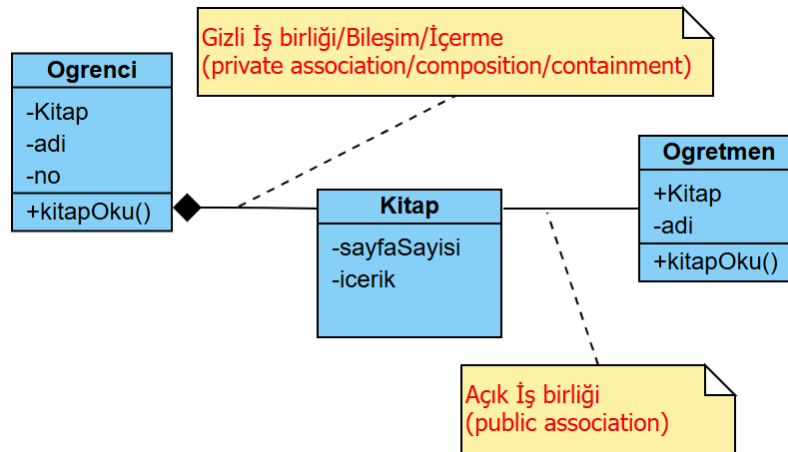
public class Kitap
{
    public int SayfaSayısı { get; set; }
    public string İçerik { get; set; }
}

public class Öğrenci
{
    public string Adı { get; set; }
    public int No { get; set; }
    public Kitap kitap;
    public Öğrenci()
    {
        kitap = new Kitap();
    }
    public void kitapOku()
    {
        Console.WriteLine($"Öğrenci Kitabını Okuyor...{kitap.İçerik}");
    }
}

public class Öğretmen
{
    public string Adı { get; set; }
    public int No { get; set; }
    private Kitap kitap;
    public Öğretmen()
    {
        kitap = new Kitap();
    }
    public void kitapOku()
    {
        Console.WriteLine($"Öğretmen Kitabını Okuyor...{kitap.İçerik}");
    }
}

```

Yukarıdaki örnekte Öğrenci ile Kitap arasındaki gizli ortaklık ile Öğretmen ve Kitap arasındaki açık iş birliği aşağıdaki UML diyagramında gösterilmiştir;

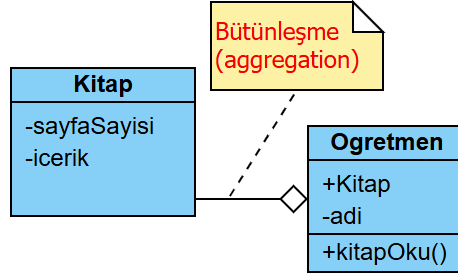


Şekil 31. Açık ve Gizli İş Birliği UML Diyagramı

UML diyagramlarında, çift yönlü ortaklıklar iki oka sahip olabilir veya hiç ok olmayabilir ve tek yönlü ortaklıkta veya kendi kendine ortaklık bir oka sahiptir.

Bütünleşme İlişkisi

Bütünleşme (**aggregation**), ilişkisi de iki farklı sınıf arasındaki sürekli olan **bütün-parça** (**whole-part**) ilişkisidir. **Açık ortaklığa** (**public association**) benzer, farklı olarak bu ilişkide kullanılan sınıfa ait başka yerde imal edilmiş nesneyi ya da hazır olan nesneyi kullanır. Tek fark kullanılacak sınıf nesnesinin kullananın iradesi dışında yaratılmış olmasıdır.



Şekil 32. Bütünleşme UML Diyagramı

```

// See https://aka.ms/new-console-template for more information
Kitap birkitap = new Kitap();
Öğretmen ilhan = new Öğretmen(birkitap); // Öğretmen Kitapsız imal edilemez!
ilhan.kitapOku();

public class Kitap
{
    public int SayfaSayısı { get; set; }
    public string İçerik { get; set; }
}

public class Öğretmen
{
    public string Adı { get; set; }
    public int No { get; set; }
    private Kitap kitap;
    public Öğretmen(Kitap kitap)
    {
        this.kitap = kitap;
    }
    public void kitapOku()
    {
        Console.WriteLine($"Öğretmen Kitabını Okuyor...{kitap.İçerik}");
    }
}
  
```

Yukarıdaki örnekte verilen Öğretmen ve Kitap arasındaki bütünleşme ilişkisi aşağıdaki UML diyagramında gösterilmiştir.

Sınıf Tasarlama İlkeleri

Nesne yönelimli programlamada (**object oriented programming**); Kodumuzda değişim yönetimi (**change management**), gösterici kullanımı yerine referansları kullanarak güvenli kod (**safe code**) aynı kodları tekrar yazmak (**duplicate code**) yerine kodların yeniden kullanımı (**reusing**) sağlayan aşağıdaki özellikleri kullanırız;

- Kod küçüldüğü için **kalıtım** (**inheritance**),
- Veri ve kontrol **soyutlaması** (**abstraction**) yaparak daha güvenli bileşen tasarladığımız için **sarmalama** (**encapsulation**),
- Yazılım mimarisinin daha esnek ve genişleyebilir olması için **çok biçimlilik** (**polymorphism**).

Ayrıca yazılım geliştirmede;

- **Zayıf bağlaşımın** (**loosely coupling**) sağlanması için yazılımın bir noktasında olabilecek değişikliğin, diğer kısımlarda değişiklik gerektirmeyecek şekilde tasarlanması gerekir. Yani yapılan değişiklik, çorap sökücü gibi yazılımın diğer kısımlarını değiştirmemelidir.
- Aynı çağrışım kümesine ilişkin bütün bileşenler bir arada geliştirilmeli diğer kümelerle karıştırılmamalıdır (**seperation of concern**). Yani bileşenler arasında **yüksek uyum** (**high cohesion**) olmalıdır.

İşte yeniden kullanım, zayıf bağlaşım ve yüksek uyum sağlamak için programcı, nesne yönelimli programlama özellikleriyle birlikte temel **ilkelere** (**principle**) uyar.

Programcı kodu tasarlarırken, yazarken ve yeniden düzenlerken bu ilkeleri aklında tutarsa; Kod çok daha temiz, genişletilebilir ve test edilebilir olur.

Bunlar ilkelerin İngilizce baş harflerinin alınarak isimlendirilen SOLID ilkeleridir (**SOLID Principles**);

Tek Sorumluluk İlkesi

Bu ilkede (**Single Responsibility Principle-SRP**): sınıflar, tasarlanırken birden fazla işten sorumlu tutulmamalıdır. Mümkün olduğunca bir işle ilgili olarak tasarlanmalıdır. Bir sınıfın tek bir şey yapması gerektiğini ve dolayısıyla değişmesi için tek bir nedeninin olması gerektiğini belirtir.

Açık-Kapalı İlkesi:

Bu ilkede (**Open Closed Principle-OCP**) sınıflar, değişikliklere kapalı (**closed for modification**) eklentilere açık (**open for extension**) şekilde tasarlanmalıdır. Bu şekilde değişikliklere açık genişleyebilir modüllere sahip oluruz. Değişikliklere kapalı olmak, mevcut bir sınıfın kodunu değiştirmemek anlamına gelirken, eklentilere açık olma yeni işlevler eklemek anlamına gelir.

Liskov İkame İlkesi

Bu ilkede (**Liskov Substitution Principle-LSP**) alt sınıflar türediği sınıfların yerine konulabilmelidir. Bu prensipten ilk olarak Barbar Liskov tarafından bahsedilmiştir. Alt sınıflar, türediği sınıf davranışlarıyla kullanılabilir. Burada amaç, alt sınıfların daha çok değişebileceği göz önüne alınarak değişimlerden an az şekilde etkilenmektir.

Türemiş sınıftan bir nesneyi, Taban sınıftan bir nesne bekleyen herhangi bir yönteme parametre olarak geçirdiğimizde yöntemin herhangi bir tuhaf çıktı vermemesi gerektiği anlamına gelir. Bu beklenen davranıştır, çünkü kalıtım kullandığımızda alt sınıfın, üst sınıfın sahip olduğu her şeyi miras aldığını varsayabiliriz. Alt sınıf davranışı genişletir ancak asla daraltmaz. Dolayısıyla bir sınıf bu ilkeye uymadığında, tespit edilmesi zor bazı kötü hatalara yol açar.

Ara Yüzleri Ayırma İlkesi

Bu ilkede (**Interface Segregation Principle-ISP**) genel amaçlı ara yüzler yerine istemciye bağlı ara yüzler tasarlanmalıdır. Yani roller, birbirinden ayrı olacak şekilde tanımlanmalıdır. Birden fazla rol tek bir rol altında olacak şekilde düşünülmemelidir. Bu nedenle buna rollerin ayrılması prensibi (**Role Segregation Principle-RSP**) olarak da adlandırılır.

Bağımlılıkları Ters Çevirme İlkesi

Bu ilkede (**Dependency Inversion Principle-DIP**) sınıflarımızın somut sınıflara ve fonksiyonlara değil, ara yüzlere veya soyut sınıflara bağlı olması gerektiğini belirtir. Nesne yönelimli programlamanın en önemli ilkesidir. Aslında bu ilke ile açık-kapalı ilkesi doğrudan birbiriyle ilişkilidir.

YAPILAR, BİRLİKLER VE NİTELİKLER

Yapı Tanımlaması

Yapısal programlamaya adını veren **yapı** (**struct**), **türetilmiş** (**derived**) veya **kullanıcı tanımlı** (**user defined**) bir veri tipidir. Farklı tiplerdeki **değer tipi** (**value type**) elemanlarının bir arada gruplandırılan özel bir veri tipi tanımlamak için **struct** anahtar kelimesini kullanırız.

Yapı bir veya birden fazla temel değer veri tipinin bir araya gelmesiyle oluşturulan yeni veri tipleridir. Yapılar farklı dipte elemanların bir araya gelmesiyle oluşabilir. Sınıflara benzer şekilde tanımlanır;

```
erişim-değiştiricisi struct yapı-kimliği {  
    veri-tipi yapı-elemanı-kimliği1; //alan (field)  
    veri-tipi yapı-elemanı-kimliği1; //alan (field)  
    ...  
}
```

Örnek olarak Öğrenci; adı, soyadı, numarası, yaşı, cinsiyeti gibi farklı öğeler ile bir **ogrenci** yapısı tanımlanabilir;

```
public struct Öğrenci {  
    public uint yas;  
    public char cinsiyet;  
    public float kilo;  
    public uint boy;  
}
```

Benzer şekilde koordinat bilgisi içeren yapı aşağıdaki şekilde tanımlanabilir;

```
public struct Koordinat {  
    public double X;  
    public double Y;  
}
```

Yapı Değişkenleri

Tanımlanan yapı kullanılarak değişkenler veya **örnekler** (**instance**) aşağıdaki gibi tanımlanabilir;

```
yapı-kimliği değişken-kimliği;  
//yada  
yapı-kimliği değişken-kimliği=new yapı-kimliği();
```

Tanımlanan yapı değişkenleri üzerinden her bir alana **nokta işleci** (**dot operator**) ile erişilir. Bu işleç de parantez işleci ile aynı önceliktedir. Ayrıca Atama (=) işleci bir **yapıyı** (**struct**) doğrudan kopyalamak için kullanılabilir. Ayrıca, bir yapının üyesinin değerini başka birine atamak için atama işlecini de kullanabiliriz. Yukarıda örneği verilen yapılardan birçok değişken kimliklendirilebilir;

```
Öğrenci ilhan;  
ilhan.yas = 20;  
ilhan.cinsiyet = 'E';  
ilhan.kilo = 80;  
ilhan.boy = 180;  
  
Öğrenci mehmet=new Öğrenci();  
mehmet.yas = 25;  
mehmet.cinsiyet = 'E';  
mehmet.kilo = 75;  
mehmet.boy = 175;  
  
Koordinat nokta1;  
nokta1.X = 10.5;  
nokta1.Y = 20.5;
```

```
Koordinat nokta2= new Koordinat();
nokta2.X = 30.5;
nokta2.Y = 40.5;

nokta2 = nokta1;
```

Yapılarda Yapıcılar

Sınıflardan ön tanımlı nesneler imal edilebildiği gibi yapılardan da ön tanımlı **örnekler** (**instance**) oluşturulabilir. Bunun için **yapıcılar** (**constructor**) kullanılır. Yapı alanları parametrelili bir yapıcı aracılığıyla veya yapı imal edildikten sonra ayrı ayarlanabilir.

```
// See https://aka.ms/new-console-template for more information
Koordinat nokta1;
nokta1.X = 10.5;
nokta1.Y = 20.5;
Console.WriteLine($"Nokta 1: ({nokta1.X}, {nokta1.Y})");
Koordinat nokta2 = new Koordinat();
nokta2.X = 30.5;
nokta2.Y = 40.5;
Console.WriteLine($"Nokta 2: ({nokta2.X}, {nokta2.Y})");
nokta2 = nokta1;
Console.WriteLine("Nokta 2: {0}, {1}", nokta2.X, nokta2.Y);

public struct Koordinat
{
    public double X;
    public double Y;
    public Koordinat()
    {
        X = 0;
        Y = 0;
    }
    public Koordinat(double x, double y)
    {
        X = x;
        Y = y;
    }
}
```

Mahrem (**private**) üyeler yalnızca yapıcı içinde ilk değer verilebilir yani **başlatılabilir** (**initialize**).

Diğer değer tipler gibi yapılar da **null** olamaz. Ancak değişken olarak **null** olabilir olarak tanımlanabilir;

```
Koordinat nokta1 = null; //HATA!: Değer tiplere null atanamaz!
Koordinat? v2 = null; //OK
Nullable<Koordinat> v3 = null // OK
```

Yapılarda Ara Yüzü Gerçekleştirme

Tanımlanan her **yapı** (**struct**), **System.ValueType** tipinden dolayı olarak miras alan **kısır** (**sealed**) bir veri tipidir. Yapılar başka herhangi bir tipten miras alamaz, ancak **ara yüzleri gerçekleştirebilirler** (**interface realization**);

```
// See https://aka.ms/new-console-template for more information
Dikdörtgen dikdörtgen = new Dikdörtgen(5, 10);
Console.WriteLine($"Dikdörtgenin alanı: {dikdörtgen.Alan()}");
Daire daire = new Daire(7);
Console.WriteLine($"Dairenin alanı: {daire.Alan()}");
public interface IŞekil // Bir arayüz
{
    double Alan();
}
```

```

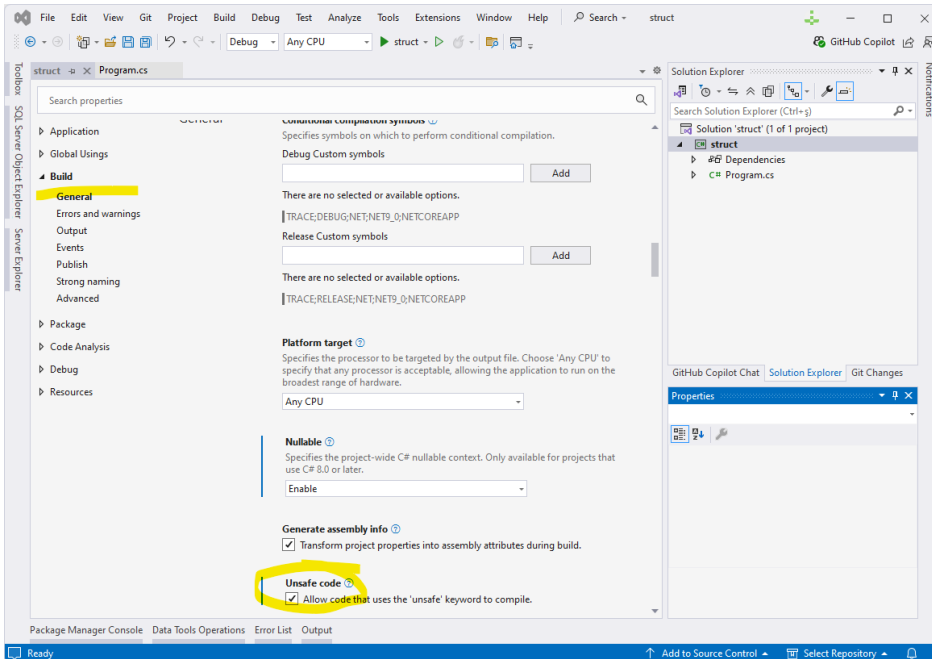
}
public struct Daire : IŞekil // Daire yapısı IŞekil arayüzünü gerçekleştiriyor
{
    public double yarıcap;
    public Daire(double yarıcap)
    {
        this.yarıcap = yarıcap;
    }
    public double Alan()
    {
        return Math.PI * yarıcap * yarıcap;
    }
}
public struct Dikdörtgen : IŞekil // Dikdörtgen yapısı IŞekil arayüzünü uyguluyor
{
    public double uzunKenar;
    public double kısaKenar;
    public Dikdörtgen(double uzunKenar, double kısaKenar)
    {
        this.uzunKenar = uzunKenar;
        this.kısaKenar = kısaKenar;
    }
    public readonly double Alan()
    {
        return uzunKenar * kısaKenar;
    }
}
}

```

Sınıflarda olduğu gibi bir yapıya üye yöntem tanımlanabilir. Eğer üye yöntem bir yapı alanını değiştirmeyecek ise yukarıdaki gibi **readonly** tanımlanabilir.

Boyutu Sabit Dizi Elemanına Sahip Yapılar

C# dilinin önceki sürümlerinde C++ dilindeki gibi sabit boyutlu bir yapı bildirmek zordur. Ancak C# dilinin gelişmiş sürümlerinde sabit boyutlu diziler mümkündür.



Şekil 33. Güvenli Olmayan Kod Derlenmesi

Artık güvenli olmayan kod blokları içerisinde dizileri sabit boyutlu olarak bildirmek mümkündür; Bunun için projenin Visual Studio içerisinde **unsafe** olarak derleneceğinin bildirilmesi gerekir. Visual Studio içerisinde ise proje özelliklerinde güvenli olmayan kodun derlenmesi için değişiklikler yapılmalıdır. Bir

projenin güvenli olmayan şekilde derlenebilmesi için komut satırında `/unsafe` argümanı derleyiciye verilmelidir;

```
C:\Users\ILHANOKZKAN>csc Program.cs /unsafe
```

Aşağıda sabit boyutlu bir diziye sahip bir yapı örneği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    StructWithFixedArray s = new StructWithFixedArray();
    s.aNumber = 5;
    s.anArray[0] = 1.0f;
    s.anArray[1] = 2.0f;
    s.anArray[2] = 3.0f;
    s.anArray[3] = 4.0f;
}
public unsafe struct StructWithFixedArray
{
    public int aNumber;
    public fixed float anArray[5];
}
```

Birlikler

Birlikler (**union**), C dili gibi çeşitli dillerde, aynı bellek bölgesinde örtüşebilen birkaç farklı türü içermek için kullanılır. Başka bir deyişle, farklı uzunluklara ve türlere sahip olsalar bile, hepsi aynı bellek adresinde başlayan farklı alanlar içerebilirler. Bunun hem bellek tasarrufu hem de otomatik dönüştürme yapma avantajı vardır. Örnek olarak bir IP adresini düşünün. Dahili olarak, bir IP adresi bir tam sayı olarak gösterilir, ancak bazen **Bayt1.Bayt2.Bayt3.Bayt4** şeklinde farklı bayt bileşenine erişmek isteriz. Bu, **Int32** veya **long** gibi ilkel değer tipler veya kendiniz tanımladığınız diğer yapılar (**struct**) olsun, herhangi bir değer tipi için işe yarar;

```
// See https://aka.ms/new-console-template for more information
using System.Runtime.InteropServices;
var ip = new IPAdresi(new Random().Next());
Console.WriteLine($"{ip} = {ip.ipAdresi}"); //33.62.229.21 = 367345185
ip.Bayt1 = 100;
Console.WriteLine($"{ip} = {ip.ipAdresi}"); //100.62.229.21 = 367345252

[StructLayout(LayoutKind.Explicit)]
struct IPAdresi
{
    // "FieldOffset" ipAdresi tamsayısının adresidir.
    // sizeof(int) 4, sizeof(byte) = 1
    [FieldOffset(0)] public int ipAdresi;
    [FieldOffset(0)] public byte Bayt1; // Yapılın 1. byte'ı offset 0
    [FieldOffset(1)] public byte Bayt2; // Yapılın 2. byte'ı offset 1
    [FieldOffset(2)] public byte Bayt3; // Yapılın 2. byte'ı offset 2
    [FieldOffset(3)] public byte Bayt4; // Yapılın 2. byte'ı offset 3
    public IPAdresi(int address) : this()
    {
        ipAdresi = address;
    }
    public override string ToString() => $"{Bayt1}.{Bayt2}.{Bayt3}.{Bayt4}";
}
```

Bir **yapıyı** (**struct**), C dilindeki **birlik** (**union**) gibi kullanmak için yapının üzerine **[StructLayout(LayoutKind.Explicit)]** niteliğini (**attribute**) eklemek gerekir. Nitelikler aslında **cephe yönelimli programlamanın** (**aspect oriented programming-AOP**) C# diline uygulanmış halidir.

Nitelikler

Nitelikler (**attribute**), C# dilinde programcıların **çalıştırma anında** (**run time**) erişip kullanabilmeleri için, **bileşenlere** (sınıf, alan, yöntem) **tanıtıcı** (**declarative**) ifadeler eklenmesini sağlarlar. Aslında nitelikler, programcıya yardımcı olan ek özelliklerdir.

Nitelik kavramını bir örnekle açıklayalım; Bazı durumlarda sınıflarda tanımlanan alanın adı, veri tabanında farklı, bir başka bileşen için farklı olabilmektedir. Böyle durumlarda bunun veri tabanındaki adını bir nitelik olarak tanımlayıp, **çalıştırma anında** programcı bu niteliği kullanarak, alana ilişkin değeri veri tabanına kaydedebilir veya veri tabanından okuyabilir.

Bu kavram aynı zamanda farklı zamanlarda ve platformlarda geliştirilmiş bileşenleri birlikte kullanmak için programcıya oldukça yardımcı olur.

Tüm nitelikler **System.Attribute** sınıfından devralınmalıdır. Nitelik kullanımınızı **System.AttributeUsage** kullanarak özelleştirebilirsiniz.

```
// See https://aka.ms/new-console-template for more information
using System.Reflection;
//Sınıfın Niteliklerini Alma:
foreach (Attribute? attribute in typeof(NitelikKullananSınıf).GetCustomAttributes())
{
    Console.WriteLine(attribute.GetType());
}
//Nitelik Detayını Alma:
var attribute2 =
typeof(NitelikKullananSınıf).GetCustomAttributes().OfType<AçıklamaNiteliği>().Single();
Console.WriteLine(attribute2.remark);
var attribute3 = typeof(NitelikKullananSınıf).GetCustomAttribute<AçıklamaNiteliği>();
Console.WriteLine(attribute3.remark);

[AttributeUsage(AttributeTargets.All)]
class AçıklamaNiteliği : Attribute
{
    string pri_remark;
    public AçıklamaNiteliği(string comment)
    {
        pri_remark = comment;
    }
    public string remark
    {
        get
        {
            return pri_remark;
        }
    }
}

[AçıklamaNiteliği("Bu sınıf özel nitelik kullanıyor..")]
class NitelikKullananSınıf
{
    // ...
}
```

Bir yöntemi çağırdığımızda iz kaydı alınacak bir nitelik aşağıdaki şekilde yazılabilir;

```
// See https://aka.ms/new-console-template for more information
var service = new MyService();
service.İzKaydıAlınmayacakYöntem(); // Bu yöntemde iz kaydı alınmaz!
service.İzKaydıAlınacakYöntem("Çay Demle"); /* Bu şekilde Çağrılırsa bu yöntemde
iz kaydı alınmaz! */
// İz kaydı alınacak yöntemi çağırmak için aşağıdaki gibi bir yöntem kullanılabilir:
İzKaydıYardımcısı.İzKaydıOlanYöntemiÇağır(service, "İzKaydıAlınacakYöntem",
"Çağrılan Yönteme Bu Parametreyi Gönder");
```

```
[AttributeUsage(AttributeTargets.Method)]
public class İzKaydıNiteliği : Attribute
{
}
public class MyService
{
    [İzKaydıNiteliği]
    public void İzKaydıAlınacakYöntem(string yapılanış)
    {
        Console.WriteLine($"Yöntem, \"{yapılanış}\" işini yapıyor...");
    }
    public void İzKaydıAlınmayacakYöntem()
    {
        Console.WriteLine("Bu yöntemde yapılan işlerin iz kaydı alınmaz!");
    }
}
public static class İzKaydıYardımcısı
{
    public static void İzKaydıOlanYöntemiÇağır(object target,
                                                string methodName,
                                                params object[] parameters)
    {
        var method = target.GetType().GetMethod(methodName);
        if (method == null) throw new ArgumentException("Böyle bir yöntem yok!");
        if (method.GetCustomAttributes(typeof(İzKaydıNiteliği), true).Any())
        {
            Console.WriteLine($"[LOG]:Çağrılacak Yöntemin;");
            Console.WriteLine($"Adı: {method.Name}");
            Console.WriteLine($"Pacapetreleri:{string.Join(", ", parameters)}");
        }
        method.Invoke(target, parameters); //Yöntemi Çağır!
    }
}
```

Bu manuel yaklaşım sınırlıdır ve büyük projeler için ideal değildir. ASP.NET Core için **ActionFilterAttribute** veya **ILogger** kullanılır. Gerçek Cephe Yönelimli Programlama (AOP) tabanlı günlük kaydı için **PostSharp/Fody** kullanılır. Gerçek **günlük (log)** çıktısı için **Serilog**, **NLog** veya **log4net** bileşenlerinden biri kullanılır.

Bu tür bileşenleri Visual Studio üzerinde projemize dahil etmek için *Tools → NuGet Package Manager* seçilerek açılan konsolda komut vererek bileşenleri projemize yüklememiz gerekir. Aşağıda **Foddy** bileşenini projemize dahil etmek için gerekli **paket yöneticisi (package manager)** komutları verilmiştir;

```
Install-Package Fody
Install-Package MethodDecorator.Fody
```

Eğer .NET Core CLI ile proje oluşturulacak ise aşağıdaki adımlar boş bir klasörde yazılarak takip edilir;

```
D:\CSharp\Samples\foddy>dotnet new console
D:\CSharp\Samples\foddy>dotnet add package Fody
D:\CSharp\Samples\foddy>dotnet add package MethodDecorator.Fody
```

Bu paket yüklendikten sonra yardımcı sınıfa gerek kalmadan nitelikler doğrudan iz kaydı alabilir olacaktır. Örnek **Program.cs** dosyası aşağıda verilmiştir;

```
// See https://aka.ms/new-console-template for more information
using MethodDecorator.Fody.Interfaces;
using System.Reflection;

var service = new MyService();
service.Process("Hello World");
try
{
}
```

```

        service.Crash();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Caught exception: {ex.Message}");
    }
}
/* Program Çıktısı:
[LOG] Entering MyService.Process with args: Hello World
Processing: Hello World
[LOG] Exiting MyService.Process
[LOG] Entering MyService.Crash with args:
[LOG] Exception in MyService.Crash: Something went wrong!
Caught exception: Something went wrong!
*/

[AttributeUsage(AttributeTargets.Method | AttributeTargets.Constructor |
AttributeTargets.Assembly | AttributeTargets.Module)]
public class LoggerAttribute : Attribute, IMethodDecorator
{
    private MethodBase _method;
    private object[] _args;
    public void Init(object instance, MethodBase method, object[] args)
    {
        _method = method;
        _args = args;
    }
    public void OnEntry()
    {
        Console.WriteLine($"[LOG] Entering {_method.DeclaringType.Name}.{_method.Name}"+
            $" with args: {string.Join(", ", _args)}");
    }
    public void OnExit()
    {
        Console.WriteLine($"[LOG] Exiting {_method.DeclaringType.Name}.{_method.Name}");
    }
    public void OnException(Exception exception)
    {
        Console.WriteLine($"[LOG] Exception in {_method.DeclaringType.Name}.{_method.Name}:"+
            $" {exception.Message}");
    }
}

public class MyService
{
    [Logger]
    public void Process(string message)
    {
        Console.WriteLine($"Processing: {message}");
    }
    [Logger]
    public void Crash()
    {
        throw new InvalidOperationException("Something went wrong!");
    }
}

```

İSTISNA YÖNETİMİ

Yapısal Programlamada Hata Yönetimi

Yapısal programlamada sorun olan, hata ayıklama kodu ile işlevsel kodun iç içe olması durumudur. Nesne yönelimli programlamanın getirdiği yenilikle bu sorun tarihe karışmıştır. Yapısal programlamada hatalı bir duruma yol açmamak için ya da hatalı bir durumla karşılaşıldığında programdan çıkılır. Aşağıda buna ilişkin bir C programı örneği verilmiştir;

```
#include <stdio.h>
int faktoriyel(int sayi) {
    if (sayi<0) return -1;
    if(sayi <= 1) return 1;
    return sayi * faktoriyel(sayi - 1);
}
int main() {
    int okunan,deger,hata;
    do {
        printf("Bir Sayı Giriniz:");
        scanf("%d",&okunan);
        deger=faktoriyel(okunan);
        printf("%d nin faktöriyeli: %d\n", okunan, deger);
        if (deger==-1) {
            hata=100;
            break; // exit(100);
        }
    } while (okunan!=0);
    return hata;
}
/* Programın Çıktısı:
Bir Sayı Giriniz:-1
-1 nin faktöriyeli: -1
...Program finished with exit code 100
*/
```

İstisna (Özel Durum) Yönetimi

Nesne Yönelimli Programlamada ise **istisna (özel durum)** (**exception**), bir programın yürütülmesi sırasında ortaya çıkan beklenmeyen bir sorundur. İstisna, programın **çalışması sırasında** (**run time**) oluşur. Beklenmeyen durumlarda imal edilen **istisna nesneleri** (**exception object**) vardır. İstisna iki türlü ortaya çıkar;

- **Eşzamanlı** (**synchronous**): Giriş verilerindeki bir hata nedeniyle bir şeylerin ters gitmesi veya programın çalıştığı mevcut veri türünü işleme durumunda oluşan istisnalar (örneğin bir sayıyı sıfıra bölmek).
- **Eşzamanlı olmayan** (**asynchronous**): Disk arızası, klavye kesintileri vb. gibi yazılan programın kontrolü dışındaki istisnalar.

İstisnai durumları yönetmek için **throw**, **try** ve **catch** anahtar kelimeleri kullanılır;

- **try** talimatı: İçerisinde istisnai bir duruma düşüleceğimizi belirten bir kod bloğunu temsil eder. Bu blokta bir veya daha fazla **catch** bloğu takip eder.
- **catch** talimatı: **try** bloğunda bir istisnai duruma düşüldüğünde yürütülecek bir kod bloğunu temsil eder. İstisnayı işlemek için kullanılan kod, **catch** bloğunun içine yazılır.
- Bir istisnai duruma **throw** anahtar sözcüğü kullanılarak da düşülebilir. Bir program bir **throw** ifadesiyle karşılaştığında hemen içinde bulunduğu **try** bloğu dışına çıkılır ve ilgili istisnayı işlemek için eşleşen bir **catch** bloğu bulmaya başlar.
- Bir **try** bloğu içinde birden fazla **throw** ifadesi bulunabilir.

```
try {
    /*
     * ...
     * İstisnai Durumun Ortaya Çıkabileceği Kod Bloğu...
     * ...
     */
    throw SomeExceptionType("İstisnai Durum Açıklaması");
    /*
     * throw an exception: istisnai bir duruma düşme
     */
} catch (Exception e) {
    /*
     * catch bloğu try bloğundaki istisnayı yakalamak ve gerekli işlemi yapmak için kullanılır.
     */
}
```

İstisnaları İşleme

Özel durumları işlemeye niçin ihtiyaç duyarız?

- Hata işleme kodunun işlevsel koddan (iş mantığını içeren koddan) ayrılması için.
- Yöntemler yalnızca seçtikleri istisnaları işleyebilir. Bir yöntemde birden çok istisnai duruma düşülebilir. Ancak bunlardan bazılarını **catch** bloğunda işlemeyi seçebilir. Geri kalanlarını yöntemi çağıran yere bırakabilir.
- Hata Türlerinin Gruplanması: C# dilinde yeni bir istisna sınıfı tanımlanabileceği gibi mevcut istisna sınıflarından da istisna nesnesi imal edilebilir.

Aşağıda dizi sınırları ile ilgili bir program örneği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
int[] array = {10, 20, 30 };
try
{
    array[4]=40;
}
catch (Exception e)
{
    Console.WriteLine("Dizi indisi sınırların dışında!");
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine($"Hata Verisi:{e.Data}");
    Console.WriteLine($"Hataya Sebep Olan İç Hata:{e.InnerException}");
}
/* Programın Çıktısı:
Dizi indisi sınırların dışında!
Hata Mesajı:Index was outside the bounds of the array.
Hata Verisi:System.Collections.ListDictionaryInternal
Hataya Sebep Olan İç Hata:

...Program finished with exit code 0
*/
```

Yukarıdaki örnekte **try** bloğunda izin verilmeyen indis istisnai bir durum fark ortaya çıkıyor ve altyapı bir istisna nesnesi imal ediliyor ve (istisna havuzuna) **fırlatılıyor** (**throw an exception**). **catch** bloğunda ise (havuzdan) **istisna nesnesi yakalanarak** (**catch an exception**) işleniyor. Örnekte standart **Exception** tipindeki **e** istisna nesnesi kullanılmıştır.

Ön Tanımlı İstisnalar

Ön tanımlı bazı istisna tipleri vardır ve aşağıdaki tabloda verilmiştir;

İstisna Tipi	Açıklama
System.ArithmeticException	ve System.OverflowException gibi System.DivideByZeroException aritmetik işlemler sırasında oluşan özel durumlar için bir temel sınıf.

İstisna Tipi	Açıklama
<code>System.ArrayTypeMismatchException</code>	Depolanmış öğenin türü dizinin türüyle uyumsuz olduğundan bir dizideki depo başarısız olduğunda oluşturulur.
<code>System.DivideByZeroException</code>	Bir sayı değerini sıfıra bölme girişi gerçekleştirildiğinde oluşturulur.
<code>System.IndexOutOfRangeException</code>	Sıfırdan küçük veya dizinin sınırlarının dışında olan bir dizin aracılığıyla bir diziye dizine ekleme girişiminde bulunuldu.
<code>System.InvalidCastException</code>	Bir temel tür veya arabirimden türetilmiş bir türe açık dönüştürme çalışma zamanında başarısız olduğunda oluşturulur.
<code>System.NullReferenceException</code>	Referansın <code>null</code> , yani başvuru nesnenin gerekli olmasına neden olacak şekilde kullanıldığında oluşturulur.
<code>System.OutOfMemoryException</code>	Bellek ayırma girişi (aracılığıyla <code>new</code>) başarısız olduğunda oluşturulur.
<code>System.OverflowException</code>	Bir bağlam içindeki <code>checked</code> aritmetik işlem taşıdığından oluşturulur.
<code>System.StackOverflowException</code>	Yürütme yığını çok fazla bekleyen çağrıya sahip olarak tükendiğinde oluşturulur; genellikle çok derin veya ilişkisiz özyinelemenin göstergesidir.
<code>System.TypeInitializationException</code>	<code>static</code> bir alan/değişken <code>başlatıcı</code> (<code>initialize</code>) veya statik alan yapıcı bir istisna oluşturur ve onu yakalamak için kullanılır

Tablo 21. İstisna (Özel Durum) Tipleri

İstisnai Durumun Belirlenmesi

Sistem tarafından istisnaların yakalanması yerine istisnaları biz `throw` anahtar kelimesiyle yeni bir istisna nesnesi imal edip yönetebiliriz. Anı örnek bu durumda aşağıdaki gibi yazılabilir;

```
// See https://aka.ms/new-console-template for more information
int[] array = {10, 20, 30 };
try
{
    int indis = 4;
    if (indis >= array.Length) // İstisnai durum
    {
        //İstisna nesnesi oluşturulup (istisna havuzuna) fırlatılıyor
        throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
    }
    array[indis]=40;
}
catch (Exception e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine($"Hata Verisi:{e.Data}");
    Console.WriteLine($"Hataya Sebep Olan İç Hata:{e.InnerException}");
}
```

Bir `try` bloğunda birden fazla istisna nesnesi (istisna havuzuna) fırlatılabilir; Aşağıdaki örnekte `try` bloğunda birden fazla istisnai nesnesi fırlatılmıştır.

```
// See https://aka.ms/new-console-template for more information
int[] array = {10, 20, 30 };
try
{
    int indis = 2;
    if (indis >= array.Length) //koşul sağlanmaz!
    {
        throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
    }
}
```

```

    array[2]=40;
    int bolen=0, bolunen=100;
    if (bolen == 0)
    {
        throw new DivideByZeroException("Bölme işlemi sıfıra bölünemez!");
    }
}
catch (IndexOutOfRangeException e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("İndis Hatası Olunca Neler Yapılmalı? Burada Yazılır.");
}
catch (DivideByZeroException e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("Sıfıra Bölme Olunca Neler Yapılmalı? Burada Yazılır.");
}
/*Program Çıktısı:
Hata Mesajı:Bölme işlemi sıfıra bölünemez!
Sıfıra Bölme Olunca Neler Yapılmalı? Burada Yazılır.
*/

```

Programda ilk karşılaşılan özel durum sıfıra bölme durumudur ve **throw** ile (istisna havuzuna) **DivideByZeroException("Bölme işlemi sıfıra bölünemez!");** nesnesi fırlatılır. Bu durumda **catch** bloklarına sırasıyla bakılır ilgili özel durum nesnesini işleyecek olan **catch** bloğu icra edilir.

when Süzgeci

Bazı durumlarda **when** süzgeci kullanılabilir;

```

// See https://aka.ms/new-console-template for more information
int[] array = { 10, 20, 30 };
try
{
    int indis = 2;
    if (indis >= array.Length) //koşul sağlanmaz!
    {
        throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
    }
    array[2] = 40;
    int bolen = 0, bolunen = 100;
    if (bolen == 0)
    {
        throw new DivideByZeroException("Bölme işlemi sıfıra bölünemez!");
    }
}
catch (Exception e) when (e is IndexOutOfRangeException || e is DivideByZeroException)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("Hata Olunca Neler Yapılmalı? Burada Yazılır.");
}

```

Kullanıcı Tanımlı İstisna Sınıfları

Aşağıda Kişi sınıfı yaş özelliğine ilişkin istisna içeren bir kod örneği verilmiştir;

```

// See https://aka.ms/new-console-template for more information
using System.Diagnostics; // StackTrace sınıfı için gerekli
int[] array = { 10, 20, 30 };
try
{
    int indis = 4;
    if (indis >= array.Length)
    {

```

```

        //throw new İndisİstisnası(); //Bu satır yada aşağıdaki
        string currentFile = new StackTrace(true).GetFrame(0).GetFileName();
        int currentLine = new StackTrace(true).GetFrame(0).GetFileLineNumber();
        throw new Dosyaİsimliİndisİstisnası(currentFile, currentLine);
    }
    array[indis] = 40;
}
catch (İndisİstisnası e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("İndis Hatası Olunca Neler Yapılmalı? Burada Yazılır.");
}
catch (Dosyaİsimliİndisİstisnası e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    // Hata mesajı: D:\CSharp\Samples\Program.cs Dosyası 12 Satırında İndis Hatası!
    Console.WriteLine($"Hata Mesajı:{e.FileName}");
    // Hata mesajı: D:\CSharp\Samples\Program.cs
    Console.WriteLine($"Hata Mesajı:{e.LineNumber}");
    // Hata mesajı: 12
}

public class İndisİstisnası : Exception
{
    public İndisİstisnası() : base("Dizide olmayan bir elemana indis belirlendi!")
    { }
}

public class Dosyaİsimliİndisİstisnası : Exception
{
    public Dosyaİsimliİndisİstisnası(string fileName, int lineNumber) :
        base($"{fileName} Dosyası {lineNumber} Satırında İndis Hatası!")
    {
        FileName = fileName;
        LineNumber = lineNumber;
    }
    public string FileName { get; private set; }
    public int LineNumber { get; private set; }
}

```

finally Bloğu

finally bloğu, bir **try** blokta gerçekleştirilen eylemleri temizlemenizi sağlar. Varsa, **finally** bloğu en son bloktur ve eşleşen **catch** bloklardan sonra icra edilir. Özel bir durum/istisna oluşturma veya özel durum/istisna tipiyle eşleşen bir **catch** blok bulunması fark etmeksizin **finally** bloğu her zaman çalışır.

```

try
{
    /* istisna olabilecek kod burada yazılır. */
}
catch (Exception e)
{
    /* istisnayı yönetme kodu buraya yazılır. */
}
finally
{
    /* Bir istisna atılmış/yakalanmış olsun veya olmasın, yürütülecek kod buraya yazılır. */
}

```

catch bloğu programcı tarafından kodlanmayabilir. Dosyalarla ilgili bir örnek aşağıda verilmiştir;

```

FileStream f = null;
try
{

```



```
f = File.OpenRead("file.txt");
}
finally
{
    f?.Close(); /* f null olabilir, bu nedenle null koşullu işlecini kullanın.
                Çünkü dosya bir şekilde açılmamış olabilir. */
}
```

İstisnayı Tekrar Fırlatma

Bazı durumlarda istisna nesnesinin yakalanıp **catch** bloğunda işlenmesi sırasında mevcut istisna nesnesini **throw** ile (istisna havuzuna) **tekrar fırlatmak** (**re-throw**) gerekir.

```
try
{
    //...
}
catch (Exception ex)
{
    //...
    throw;
    //yada throw ex;
    //derleyiciye icra akışının asla try bloğundan ayrılmadığını bildirmek için yapılır.
}
```

Katmanlı mimaride bir katmandan diğerine istisnaları iletmek için bu yöntem kullanılır. Genellikle iç katmanlarda hatanın gösterileceği bir kullanıcı ara yüzü bulunmaz.

Özel Durum İşleme Kuralları

Kontrol Akışını İstisna Kodları ile Gerçekleştirmeyin

İstisna/özel durumları işleyen kod ile iş mantığını içeren kod nesne yönelimli paradigmada her zaman ayrı olmalıdır! Bunun için;

- İş mantığını içeren **kontrol akışı** (**control flow**) istisna kodlarıyla yapılmamalıdır. Bunun yerine **kontrol işlemleri** (**control operations**) ile iş mantığı gerçekleştirilmelidir.
- Ayrıca Bir kontrol akışı, if-else ifadeleriyle açıkça yapılabiliriyorsa, okunabilirliği ve performansı azalttığı için istisnalar kullanılmaz.

Aşağıdaki örneği göz önüne alalım;

```
//Bunu YAPMA!
object myObject;
void NesneYaz()
{
    Console.WriteLine(myObject.ToString());
}
```

Bu örnekle **Console.WriteLine** satırı icra edilmeye çalışıldığında **NullReferenceException** istisnası oluşur. Bunu yönetmek için aşağıdaki gibi bir kod yazmamalıyız;

```
// Bunu YAPMA!
object myObject;
void NesneYaz()
{
    try
    {
        Console.WriteLine(myObject.ToString());
    }
    catch (NullReferenceException ex)
    {
        // eğer object'in yeni bir örneğini oluşturup myObject'e atarsam sorunu çözebilirim.
        myObject = new object();
        // artık myObject ile çalışmaya devam edebilirim
    }
}
```

```
NesneYaz();
}
}
```

Doğru olan kodlama aşağıdaki gibi olmalıdır;

```
// Bunu YAP!
object myObject;
void NesneYaz()
{
    if(myObject == null)
        myObject = new object();
    // Programın icrası bu noktaya ulaştığında, myObject'in null olmadığından emin oluruz
    NesneYaz();
}
```

Zorunlu Olmadıkça İstisnayı Tekrar İmal Etmeyin

İstisnaları tekrar imal etmek (re-throw) pahalıdır. Performansı olumsuz etkiler! Rutin olarak başarısız olan kodlar için, performans sorunlarını en aza indirmek için **tasarım desenlerini** (design pattern) kullanabilirsiniz. Microsoft sitesinde yer alan İstisnalar ve Performans konusu istisnaların performansı önemli ölçüde etkileyebileceği zamanlarda faydalı olan iki tasarım desenini açıklamaktadır.²¹

İz Kaydı Olmadan İstisnaları Görmezden Gelmeyin

Aşağıdaki gibi bir istisna işleme kötü bir uygulamadır;

```
try
{
    //...
}
catch(Exception ex) { } // Boş catch bloğu kötü uygulamadır.
```

İstisnaları görmezden gelmek o anı kurtaracaktır ancak daha sonra sürdürülebilirlik için bir kaos yaratacaktır. İstisnaların **iz kaydını** (log) günlüğe kaydederken, istisna örneğini her zaman kaydetmelisiniz. Böylece tüm yığın izi günlüğe kaydedilir ve yalnızca istisna mesajı kaydedilmez! Aşağıdaki gibi bir kod daha doğru olabilir;

```
try
{
    //...
}
catch(NullException ex)
    LogManager.Log(ex.ToString());
}
```

İşleyemeyeceğiniz İstisnaları Yakalamayın

İşlemeyeceğiniz istisnaları yakalamayın. Sadece yakalamayın. Yalnızca o konumda işleyebildiğiniz bir istisnayı yakalamalısınız. Bunun yerine, iyi bir çözümünüz olana veya ana fonksiyona ulaşana kadar bunların çağrı ağacında yukarı doğru ilerlemesine izin verin, bu durumda hata bilgilerinin doğru şekilde yönlendirildiğinden emin olabilirsiniz.

²¹ [https://learn.microsoft.com/en-us/previous-versions/dotnet/netframework-4.0/ms229009\(v=vs.100\)?redirectedfrom=MSDN](https://learn.microsoft.com/en-us/previous-versions/dotnet/netframework-4.0/ms229009(v=vs.100)?redirectedfrom=MSDN)

TIP DÖNÜŞÜMLERİ VE GÖSTERİCİLER

Tip Dönüşümleri

C# dilinde derleme öncesi bir değişken bildirildikten sonra, bu tip örtük olarak bir başka değişkenin tipine dönüştürülebilir olmadığı sürece tekrar bildirilemez veya başka bir tipte bir değer atanamaz. Örneğin, `string` örtük olarak `int` veri tipine dönüştürülemez. Bu nedenle, bir değişken `int` olarak bildirdikten sonra, aşağıdaki koddaki gibi, ona `"Merhaba"` dizgisini atayamazsınız;

```
int i;
i = "Merhaba"; // error CS0029: can't implicitly convert type 'string' to 'int'
```

Ancak bazen bir değeri başka bir tipin değişkenine veya yöntem parametresine kopyalamamız gerekebilir. Örneğin, `double` parametresi bekleyen bir yöntemde geçirmemiz gereken bir tam sayı değişkenimiz olabilir. Bir başka örnek olarak bir `sınıf örneğini` (instance) bir `ara yüz` (interface) tipinin değişkenine atamamız gerekebilir. Bu tür işlemlere tip dönüşümleri denir;

- **Örtük dönüşümler** (implicit conversion): Dönüştürme her zaman başarılı olduğundan ve veri kaybı olmadığından özel bir söz dizimi gerekmez. Örnek olarak küçükten daha büyük tam sayı türlerine dönüştürmeler ve türetilmiş sınıflardan temel sınıflara dönüştürmeler verilebilir.
- **Açık dönüşümler** (explicit conversion) veya **zorlama** (cast): Açık dönüşümler bir atama ifadesi gerektirir. Dönüşümde bilgi kaybolabileceği veya dönüştürmenin başka nedenlerle başarılı olmayabileceği durumlarda atama gereklidir. Buna örnek olarak da daha az duyarlıya veya daha küçük bir belleğe sahip bir tipe sayısal dönüştürme ve temel sınıf örneğinin türetilmiş bir sınıfa dönüştürülmesi verilebilir.
- Kullanıcı tanımlı dönüşümler: Kullanıcı tanımlı dönüştürmeler, temel sınıf türetilmiş sınıf ilişkisi olmayan özel türler arasında açık ve örtük dönüştürmeleri etkinleştirmek için tanımlayabileceğiniz özel yöntemler kullanır.
- Yardımcı sınıflarla dönüşümler: Uyumlu olmayan tipler arasında dönüştürme yapmak için yardımcı sınıflar kullanılır. `System.Convert`, `Parse`, `Int32.Parse` ve `System.BitConverter` bu yardımcı sınıf ve yöntemleridir.

Örtük Dönüşümler

Yerleşik sayısal tipler için, depolanacak değer kırılmadan veya yuvarlanmadan değişkene sığabildiğinde **örtük** (implicit) dönüştürme yapılabilir. Tam sayı tipleri için bu, kaynak tipi aralığının hedef tipi için aralığın uygun bir alt kümesi olduğu anlamına gelir. Örneğin, `long` (64 bit tamsayı) türünde bir değişken, `int` (32 bit tamsayı) tarafından depolanabilir herhangi bir değeri depolayabilir.

Aşağıdaki örnekte, derleyici `uzunTamsayı` değişkenine `tamsayı` değerini atamadan önce örtük olarak `long` tipine dönüştürür. Tüm örtük sayısal dönüştürmelerin tam listesi için dipnottaki linke bakabilirsiniz²².

```
int tamsayı = 2147483647;
long uzunTamsayı = tamsayı;
```

Referans tipler için, örtük dönüştürme her zaman bir sınıftan doğrudan veya dolaylı temel sınıflarından veya arabirimlerinden herhangi birine var olur. Türetilmiş bir sınıf her zaman bir temel sınıfın tüm üyelerini içerdiğinden özel söz dizimi gerekmez.

```
Derived d = new Derived();
Base b = d; // Türetilmiş sınıf, her zaman taban sınıfa örtük olarak dönüştürülür.
```

²² <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/numeric-conversions>

Açık Dönüşümler

Bilgi kaybı riski olmadan dönüştürme yapılamazsa, derleyiciye açık bir dönüştürme gerçekleştirmenizi bildirmeniz gerektirir. Açık (explicit) dönüşümler, Derleyiciye dönüştürmeyi amaçladığınızı ve veri kaybı olabileceğini veya atamanın çalışma zamanında başarısız olabileceğini fark ettiğinizi açıkça bildirmenin bir yoludur. Bir atama gerçekleştirmek için, dönüştürülecek değerin veya değişkenin önündeki parantez içinde atama yaptığınız veri tipi belirtilir;

```
double x = 1234.7;
int a;
a = (int) x; // double tipini int tipine dönüştürme için zorlanıyor (cast).
Console.WriteLine(a); //1234 Ondalık kısmı göz ardı edildi!
```

Bu program, zorlama (cast) olmadan derlenmez. Referans tipler için, bir taban tipten türetilmiş tipe dönüştürmeniz gerekiyorsa zorlama yine gereklidir;

```
Derived d = new Derived();// Türetilmiş tipten bir nesne imal ediliyor.
Base b = d; // Örtük (implicit) olarak tanban tipe dönüştürülebiliyor.

/*
Ancak bu taban tipteki b nesnesi türetilmiş tipe dönüştürülecek ise açık olarak dönüştürülmesi
gerekir.
*/
Derived d2 = (Derived)b;
```

Referans tipleri arasındaki dönüştürme işlemi, temel alınan nesnenin çalışma zamanı (run time) tipini değiştirmez. Yalnızca bu nesneye referans olarak kullanılan değerin tipini değiştirir. Yani çok biçimlilik (polymorphism) burada uygulanır.

Bazı referans tipi dönüştürmelerinde, derleyici bir zorlamanın geçerli olup olmadığını belirleyemez. Doğru derlenen bir açık dönüşüm işleminin çalışma zamanında başarısız olması mümkündür;

```
// See https://aka.ms/new-console-template for more information
Canlı canlı = new Hayvan();
Bitki bitki = (Bitki)canlı; // Derlenir ama çalıştırma anında hata olur:InvalidCastException

class Canlı
{
    public virtual string ToString() => "Ben Bir Canlıyım.";
}
class Hayvan : Canlı {
    public void HareketEt() => System.Console.WriteLine("Hareket Ediyorum..");
}
class Bitki : Canlı {
    public void FotosentezYap() => System.Console.WriteLine("Fotosentez Yapıyorum..");
}
```

Kullanıcı Tanımlı Dönüşümler

Kullanıcı tanımlı bir veri tipi, aynı iki tip arasında standart bir dönüşüm olmadığı sürece, başka bir tipe özel olarak örtük (implicit) veya açık (explicit) dönüşüm tanımlayabilir. Örtük dönüşümler, çağırılması için özel bir sözdizimi gerektirmez. Bu dönüşümler atamalarda ve yöntem çağrılarında meydana gelebilir. Önceden tanımlanmış örtük dönüşümleri her zaman başarılı olur ve asla bir istisna oluşturmaz. Kullanıcı tanımlı örtük dönüşümler de bu şekilde davranmalıdır.

Özel bir dönüşüm bir istisna oluşturabilir veya bilgi kaybedebilirse, bunu açık bir dönüşüm olarak tanımlamamız gerekir.

is ve **as** işleçleri kullanıcı tanımlı dönüşümleri dikkate almaz. Kullanıcı tanımlı açık bir dönüşümü çağırmaq için bir dönüştürme ifadesi kullanılması gerekir.

Sırasıyla örtük veya açık bir dönüşümü tanımlamak için **implicit operator** veya **explicit operator** anahtar sözcükleri kullanılır. Bir dönüşümü tanımlayan tip, o dönüşümün kaynak tipi veya hedef tipi olmalıdır. İki kullanıcı tanımlı tip arasındaki bir dönüşüm, iki tipten herhangi birinde tanımlanabilir.

```
// See https://aka.ms/new-console-template for more information
var d = new OndalıkRakam(8);
byte sayı = d;
Console.WriteLine(sayı); // 8
OndalıkRakam rakam = (OndalıkRakam) sayı;
Console.WriteLine(rakam); // 8

public readonly struct OndalıkRakam
{
    private readonly byte rakam;
    public OndalıkRakam(byte b)
    {
        if (b > 9)
        {
            throw new ArgumentOutOfRangeException(nameof(b),
                "Ondalık Rakam 9 dan büyük olamaz..");
        }
        this.rakam = b;
    }
    public static implicit operator byte(OndalıkRakam d) => d.rakam;
    public static explicit operator OndalıkRakam(byte b) => new OndalıkRakam(b);
    public override string ToString() => $"{rakam}";
}
```

Yardımcı Sınıflarla Dönüşümler

Çoğu programımızda klavyeden girilen metni bir **dizgi** (**string**) veri tipinde değişkenlerle işleriz. Bazı durumlarda da bu dizgileri diğer tiplere dönüştürürüz. Bunun için **System.Convert** sınıfını kullanırız²³.

```
// See https://aka.ms/new-console-template for more information
using System.Globalization;

DateTime zaman = new(1971,9 ,1, 11, 53, 0);
Console.WriteLine(zaman.ToString("yyyy-MM-dd HH:mm")); // 1971-09-01 11:53
Console.WriteLine(zaman.ToString(CultureInfo.InvariantCulture)); // 09/01/1971 11:53:00
Console.WriteLine(zaman.ToString(new CultureInfo("en-us"))); // 9/1/1971 11:53:00 AM
Console.WriteLine(zaman.ToString(new CultureInfo("tr-TR"))); // 1.09.1971 11:53:00
Console.WriteLine(zaman.ToString(new CultureInfo("ja-JP"))); // 1971/09/ 01 11:53:00

string doubleMetin = "12,53";
double sayı = Convert.ToDouble(doubleMetin);
Console.WriteLine(sayı); // 12,53
string intMetin = "6161";
int num = int.Parse(intMetin);
Console.WriteLine(num); // 6161
if (int.TryParse(intMetin, out int result))
{
    Console.WriteLine(result); // 6161
}
else
{
    Console.WriteLine("Dönüştürme Başarısız Oldu.");
}
```

²³ <https://learn.microsoft.com/en-us/dotnet/api/system.convert?view=net-9.0>

Kutulama

Kutulama (**boxing**), bir değer tipini bir nesneye veya bu değer tipi tarafından uygulanan herhangi bir **ara yüz** (**interface**) tipine dönüştürme işlemidir.

```
int i = 123;
object o = i; // o nesnesi i değeri tipini kutulamıştır.
o = 123;
i = (int)o; // i değişkeni kutudan çıkarılmıştır.
object obj = 61;
if (obj is int number)
{
    Console.WriteLine("obj bir tamsayıdır:" + number);
}
```

Göstericiler

C# göstericileri sınırlı bir ölçüde destekler. **Gösterici** (**pointer**), bir bellek adresini tutan bir değişkenden başka bir şey değildir. C# göstericisi yalnızca değer tiplerin ve dizilerin bellek adresini tutmak için bildirilebilir (**declaration**).

1990 yıllarının sonlarına kadar, geliştirme çalışmalarımın çoğu C programlama dilini kullanarak yapıldı. C veya C++ dillerini kullanan programcılar göstericileri kullanmaktan kaçınmazdı. C# ve Java gibi saf nesne yönelimli dilleri kullanmaya başladığımızdan beri göstericiler çok nadir kullanılmaya başlandı ve genellikle buna gerek duyulmamaktadır. Çünkü göstericilerin çokça kullanıldığı dinamik veri yapılarını sağlayan kütüphaneler doğrudan bize altyapı tarafından sunulmuştur. Ancak göstericiler, bazı durumlarda düşük düzey iş ve işlemler için gerekli olabilir. Buna örnek olarak P/Invoke verilebilir. P/Invoke, **yönetilen** (**managed**) kodunuzdan **yönetilmeyen** (**unmanaged**) kütüphanelerdeki yapılara, **geri aramalara** (**callback**) ve fonksiyonlara erişmenizi sağlayan bir teknolojidir²⁴.

Referans tiplerin aksine, gösterici tipleri varsayılan **çöp toplama** (**garbage collection**) mekanizması tarafından izlenmez. Aynı nedenden dolayı göstericilerin bir referans tipe veya hatta bir referans tipi içeren bir **yapı** (**struct**) türüne işaret etmesine izin verilmez. Göstericilerin yalnızca **yönetilmeyen** (**unmanaged**) tipleri, yani tüm temel veri tiplerini (**sbyte**, **byte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**, **char**, **float**, **double**, **decimal**, **bool**), **enum** tiplerini, diğer gösterici tiplerini ve yalnızca yönetilmeyen tipleri içeren yapıları (**struct**) işaret edebileceğini söyleyebiliriz.

Gösterici Tipi Bildirme

Bir gösterici tipi bildirmenin genel biçimi aşağıda gösterildiği gibidir:

```
veritipi *gösterici-kimliği;
```

Yıldız (*) karakteri, **referanstan çıkarma işleci** (**de-reference operator**) olarak bilinir. Bir **int** tipinin adresini tutabilen bir gösterici değişkeni **ptrToInt** aşağıdaki gibi bildirir;

```
int* ptrToInt;
```

Referans işleci (&) bir değişkenin bellek adresini almak için kullanılabilir. Bir göstericiye ilke değer vermek için kullanılabilir. Aşağıdaki örnekte gösterici veri tipiyle, değişkenin veri tipinin aynı olduğuna dikkat edilmelidir.

```
int x = 100;
int* ptrToInt = &x;
Console.WriteLine((int) ptrToInt) // Konsola ptrToInt göstericisinin tuttuğu adres yazılır
Console.WriteLine(*ptr) // Konsola ptrToInt göstericisinin gösterdiği adresteki değer yazılır.
```

C# ifadeleri **güvenli** (**safe**) veya **güvenli olmayan** (**unsafe**) bir bağlamda yürütülebilir. **unsafe** anahtar sözcüğü kullanılarak güvenli olmayan olarak işaretlenen ifadeler **çöp toplayıcısının** (**garbage**

²⁴ <https://learn.microsoft.com/en-us/dotnet/standard/native-interop/pinvoke>

collector) denetimi dışında çalışır. C# dilinde göstericiler içeren herhangi bir kodun güvenli olmayan bir bağlam gerektirdiğini unutulmamalıdır. **Main** dahil tüm yöntemleri güvenli olmayan olarak işaretleyebilir kullanabiliriz.

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    int x = 1;
    int y = 2;
    int* ptr1 = &x;
    int* ptr2 = &y;
    Console.WriteLine((int)ptr1); //871885156
    Console.WriteLine((int)ptr2); //871885152
    Console.WriteLine(*ptr1); //1
    Console.WriteLine(*ptr2); //2
}
Sınıf nesne = new Sınıf();
nesne.GüvenliOmlayanYöntem();
class Sınıf
{
    public unsafe void GüvenliOmlayanYöntem()
    {
        int x = 3;
        int y = 4;
        int* ptr1 = &x;
        int* ptr2 = &y;
        Console.WriteLine((int)ptr1); //871885052
        Console.WriteLine((int)ptr2); //871885048
        Console.WriteLine(*ptr1); //3
        Console.WriteLine(*ptr2); //4
    }
}
```

C# **çöp toplayıcısı** (**garbage collector**), çöp toplama işlemi sırasında nesneleri istediği gibi bellekte taşıyabilir. Çöp toplayıcısına bir nesneyi taşımamasını söylemek için özel bir **fixed** anahtar sözcüğü kullanılır. Böylece işaret edilen değer türlerinin konumunu bellekte sabitlenir. Bu durum **sabitleme** (**pinning**) olarak bilinir. Aşağıda C dili göstericileri gibi bir dizi elemanlarına erişim ve nesne durumuna erişim örneği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    var buffer = new int[1024];
    fixed (int* p = &buffer[0])
    {
        for (var i = 0; i < buffer.Length; i++)
        {
            *(p + i) = i;
        }
    }

    Sınıf nesne = new Sınıf();
    nesne.alan = 32;
    nesne.KonsolaAlanıYaz();
    fixed (int* alan = &nesne.alan)
    {
        *alan += 3;
    }
    nesne.KonsolaAlanıYaz();
}

class Sınıf
```



```
{
    public Sınıf() { }
    public int alan; //field
    public void KonsolaAlanıYaz()
    {
        Console.WriteLine("alan = {0}", alan);
    }
}
```

Gösterici Aritmetiği

Göstericilerde toplama ve çıkarma tamsayılardan farklı şekilde çalışır. Bir gösterici artırıldığında veya azaltıldığında, işaret ettiği adres, referans tipinin bellek boyutu kadar artırılır veya azaltılır.

Örneğin, `System.Int32` için takma ad olan `int` tipi 4 bayt boyutundadır. Bir `int`, 0 adresinde saklanabiliyorsa, sonraki `int` 4 adresinde saklanır;

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    int* ptr = (int*)IntPtr.Zero;
    Console.WriteLine(new IntPtr(ptr)); // 0
    ptr++;
    Console.WriteLine(new IntPtr(ptr)); // 4
    ptr++;
    Console.WriteLine(new IntPtr(ptr)); // 8
}
```

Benzer şekilde, `System.Int64` için takma ad olan `long` tipi 8 bayt boyutundadır. Eğer bir `long` 0 adresinde saklanabiliyorsa, sonraki `long` 8 adresinde saklanır;

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    long* ptr = (long*)IntPtr.Zero;
    Console.WriteLine(new IntPtr(ptr)); // 0
    ptr++; // bellekteki bir sonraki long adresini alır
    Console.WriteLine(new IntPtr(ptr)); // 8
    ptr+=4; // bellekteki 4 sonraki long adresini alır
    Console.WriteLine(new IntPtr(ptr)); // 40
}
```

Hiçbir değeri olmayan veri tipi anlamındaki `void` türü özeldir ve `void` göstericileri de özeldir. Veri tipi bilinmediğinde veya önemli olmadığında her şeyi ifade eden `void` göstericileri kullanılırlar. C ve C++ dilinin aksine boyuttan bağımsız yapıları nedeniyle, `void` göstericileri artırılmaz veya azaltılamaz:

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    void* ptr = (void*)IntPtr.Zero;
    Console.WriteLine(new IntPtr(ptr));
    ptr++; // Hata: The operation in question is undefined on void pointers
    Console.WriteLine(new IntPtr(ptr));
    ptr++; // Hata: The operation in question is undefined on void pointers
    Console.WriteLine(new IntPtr(ptr));
}
```

Herhangi bir gösterici tipi, **örtük** (**implicit**) bir dönüşüm kullanılarak `void*` tipine atanabilir;

```
int* p1 = (int*)IntPtr.Zero;
void* ptr = p1; //örtük olarak void* tipne atama.
```

Bunun tersi durum ancak **açık** (**explicit**) olarak mümkündür;

```
int* p1 = (int*)IntPtr.Zero;
void* ptr = p1; //örtük olarak void* tipne atama.
```



```
int* p2 = (int*)ptr; //void* tip int* tipine örtük olarak atanamaz!
```

Referanstan Çıkarma İşleci Veri Tipinin Bir Parçasıdır

C ve C++ dillerinde, bir gösterici değişkeninin bildirimindeki yıldız işareti, bildirilen ifadenin bir parçasıdır. C# dilindeki bildirimdeki yıldız işareti **referanstan çıkarma işleci** (de-reference operator) olup veri tipinin bir parçasıdır;

Aşağıdaki örnekte verilen kod C# dilinde iki **int** göstericisi bildirmesine rağmen C ve C++ dillerinde ilki gösterici, ikincisi değişken olarak bildirilir.

```
int* a,b;
/*
 C ve C++ olarak eşdeğeri: int* a; int b;
 C# eşdeğeri: int* a; int* b;
*/
int *bir, *iki; //Hata:C# dilinde derlenmez.
/*
 Ancak C ve C++ dillerinde geçerli iki farklı göstericidir.
*/
```

Gösterici Üye Erişim İşleci

C ve C++ gibi C# dilinde de bir yapı ya da sınıf örneğinin üyelerine bir gösterici aracılığıyla erişmenin bir yolu olarak **->** işlecini kullanılır.

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    Koordinat v;
    v.X = 5;
    v.Y = 10;
    Koordinat* ptr = &v;
    int x = ptr->X;
    int y = ptr->Y;
    string s = ptr->ToString();
    Console.WriteLine(x); // 5
    Console.WriteLine(y); // 10
    Console.WriteLine(s); // Koordinat
}

struct Koordinat
{
    public int X;
    public int Y;
}
```

KOLEKSİYONLAR

Şu ana kadar en basit **veri yapıları** (**data structure**) olan değişkenlerle karşılaştık. Programlamada en çok kullanılan bir başka veri yapısı da **dizilerdir** (**array**). Diziler **koleksiyonlardan** (**collection**) biridir ve daha önce anlatılmıştır. Koleksiyonlar, dinamik veri yapıları olup **new** anahtar kelimesiyle çalıştırma anında bellekte oluşturulan nesnelerdir.

ArrayList

Dizilerin çalıştırma anında boyutu belirlenir ve daha sonra boyutu değiştirilmez. Gerekğinde boyutu dinamik olarak artırılan bir dizi kullanmak istediğimizde **ArrayList** kullanırız. **ArrayList**, .Net için standart olarak tanımlı **ICollection** **ara yüzünü** (**interface**) uygulayan bir koleksiyondur.

```
// See https://aka.ms/new-console-template for more information
using System.Collections;

ArrayList liste = new ArrayList();
liste.Add("Renault");
liste.Add("Fiat");
liste.Add("Toyota");
Console.WriteLine("Araçlar");
Console.WriteLine("    Adet:    {0}", liste.Count); //3
Console.WriteLine("    Kapasite: {0}", liste.Capacity); //4
Console.WriteLine("    Değerler:");
foreach (Object obj in liste)
    Console.WriteLine("    {0}", obj); //    Renault    Fiat    Toyota
```

Jenerik

C# dili farklı veri tipleriyle çalışan yöntemler, sınıflar, algoritmaları tekrar tekrar yazmamak için bir programlama türü olan **jenerik programlamayı** (**generic programming**) destekler. Dizi dışındaki tüm koleksiyonlar jenerik programa ile çalışır.

Jenerikler (**generic**), veri tiplerine bağlı olmadan algoritma içeren bir yöntem veya sınıf oluşturmamızı sağlar! Jenerik, aynı mantığı yeniden yazmadan bir yöntemin farklı veri tipleri üzerinde çalışmasını sağlayan bir sınıf, yöntem ve algoritma planı tanımlar. Jenerikler, kodunuzda sınıfı veya yöntemi kullanana kadar bir veya daha fazla tip parametresinin belirtilmesini erteleyen sınıflar ve yöntemler tasarlamayı mümkün kılar.

Genel bir veri tipini temsil eden **T** veya **V** parametresi kullanarak, çalışma zamanı dönüştürmelerinin veya **kutulama** (**boxing**) işlemlerinin maliyetine veya riskine katlanmadan diğer istemci kodunun kullanabileceği tek bir sınıf yazabilirsiniz;

```
// See https://aka.ms/new-console-template for more information
GenericList<int> list1 = new(); //int veri tipinde elemanlardan oluşan liste
list1.Add(1);

GenericList<string> list2 = new(); //string veri tipinde elemanlardan oluşan liste
list2.Add("");

GenericList<AClass> list3 = new(); //AClass veri tipinde elemanlardan oluşan liste
list3.Add(new AClass());

public class GenericList<T>
{
    public void Add(T item) { }
}

public class AClass { } // Örnek Bir Sınıf
```

IEnumerator, numaralandırılabilen **ArrayList** gibi tüm genel olmayan koleksiyonlar için temel **ara yüzüdür** (**interface**). **IEnumerator<T>** ise **List<T>** gibi tüm genel numaralandırıcılar için temel ara yüzüdür.

IEnumerable, **GetEnumerator** metodunu uygulayan bir ara yüzdür. **GetEnumerator** metodu, **foreach** gibi koleksiyonda yineleme yapmak için seçenekler sağlayan bir **IEnumerator** döndürür.

```
// See https://aka.ms/new-console-template for more information
using System.Collections;
foreach (var araç in new AraçKoleksiyonu())
{
    Console.WriteLine($"{araç} "); //sedan heçbek siteyşın limuzin
}

public class AraçTipiNumaralandırma : IEnumerator<string>
{
    string[] araçTipleri = new string[4] { "sedan", "heçbek", "siteyşın", "limuzin" };
    int currentIndex = -1;
    public string Current
    {
        get
        {
            if (currentIndex < 0 || currentIndex >= araçTipleri.Length)
                throw new InvalidOperationException();
            return araçTipleri[currentIndex];
        }
    }
    object IEnumerator.Current // Explicit implementation of IEnumerator.Current
    {
        get
        {
            return Current;
        }
    }
    public void Dispose()
    {
    }
    public bool MoveNext()
    {
        currentIndex++;
        if (currentIndex < araçTipleri.Length)
        {
            return true;
        }
        return false;
    }
    public void Reset()
    {
        currentIndex = 0;
    }
}

public class AraçKoleksiyonu : IEnumerable<string>
{
    private AraçTipiNumaralandırma enumerator;
    public AraçKoleksiyonu()
    {
        enumerator = new AraçTipiNumaralandırma();
    }
    public IEnumerator<string> GetEnumerator()
    {
        return enumerator;
    }
    IEnumerator IEnumerable.GetEnumerator()
    // Explicit implementation of IEnumerable.GetEnumerator
    {
        return GetEnumerator();
    }
}
```

```
}  
}
```

yield Anahtar Kelimesi

Bir ifadede **yield** anahtar kelimesini kullandığınızda, içinde görüldüğü yöntem, operatör veya **get** erişimcisinin bir **yineleyici** (**iterator**) olduğunu belirtirsiniz. Bir yineleyiciyi tanımlamak için **yield** kullanmak, özel bir koleksiyon türü için **IEnumerable** ve **IEnumerator** desenini uyguladığınızda açık bir ek sınıfa (bir sayım için durumu tutan sınıf) olan ihtiyacı ortadan kaldırır.

```
// See https://aka.ms/new-console-template for more information  
using System.Collections.Generic;  
public class Program  
{  
    public static void Main()  
    {  
        foreach (int value in Count(start: 4, count: 10))  
        {  
            Console.Write(value+" ");  
        }  
    }  
    public static IEnumerable<int> Count(int start, int count)  
    {  
        for (int i = 0; i <= count; i++)  
        {  
            yield return start + i;  
        }  
    }  
}  
/*Program Çıktısı:  
4 5 6 7 8 9 10 11 12 13 14  
*/
```

Yukarıdaki örnekte **foreach** ifadesi gövdesinin her **yinelemesi** (**iteration**), **Count** yineleyici yöntemine bir çağrı oluşturur. Yineleyici yöntemine yapılan her çağrı, for döngüsünün bir sonraki yinelemesi sırasında gerçekleşen **yield return** ifadesinin bir sonraki icrasına ilerler.

Aşağıda bağlı listelere örnek olarak örnek bir **GenericList** jeneriği verilmiştir;

```
// See https://aka.ms/new-console-template for more information  
public class GenericList<T>  
{  
    private class Node(T t) // T tipinde Düğüm verileri içeren iç sınıf  
    {  
        public T Data { get; set; } = t; // T tipinde özellik (property)  
        public Node? Next { get; set; }  
    }  
    private Node? head; // Bağlı Listede İlk Düğüm  
    public void AddHead(T t)  
    {  
        Node n = new(t);  
        n.Next = head;  
        head = n;  
    }  
    public IEnumerator<T> GetEnumerator()  
    {  
        Node? current = head;  
        while (current is not null)  
        {  
            yield return current.Data;  
            current = current.Next;  
        }  
    }  
}
```

```

}
public class Client
{
    public static void Main()
    {
        GenericList<int> list = new();
        for (int x = 0; x < 10; x++)
        {
            list.AddHead(x);
        }
        foreach (int i in list)
        {
            Console.WriteLine(i);
        }
    }
}

```

Mevcut yineleyici yöntemlerinin işlevselliğinde bir sonlandırma koşulu tanımlanırsa iç döngünün yürütülmesini durdurmak için bir **yield break** çağırılır;

```

public static IEnumerable<int> CountUntilAny(int start, HashSet<int> earlyTerminationSet)
{
    int curr = start;
    while (true)
    {
        if (earlyTerminationSet.Contains(curr))
        {
            yield break; // Yinelemeyi durduracak bir değerle karşılaştık
        }
        yield return curr;
        if (curr == Int32.MaxValue)
        {
            yield break; // taşma olmasın diye yinelemeyi durdurduk.
        }
        curr++;
    }
}

```

HashSet<T> Karma Kümesi

Karma kümesi (**hashset**), üyeleri tekil olan bir jenerik koleksiyondur. Dizilerden farklı olarak karma küme içinde bir değer yalnızca bir defa yer alabilir.

```

// See https://aka.ms/new-console-template for more information
HashSet<int> aidatVerenDaireler = new HashSet<int>();
aidatVerenDaireler.Add(4);
aidatVerenDaireler.Add(6);
aidatVerenDaireler.Add(9);
aidatVerenDaireler.Add(10);
Console.WriteLine($"Aidat Veren Daire Sayısı:{aidatVerenDaireler.Count}");
//Karma Küme Başlatma (İlk Değer Verme)
HashSet<int> aidatVermeyenDaireler = new HashSet<int>() { 1, 2, 3, 5, 8 };
bool vermemişmi = aidatVermeyenDaireler.Contains(8);
Console.WriteLine($"8 Numaralı Daire Aidat Vermemiş Mi:{vermemişmi}");

```

Dictionary<TKey, TValue> Sözlük

Sözlük (**dictionary**) koleksiyonu anahtar-değer ikilisi şeklinde sözlük nesnesi oluşturmak için kullanılan jenerik bir koleksiyondur.

```

// See https://aka.ms/new-console-template for more information
var kişiler = new Dictionary<int, string> { { 10, "İlhan" },

```

```

        { 20, "Mehmet" },
        {30, "Abdullah"} };

Console.WriteLine(kişiler[10]); // "İlhan"
//Console.WriteLine(kişiler[50]); // HATA!:throws KeyNotFoundException
string? adı; //null olabilen string
if (kişiler.TryGetValue(20, out adı))
{
    Console.WriteLine(adı); // "Mehmet"
}
kişiler[40]="Abdullah";
kişiler.Add(50,"Damla");
//kişiler.Add(40, "Damla"); // HATA!:Throws ArgumentException since 40 already exists
foreach (KeyValuePair<int,string> kişi in kişiler)
{
    Console.WriteLine("No={0}, Adı={1}", kişi.Key, kişi.Value);
}
foreach (int no in kişiler.Keys)
{
    Console.WriteLine("No={0}", no);
}
foreach (string adı in kişiler.Values)
{
    Console.WriteLine("Adı={0}", adı);
}

```

SortedSet<T> Sıralı Küme

Sıralı küme (**sortedset**), kümenin elemanlarını sıralı bir şekilde saklamak için kullanılan jenerik bir koleksiyondur. Aynı eleman kümede ikinci defa yer almaz!

```

// See https://aka.ms/new-console-template for more information

var sıralıListe = new SortedSet<int>();
sıralıListe.Add(4);
sıralıListe.Add(3);
sıralıListe.Add(3);// Kümede var eklenmez!
sıralıListe.Add(2);
sıralıListe.Add(1);
foreach (var item in sıralıListe)
{
    Console.Write(item+" ");
}
/*Program Çıktısı:
1 2 3 4
*/

```

List<T> Liste

Öğeleri bir **liste** (**list**) olarak tutar ve öğeler indekse göre eklenebilir, yerleştirilebilir, kaldırılabilir ve adreslenebilir. Listeler bu amaçla kullanılan jenerik bir koleksiyondur.

```

// See https://aka.ms/new-console-template for more information
var liste = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
liste.Add(6);
Console.WriteLine(liste.Count); // 6
liste.RemoveAt(3);
Console.WriteLine(liste.Count); // 5
Console.WriteLine(liste[3]); // 5

```

Stack<T> Yığın

Yığın (**stack**), koleksiyona son giren ilk geri alınacak şekilde dinamik bir veri yapısı sunan ve bu amaçla kullanılan jenerik bir koleksiyondur.

```
// See https://aka.ms/new-console-template for more information
var yığın = new Stack<int>();
yığın.Push(10);
yığın.Push(3);
yığın.Push(5);
yığın.Push(8);
Console.WriteLine(yığın.Peek()); // 8
Console.WriteLine(yığın.Pop()); // 8
Console.WriteLine(yığın.Pop()); // 5
Console.WriteLine(yığın.Pop()); // 3
```

LinkedList<T> Bağlı Liste

Bu koleksiyon **bağlı liste** (**linkedlist**) dinamik veri yapısını sunan ve bu amaçla kullanılan jenerik bir koleksiyondur.

```
// See https://aka.ms/new-console-template for more information
LinkedList<int> bağlıListe = new LinkedList<int>();
bağlıListe.AddLast(3);
bağlıListe.AddLast(5);
bağlıListe.AddLast(8);
bağlıListe.AddFirst(10);
bağlıListe.AddFirst(2);
bağlıListe.RemoveFirst();
bağlıListe.RemoveLast();
foreach (int i in bağlıListe)
    Console.Write(i+" ");
/* Program Çıktısı:
10 3 5
*/
```

Queue<T> Kuyruk

Bu koleksiyon **kuyruk** (**queue**) dinamik veri yapısını sunan ve kullanılan jenerik bir koleksiyondur.

```
// See https://aka.ms/new-console-template for more information
var kuyruk = new Queue<int>();
kuyruk.Enqueue(10);
kuyruk.Enqueue(6);
kuyruk.Enqueue(4);
kuyruk.Enqueue(9);
Console.WriteLine(kuyruk.Peek()); // 10
Console.WriteLine(kuyruk.Dequeue()); // 10
Console.WriteLine(kuyruk.Dequeue()); // 6
Console.WriteLine(kuyruk.Dequeue()); // 4
```

İndisleyiciler

İndisleyiciler (**indexer**), nesneleri bir dizi gibi kullanmayı sağlarlar. İndisleyiciler, köşeli parantez içinde parametresi olan **this** kelimesiyle tanımlanırlar. Özellikler gibi indisleyiciler de **get** ve **set** yöntemleri kullanılır ve yalnız-okunur, yalnız-yazılır olabilirler. Bunların dışında indisleyiciler sanal olarak da tanımlanabilir ve parametresi farklı olmak kaydıyla indisleyicilere **aşırı yükleme** (**overload**) yapılabilir.

```
// See https://aka.ms/new-console-template for more information
List<string> names = new List<string>();
names.Add("Ali");
names.Add("Veli");
```

```
names.Add("İlhan");
for (int i = 0; i < names.Count; i++)
{
    //İndis kullanımı:
    string s = names[i];
    names[i] = s.ToUpper();
    Console.WriteLine(names[i]);
}

public class List<T>
{
    //Sabitler:
    const int defaultCapacity = 4;
    //Alanlar:
    T[] items;
    int count;
    //Yapıcılar:
    public List() : this(defaultCapacity) { }
    public List(int capacity)
    {
        items = new T[capacity];
    }
    //Özellikler:
    public int Count
    {
        get { return count; }
    }
    public int Capacity
    {
        Get { return items.Length; }
        set
        {
            if (value < count) value = count;
            if (value != items.Length)
            {
                T[] newItems = new T[value];
                Array.Copy(items, 0, newItems, 0, count);
                items = newItems;
            }
        }
    }
    //İndisleyici Tanımı:
    public T this[int index]
    {
        get { return items[index]; }
        set { items[index] = value; }
    }
    //Yöntemler:
    public void Add(T item)
    {
        if (count == Capacity) Capacity = count * 2;
        items[count] = item;
        count++;
    }
}
```


LINQ SORGULARI

LINQ Nedir?

Dile Entegre Sorgu (**Language INtegrated Query-LINQ**) anlamına gelir. Çeşitli veri kaynakları ve biçimleri arasında verilerle çalışmak için tutarlı bir model sunarak bir sorgu dilini programlama diline entegre eden bir kavramdır; XML belgelerinde, SQL veri tabanlarında, ADO.NET Veri Kümelerinde, .NET koleksiyonlarında ve LINQ sağlayıcısının sunmuş olduğu biçimlerde verileri sorgulamak ve dönüştürmek için aynı temel kodlama kalıplarını kullanırsınız.

LINQ, çeşitli veri kaynaklarından gelen verileri tek tip bir şekilde sorgulamamıza olanak tanıyan ortak bir söz dizimi sağlar. Bu, tek bir LINQ sorgusu kullanarak SQL Server veri tabanı, XML belgeleri, ADO.NET Veri Kümeleri ve Koleksiyonlar, Jenerik gibi çeşitli veri kaynaklarından veri alabileceğimiz anlamına gelir.

```
// See https://aka.ms/new-console-template for more information
List<İl> iller = new List<İl> {
    new İl { ilTrafikKodu = 1, Adı = "Adana" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul" },
    new İl { ilTrafikKodu = 5, Adı = "Amasya" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara" },
    new İl { ilTrafikKodu = 7, Adı = "Antalya" },
    new İl { ilTrafikKodu = 16, Adı = "Bursa" },
    new İl { ilTrafikKodu = 10, Adı = "Balıkesir" },
    new İl { ilTrafikKodu = 61, Adı = "Trabzon" }
};

// Sorgular Buraya Gelecek

struct İl
{
    public int ilTrafikKodu;
    public string Adı;
}
```

Yukarıdaki gibi bir veri kaynağını göz önüne aldığımızda LINQ sorguları iki şekilde yazılabilir:

1. **Sorgu Şeklinde:** SQL'e benzer ve genellikle SQL'e aşina olanlar için daha okunabilir. Bir **from** ifadesiyle başlar, ardından bir aralık değişkeni gelir ve **where**, **select**, **group**, **join** vb. gibi standart sorgu işlemlerini içerir.

```
from object in datasource
where condition
select object
```

Yukarıdaki sözdizimine sahip sorguya örnek aşağıdaki kod verilebilir;

```
var query = from i in iller
             where i.Adı == "Ankara"
             select i.Adı;
```

2. **Akıcı (fluent) Şekilde:** Uzantı yöntemleri ve lamda ifadeleri kullanır. Daha öz olabilir ve karmaşık sorgular yazarken tercih edilir çünkü okunması ve oluşturulması daha kolay olabilir.

```
datasource.ConditionMethod().SelectionMethod()
```

Yukarıdaki sözdizimine sahip sorguya örnek aşağıdaki kod verilebilir;

```
var query = iller.Where(i => i.Adı == "Ankara").Select(i => i.Adı);
```

IEnumerable ve **IQueryable**, veri işleme ve sorgulama için sıklıkla kullanılan iki **ara yüzdür (interface)**, ancak farklı amaçlara hizmet ederler ve farklı davranışlara sahiptirler. Verilen örnekteki **query** değişkeni **IEnumerable<string>?** veri tipine sahiptir.

```
IEnumerable<int> queryILKodları = iller.Select(i => i.ilTrafikKodu);
```

```
IEnumerable<string> queryIlListesi = iller.Where(i => i.Adı.Contains("an"))
    .Select(i => i.Adı);
```

IEnumerable ve IEnumerator Arayüzleri

LINQ fonksiyonları hem bir **IEnumerable<TSource>** üzerinde çalışır hem de bir **IEnumerable<TResult>** döndürür.

IEnumerable, bir **IEnumerator** nesnesi döndüren **GetEnumerator()** adlı bir yöntemi tanımlayan bir ara yüzdür. Bu ara yüz, **System.Collections** isim uzayında bulunur. .NET Framework'ün önemli bir parçasıdır ve bir nesne koleksiyonu üzerinde **yineleme** (**iteration**) yapmak için kullanılır.

GetEnumerator() yöntemi, **IEnumerable** ara yüzünde tanımlanan tek yöntemdir. Bir **Current** özelliğini (**property**) ve **MoveNext()** ve **Reset()** yöntemlerini açığa çıkararak koleksiyonda yineleme yapma olanağı sağlayan bir **IEnumerator** nesnesi döndürür. Böylece **foreach** ile sorgu sonucunu içeren koleksiyon üzerinde yineleme yapabilmış oluruz.

```
foreach (var item in queryIlKodları)
{
    Console.Write($"{item} ");
} // 1 34 5 6 7 16 10 61
foreach (var item in queryIlListesi)
{
    Console.Write($"{item} ");
} // Adana İstanbul
```

IQueryable Arayüzü

IQueryable, bir veri kaynağından veri sorgulamak için kullanılan bir ara yüzdür. **System.Linq** isim uzayının bir parçasıdır ve önemli bir bileşendir. Bellek içi koleksiyonlar üzerinde yineleme yapmak için kullanılan **IEnumerable** ara yüzünün aksine, **IQueryable**, nesne numaralandırılana kadar sorgunun yürütülmediği veri kaynaklarını sorgulamak için tasarlanmıştır. Bu, özellikle veri tabanları gibi uzak veri kaynakları için yararlıdır ve sorgunun sonucu tarafında yürütülmesine izin vererek verimli sorgulamayı mümkün kılar.

```
IQueryable<İl> bazıİller = iller.AsQueryable().Where(i => i.ilTrafikKodu <30);
foreach (var item in bazıİller) //Sorgu sonucu İl nesneleri döner
{
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
}
/*Program Çıktısı:
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
*/
```

IEnumerable istemci belleğinde yürütülürken, **IQueryable** veri kaynağında yürütülür.

IEnumerable nesneler ve bellek içi verilerle çalışmak için uygundur. **IQueryable** SQL veri tabanları ile etkileşim kurması için uygundur.

IQueryable, veri tabanının verileri optimize etmesine ve filtrelemesine izin verdiği için büyük veri kümeleri için daha iyi performans gösterebilir.

LINQ Genişleme Yöntemleri

Genişleme yöntemleri (**extension method**), yeni bir türetilmiş tip oluşturmadan, yeniden derlemeden veya orijinal türü başka şekilde değiştirmeden mevcut türlere davranışı genişletir ve ekler

Where

```
var filteredResult = iller.Where(item => item.Adı == "Ankara");
foreach (var item in filteredResult)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 6, Adı: Ankara
*/
```

Select

```
var projectedResult = iller.Select(item => new { ilTrafikKodu = 61, Adı = "Trabzon" });
foreach (var item in projectedResult)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/* iller koleksiyonunda ne kadar eleman varsa o kadar yeni il ekler
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
*/
foreach (var item in iller)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 61, Adı: Trabzon
*/
```

OrderBy ve OrderByDescending

```
var sortedResult = iller.OrderBy(item => item.ilTrafikKodu);
foreach (var item in sortedResult)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 61, Adı: Trabzon
*/
var sortedDescendingResult = iller.OrderByDescending(item => item.Adı);
foreach (var item in sortedDescendingResult)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 1, Adı: Adana
*/
```

*/

Group By

```
// See https://aka.ms/new-console-template for more information
List<İl> iller = new List<İl> {
    new İl { ilTrafikKodu = 1, Adı = "Adana" , Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 5, Adı = "Amasya", Bölge = "Karadeniz" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
    new İl { ilTrafikKodu = 7, Adı = "Antalya", Bölge = "Akdeniz"},
    new İl { ilTrafikKodu = 16, Adı = "Bursa", Bölge = "Marmara"},
    new İl { ilTrafikKodu = 10, Adı = "Balıkesir", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 61, Adı = "Trabzon", Bölge = "Karadeniz" }
};
IEnumerable<IGrouping<string, İl>> groupedResult = iller.GroupBy(item => item.Bölge);
foreach (var item in groupedResult)
{
    Console.WriteLine($"{item.Key} "); //Akdeniz Marmara Karadeniz İç Anadolu
}

struct İl
{
    public int ilTrafikKodu;
    public string Adı;
    public string Bölge;
}
```

First ve Last

Bir dizinin ilk öğesini veya sağlanan öngörüyü uyan ilk veya son elemanı döndürür. Dizi hiçbir öğe içermiyorsa, şu iletiyle bir **InvalidOperationException** istisnası oluşur.

```
// See https://aka.ms/new-console-template for more information
new[] { "a" }.First();//"a"
new[] { "a", "b" }.First();//"a"
new[] { 1, 2 }.First(); // 1
new[] { "a", "b" }.First(x => x.Equals("b"));// "b"
new[] { 1, 2 }.First(x => x > 1); // 2
new[] { "ba", "be" }.First(x => x.Contains("b"));// "ba"
//new[] { "ca", "ce" }.First(x => x.Contains("b"));// Hata:InvalidOperationException
new[] { "a" }.Last();// "a"
new[] { "a", "b" }.Last();// "b"
Console.WriteLine(new[] { 1, 2 }.Last()); // 2
new[] { "a", "b" }.Last(x => x.Equals("a"));// "a"
Console.WriteLine(new[] { 1, 2 }.Last(x => x < 2)); // 1
new[] { "ba", "be" }.Last(x => x.Contains("b"));// "be"
//new[] { "ca", "ce" }.Last(x => x.Contains("b"));// Hata: InvalidOperationException
```

FirstOrDefault ve LastOrDefault

Bir dizinin ilk öğesini veya sağlanan süzgece uyan ilk elemanı veya son elemanı döndürür. Dizi hiçbir öğe içermiyorsa veya sağlanan fonksiyona uyan hiçbir öğe yoksa, **default(T)** kullanılarak dizi veri tipinin varsayılan değeri döndürülür.

```
// See https://aka.ms/new-console-template for more information
new[] { "a" }.FirstOrDefault();// "a"
new[] { "a", "b" }.FirstOrDefault(); // "a"
new[] { 1, 2 }.FirstOrDefault(); //1
new[] { "a", "b" }.FirstOrDefault(x => x.Equals("b"));// "b"
new[] { 1, 2 }.FirstOrDefault(x => x > 1); //2
new[] { "ba", "be" }.FirstOrDefault(x => x.Contains("b"));// "ba"
new[] { 1, 2 }.FirstOrDefault(x => x > 2); //0
new[] { "ca", "ce" }.FirstOrDefault(x => x.Contains("b"));// null
```

```
new[] { "a" }.LastOrDefault(); // "a"
new[] { "a", "b" }.LastOrDefault(); // "b"
new[] { 1, 2 }.LastOrDefault(); // 2
new[] { "a", "b" }.LastOrDefault(x => x.Equals("a")); // "a"
new[] { 1, 2 }.LastOrDefault(x => x < 2); // 1
new[] { "ba", "be" }.LastOrDefault(x => x.Contains("b")); // "b3"
new[] { 1, 2 }.LastOrDefault(x => x > 2); // 0
new[] { "ca", "ce" }.LastOrDefault(x => x.Contains("b")); // null
```

Single

Bir dizinin tek elemanı varsa veya sağlanan süzgece uyan tek elemanı döndürür. Dizi hiçbir öge içermiyorsa, şu iletille bir `InvalidOperationException` istisnası oluşur.

```
// See https://aka.ms/new-console-template for more information
new[] { "a" }.Single(); // "a"
//new[] { "a", "b" }.Single(); // InvalidOperationException
new[] { 1 }.Single(); // 1
new[] { "a", "b" }.Single(x => x.Equals("b")); // "b"
new[] { 1, 2 }.Single(x => x < 2); // 1
//new[] { "a", "b" }.Single(x => x.Equals("c")); // InvalidOperationException
//new[] { 1, 2 }.Single(x => x > 2); // InvalidOperationException
//new string[0].Single(); // InvalidOperationException
```

SingleOrDefault

Bir dizinin son ögesini veya sağlanan süzgece uyan son ögeyi döndürür. Dizi hiçbir öge içermiyorsa veya sağlanan fonksiyona uyan hiçbir öge yoksa, `default(T)` kullanılarak dizi veri tipinin varsayılan değeri döndürülür.

```
// See https://aka.ms/new-console-template for more information
new[] { "a" }.SingleOrDefault(); // "a"
//new[] { "a", "b" }.SingleOrDefault(); // InvalidOperationException
new[] { 1 }.SingleOrDefault(); // 1
new[] { "a", "b" }.SingleOrDefault(x => x.Equals("b")); // "b"
new[] { 1, 2 }.SingleOrDefault(x => x > 1); // 2
new[] { "a", "b" }.SingleOrDefault(x => x.Equals("c")); // null
new[] { 1, 2 }.SingleOrDefault(x => x > 2); // 0
```

Sum, Min, Max ve Average

```
// See https://aka.ms/new-console-template for more information
var total = iller.Sum(item => item.ilTrafikKodu);
Console.WriteLine($"Toplam İl Trafik Kodu: {total}");
var average = iller.Average(item => item.ilTrafikKodu);
Console.WriteLine($"Ortalama İl Trafik Kodu: {average}");
var min = iller.Min(item => item.ilTrafikKodu);
Console.WriteLine($"En Küçük İl Trafik Kodu: {min}");
var max = iller.Max(item => item.ilTrafikKodu);
Console.WriteLine($"En Büyük İl Trafik Kodu: {max}");
```

Any ve All

Herhangi bir eleman veya tüm dizi elemanlarını bir koşulu karşılayıp karşılamadığını kontrol eder.

```
// See https://aka.ms/new-console-template for more information
bool hasItems = iller.Any();
Console.WriteLine(hasItems); // true
bool allMatchCondition = iller.All(item => item.ilTrafikKodu < 70);
Console.WriteLine(allMatchCondition); // true
bool anotherMatchCondition = iller.All(item => item.ilTrafikKodu > 70);
Console.WriteLine(anotherMatchCondition); // false
```

Count

Bir dizideki öğeleri sayar, isteğe bağlı olarak koşulla verilebilir.

```
// See https://aka.ms/new-console-template for more information
int count = iller.Count();
Console.WriteLine($"Toplam İl Sayısı: {count}"); // 8
int conditionalCount = iller.Count(item => item.Bölge == "Karadeniz");
Console.WriteLine($"Karadeniz Bölgesindeki İl Sayısı: {conditionalCount}"); // 2
```

FindAll

Verilen ifadeye uygun bir dizideki öğeleri döndürür.

```
// See https://aka.ms/new-console-template for more information
List<int> girdiler = new List<int>() { 10, 20, 30, 40, 50, 60 };
Console.WriteLine("Girdiler: ");
foreach (var değer in girdiler)
{
    Console.WriteLine("{0} ", değer);
} // Girdiler: 10 20 30 40 50 60
Console.WriteLine();
List<int> üçeBölünenler = girdiler.FindAll(x => (x % 3) == 0);
Console.WriteLine("Üçe Bölünen Girdiler: ");
foreach (var değer in üçeBölünenler)
{
    Console.WriteLine("{0} ", değer);
} // Üçe Bölünen Girdiler: 30 60
```

Distinct

Bir diziden tekil olan veya diğerlerinden farklı elemanları döndürür.

```
// See https://aka.ms/new-console-template for more information
List<İl> iller = new List<İl> {
    new İl { ilTrafikKodu = 1, Adı = "Adana", Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
    new İl { ilTrafikKodu = 1, Adı = "Adana", Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
    new İl { ilTrafikKodu = 1, Adı = "Adana", Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
};

Console.WriteLine("Listenin Tekil Olan Elemanları:");
var tekiliste = iller.Distinct();
foreach (var item in tekiliste)
{
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
}
/*
Listenin Tekil Olan Elemanları:
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 6, Adı: Ankara
*/

struct İl
{
    public int ilTrafikKodu;
    public string Adı;
    public string Bölge;
}
```

}

Intersect, Except ve Union

Diziler üzerinde kesişim, fark ve birleşim gibi küme işlemlerini gerçekleştirir.

```
// See https://aka.ms/new-console-template for more information
// See https://aka.ms/new-console-template for more information
List<İl> iller1 = new List<İl> {
    new İl { ilTrafikKodu = 1, Adı = "Adana" , Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 5, Adı = "Amasya", Bölge = "Karadeniz" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
    new İl { ilTrafikKodu = 7, Adı = "Antalya", Bölge = "Akdeniz"},
    new İl { ilTrafikKodu = 16, Adı = "Bursa", Bölge = "Marmara"},
    new İl { ilTrafikKodu = 10, Adı = "Balıkesir", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 61, Adı = "Trabzon", Bölge = "Karadeniz" }
};
List<İl> iller2 = new List<İl> {
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
    new İl { ilTrafikKodu = 7, Adı = "Antalya", Bölge = "Akdeniz"},
    new İl { ilTrafikKodu = 16, Adı = "Bursa", Bölge = "Marmara"},
    new İl { ilTrafikKodu = 10, Adı = "Balıkesir", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 61, Adı = "Trabzon", Bölge = "Karadeniz" },
    new İl { ilTrafikKodu = 35, Adı = "İzmir", Bölge = "Ege" },
    new İl { ilTrafikKodu = 45, Adı = "Manisa", Bölge = "Ege" },
    new İl { ilTrafikKodu = 67, Adı = "Zonguldak", Bölge = "Karadeniz" }
};
Console.WriteLine("İki Listenin Kesişim Kümesi:");
var kesişimKümesi = iller1.Intersect(iller2);
foreach (var item in kesişimKümesi)
{
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
}
/*
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 61, Adı: Trabzon
*/
Console.WriteLine("İki Listenin Birleşim Kümesi:");
var birleşimKümesi = iller1.Union(iller2);
foreach (var item in birleşimKümesi)
{
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
}
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 35, Adı: İzmir
İl Trafik Kodu: 45, Adı: Manisa
İl Trafik Kodu: 67, Adı: Zonguldak
*/
Console.WriteLine("İki Listenin Fark Kümesi:");
```



```

var farkKümesi = iller1.Except(iller2);
foreach (var item in farkKümesi)
{
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
}
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 5, Adı: Amasya
*/

struct İl
{
    public int ilTrafikKodu;
    public string Adı;
    public string Bölge;
}

```

OfType

Elmanları değişik veri tiplerinde olan koleksiyonlarda filtreleme yapar.

```

// See https://aka.ms/new-console-template for more information
System.Collections.ArrayList meyveler = new()
{
    "Mango",
    "Portakal",
    null,
    "İğde",
    3.0,
    "Armut",
    10,
    "Çağla"
};
IEnumerable<string> query1 = meyveler.OfType<string>();
Console.WriteLine("'string' veri tipli elemanlar:");
foreach (string meyve in query1)
{
    Console.WriteLine(meyve);
}
/*
'string' veri tipli elemanlar:
Mango
Portakal
İğde
Armut
Çağla
*/
IEnumerable<string> query2 =
    meyveler.OfType<string>().Where(fruit =>
        fruit.Contains('ğ', StringComparison.CurrentCultureIgnoreCase));

Console.WriteLine("\n'ğ' harfli meyveler:");
foreach (string meyve in query2)
{
    Console.WriteLine(meyve);
}
/*
'ğ' harfli meyveler:
İğde
Çağla
*/

```


SelectMany

Bu yöntem bir `IEnumerable<IEnumerable<T>>` ögesini bir `IEnumerable<T>` ögesine olarak değiştirir. Kaynak `IEnumerable`'da bulunan `IEnumerable` örnekleri içindeki tüm T öğeleri tek bir `IEnumerable`'da birleştirilir.

```
// See https://aka.ms/new-console-template for more information
var kelimeler = new[] { "ali,ahmet,cemal", "damla,efe", "fatma" };
var virgilleayrilmiskelimeler = kelimeler.SelectMany(x => x.Split(','));
foreach (var item in virgilleayrilmiskelimeler)
{
    Console.Write($"{item} ");
} //ali ahmet cemal damla efe fatma
Console.WriteLine();
Yemek[] yemekler = new[] {
    new Yemek() {
        Meyveler = new [] { new Meyve { Adı="Elma"}, new Meyve { Adı = "Armut"} }
    },
    new Yemek() {
        Meyveler = new [] { new Meyve { Adı = "Çilek"}, new Meyve { Adı = "Ahududu"} }
    }
};
var tümYemekler = yemekler.SelectMany(y => y.Meyveler.Select(m => m.Adı));
foreach (var item in tümYemekler)
{
    Console.Write($"{item} ");
} //Elma Armut Çilek Ahududu

class Meyve
{
    public required string Adı { get; set; }
}
class Yemek
{
    public required Meyve[] Meyveler { get; set; }
}
```

Join

Birleştirmeler (`join`), ortak bir anahtar aracılığıyla veri tutan farklı listeleri veya tabloları birleştirmek için kullanılır. SQL'de olan türde birleştirmeler desteklenir: İç (`inner`), Sol (`left`), Sağ (`right`), Çapraz (`cross`) ve Tam Dış (`full outer`) birleştirmeler. Aşağıdaki iki tabloyu göz önüne alalım.

```
var sol = new List<string>() { "armut", "muz", "elma" }; // Sol (Left) tablo
var sağ = new List<string>() { "armut", "mango", "elma", "çilek" }; // Sağ (Right)
```

İç Birleştirme (Inner Join):

```
var innerjoin = from l in sol
                join s in sağ on l equals s
                select new { İLK = l, İKİNCİ = s };
var innerjoinfluent = sol.Join(sağ,
                              l => l,
                              s => s,
                              (l, s) => new { İLK = l, İKİNCİ = s });
foreach (var item in innerjoinfluent)
{
    Console.Write($"{item.İLK}-{item.İKİNCİ} ");
} //armut-armut elma-elma
```

Sol Bileştirme (Left Outer Join):

```
var leftOuterJoin = from l in sol
                    join s in sağ on l equals s into tmp
```

```

        from t in tmp.DefaultIfEmpty()
        select new { İLK = l, İKİNCİ = t };
var leftOuterJoin2 = from l in sol
                    from s in sağ.Where(x => x == l).DefaultIfEmpty()
                    select new { İLK = l, İKİNCİ = s };
var leftOuterJoinFluent = sol.GroupJoin(sağ,
                                     l => l,
                                     s => s,
                                     (l, s) => new { İLK = l, İKİNCİ= s })
                           .SelectMany(temp => temp.İKİNCİ.DefaultIfEmpty(),
                                     (l, s) => new { İLK = l.İLK, İKİNCİ = s });
foreach (var item in leftOuterJoinFluent)
{
    Console.WriteLine($"{item.İLK}-{item.İKİNCİ} ");
} //armut-armut muz-(null) elma-elma

```

Sağ Bileştirme (Right Outer Join):

```

var rightOuterJoin = from s in sağ
                    join l in sol on s equals l into temp
                    from t in temp.DefaultIfEmpty()
                    select new { İLK = t, İKİNCİ = s };
foreach (var item in rightOuterJoin)
{
    Console.WriteLine($"{item.İLK}-{item.İKİNCİ} ");
} //armut-armut (null)-mango elma-elma (null)-çilek

```

Çapraz Birleştirme (Cross Join):

```

var CrossJoin = from f in sol
                from s in sağ
                select new { İLK=f, İKİNCİ=s };
foreach (var item in CrossJoin)
{
    Console.WriteLine($"{item.İLK}-{item.İKİNCİ}");
}
/*
armut-armut
armut-mango
armut-elma
armut-çilek
muz-armut
muz-mango
muz-elma
muz-çilek
elma-armut
elma-mango
elma-elma
elma-çilek
*/

```

Tam Dış Birleşme (Full Outer Join):

```

var leftOuterJoin = from l in sol
                    join s in sağ on l equals s into tmp
                    from t in tmp.DefaultIfEmpty()
                    select new { İLK = l, İKİNCİ = t };
var rightOuterJoin = from s in sağ
                    join l in sol on s equals l into temp
                    from t in temp.DefaultIfEmpty()
                    select new { İLK = t, İKİNCİ = s };
var fullOuterjoin = leftOuterJoin.Union(rightOuterJoin);
foreach (var item in fullOuterjoin)
{
    Console.WriteLine($"{item.İLK}-{item.İKİNCİ}");
}

```

```

}
/*
armut-armut
muz-(null)
elma-elma
(null)-mango
(null)-çilek
*/

```

Skip ve Take

Skip yöntemi, kaynak koleksiyonun başlangıcından itibaren belirli sayıda elemanı hariç tutan bir koleksiyon döndürür. Hariç tutulan eleman sayısı, argüman olarak verilen sayıdır. Koleksiyonda argümanda belirtilenden daha az öğe varsa, boş bir koleksiyon döndürülür.

Take yöntemi, kaynak koleksiyonun başlangıcından itibaren bir dizi eleman içeren bir koleksiyon döndürür. Dahil edilen eleman sayısı, argüman olarak verilen sayıdır. Koleksiyonda argümanda belirtilenden daha az eleman varsa, döndürülen koleksiyon kaynak koleksiyonuyla aynı elemanları içerecektir.

```

// See https://aka.ms/new-console-template for more information
var values = new[] { 5, 4, 3, 2, 1 };
var skipTwo = values.Skip(2); // { 3, 2, 1 }
var takeThree = values.Take(3); // { 5, 4, 3 }
var skipOneTakeTwo = values.Skip(1).Take(2); // { 4, 3 }
var takeZero = values.Take(0); // IEnumerable<int> fakat elemanı yok

```

Range ve Repeat

Enumerable'daki **Range** ve **Repeat** statik yöntemleri basit diziler oluşturmak için kullanılabilir.

```

// See https://aka.ms/new-console-template for more information
var range = Enumerable.Range(1,100); // [1, 2, 3, ..., 98, 99, 100]
/* elemanları 1 den başlayıp 100'e kadar devam eden bir tamsayı dizisi oluşturur. */
var repeatedValues = Enumerable.Repeat("a", 3); // ["a","a","a"]
/* aynı elemandan birden fazla içeren dizi oluşturmak için kullanılır*/

```

Contains

Belirtilen bir **IEqualityComparer<T>** kullanarak bir dizinin belirtilen bir elemanı içerip içermediğini belirler.

```

List<int> sayılar = new List<int> { 1, 2, 3, 4, 5 };
var sonuç1 = sayılar.Contains(4); // true
var sonuç2 = sayılar.Contains(8); // false
List<int> başkaSayılar = new List<int> { 4, 5, 6, 7 };
var sonuç3 = başkaSayılar.Where(item => sayılar.Contains(item)); //true (4,5)

```

Zip

Zip yöntemi iki koleksiyon üzerinde etki eder. İki serideki her bir elemanı indise (konuma) göre eşleştirir. İki koleksiyondan gelen elemanları çiftler halinde işlemek için kullanırız. Seriler boyut olarak farklıysa, daha büyük serinin ekstra elemanları yok sayılır.

```

int[] sayılar = { 3, 4, 5, 7 };
string[] sözcükler = { "üç", "dört", "beş", "yedi", "göz ardı edilecek" };
IEnumerable<string> zip = sayılar.Zip(sözcükler, (n, s) => n + "=" + s);
foreach (var item in zip)
    Console.WriteLine(item);
/*
3=üç
4=dört
5=beş
7=yedi

```

*/

Aggregate

Bir dizi üzerinde bir biriktirici fonksiyonu uygular.

```
// See https://aka.ms/new-console-template for more information
int[] sayıListesi = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
int toplam = sayıListesi.Aggregate((aratoplama, şimdiki) => aratoplama + şimdiki);
Console.WriteLine(toplam); // 55
string[] stringList = { "Merhaba", "C#", "!" };
string birleşikmetin = stringList.Aggregate((prev, current) => prev + " " + current);
Console.WriteLine(birleşikmetin); // Merhaba C# !
```

Let Anahtar Kelimesi

Bir LINQ ifadesinin içinde bir değişken tanımlamak için **let** anahtar sözcüğünü kullanabilirsiniz. Bu genellikle ara alt sorguların sonuçlarını depolamak için yapılır.

```
// See https://aka.ms/new-console-template for more information
int[] sayılar = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
var ortalamaÜstüKareler = from sayı in sayılar
                           let ortalama = sayılar.Average()
                           let kare = Math.Pow(sayı, 2)
                           where kare > ortalama
                           select sayı;
Console.WriteLine("Ortalama: {0}", sayılar.Average());
foreach (int n in ortalamaÜstüKareler)
    Console.WriteLine("{0} sayısının karesi {1} Ortalamayı Geçti!", n, Math.Pow(n,2));
/*
Ortalama: 4,5
3 sayısının karesi 9 Ortalamayı Geçti!
4 sayısının karesi 16 Ortalamayı Geçti!
5 sayısının karesi 25 Ortalamayı Geçti!
6 sayısının karesi 36 Ortalamayı Geçti!
7 sayısının karesi 49 Ortalamayı Geçti!
8 sayısının karesi 64 Ortalamayı Geçti!
9 sayısının karesi 81 Ortalamayı Geçti!
*/
```

DOSYALAR VE AKIŞLAR

Dosyalar ve Klasörler

Kısaca **dosya** (**file**), verilerin bir araya geldiği (**collection of data**) ikincil saklama ortamıdır (**seconder storage**). Bu ikincil ortam; Bir bilgisayar **belleği** (**memory**) olabileceği gibi, verilerin elektrikler kesildiğinde kaybolmayacağı **sabit disk** (**hard disc**), ya da bir başka ortama/bilgisayara veri **gönderen** (**send**) veya **alan** (**receive**) modem ve benzeri bir **cihaz** (**device**) olabilir.

Dosyalar söz konusu olduğunda bazı kavramları bilmemiz gerekir;

- **Yol** (**Path**): Dosyanın dosya sistemindeki konumu. Örneğin; **C:\Users\ILHANOZKAN\Okubeni.txt**
- **Sonuna Ekleme** (**Append**): Dosya mevcutsa, dosyanın **sonuna veri ekler** (**append**) değilse yeni dosya üzerine yazar.
- **Metin** (**Text**): Yazılacak veya saklanacak verinin metin yani gözle okunabilir olduğunu ifade eder. Yani okunup yazılan veri basit metin editörleriyle görülebilir.
- **İçerik** (**Contents**): Yazılacak **dizgilerin** (**string**) bir koleksiyonu.
- **Kaynak** (**Source**): Kaynak dosyanın dosya sistemindeki konumu.
- **Hedef** (**Destination**): Hedef dosyanın dosya sistemindeki konumu

Veriler dosyalara aynı nehirde birbirinin peşi sıra giden tekneler gibi bayt-bayt gönderilir veya alınırlar. Bu nedenle dosyalara veri taşıyan ve veri getiren nesnelere **akış** (**stream**) adı verilir. Dosyaya giden nehre hangi veriyi koyarsak onu nehrin sonunda onu dosyaya koyar. Benzer şekilde nehirde hangi sırada veri gelirse o sırada verileri nehirde alırız.

Dosya Oluşturma

File statik sınıfının **Create** yöntemini kullanarak dosyalar oluşturabiliriz. Bu yöntem, verilen yolda dosyayı oluşturur, aynı zamanda dosyayı açar ve bize dosyanın **FileStream** nesnesini verir. İşimiz bittikten sonra dosyayı kapattığınızdan emin olmalıyız;

```
// See https://aka.ms/new-console-template for more information
FileStream dosyaAkışı = File.Create("..\Metin.txt");
// Programın çalıştığı dizine dosya oluşturur.
dosyaAkışı.Close();
```

Buna eşdeğer olarak **Open** Yöntemini kullanabiliriz:

```
FileStream dosyaAkışı = File.Open("..\Metin.txt", FileMode.Create);
// Programın çalıştığı dizine dosya oluşturur.
dosyaAkışı.Close();
```

using anahtar kelimesi ile açılan bir kaynağın kapatılmasını otomatik olarak yapabiliriz. Dosya yolundaki ters bölü karakteri metinler için özel karakter olduğundan iki kez yazılır;

```
using (var dosyaAkışı2 = File.Create("C:\\Users\\ILHANOZKAN\\YeniDosya.txt"))
{
    //dosyaAkışı kullanılmıyorsa blok dışında otomatik olarak kapatılır.
}
```

Tam metin (**verbatim string**) kullanarak ters bölü karakterini iki kez yazmaktan kurtulabiliriz;

```
File.Create(@"C:\Users\ILHANOZKAN\YeniDosya2.txt").Close();
/*
*/
```

Dosya oluşturma bir diğer yolu da **FileStream** sınıfını kullanmaktır;

```
FileStream dosyaAkışı3 = new FileStream("..\Metin.txt",
    FileMode.OpenOrCreate,
    FileAccess.ReadWrite,
    FileShare.None);
```

FileMode: "Dosya oluşturulmalı mı? açılmalı mı? mevcut değilse oluşturulmalı mı, açılmalı mı?" türündeki soruları yanıtlar²⁵.

FileAccess: "Dosyayı okuyabilmeli miyim, dosyaya yazabilmeli miyim yoksa her ikisini de yapabilmeli miyim?" türündeki soruları yanıtlar²⁶.

FileShare: "Diğer kullanıcılar dosyayı aynı anda kullanırken okuyabilmeli, yazabilmeli mi vb.?" türündeki soruları yanıtlar²⁷.

Dosya Taşıma

File statik sınıfının **Move** yöntemini kullanarak dosyaları taşıyabiliriz.

```
//File.Move("kaynakyol", @"hedefyol");
File.Move(@"C:\Users\ILHANÖZKAN\YeniDosya2.txt", @"D:\YeniDosya.kopya.txt");
```

Yalnızca dosyanın dizinini değiştirir. Bu işlem dosya boyutuna göre zaman almaz. Ayrıca aynı dosya hedefte var ise kopyalama olmaz.

Dosya Silme

```
string path = @"D:\YeniDosya.kopya.txt";
File.Delete(path);
```

Delete yöntemi dosya mevcut değilse istisna oluşturmaz, ancak belirtilen yol geçersizse veya çağıran gerekli izinlere sahip değilse istisna oluşturur. **Delete** çağrılarını her zaman **try-catch** bloğunun içine koymalı ve tüm beklenen istisnaları işlemelisiniz. Birden fazla program aynı dosyayı silmeye kalkışabilir ve bu durumda **yarış durumu** (**race condition**) ortaya çıkar. Bunu önlemek için silme mantığını **lock** ifadesinin içine koymalıyız.

Bir Klasördeki Dosyaları Öğrenme

Klasördeki tüm dosyaları öğrenmek için aşağıdaki kodu yazarız;

```
var FileSearchRes = Directory.GetFiles(Path, "*.*", SearchOption.AllDirectories);
```

Belirtilen dizindeki tüm dosyaları (*.*) adı ve uzantısı ne olursa olsun demektir) temsil eden bir **FileInfo** dizisi döndürür. Eğer belirtilen dizin ve alt dizinlerindeki PDF uzantılı dosyaları arayacak isek aşağıdaki kodu yazmalıyız;

```
// See https://aka.ms/new-console-template for more information
string Path = @"D:\CSharp";
string[] FileSearchRes = Directory.GetFiles(Path, "*.pdf", SearchOption.AllDirectories);
foreach (var item in FileSearchRes)
{
    Console.WriteLine(item);
}
```

Metin Dosyasını Okuma

Bir dosyanın tüm içeriğini bir **dizgiye** (**string**) okumak için **System.IO.File.ReadAllText** ve yazmak için **System.IO.File.WriteAllText** fonksiyonunu kullanabilirsiniz;

```
// See https://aka.ms/new-console-template for more information
System.IO.File.WriteAllText(@".\Metin.txt",
    "Bu bir deneme metnidir.\nBu metin ikinci satırdır.");
string text = System.IO.File.ReadAllText(@".\Metin.txt");
string[] lines = text.Split(new[] { '\n' }, StringSplitOptions.None);
int lineCount = lines.Length;
Console.WriteLine($"Toplam {lineCount} satır var.");
```

²⁵ <https://learn.microsoft.com/en-us/dotnet/api/system.io.filemode?view=net-9.0&redirectedfrom=MSDN>

²⁶ <https://learn.microsoft.com/en-us/dotnet/api/system.io.fileaccess?view=net-9.0&redirectedfrom=MSDN>

²⁷ <https://learn.microsoft.com/en-us/dotnet/api/system.io.fileshare?view=net-9.0&redirectedfrom=MSDN>

Tüm satırları bir dizgi dizisine (`string[]`) okumak için `System.IO.File.ReadAllLines` ve yazmak için `System.IO.File.WriteAllLines` fonksiyonu kullanılır;

```
// See https://aka.stream/new-console-template for more information
string[] lines = System.IO.File.ReadAllLines(@".\Metin.txt");
foreach (string line in lines)
{
    Console.WriteLine(line);
}
```

Büyük Metin Dosyaları

Büyük dosyalarla çalışırken, bir dosyadaki tüm satırları `IEnumerable<string>` jenerik ara yüzüne okumak için `System.IO.File.ReadLines` yöntemini kullanabilirsiniz;

```
// See https://aka.stream/new-console-template for more information
using System.Text; //Encoding için gerekli
IEnumerable<string> AllLines = File.ReadLines(@".\Metin.txt", Encoding.Default);
foreach (string line in AllLines)
{
    Console.WriteLine(line);
}
```

Burada farklı [yerel kodlamaya](#) ([encoding](#)) sahip metin dosyasının okuma imkanına da sahip oluruz²⁸. Ayrıca `File.WriteLine` fonksiyonunu yazmak için de kullanabiliriz.

Akış Nedir?

Bir akış, veri aktarımı için düşük seviyeli bir araç sağlayan bir nesnedir. Kendileri veri kapsayıcıları olan koleksiyonlar gibi hareket etmezler. Akışlarda ele aldığımız veriler bayt dizisi (`byte []`) biçimindedir. Okuma ve yazma işlevlerini gerçekleştirecek akışlarda doğrudan tamsayılar, [dizgiler](#) (`string`) için hiçbir [yöntem](#) (`method`) yoktur. Bu da akışı genel amaçlı yapar, ancak yalnızca metin aktarmak istiyorsanız çalışması daha az basittir.

Akışlar, özellikle büyük miktarda veriyle uğraşırken çok yararlıdır. Hepsi n adet bayt üzerinden işlem yapar. Yazılması/okunması gereken yere (yani yedekleme deposuna) göre farklı türde [akış](#) (`stream`) kullanmamız gerekecek. Örneğin, kaynak bir dosyaysa, `FileStream` kullanmamız gerekir;

```
string filePath = @".\Metin.txt";
using (FileStream fs = new FileStream(filePath,
                                     FileMode.Open,
                                     FileAccess.Read,
                                     FileShare.ReadWrite))
{
    // Dosya Akışı ile Yapılacak İşler
    fs.Close();
}
```

Benzer şekilde, yedekleme deposu bellek ise `MemoryStream` kullanılır;

```
// Diskte dosyada bulunan tüm batırları oku.
byte[] dosya = File.ReadAllBytes(@".\Metin.txt");
// Okunan baytlar için hafızada bir akış oluştur.
using (MemoryStream memory = new MemoryStream(dosya))
{
    // Hafızadaki akış ile Yapılacak İşler
}
```

Benzer şekilde `System.Net.Sockets.NetworkStream` ise ağ işlemleri için kullanılır.

Tüm Akışlar, `System.IO.Stream` sınıfından türetilmiştir. Veriler doğrudan akışlardan okunamaz veya akışlara yazılamaz. .NET Framework içinde, temel veri tipler ve düşük seviyeli akış ara yüzü arasında

²⁸ <https://learn.microsoft.com/en-us/dotnet/api/system.text.encoding?view=net-9.0>

dönüşüm yapan ve verileri sizin için akışa veya akıştan size aktaran **StreamReader**, **StreamWriter**, **BinaryReader** ve **BinaryWriter** gibi yardımcı sınıflar bulunur.

Akışlara Veri Yazma ve Okuma

StreamWriter aracılığıyla akışları kullanırken ve bunları kapatırken dikkatli olmak gerekir.

```
// See https://aka.stream/new-console-template for more information
FileStream fs = new FileStream(@".\Metin.txt", FileMode.Create);
StreamWriter sw = new StreamWriter(fs);
string NextLine = "Bu metin dosyanın sonuna eklenir.";
sw.WriteLine(NextLine);
sw.Close();
//fs.Close(); Bu talimata gerek yoktur çünkü örtük olarak sw.Close() bu yöntemi çalıştırır.
```

Benzer şekilde **StreamReader** kullanırken aynı şeye dikkat etmemiz gerekir;

```
// See https://aka.stream/new-console-template for more information
using (var stream = new MemoryStream())
{
    StreamWriter writer = new StreamWriter(stream);
    writer.WriteLine(123);
    writer.WriteLine("Bu bir deneme metnidir.");
    //writer.Close(); Bu akışı kapatır.
    writer.Flush(); //Kalan verileri akışa gönderebilirsiniz.
    //Kapatma yapılırsa bu otomatik olarak yapacaktır
    stream.Position = 0; // Akışın başına geri dön
    StreamReader sr = new StreamReader(stream);
    var myStr = sr.ReadToEnd();
    Console.WriteLine(myStr);
    /*
    123
    Bu bir deneme metnidir.
    */
    sr.Close();
    stream.Close();
}
```

Akış Olmadan Metin Dosyaları Oluşturma

StreamWriter nesnesinin kapsam dışına çıktığı anda elden çıkarılmasını ve böylece dosyanın kapatılmasını sağlayan **using** anahtar sözcüğünün kullanılması gerekir.

```
// See https://aka.ms/new-console-template for more information
string[] lines = { "Birinci Metin Satırı.", "İkinci Metin Satırı.", "Üçüncü Metin Satırı" };
using (System.IO.StreamWriter sw = new System.IO.StreamWriter(@"D:\Metin.txt"))
{
    foreach (string line in lines)
    {
        sw.WriteLine(line);
    }
} //<-Bu bloktan sonra sw çöpe gönderilir.
```

StreamWriter sınıfı yapıcısında ikinci bir bool parametresi alabileceğini ve bu sayede dosyaya ekleme yapılabilir.

```
bool appendExistingFile = true;
using (System.IO.StreamWriter sw2 = new
System.IO.StreamWriter(@"D:\Metin.txt", appendExistingFile))
{
    sw2.WriteLine("Bu satır son satır olarak dosya sonuna eklenecek.");
}
```


İkili Dosyaya Yazma ve Okuma

Metin dosyalarından farklı olarak ikili olarak sayı ve yapıları dosyalara yazabilir ve okuyabiliriz. Bu şekilde oluşan dosyalar gözle okunamazlar. Çünkü işlemcinin doğal olarak anladığı ikili sayı sisteminde veriler dosyalara yazılır ve okunur.

```
// See https://aka.stream/new-console-template for more information
using System.Text;
string fileName = "ikili.bin";
using (var stream = File.Open(fileName, FileMode.Create))
{
    using (var writer = new BinaryWriter(stream, Encoding.UTF8, false))
    {
        writer.Write(1.250F);
        writer.Write(@"c:\Temp");
        writer.Write("İlhan Özkan");
        writer.Write(10);
        writer.Write(true);
    }
}
float aspectRatio;
string tempDirectory;
string tempUserName;
int autoSaveTime;
bool showStatusBar;
if (File.Exists(fileName))
{
    using (var stream = File.Open(fileName, FileMode.Open))
    {
        using (var reader = new BinaryReader(stream, Encoding.UTF8, false))
        {
            aspectRatio = reader.ReadSingle();
            tempDirectory = reader.ReadString();
            tempUserName = reader.ReadString();
            autoSaveTime = reader.ReadInt32();
            showStatusBar = reader.ReadBoolean();
        }
    }
    Console.WriteLine("Aspect ratio set to: " + aspectRatio);
    Console.WriteLine("Temp directory is: " + tempDirectory);
    Console.WriteLine("User Name is: " + tempUserName);
    Console.WriteLine("Auto save time set to: " + autoSaveTime);
    Console.WriteLine("Show status bar: " + showStatusBar);
}
```

Oluşan **ikili.bin** dosyasına ilk önce **kayan noktalı sayı** (**float**) değeri sonra iki adet **dizgi** (**string**) ardından bir **tamsayı** (**int**) ve **bool** değer ikili olarak yazılmıştır. Bellekte saklandığı şekliyle dosyaya kaydedildiğinden içeriği gözle okunamaz. İkili dosyaları açan görüntüleyiciler ile açılıp görülebilir. Bir ikili dosyayı onaltılık rakamlara çevirip metin olarak görüntülemek için Windows ortamında aşağıdaki komut verilebilir;

```
C:\Users\ILHANÖZKAN>certutil -encodehex ikili.bin ikili.txt
Input Length = 31
Output Length = 147
CertUtil: -encodehex command completed successfully.
```

Artık dönüştürülen dosyayı (**ikili.txt**) istediğimiz metin düzenleyicisinde (**notepad**) açıp okuyabiliriz.

```
0000  00 00 a0 3f 07 63 3a 5c  54 65 6d 70 0d c4 b0 6c  ...?.c:\Temp...l
0010  68 61 6e 20 c3 96 7a 6b  61 6e 0a 00 00 00 01  han ..zkan.....
```

VERİ TABANI İŞLEMLERİ

Veri Tabanı Nedir?

Veri tabanları (database), tuttukları veriler hakkında bilgi sahibi olan programlardır. Veri tabanları, yazdığımız uygulamadan bağımsız olarak verilerimizi sistematik olarak saklar, erişilebilir kılar ve yönetmemizi sağlar. Veri tabanı kullanmak bize veri tutarlılığı, veri paylaşımı, veri güvenliği ve veri analitiği için bir altyapı sağlar.

Veri tabanı;

- Kullanıcılardan gelen sorgu taleplerini analiz eder,
- Analiz sonrasında varsa mevcut veri üzerinde değişiklik yapar veya istenen veriyi belirler
- Sorgu sonucunu kullanıcının erişebileceği formda sunar.

Veri tabanında sorgular, **Structured Query Language-SQL** dili ile ilişkisel veri tabanında veya NoSQL dilile ilişkisel olmayan veri tabanlarında yapılmaktadır.

Veri tabanı yönetim sistemleri (VTYS) aracılığıyla veri tabanları güvenli ve ölçeklenebilir şekilde yönetilebilmektedir. Veri tabanı yönetim sistemi verinin fiziksel veya mantıksal alanda bulunduğunu belirler ve gelen sorguları bu kapsamda ayrıştırmaktadır. Sorguları hızlandırmak için indeksler oluşturulup veriye hızlıca erişilmesi sağlanmaktadır.

Bir uygulama geliştirmeye başladığınızda yapacağınız ilk tercihlerden biri SQL veya NoSQL Veritabanı kullanıp kullanmayacağınızdır. Verileri düzenlemek çok zor bir iştir ve düzenlediğimizde verilerimizi çeşitli türlere göre kategorize ederiz.

PostgreSQL, MySQL, Microsoft SQL Server, Oracle Database veya SQLite gibi geleneksel bir veri tabanı şema kullanır. SQL veri tabanları şema kullanır ve saklanan verileri düzenlemek daha kolaydır.

MongoDB, Redis, CouchDB, Cassandra, Elasticsearch, Couchbase ve LiteDB gibi NoSQL veri tabanları SQL veri tabanları kadar veri tutarlılığı sağlamaz ve verileri düzenlemek çok daha zordur. Daha çok **iz kayıtları (log)**, belge ve resim saklamak için kullanılırlar.

SQL ve NoSQL arasındaki temel fark, verilerin nasıl depolandığı ve nasıl dışarıya aktarıldığıdır. Bir veri tabanı projesini yazabilmeniz için ilk önce ilgili veri tabanına karar verip projenizin çalıştığı bilgisayara ya da projenizle aynı ağda ve erişilebilir olan bir bilgisayara kurmanız gerekir.

Erişilebilir bir veri tabanınız varsa o veri tabanına ilişkin bileşenleri projemize dahil ederek veri tabanına bağlanıp gerekli veri işlemlerini gerçekleştirebiliriz.

Bu tür bileşenleri projemize dahil etmek için *Tools* → *NuGet Package Manager* → *Package Manager Console* üzerinden komut vererek bileşenleri projemize dahil ederiz. Veri Tabanı Sunucuları için örnek;

```
Install-Package Microsoft.Data.Sqlite
Install-Package Microsoft.Data.SqlClient
Install-Package LiteDb
```

Bağlantı Dizgisi

Projemizde kullanacağımız veri tabanları uygulamamızdan ayrı bir programdır. Bu veri tabanlarına bağlanmamızı sağlayan metinleri **dizgi (string)** değişkenleri olarak tanımlarız. Bu değişkenlere **bağlantı dizgisi (connection string)** adı veririz. Bu değişken genellikle veri tabanı yönetim sistemine iletilen kullanıcı adı, şifre ve şema bilgileri içerir.

```
string sqliteConnectionString="Data Source=hello.db";
var sqliteConnection = new SqliteConnection(sqliteConnectionString);

string nosqlConnectionString=@"MyData.db";
var nosqlConnection = new LiteDatabase(nosqlConnectionString);
```

Küçük ve yerel bilgisayarınızda çalışabilecek Veri tabanlarının bağlantı dizgileri kısa ve öz olur. Ancak kurumsal ve ölçeklenebilir Veri tabanlarının bağlantı dizgisi daha karmaşık olabilir. Bu gibi bağlantı

dizgilerini oluşturmak için `DbConnectionStringBuilder`, `SqlConnectionStringBuilder` gibi yardımcı sınıflar kullanırız;

```
DbConnectionStringBuilder builder1 = new DbConnectionStringBuilder();
builder1.ConnectionString = @"Data Source=D:\Data\MyDb.mdb"; //Access DataBase
builder1.Add("Provider", "Microsoft.Jet.Oledb.4.0");
builder1.Add("Jet OLEDB:Database Locking Mode", 1);
OleDbConnection oledbConnect = new OleDbConnection(builder1.ConnectionString);
var builder2 = new SqlConnectionStringBuilder
{
    DataSource = "(local)",
    UserID = "sa",
    Password = "123456*",
    InitialCatalog = "CSharpDB"
};
var SqlConnection = new SqlConnection(builder2.ConnectionString);
var SqlConnection2 = new SqlConnection("server=(local);initial catalog=CSharpDB");
```

Veri kaynakları genellikle hem mali hem de tükettikleri kaynak açısından pahalı sistemlerdir. Pahalı olan bu kaynaklar başka yazılımlar ile ortak kullanılırlar. Bu nedenle veri kaynaklarına olan bağlantılarımızı sınırlı tutmalıyız. Bu tür kaynakları etkin kullanmak için `using` saklı kelimesi kullanılır. `using` kelimesi veri kaynağı gibi değerli kaynaklara ilişkin işlem yapılırken bir hata meydana geldiğinde, bağlantıya ilişkin tüm nesneleri `Dispose` yöntemi ile bellekten siler yani `Dispose` yönteminin çağırılmasını garanti eder.

İlişkisel Veri Tabanı İşlemleri

Veri tabanını kullanabilmek için ilk önce bir `bağlantı` (`connection`) kurmak gerekir. Sonrasında veri tabanı üzerinde işlem yapacak `SQL komutu` (`command`) nesnesi hazırlanır ve hazırlanan komut `icra` (`execute`) edilir.

Veri Tabanında Tablolar Oluşturma

```
// See https://aka.ms/new-console-template for more information
using Microsoft.Data.Sqlite;
var connectionString = @"Data Source=D:\CSharp.sqlite";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText =
@"
DROP TABLE IF EXISTS Ogrenci;
CREATE TABLE Ogrenci (
    OgrenciId INTEGER PRIMARY KEY AUTOINCREMENT,
    Adi text NOT NULL,
    Soyadi text NOT NULL
);
DROP TABLE IF EXISTS Ders;
CREATE TABLE Ders(
    DersId INTEGER PRIMARY KEY AUTOINCREMENT,
    DersAdi NVARCHAR(50)
);
";
    /*
    Yukarıdaki gibi Noktalı virgül karakteriyle ayırarak
    birden fazla sorgu çalıştırabiliriz.
    */
    int rowsAffected = command.ExecuteNonQuery(); //Veri getirmeyecek bir sorgu!
    Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
    command.CommandText =
@"
```

```

        INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ali', 'Veli');
        INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ayşe', 'Fatma');
        INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Mehmet', 'Can');
        INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Zeynep', 'Yılmaz');
        INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ahmet', 'Demir');
        INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Fatma', 'Kaya');
        INSERT INTO Ders (DersAdi) VALUES ('Matematik');
        INSERT INTO Ders (DersAdi) VALUES ('Fizik');
    ";
    rowsAffected = command.ExecuteNonQuery();
    Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
    connection.Close();
}

```

SQLite/SQL Compact Toolbox eklentisini Visual Studio'ya kurarak oluşturduğumuz SQLite veri tabanını açıp sorgulayabiliriz²⁹.

Sorgu Parametreleri

Veri tabanı için yazılacak sorgulara parametre metin olarak yazılma yerine bir değişkene bağlanabilir;

```

// See https://aka.ms/new-console-template for more information
using Microsoft.Data.Sqlite;
var connectionString = @"Data Source=D:\CSharp.sqlite";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText =
        @"
        INSERT INTO Ogrenci (Adi, Soyadi) VALUES ($Ad, $Soyad);
    ";
    string ad="İlhan", soyad="Özkan";
    command.Parameters.AddWithValue("$Ad", ad);
    command.Parameters.AddWithValue("$Soyad", soyad);
    int rowsAffected= command.ExecuteNonQuery();
    Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
    connection.Close();
}
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");

```

Veri Tabanından Veri Okuma

Veri tabanına eklenmiş kayıtları da sorgulatıp öğrenebiliriz;

```

// See https://aka.ms/new-console-template for more information
using Microsoft.Data.Sqlite;
var connectionString = @"Data Source=D:\CSharp.sqlite";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText = @"SELECT OgrenciId, Adi, Soyadi FROM Ogrenci;";
    using (SqliteDataReader reader = command.ExecuteReader()) //Veri getirecek bir sorgu!
    {
        while (reader.Read())
        {
            var öğrenciId = reader.GetString(0);
            var adı = reader.GetString(1);
            var soyadı = reader.GetString(2);
            Console.WriteLine($"{öğrenciId} {adı} {soyadı}");
        }
    }
}

```

²⁹ <https://marketplace.visualstudio.com/items?itemName=ErikEJ.SQLServerCompactSQLiteToolbox>

```

    }
}
connection.Close();
}

```

İşlemler

İşlemler (**transaction**), birden fazla SQL ifadesini tek bir iş biriminde gruplandırmanıza ve bu iş biriminin veri tabanına tek bir atomik birim olarak kaydedilmesine olanak tanır. İş birimindeki herhangi bir ifade başarısız olursa, önceki ifadeler tarafından yapılan değişiklikler geri alınabilir. İşlem başlatıldığında veri tabanının ilk durumu korunur. Veri tabanında aynı anda çok sayıda değişiklik yapıldığı durumlarda faydalıdır.

Kurumsal veri tabanlarının aksine SQLite veri tabanında, aynı anda yalnızca bir işlemin beklemede olan değişikliklere sahip olmasına izin verilir. Bu durum **eş zamanlılık** (**concurrency**) olarak adlandırılır. Bu nedenle, **SQLiteCommand** ile başlanan işte **BeginTransaction** ve **Execute** yöntemlerine yapılan çağrılar, başka bir işlemin tamamlanması çok uzun sürerse zaman aşımına uğrayabilir³⁰.

```

// See https://aka.ms/new-console-template for more information
using Microsoft.Data.Sqlite;
var connectionString = @"Data Source=D:\CSharp.sqlite";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText =
@"
        DROP TABLE IF EXISTS OgrencilerinAldigiDersler;
        CREATE TABLE Burslar (
            OgrenciId INTEGER NOT NULL,
            BursMiktari REAL NOT NULL
        );
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (1,1000.0);
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (2,2000.0);
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (3,3000.0);
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (4,4000.0);
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (5,3000.0);
";
    int rowsAffected = command.ExecuteNonQuery();
    Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
    using (var transaction = connection.BeginTransaction(deferred: true))
    {
        var readCommand = connection.CreateCommand();
        long öğrenciId = 1;
        readCommand.CommandText = @"SELECT BursMiktari FROM Burslar Where (OgrenciId=@Id)";
        readCommand.Parameters.AddWithValue("@Id", öğrenciId);
        var value = (double) readCommand.ExecuteScalar();
        var writeCommand = connection.CreateCommand();
        writeCommand.CommandText =
@"UPDATE Burslar SET BursMiktari = $newValue Where OgrenciId = @Id";
        writeCommand.Parameters.AddWithValue("$newValue", value + 6000.0);
        writeCommand.Parameters.AddWithValue("@Id", öğrenciId);
        writeCommand.ExecuteNonQuery();
        transaction.Commit();
    }
    connection.Close();
}

```

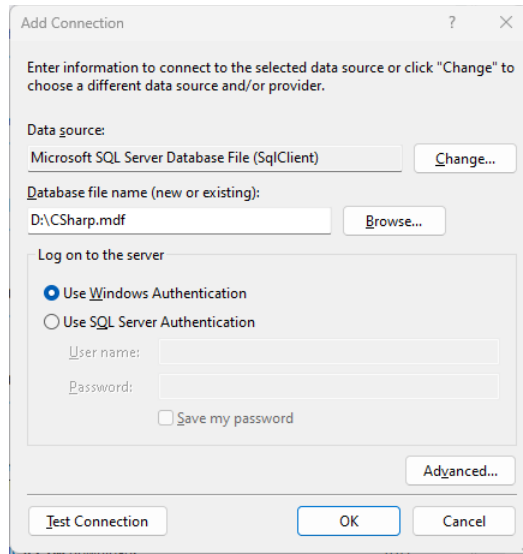
³⁰ <https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/transactions>

Veri Kaynaklarına Erişim Yöntemleri

.Net Framework ile veri kaynaklarına iki stratejiyle erişim yapılır;

1. **Bağlı (connected)**: Bu yöntemde veri üzerindeki değişiklikler doğrudan veri kaynağı üzerinde yapılır. Veri kaynakları pahalı yatırımlarla kurulduğundan, kaynağı en az meşgul edecek işlemler için veya zorunlu durumlarda kullanılır. .NET teknolojisinde bunu yukarıdaki örneklerde verildiği gibi, **DataReader** nesnesi ile yaparız.
2. **Kopuk (disconnected)**: Bu yöntemde veriler bellekte geçici bir yere aktarılırlar. **Veri seti (DataSet)** olarak adlandırılan bu yer, sanki bir veri kaynağıymış gibi kullanılır. Veri üzerinden işlemler tamamlandığında, bellekteki değişiklikler veri kaynağına kaydedilir. Veri setleri, hafızada oluşturulan bir veri tabanlarıdır.

Veri Seti için *Microsoft SQL Server LocalDB* sürümünü kullanacağız. Bunun için *Tools → Connect To Database...* seçilir. Açılan pencerede yeni bir veri tabanı oluşturulur;



Şekil 34. MSSQL Veritabanı Oluşturma

Oluşan veri tabanına *View → Server Explorer* menüsünden ulaşılabilir. Eğer .NET Core CLI ile proje oluşturmuş iseniz veri tabanını yönetmek için *SQL Server Management Studio* yazılımını kullanabilirsiniz³¹.

```
// See https://aka.ms/new-console-template for more information
using Microsoft.Data.SqlClient;
using System.Data;
var connectionString = @"Data Source=(LocalDB)\MSSQLLocalDB;"
    + @"AttachDbFilename=D:\CSharp.mdf;"
    + @"Integrated Security=True;Connect Timeout=30";
using (var connection = new SqlConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText =
        @"
        DROP TABLE IF EXISTS Ogrenci;
        CREATE TABLE Ogrenci (
            OgrenciId INT PRIMARY KEY IDENTITY (1, 1),
            Adi NVARCHAR (50) NOT NULL,
            Soyadi NVARCHAR (50) NOT NULL
        );
        DROP TABLE IF EXISTS Ders;
        CREATE TABLE Ders(
            DersId INT PRIMARY KEY IDENTITY (1, 1),
```

³¹ <https://learn.microsoft.com/en-us/ssms/install/install>

```

        DersAdi NVARCHAR(50) NOT NULL
    );
";
int rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
command.CommandText =
@"
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ali', 'Veli');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ayşe', 'Fatma');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Mehmet', 'Can');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Zeynep', 'Yılmaz');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ahmet', 'Demir');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Fatma', 'Kaya');
    INSERT INTO Ders (DersAdi) VALUES ('Matematik');
    INSERT INTO Ders (DersAdi) VALUES ('Fizik');
";
rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
DataSet dataSet = new DataSet();
dataSet.Tables.Add("Ogrenci");
dataSet.Tables.Add("Ders");
SqlDataAdapter dataAdapterOgrenci = new SqlDataAdapter("Select * From Ogrenci",
                                                    connection);

dataAdapterOgrenci.Fill(dataSet.Tables[0]);
SqlDataAdapter dataAdapterDers = new SqlDataAdapter("Select * From Ders",
                                                    connection);

dataAdapterDers.Fill(dataSet.Tables[1]);
/* veriler veritabanından okunup verisetine yüklendi. veritabanına bağlatı
   (connection) sadece bu an için yapıldı. */
connection.Close();
Console.WriteLine("OgrenciId,Adı,Soyadı");
foreach (DataRow row in dataSet.Tables[0].Rows)
{
    Console.WriteLine("{0},{1},{2}",row[0], row[1], row[2]);
}
Console.WriteLine("DersId,DersAdi");
foreach (DataRow row2 in dataSet.Tables[1].Rows)
{
    Console.WriteLine("{0},{1}",row2[0], row2[1]);
}
}

```

NoSQL Veri Tabanı

NoSQL veri tabanları ilişkisel veri tabanları kadar veri tutarlılığı sağlamaz ve verileri düzenlemek çok daha zordur. Bu nedenle genellikle veri yazmak ve okumak için kullanılır. Aşağıda bir okuldaki öğrenciler ve diplomalarının saklandığı bir NoSQL örneği verilmiştir;

```

// See https://aka.ms/new-console-template for more information
using LiteDB;
using (var db = new LiteDatabase(@"D:\CSharp.litedb"))
{
    var öğrenciler = db.GetCollection<Ogrenci>("Ogrenciler");
    var öğrenci = new Ogrenci
    {
        Adı = "İlhan Özkan",
        EPosta = "ilhanozkan@outlook.com",
        CepTelefonu = "03121234567"
    };
    // Yeni Öğrenci koleksiyona eklenir (Id otomatik artar)
    öğrenciler.Insert(öğrenci);
    var result = öğrenciler.Find(x => x.EPosta == "ilhanozkan@outlook.com").FirstOrDefault();
}

```



```

        Console.WriteLine($"{result.Id}-{result.Adı}");
        öğrenci.CepTelefonu = "04621234567";
        öğrenciler.Update(öğrenci);
        var diplomalar = db.GetCollection<Diploma>("Diplomalar");
        byte[] fileBytes = File.ReadAllBytes(@"D:\lisans.pdf"); //PDF dosyayı veritabanına
        var diploma = new Diploma
        {
            ÖğrenciId = öğrenci.Id,
            PdfDocument = fileBytes
        };
        diplomalar.Insert(diploma);
    }

    public class Diploma
    {
        public int Id { get; set; }
        public int ÖğrenciId { get; set; }
        public byte[] PdfDocument { get; set; }
    }

    public class Öğrenci
    {
        public int Id { get; set; }
        public string Adı { get; set; }
        public string EPosta { get; set; }
        public string CepTelefonu { get; set; }
    }

```

Veri tabanına yazdırılan nesneleri okumak için aşağıdaki kod kullanılabilir;

```

// See https://aka.ms/new-console-template for more information
using LiteDB;
using (var db = new LiteDatabase(@"D:\CSharp.litedb"))
{
    var öğrenciler = db.GetCollection<Öğrenci>("Öğrenciler");
    var liste = öğrenciler.FindAll();
    foreach (var o in liste)
        Console.WriteLine($"{o.Id}-{o.Adı}-{o.EPosta}-{o.CepTelefonu}");
    var diplomalar = db.GetCollection<Diploma>("Diplomalar");
    var result = diplomalar.Find(x => x.ÖğrenciId == 1).FirstOrDefault();
    File.WriteAllBytes(@"D:\lisans.new.pdf", result.PdfDocument);
    //veritabanından dosya sistemine dosyayı yazdık.
}

public class Diploma
{
    public int Id { get; set; }
    public int ÖğrenciId { get; set; }
    public byte[] PdfDocument { get; set; }
}

public class Öğrenci
{
    public int Id { get; set; }
    public string? Adı { get; set; }
    public string? EPosta { get; set; }
    public string? CepTelefonu { get; set; }
}

```


DESENLER

Desen Nedir?

Nesne yönelimli programlamanın ortaya çıkışıyla beraber, daha önceden yazılmış hazır **bileşenlerin** (**component**) **yeniden kullanımı** (**reusing**) konusunda oldukça önemli ilerlemeler sağlamıştır. Bunun paralelinde **tecrübelerin** (**experience**) yeniden kullanımı konusunda da çeşitli çalışmalar yapılmış ve **desen** (**pattern**) kavramı ortaya çıkmıştır.

Yazılım geliştirme öncesi, geliştirme aşması ve sonrasında daha önce yaşanmış çeşitli problemlere karşı birçok kimse tarafından çeşitli çözümler getirilmiştir. Desenler, daha önce geliştirme yapan programcıların yanlış yaparak doğru yapmayı öğrendikleri başarılı tecrübelerin anlatılması için bir araçtır. Gerçekleştirilen bu çözümlerin, daha sonradan yaşanacak benzer nitelikli problemlerde kullanılması amacıyla desenler kullanılır. Desen kavramının temelleri Mimar Christopher Alexander'ın 1970 sonlarında başlattığı çalışmalara dayanmaktadır³².

1987 yılında uluslararası “Nesne Yönelimli Programlama, Sistemler, Diller ve Uygulamalar” konferansına kadar desenlerle ilgili bir çalışma ortaya çıkmamıştır³³. Bu tarihten sonra ise başta *Grady Booch*, *Richard Helm*, *Erich Gamma* ve *Kent Beck* olmak üzere pek çok bilim adamı desenlerle ilgili makale ve sunumlar yayınlamışlardır. 1994 yılında *Erich Gamma*, *Richard Helm*, *Ralph Johnson* ve *John Vlissides* tarafından yayınlanan “Tasarım Desenleri: Tekrar kullanılabilir Nesne Yönelimli Yazılımın Temelleri” kitabı, tasarım desenlerin yazılımda kullanılmasında dönüm noktası olmuştur³⁴.

Desenlerin aksine anti-desen (**anti-pattern**) ise başarısızlık tecrübelerini anlatır

Yazılımlarda kullanılan desenler yedi ana başlıkta toplanırlar. Bunlar;

1. Mimari desenler (**architectural pattern**)
2. Analiz desenleri (**analysis pattern**)
3. Tasarım desenleri (**design pattern**)
4. Kodlama desenleri (**implementation pattern**)
5. Test desenleri (**test pattern**)
6. Çözüm desenleri (**solution pattern**)
7. Veri desenleri (**data pattern**)

Bu desenlerin en önemlisi ve ilk gelişenlerden birisi, **tasarım desenleridir** (**design pattern**). Tasarım desenleri aslında, Nesne Yönelimli Tasarım Prensiplerini, yazılımlara uygulamanın bir başka ifadesidir. Bu nedenle çok önemlidir. Tasarım desenleri ise üç ana başlıkta toplanmaktadır.

1. Nesne imali ile ilgili desenler (**creational patterns**)
2. Yapısal desenler (**structural patterns**)
3. Davranışlarla ilgili desenler (**behavioral patterns**)

Desenlerin çok sık kullanıldığı programlama yöntemine günümüzde desen yönelimli programlama (**pattern oriented programming**) adı verilmektedir³⁵. İzleyen sayfalarda verilen desen UML diyagramlarının birçoğu Erich Gamma'nın Design Pattern CD yayınından alınmıştır³⁶.

Bu bölümde verilen örnekleri iyi anlamak için *Sınıflar Arası İlişkiler* başlığını iyi özümsemek gerekir. Bu bölümde oluşturulan projeler eski tip proje oluşturularak yapılmıştır. Bunun için: *Visual Studio* açılır ve şu adımlar takip edilir;

1. Programı ilk defa çalıştırıyorsanız, karşılama sayfasından *Create a new project* seçiniz. Eğer program açık ise *File* menüsüne giderek *File* → *New Project* seçiniz.
2. Proje tipi olarak *Console Application* seçilir.
3. *Project Name*, *Location* ve *Solution Name* belirlenir

³² <http://gee.cs.oswego.edu/dl/ca/ca/ca.html>

³³ OOPSLA, Object Oriented Programming, Systems, Languages, and Applications

³⁴ Design Patterns: Elements of Reusable Object-Oriented Software, ISBN 0-201-63361-2

³⁵ www.mathcs.sjsu.edu/faculty/pearce/patterns.html

³⁶ Design Pattern CD, Addison-Wesley, 1998

4. *Additional Information* sayfasında *Do not use top-level statemets* kutucuğu SEÇİLİ bırakılır. Böylece geleneksel proje şablonu ile proje oluşacaktır.
5. *Solution Explorer* üzerinde *Program.cs* dosyası açılır.

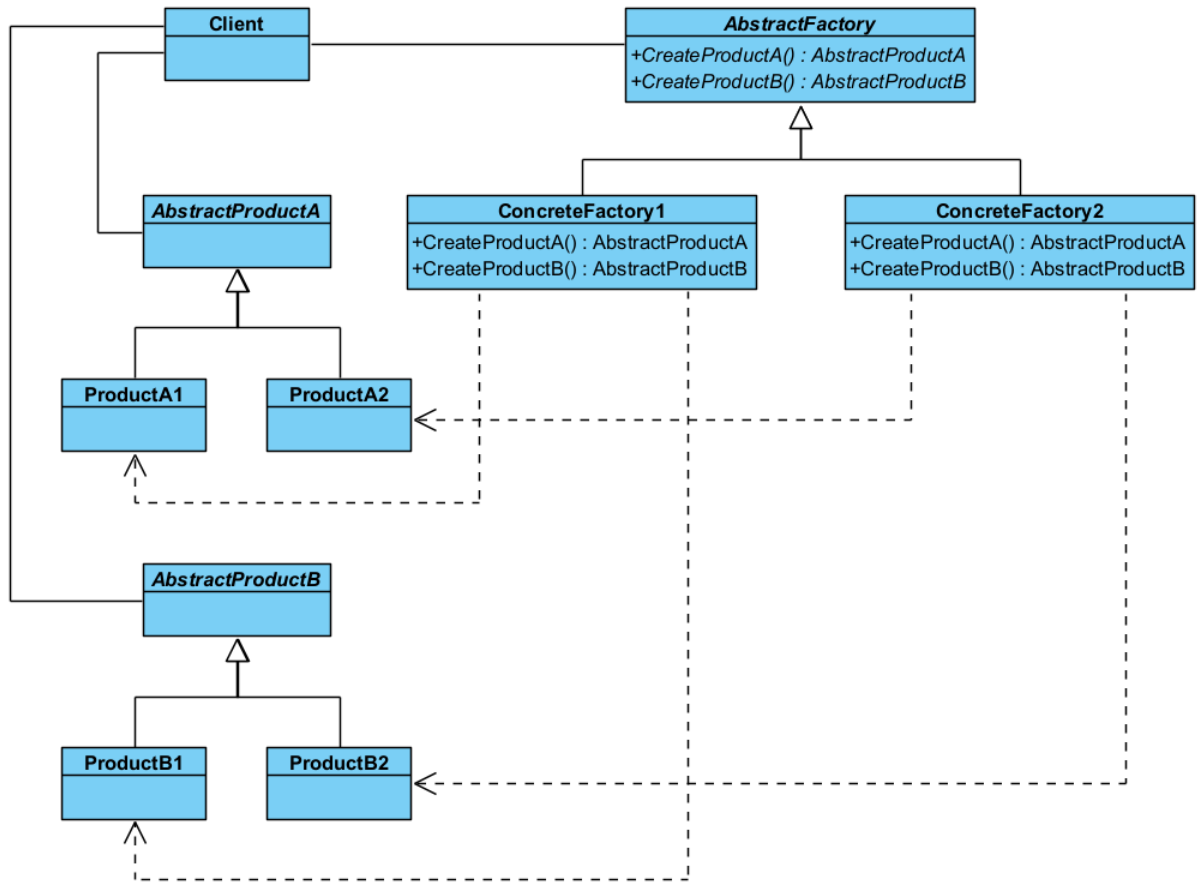
Nesne İmalatı ile İlgili Tasarım Desenleri

Nesne imalatı ile ilgili desenler (**creational patterns**), nesne (**object**) örnekleme ile ilgili ortaya çıkabilecek çeşitli problemlere çözüm getirmektedirler. Bu desenlerde nesneler dinamik olarak imal edilirler, çünkü değişim yönetiminde statik nesneler kullanılmazlar.

Soyut Fabrika Deseni

Soyut fabrika deseninde (**abstract factory pattern**) amaç, nesnelerin imal edilmesi ve kullanılması ile ilgili olan kısımlarının birbirinden ayrılmasıdır. Bu amaca, birbiriyle ilgili ya da birbirine bağımlı olabilecek nesnelerin imal edilmesinde, nesnelerin somut (**concrete**) sınıflarını değil soyut (**abstract**) sınıfları kullanılarak ulaşılır. Somut sınıflar istemciden izole edilir ve programcıya somut sınıfları daha kolay değiştirme için olanak sağlar.

Bu başlıktaki desenlerde, nesnelerin (**objects**) yapılandırılıp imal edildiği (**creation**) yer için fabrika (**factory**) kavramı, üretilen nesneler için ise ürün (**product**) kavramı kullanılmaktadır. Aşağıdaki UML diyagramında görüldüğü üzere istemci yalnızca soyut sınıflara bağımlı olduğundan somut ürüne olan bağımlılık azaltılmıştır (**loosely coupling**).



Şekil 35. Soyut Fabrika Deseni UML Diyagramı

Şimdi bu deseni kodlamaya başlayalım. İlk önce **AbstractProductA** ve alt sınıflarını kodlayalım.

```

abstract class AbstractProductA
{
}
class ProductA1 : AbstractProductA
{
}
class ProductA2 : AbstractProductA

```

```
{  
}
```

Ardından **AbstractProductB** ve alt sınıflarını kodlayalım. Bu sınıftan imal edilecek ürünler **AbstractProductA** ürünleriyle etkileşim yaptığı varsayılarak bir **interact** yöntemi eklenmiştir.

```
abstract class AbstractProductB {  
    public abstract void Interact(AbstractProductA a);  
}  
class ProductB1 : AbstractProductB {  
    public override void Interact(AbstractProductA a)  
    {  
        Console.WriteLine("{0} ile {1} etkileşiyor.",  
            this.GetType().Name, a.GetType().Name);  
    }  
}  
class ProductB2 : AbstractProductB  
{  
    public override void Interact(AbstractProductA a)  
    {  
        Console.WriteLine("{0} ile {1} etkileşiyor.",  
            this.GetType().Name, a.GetType().Name);  
    }  
}
```

Ardından ürünleri imal edecek fabrika sanal sınıfı ve somut sınıflarını tanımlayalım;

```
abstract class AbstractFactory  
{  
    public abstract AbstractProductA CreateProductA();  
    public abstract AbstractProductB CreateProductB();  
}  
class ConcreteFactory1 : AbstractFactory  
{  
    public override AbstractProductA CreateProductA()  
    {  
        return new ProductA1();  
    }  
    public override AbstractProductB CreateProductB()  
    {  
        return new ProductB1();  
    }  
}  
class ConcreteFactory2 : AbstractFactory  
{  
    public override AbstractProductA CreateProductA()  
    {  
        return new ProductA2();  
    }  
    public override AbstractProductB CreateProductB()  
    {  
        return new ProductB2();  
    }  
}
```

Son olarak fabrika ve ürün sınıflarını sanal olarak kullanan istemci programı kodlayalım;

```
class Client  
{  
    static void Main(string[] args)  
    {  
        AbstractFactory factory1 = new ConcreteFactory1();  
        AbstractFactory factory2 = new ConcreteFactory2();  
        // Bu istemcide ürünlerin yaratılmasına
```

```

// ilişkin bir işlem yapılmamaktadır.
// Ürünler birbirlerine bağlı (depended) yada
// ilişkilidir (relation).
AbstractProductA producta = factory1.CreateProductA();
AbstractProductB productb = factory2.CreateProductB();
productb.Interact(producta);
}
}
/* Program Çalıştığında;
ProductB2 ile ProductA1 etkileşiyor.

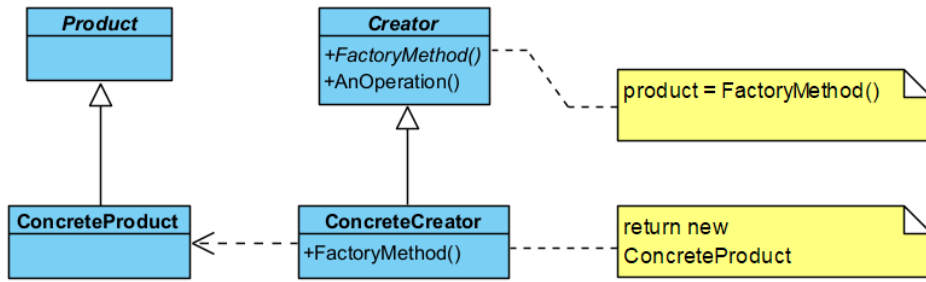
...Program finished with exit code 0
*/

```

Fabrika Yöntemi Deseni

Fabrika yöntemi deseninde (factory method pattern) amaç, imal edilecek nesnenin sınıfını kesin olarak belirtmeden nesne yaratma işleminin gerçekleştirilmesidir. Bunu yapmak için fabrika yöntemi adında soyut (abstract) bir yöntem tanımlanır, fakat nesneleri imal etme (instantiation) işlemi alt sınıflara bırakılır. Yani sanal bir yapıcı (virtual constructor) kullanılır.

Soyut fabrika deseninde olduğu gibi dinamik nesne imal etme ile ilgili **new** anahtar kelimesi, fabrika yöntemi içinde kullanıldığından, istemcinin, ürünün yapıcısına (constructor) bağımlılığı ortadan kalkar. Aşağıdaki UML diyagramında görüldüğü üzere istemcinin somut ürünün yapıcısına bağımlılığı kaldırılmış ve nesne imal etme süreci kontrol altına alınmıştır.



Şekil 36. Fabrika Yöntemi Deseni UML Diyagramı

Şimdi bu deseni kodlamaya başlayalım. İlk önce **Product** ve alt sınıflarını kodlayalım.

```

abstract class Product { } // "ConcreteProduct", Somut ürün
class ConcreteProduct : Product { } // "Creator", Soyut Üretici
abstract class Creator
{
    // Ürün Üretecek Soyut Fabrika İşlevi
    // Bu desende bu işlev; Ürün yaratacak bir arayüzdür.
    // Bu arayüz istemcinin (client) ürün yaratmaya
    // (creation of objects)
    // ilişkin bağımlılığı ortadan kaldırır.
    public abstract Product FactoryMethod();
    public void AnOperation() { }
}

```

Ardından ürün imal edecek olan imalatçı **Creator** sınıfını kodluyoruz;

```

class ConcreteCreator : Creator // "ConcreteCreator", Somut Ürün Üreticisi
{
    // somut ürününü üreten Fabrika işlevi
    public override Product FactoryMethod()
    {
        return new ConcreteProduct();
    }
}

```

Son olarak ürün ve imalatçı sınıflarını sanal olarak kullanan istemci programı kodlayalım;

```

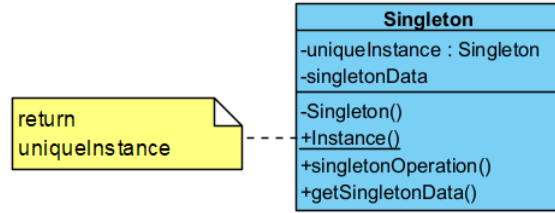
class Client
{
    static void Main(string[] args)
    {
        // Üreticileri soyut olarak kullanacağız.
        Creator creator = new ConcreteCreator();
        // Ürünleri de new kullanmadan yaratıp kullanıyoruz. Bu durumda kullanıcı ürünün
        // yaratıcı işlevine bağımlı kalmaz. Bu desenin amacı ürüne ilişkin new kelimesini
        // kullanmamaktır. Product product=new ConcreteProductA();
        // Kullanılabilir ama; Burada ürünün yapıcı işlevinin değişmesi durumunda
        // istemcinin (client) yeniden derlenmesi gerekir. İstemci ürünün nasıl yaratıldığını
        // ile değil nasıl kullanılacağı ile ilgilenir.
        Product product = creator.FactoryMethod();
        Console.WriteLine("{0} created.",product.GetType().Name);
    }
}
/* Program Çıktısı:
ConcreteProduct created.

...Program finished with exit code 0
*/

```

Tekil Nesne Deseni

Tekil Nesne deseninde (singleton pattern) amaç, bir sınıftan yalnızca bir örnek (instance) imal etmektir. Bir sınıfın yalnızca bir nesnesinin olmasına izin verilir, birden fazla olmasına izin verilmez. Yaratılan nesneye genel olarak evrensel (global) olarak erişilir.



Şekil 37. Tekil Deseni UML Diyagramı

Verilen UML diyagramı incelendiğinde, **Singleton** sınıfının **instance** yöntemi her seferinde aynı nesneye ilişkin referans geri döndürülür. Bu desen aynı zamanda, nesneye ilk değerlerin verilerek oluşturulduğu, imal edilme sürecini ve ilk kullanımını **izole** (encapsulate) eder. Böylece istemcilerin her biri aynı nesne ile muhatap olurlar. Şimdi **Singleton** sınıfını kodlayalım;

```

// Singleton sınıfı öyle bir sınıftır ki yalnızca bir
// örneği(instance) vardır.
class Singleton
{
    // Sınıftan yaratılan örneklerle değil,
    // sınıfa ait üye değişken (class scope member variable)
    // tanımlanıyor.
    private static Singleton uniqueInstance;
    private int singletonData;
    // Birden fazla örneğe(instance) izin vermemek için
    // yapıcı(constructor) işlevin görünürlüğü(visibility)
    // özel(private) olmalıdır.
    private Singleton() { }
    public static Singleton Instance()
    {
        if (uniqueInstance == null)
            uniqueInstance = new Singleton();
        return uniqueInstance;
    }
    public void SingletonOperation()
    {

```

```

        Console.WriteLine("Singleton İşlevi Çağrıldı.");
    }
    public int GetSingletonData()
    {
        return singletonData;
    }
}

```

Bu sınıfı kullanan istemci aşağıdaki gibi yazılabilir;

```

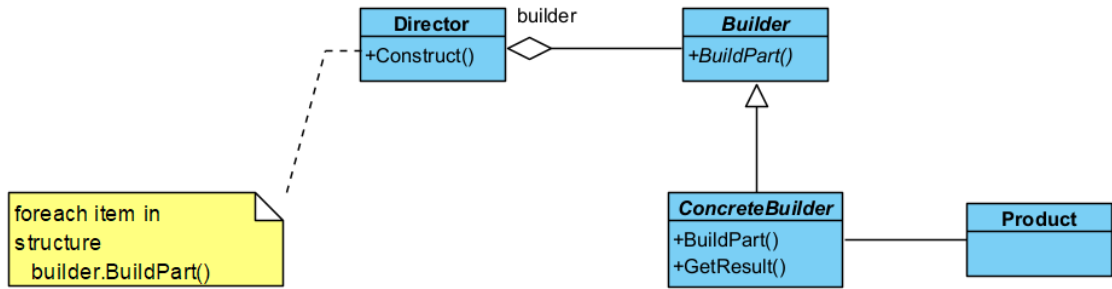
class Client
{
    static void Main(string[] args)
    {
        Singleton object1 = Singleton.Instance();
        Singleton object2 = Singleton.Instance();
        if (object1 == object2)
            Console.WriteLine
                ("object1 ve object2 AYNI nesnedir.");
        else
            Console.WriteLine
                ("object1 ve object2 FARKLI nesnedir.");
        object1.SingletonOperation();
    }
}
/* Program Çıktısı:
object1 ve object2 AYNI nesnedir.
Singleton İşlevi Çağrıldı.

...Program finished with exit code 0
*/

```

Kurucu Deseni

Kurucu deseninde (builder pattern) amaç, çok karmaşık bir nesnenin imal edilmesiyle ilgili işlemleri bir başka sınıfa bırakmaktır. Böylece nesnenin gösterimi ve kullanılması ilişkin kodlar ile yaratılmasına ilişkin kodlar birbirinden ayrılmış olur.



Şekil 38. Kurucu Deseni UML Diyagramı

Yukarıda verilen UML diyagramı incelendiğinde, Ürün imalatını yönetecek olan **Director** nesnesi, karmaşık nesneyi oluşturacak küçük **ürünlerin** (product) yapımını **Builder** nesnesine bırakır. **Director** nesnesi, oluşturulan bu küçük parçaları **Construct** yöntemi ile bir araya getirir. İlk önce Product sınıfını tanımlayalım.

```

class Product
{
    ArrayList parts = new ArrayList();
    public void Add(string part)
    {
        parts.Add(part);
    }
    public void Show()
    {
    }
}

```

```
        Console.WriteLine("Ürün Parçaları:");
        foreach (string part in parts) Console.WriteLine(part);
    }
}
```

Buna ek olarak **Builder** ve alt sınıfını tanımlayalım;

```
abstract class Builder
{
    public abstract void BuildPartA();
    public abstract void BuildPartB();
    public abstract Product GetResult();
}
class ConcreteBuilder1 : Builder
{
    private Product product = new Product();
    public override void BuildPartA()
    {
        product.Add("A Parçası");
    }
    public override void BuildPartB()
    {
        product.Add("B Parçası");
    }
    public override Product GetResult()
    {
        return product;
    }
}
class ConcreteBuilder2 : Builder
{
    private Product product = new Product();
    public override void BuildPartA()
    {
        product.Add("X Parçası");
    }
    public override void BuildPartB()
    {
        product.Add("Y Parçası");
    }
    public override Product GetResult()
    {
        return product;
    }
}
```

Şimdi de **Director** sınıfını tanımlayalım;

```
class Director
{
    public void Construct(Builder builder)
    {
        builder.BuildPartA();
        builder.BuildPartB();
    }
}
```

Son olarak istemciyi tanımlayalım;

```
class Client
{
    static void Main(string[] args)
    {
        Director director = new Director();
    }
}
```

```

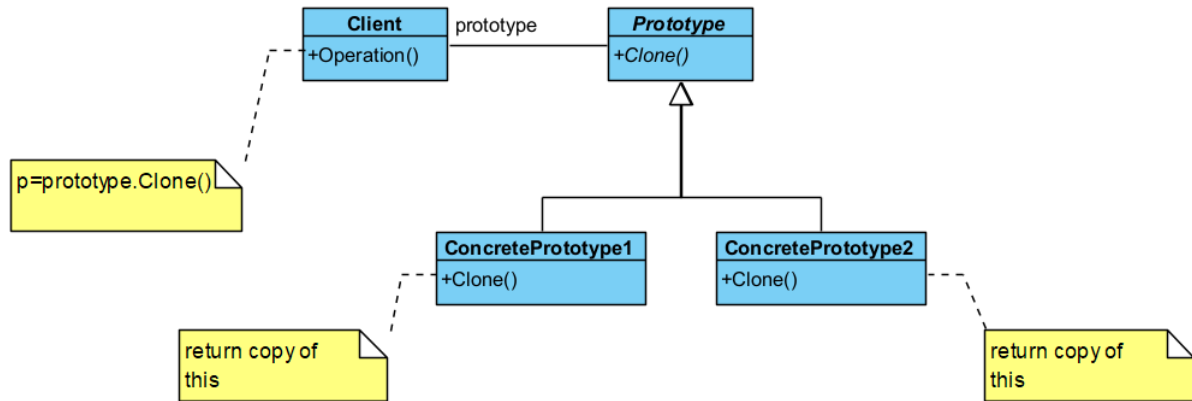
        Builder b1 = new ConcreteBuilder1();
        Builder b2 = new ConcreteBuilder2();
        director.Construct(b1);
        Product p1 = b1.GetResult();
        p1.Show();
        director.Construct(b2);
        Product p2 = b2.GetResult();
        p2.Show();
    }
}
/*Programın Çıktısı:
Ürün Parçaları:
A Parçası
B Parçası
Ürün Parçaları:
X Parçası
Y Parçası

...Program finished with exit code 0
*/

```

Prototip Deseni

Prototip deseni (prototype pattern), hâlihazırda mevcut olan nesnenin bir kopyasını çıkarmak için kullanılır. Bu desen, imal edilmesi güç olan nesnelere karar verilerek, bu nesneleri tekrar imal etmek için bir sürü işlem yapılması yerine, mevcut nesneden bir kopya çıkararak kullanılmasını sağlamaktadır.



Şekil 39. Prototip Deseni UML Diyagramı

Klonlama yoluyla var olan nesnelerin bir şablonuna dayalı yeni nesneler oluşturmak için kullanılan bu deseni kodlaması aşağıdaki şekilde yapılabilir;

```

abstract class Prototype
{
    private string id;
    public Prototype(string id)
    {
        this.id = id;
    }
    public string Id
    {
        get { return id; }
    }
    public abstract Prototype Clone();
}
class ConcretePrototype1 : Prototype
{
    public ConcretePrototype1(string id) : base(id) { }
    public override Prototype Clone()

```



```

    {
        return (Prototype)this.MemberwiseClone();// deep copy
    }
}
class ConcretePrototype2 : Prototype
{
    public ConcretePrototype2(string id) : base(id) { }
    public override Prototype Clone()
    {
        return (Prototype)this.MemberwiseClone();// deep copy
    }
}

```

Şimdi de istemciyi yazalım;

```

class Client
{
    static void operation() {
        ConcretePrototype1 p1 = new ConcretePrototype1("I");
        ConcretePrototype1 c1 = (ConcretePrototype1)p1.Clone();
        Console.WriteLine("{0} Klonandı.", c1.Id);
        ConcretePrototype2 p2 = new ConcretePrototype2("II");
        ConcretePrototype2 c2 = (ConcretePrototype2)p2.Clone();
        Console.WriteLine("{0} Klonlandı.", c2.Id);
    }
    static void Main(string[] args)
    {
        operation();
    }
}
/* Program Çıktısı:
I Klonandı.
II Klonlandı.

...Program finished with exit code 0
*/

```

Nesneleri kopyalama söz konusu olduğunda, genel olarak üç tür kavramdan bahsedilir;

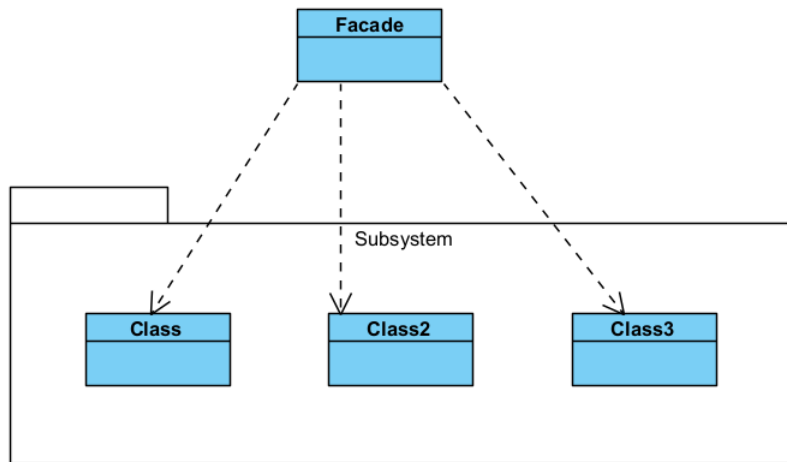
- **Üstünkörü kopyalamada** (*shallow copy*) yeni nesne eskisini referans gösterir. Bu durumda birinci nesnenin bir **alanı** (*field*) değiştiğinde ikincisinin ki de değişmiş olur. Yani birinde olan değişiklik diğerini etkiler. Bu kopyalamanın diğer adı da **bit düzeyi kopyalamadır** (*bitwise copy*).
- **Derinlemesine kopyalamada** (*deep copy*) ise birinci nesnenin aynısı bir başka bellek bölgesinde oluşturulur ve ikinci nesne yeni bellek bölgesini referans gösterir. Bu kopyalamanın diğer adı **da üye düzeyi kopyalamadır** (*memberwise copy*). Kopyalama sonrasında bir nesne üzerinde olan değişiklik yeni nesneyi etkilemez.
- Üçüncü tip bir kopyalama ise **tembel kopyalamadır** (*lazy copy*). Yukarıda bahsi geçen iki kopyalamanın iç içe girmiş şeklidir.

Yapısal Tasarım Desenleri

Yapısal desenler (*structural patterns*), sınıflar ve nesnelerin çok olduğu daha karmaşık programlarda getirilen yapısal çözüm yöntemleridir.

Vitrin Deseni

Vitrin deseninde (*facade pattern*) alt sistemlerin değişmesi halinde tüm sistemin yeniden derlenme olasılığını bertaraf etmek amacıyla alt sistemlere erişim tek bir sınıf üzerinden yapılır. Çok katmanlı yazılım mimarisinde, alt katmanların içinde olan değişiklikler sistem genelini etkilemez. Çünkü katmanlar birbirlerinin ön yüzünü (*facade*) görür ve kullanırlar.



Şekil 40. Vitrin Deseni UML Diyagramı

Bir sistem içindeki bir dizi ara yüze tek bir ara yüz üzerinden erişim sağlayacağımız program için bir dizi alt sistem kodunu yazalım;

```

class SubSystemOne // Subsystem ClassA"
{
    public void MethodOne()
    {
        Console.WriteLine(" SubSystemOne Yöntemi");
    }
}
class SubSystemTwo // Subsystem ClassB"
{
    public void MethodTwo()
    {
        Console.WriteLine(" SubSystemTwo Yöntemi");
    }
}
class SubSystemThree // Subsystem ClassC"
{
    public void MethodThree()
    {
        Console.WriteLine(" SubSystemThree Yöntemi");
    }
}
class SubSystemFour // Subsystem ClassD"
{
    public void MethodFour()
    {
        Console.WriteLine(" SubSystemFour Yöntemi");
    }
}
  
```

Şimdi de **Facade** sınıfını kodlayalım;

```

class Facade
{
    SubSystemOne one;
    SubSystemTwo two;
    SubSystemThree three;
    SubSystemFour four;
    public Facade()
    {
        one = new SubSystemOne();
        two = new SubSystemTwo();
        three = new SubSystemThree();
        four = new SubSystemFour();
    }
}
  
```

```

    }
    public void MethodA()
    {
        Console.WriteLine("MethodA() ---- ");
        one.MethodOne();
        two.MethodTwo();
        four.MethodFour();
    }
    public void MethodB()
    {
        Console.WriteLine("MethodB() ---- ");
        two.MethodTwo();
        three.MethodThree();
    }
}

```

Son olarak istemci kodumuzu yazalım;

```

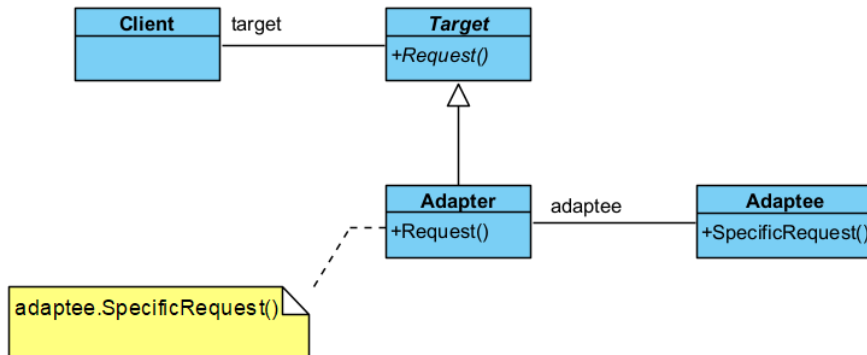
class Client
{
    static void Main(string[] args)
    {
        Facade facade = new Facade();
        facade.MethodA();
        facade.MethodB();
    }
}
/*Program Çalıştığında:
MethodA() ----
SubSystemOne Yöntemi
SubSystemTwo Yöntemi
SubSystemFour Yöntemi
MethodB() ----
SubSystemTwo Yöntemi
SubSystemThree Yöntemi

...Program finished with exit code 0
*/

```

Adaptör Deseni

Adaptör deseni (**adapter pattern**), yabancı bir ara yüzü istemcinin anlayacağı bir ara yüze dönüştürür. Yani farklı ara yüzlere sahip sınıfların, iletişim ve etkileşim kurabilecekleri ortak bir nesne oluşturarak birlikte çalışmalarına izin verir. Bu desen, yabancı ara yüze yaptırılacak işin bir şekilde yapıldığı ve zaten yapılan bu işlemin istemcinin isteğine uygun hale getirildiği düşünülmelidir.



Şekil 41. Adaptör Deseni UML Diyagramı

Aşağıdaki örnekte **Target** nesnesi, istemci tarafından gelen isteği, bu işi yapabilme yeteneğine sahip **Adaptee** nesnesine yaptıracaktır. İşte burada **Adapter** nesnesi devreye girerek **Target** nesnesine gelen istekleri, **Adaptee** nesnesinin yapabileceği şekilde uyarlar.

İlk önce yabancı ara yüzü temsil eden **Adaptee** sınıfını kodluyoruz;

```
class Adaptee
{
    public void SpecificRequest()
    {
        Console.WriteLine(
            "Adaptee.SpecificRequest() işlevi çağrıldı.");
    }
}
```

Daha sonra soyut olan **Target** ve somut olan **Adapter** sınıfını kodluyoruz;

```
class Target
{
    public virtual void Request()
    {
        Console.WriteLine("Target.Request() işlevi çağrıldı.");
    }
}
class Adapter : Target
{
    private Adaptee adaptee = new Adaptee();
    public override void Request()
    {
        adaptee.SpecificRequest();
    }
}
```

Son olarak istemci tarafını kodluyoruz;

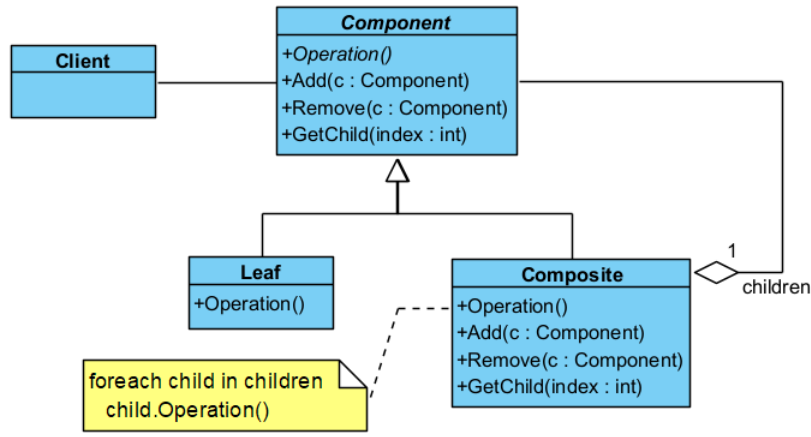
```
class Client
{
    static void Main(string[] args)
    {
        Target target = new Adapter();
        target.Request();
        // İstemci target.Request() işlevine bağımlıdır
        // Adaptöre uyan Adaptee.SpecificRequest() işlevine
        // bağımlı değildir.
        // Böylece adaptöre uyum gösteren neslelerdeki işlevler
        // istemciden bağımsız olarak değiştirilebilir.
    }
}
/* Program Çıktısı:
Adaptee.SpecificRequest() yöntemi çağrıldı.

...Program finished with exit code 0
*/
```

Bileşik Deseni

Bileşik desen (**composite pattern**), özyinelemeli (**recursive**) ya da hiyerarşik yapıya sahip nesne oluşturmak gerektiğinde yapısal olarak nasıl bir kodlama yöntemi izleneceğini söyler.

Bu desen, her nesnenin bağımsız olarak veya aynı ara yüz üzerinden iç içe geçmiş nesneler kümesi olarak ele alınabileceği nesne hiyerarşilerinin oluşturulmasını kolaylaştırır. Yukarıda verilen UML diyagramını incelendiğinde; **Leaf**, ağaç yapısı içinde kullanılabilecek, parçalara ayrılamayan en küçük bileşen olan **en ilkel** (**primitive**) yapıdır. **Component**, bu ilkel yapılardan oluşturulacak karmaşık **Composite** nesnesine ilkel yapıların eklenip çıkarılmasını sağlayacak bileşendir.



Şekil 42. Bileşik Deseni UML Diyagramı

Şimdi sanal **Component** sınıfı ile somut sınıflarını tanımlayalım;

```

abstract class Component
{
    protected string name;
    public Component(string name)
    {
        this.name = name;
    }
    public abstract void Add(Component c);
    public abstract void Remove(Component c);
    public abstract void Display(int depth);
}

class Leaf : Component
{
    public Leaf(string name) : base(name) { }
    public override void Add(Component c)
    {
        Console.WriteLine("Bileşen eklenemiyor.");
    }
    public override void Remove(Component c)
    {
        Console.WriteLine("Bileşen Silinemiyor.");
    }
    public override void Display(int depth)
    {
        Console.WriteLine(new String('-', depth) + name);
    }
}

class Composite : Component
{
    private ArrayList children = new ArrayList();
    public Composite(string name) : base(name) { }
    public override void Add(Component component)
    {
        children.Add(component);
    }
    public override void Remove(Component component)
    {
        children.Remove(component);
    }
    public override void Display(int depth)
    {
        Console.WriteLine(new String('-', depth) + name);
        foreach (Component component in children) // Recursively display child nodes
        {
            component.Display(depth + 2);
        }
    }
}

```

```

    }
}

```

Şimdi istemci kodunu yazabiliriz;

```

class Client
{
    static void Main(string[] args)
    {
        Composite root = new Composite("kök");
        root.Add(new Leaf("A Yaprığı"));
        root.Add(new Leaf("B Yaprığı"));
        Composite comp = new Composite("X Bileşimi");
        comp.Add(new Leaf("XA Yaprığı"));
        comp.Add(new Leaf("XB Yaprığı"));
        root.Add(comp);
        root.Add(new Leaf("C Yaprığı"));
        Leaf leaf = new Leaf("D Yaprığı");
        root.Add(leaf);
        root.Remove(leaf);
        // İçi içe (Recursively) ağacı göster
        root.Display(1);
    }
}

/*Program Çıktısı:
-kök
---A Yaprığı
---B Yaprığı
---X Bileşimi
-----XA Yaprığı
-----XB Yaprığı
---C Yaprığı

...Program finished with exit code 0
*/

```

Dekorator Deseni

Dekorator deseninde (**decorator pattern**), bir nesneye dinamik olarak yeni durum ve davranışları eklemek mümkündür. Yani çalıştırma anında, bir nesnenin sahip olduğu yeteneklere, yeni yeteneklerin eklenmesini sağlar. Aşağıdaki UML diyagramı incelendiğinde, **Component** nesnesi, **Decorator** nesnesi aracılığı ile **çalıştırma anında** (**run time**), **addedBehavior** yöntemine ve **addedState** durumuna (**state**) sahip olur.

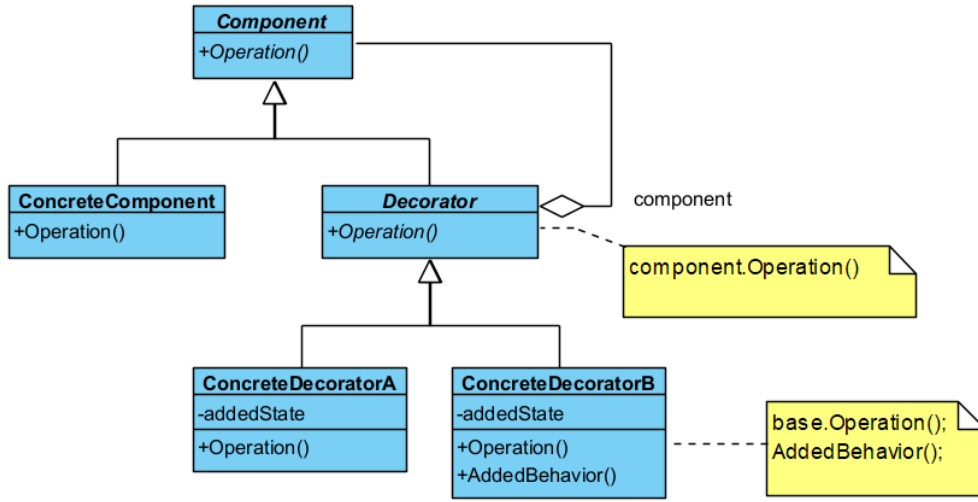
İlk olarak sanal sınıf olan **Component** ve somut **ConcreteComponent** bileşenlerini kodlayalım;

```

abstract class Component
{
    public abstract void Operation();
}

class ConcreteComponent : Component
{
    public override void Operation()
    {
        Console.WriteLine("ConcreteComponent.Operation()");
    }
}

```



Şekil 43. Dekorator Deseni UML Diyagramı

Şimdi de somut **Decorator** sınıfı ve alt sınıflarını kodlayalım;

```

abstract class Decorator : Component
{
    protected Component component;
    public Component SetComponent(Component component)
    {
        this.component = component;
        return this;
    }
    public override void Operation()
    {
        if (component != null)
        {
            component.Operation();
        }
    }
}

class ConcreteDecoratorA : Decorator
{
    private string addedState;
    public ConcreteDecoratorA()
    {
        addedState = "AddedState";
    }
    public string AddedState
    {
        get { return addedState; }
        set { addedState = value; }
    }
    public override void Operation()
    {
        base.Operation();
        Console.WriteLine(
            "Bileşen Yeni bir Duruma Sahiptir:{0}",
            this.AddedState);
    }
}

class ConcreteDecoratorB : Decorator
{
    void AddedBehavior()
    {
        Console.WriteLine(
            "Bileşen Yeni Bir Davranışa Sahiptir:AddedBehaviour()");
    }
}
  
```

```

public override void Operation()
{
    base.Operation();
    AddedBehavior();
}
}

```

Son olarak istemcimizi kodluyoruz;

```

class Client
{
    static void Main(string[] args)
    {
        Component c = new ConcreteComponent();
        ConcreteDecoratorA d1 = new ConcreteDecoratorA();
        ConcreteDecoratorB d2 = new ConcreteDecoratorB();
        c = d1.SetComponent(c); // "c" dinamik olarak yeni bir duruma sahip oldu.
        c.Operation();
        c = d2.SetComponent(c); // "c" dinamik olarak yeni bir davranışa sahip oldu.
        c.Operation();
    }
}

/* Program Çıktısı:
ConcreteComponent.Operation()
Bileşen Yeni bir Duruma Sahiptir:AddedState
ConcreteComponent.Operation()
Bileşen Yeni bir Duruma Sahiptir:AddedState
Bileşen Yeni Bir Davranışa Sahiptir:AddedBehaviour()

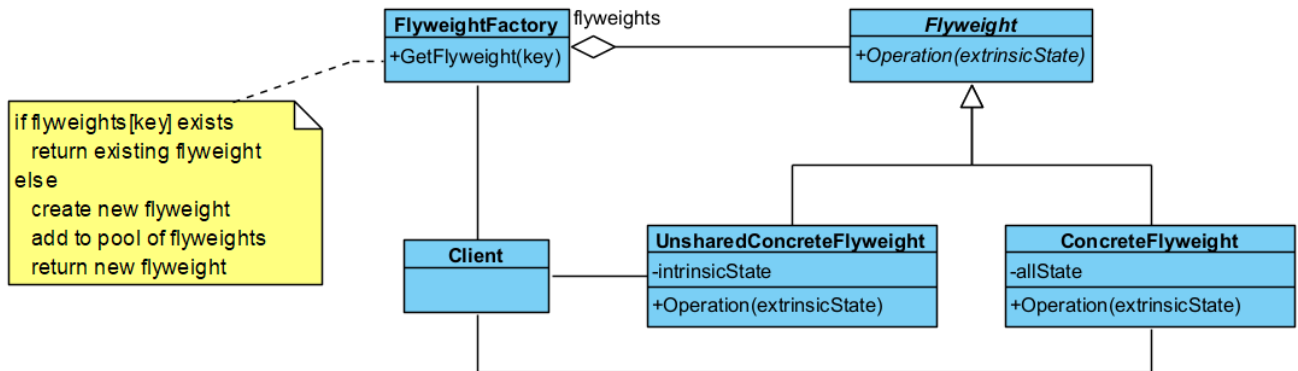
...Program finished with exit code 0
*/

```

Sineksiklet Deseni

Sineksiklet deseninde (flyweight pattern), birçok küçük nesneden oluşan bir sistemi paylaşarak kullanma hedeflenmiştir. Temel olarak sineksiklet, paylaşılan bir nesnedir ve eş zamanlı olarak küçük nesneleri kullanır. Yani sineksiklet nesnesi sözü geçen her bir küçük nesne gibi davranır.

Bu desen, bir metin editörü ile açıklanabilir. Metin editöründe yazılacak her bir harf, font bilgisine ve büyüklüğe sahiptir. İşte buradaki her bir harf ile kastedilen, sineksiklet deseninde konu olan küçük nesnelerdir. Bu nesneler bir araya gelerek satırları ve kolonları oluşturur. Sineksiklet nesnesi ise yerine göre bu harflerin her birinin yerine geçer.



Şekil 44. Sineksiklet Deseni UML Diyagramı

Yukarıdaki UML dokümanı incelendiğinde; **FlyweightFactory** küçük nesnelerden büyük resmi oluşturan fabrikayı temsil etmektedir. Birçok uygulamada durumlar geçicidir (extrinsic state) ve bu geçici durumlar saklanmazlar. Geçici durumlarla yapılan işlemler sonunda, kalıcı durumlar (intrinsic state) olur ve nesnelerde saklanır. Geçici durumlarla şekillenen ortak bir nesne (shared object) tanımıyla küçük nesneler tanımlanır. Bu küçük nesnelerin her biri sineksiklet (flyweight) olarak

adlandırılır. Ortak özellik taşımayan nesneler (**unshared object**) de büyük resim içinde yer alabilir. Bu nesneler de ortaklık dışı sineksiklet (**unshared flyweight**) olarak adlandırılır. Fabrika geçici durumlarla şekillenen sineksiklet nesneler üretir ve büyük resmi oluşturur.

Sineksiklet nesneleri ortak kullanıldığından, uygulama nesnenin kimliğine bağımlı değildir. Bu desen çok sık kullanılmaz ancak aşağıdaki durumların çoğu varsa kullanılabilir;

- Bir uygulama geniş çaplı olup çok miktarda nesne içeriyorsa,
- Nesneleri durumlarıyla olduğu gibi saklamanın maliyeti yüksekse,
- Nesnelerin birçok durumu geçici ve saklanmasına gerek yoksa,
- Birçok nesneden oluşan bir nesne grubu, ortak kullanılan bir nesnedeki geçici değişime bağlı olarak değişebiliyorsa,

İlk önce soyut **Flyweight** sınıfı ve somut sınıflarını kodlayalım;

```
abstract class Flyweight
{
    public abstract void Operation(int extrinsicstate);
}
class ConcreteFlyweight : Flyweight
{
    public override void Operation(int extrinsicstate)
    {
        Console.WriteLine("ConcreteFlyweight: " + extrinsicstate);
    }
}
class UnsharedConcreteFlyweight : Flyweight
{
    public override void Operation(int extrinsicstate)
    {
        Console.WriteLine("UnsharedConcreteFlyweight: " + extrinsicstate);
    }
}
```

Şimdi de **FlyweightFactory** sınıfını kodlayalım;

```
class FlyweightFactory
{
    private Hashtable flyweights = new Hashtable();
    public FlyweightFactory()
    {
        flyweights.Add("X", new ConcreteFlyweight());
        flyweights.Add("Y", new ConcreteFlyweight());
        flyweights.Add("Z", new ConcreteFlyweight());
    }
    public Flyweight GetFlyweight(string key)
    {
        return ((Flyweight)flyweights[key]);
    }
}
```

Son olarak de istemci sınıfı yazalım;

```
class Client
{
    static void Main(string[] args)
    {
        int extrinsicstate = 22; // Arbitrary extrinsic state
        FlyweightFactory f = new FlyweightFactory();
        // Work with different flyweight instances
        Flyweight fx = f.GetFlyweight("X");
        fx.Operation(--extrinsicstate);
        Flyweight fy = f.GetFlyweight("Y");
        fy.Operation(--extrinsicstate);
    }
}
```

```

        Flyweight fz = f.GetFlyweight("Z");
        fz.Operation(--extrinsicstate);
        UnsharedConcreteFlyweight uf = new UnsharedConcreteFlyweight();
        uf.Operation(--extrinsicstate);
    }
}
/*Program çalıştığında:
ConcreteFlyweight: 21
ConcreteFlyweight: 20
ConcreteFlyweight: 19
UnsharedConcreteFlyweight: 18

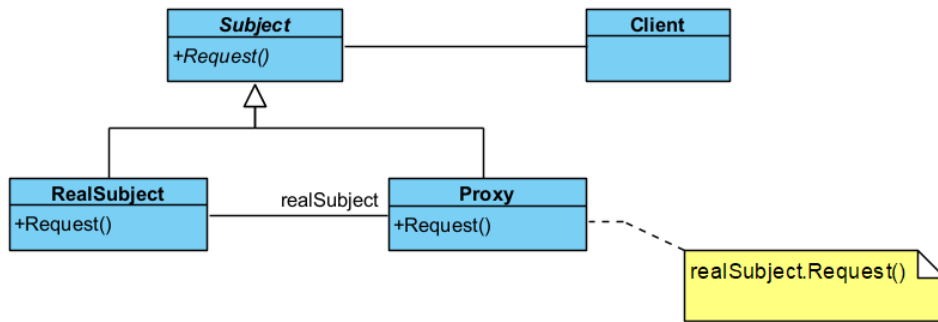
...Program finished with exit code 0
*/

```

Vekil Deseni

Vekil deseni (proxy pattern), kullanılacak ve erişilecek bir nesnenin yerine geçen bir başka nesneye ihtiyaç duyulduğunda kullanılır. İstemci (client), nesnenin doğrudan kendisine değil, vekil (proxy) nesneye ulaşır ve bu nesneye isteklerini iletir. Kısaca elinden her iş gelen veya diğer nesneler hakkında bilgi sahibi olan bir nesneye ihtiyaç duyulduğunda kullanılır.

- Bir nesne, bulunduğu yerden uzaktaki bir nesneyi kullanacaksa uzak vekil (remote proxy) kullanılır.
- Bir nesneye başka nesneler tarafından erişim kısıtlanacaksa koruma vekili (protection proxy) olarak adlandırılır.
- Bir nesneye erişim sırasında başka işlemler yapılacaksa vekil, akıllı referans (smart reference) olarak adlandırılır. Böyle durumda vekil, nesneye bir erişim yapıldığında, diğer nesnelerin erişmemesi için kullanılacak nesneye kilit koyabilir. Akıllı referans, akıllı gösterici (smart pointer) olarak da adlandırılır.



Şekil 45. Vekil Deseni UML Diyagramı

UML diyagramındaki **RealSubject**, bazı temel iş mantıklarını içerir. Genellikle, giriş verilerini düzeltme gibi hassas olabilen bazı yararlı işler yapma yeteneğine sahiptir. Bir **Proxy**, **RealSubject** kodunda herhangi bir değişiklik yapmadan bu sorunları çözebilir.

İstemci kodunun hem **RealSubject** hem de **Proxy** çalışabilmesi için tüm nesnelerle **Subject** ara yüzü üzerinden çalışması gerekir. Ancak gerçek hayatta, istemciler çoğunlukla **RealSubject** ile doğrudan çalışır. Bu durumda, deseni daha kolay uygulamak için **Proxy**, **RealSubject** sınıfından genişletebilirsiniz.

Şimdi **Subject** sanal sınıfı ve somut sınıflarını kodlayalım;

```

abstract class Subject
{
    public abstract void Request();
}
class RealSubject : Subject
{
    public override void Request()
    {
        Console.WriteLine("Called RealSubject.Request()");
    }
}

```

```

class Proxy : Subject
{
    RealSubject realSubject;
    public override void Request()
    {
        if (realSubject == null)
        {
            realSubject = new RealSubject();
        }
        realSubject.Request();
    }
}

```

Son olarak istemci kodunu kodlayalım;

```

class Client
{
    static void Main(string[] args)
    {
        Proxy proxy = new Proxy();
        proxy.Request();
    }
}
/* Program Çıktısı:
Called RealSubject.Request()

...Program finished with exit code 0
*/

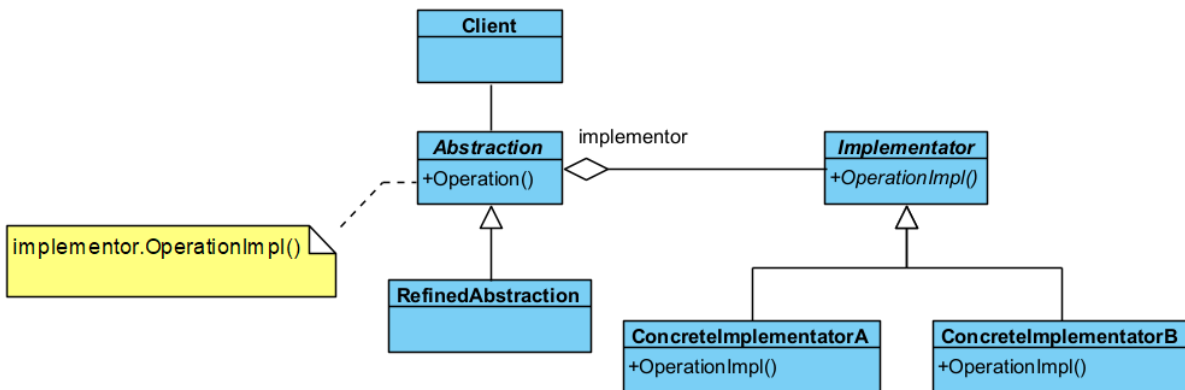
```

Köprü Deseni

Köprü deseni (bridge pattern), işi yapan alt yüklenici veya taşeron (implementator) nesne ile işi yaptıran nesne (client) arasındaki bağımlılığı soyutlama ile ortadan kaldırır. Yani birbiriyle oldukça iç içe olan kodlama ile soyutlamayı ortak ara yüz ile birbirinden uzaklaştırır. Sistemin genişlemesine izin verir.

“Bu desen, yazılımlarda oldukça çok kullanılan bir desendir. Çünkü taşeronlar değişse de yeni taşeron eklense de istemci tarafında bir şey değişmez. Köprü deseni aşağıdaki durumlarda kullanılır;

- Taşeron nesneye kalıcı bir bağımlılık istenmediği durumlar. Çalıştırma anında (run time) taşeronlar arasında geçiş yapmayı sağlar.
- Birden çok taşeron kullanarak sistemin genişlemesine ihtiyaç olduğu durumlar.
- Taşeronlar üzerindeki değişiklikler, istemciyi etkilemez. İstemci tarafında derleme olmadan yazılımın değiştirilmesi sağlanır.
- Projede aynı anda farklı birden çok taşeron tasarımı yapılabildiğinden, proje parçalara ayrılarak kendi içinde hızla sınıf tasarımı yapılabilir. Bu durum Rumbaugh tarafından iç içe genelleştirme (nested generalization) olarak adlandırılmıştır.



Şekil 46. Köprü Deseni UML Diyagramı

İlk önce taşeron olan soyut **Implementator** sınıfı ve somut alt sınıflarını tanımlayalım;

C# ile Nesne Yönelimli Programlama ve Entity Framework: <https://github.com/HoydaBre/csharp>

```
abstract class Implementator
{
    public abstract void OperationImpl();
}
class ConcreteImplementatorA : Implementator
{
    public override void OperationImpl()
    {
        Console.WriteLine("ConcreteImplementatorA İşlemi");
    }
}
class ConcreteImplementatorB : Implementator
{
    public override void OperationImpl()
    {
        Console.WriteLine("ConcreteImplementatorB İşlemi");
    }
}
```

Şimdi de yüklenici olan **Abstraction** sınıfını ve alt somut sınıfını kodlayalım;

```
abstract class Abstraction
{
    protected Implementator implementator;
    public Implementator Implementator
    {
        set { implementator = value; }
    }
    public virtual void Operation()
    {
        implementator.OperationImpl();
    }
}
class RefinedAbstraction : Abstraction
{
    public override void Operation()
    {
        implementator.OperationImpl();
    }
}
```

Son olarak istemi tarafını kodlayalım;

```
class Client
{
    static void Main(string[] args)
    {
        Abstraction ab = new RefinedAbstraction();
        // Set implementation and call
        ab.Implementator = new ConcreteImplementatorA();
        ab.Operation();
        ab.Implementator = new ConcreteImplementatorB();
        ab.Operation();
    }
}
/* Program Çıktısı:
ConcreteImplementatorA İşlemi
ConcreteImplementatorB İşlemi

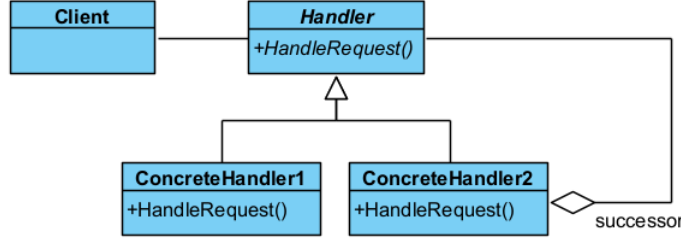
...Program finished with exit code 0
*/
```

Davranışla İlgili Tasarım Desenleri

Davranışla ilgili desenler (**behavioral patterns**) nesnelerin çeşitli olaylar karşısında gösterecekleri davranışlara çözüm getirmektedirler.

Sorumluluk Zinciri Deseni

Bazı durumlarda bir nesne, kendinin yapmayacağı bir işlemi **halefine** (**successor**) devredebilir. Ya da nesnenin kendi sorumluluğunu aşan durumlarda ilgili diğer nesneye işlem yaptırılır. İşte bu durumda **sorumluluk zinciri deseni** (**chain of responsibility pattern**) kullanılır.



Şekil 47. Sorumluluk Zinciri Deseni UML Diyagramı

Şimdi soyut **Handler** sınıfı ve somut sınıflarını kodlayalım;

```

abstract class Handler
{
    public abstract void handleRequest(int pRequest);
}

class ConcreteHandler1: Handler
{
    public override void handleRequest(int pRequest)
    {
        Console.WriteLine("Yetki Devretmeyen ConcreteHandler1");
        Console.WriteLine("{0} ile gelen istek yerine getirildi!", pRequest);
    }
}

class ConcreteHandler2 : Handler
{
    private Handler successor;
    private int threshold = 10;
    public ConcreteHandler2(Handler pSuccessor)
    {
        successor = pSuccessor;
    }
    public override void handleRequest(int pRequest)
    {
        if (pRequest < threshold)
        {
            Console.WriteLine("Yetki Devretmeyen ConcreteHandler2");
            Console.WriteLine("{0} ile gelen istek yerine getirildi!", pRequest);
        }
        else
        {
            Console.WriteLine("ConcreteHandler2 Yetkisini Devretti! ");
            successor.handleRequest(pRequest);
        }
    }
}
  
```

Son olarak istemci kısmını kodlayalım;

```

class Client
{
    static void Main(string[] args)
  
```

```

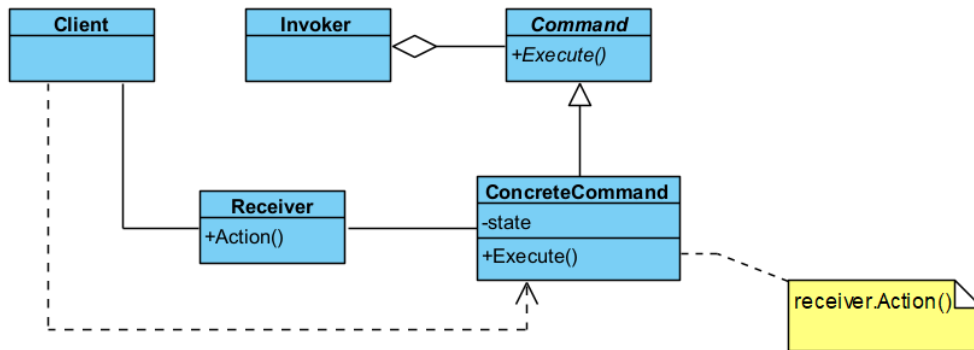
{
    int[] requests = { 5, 26, 10, 15, 9, 45 };
    Console.Write("Requests:");
    foreach (int request in requests)
        Console.Write(request + " ");
    Console.WriteLine();
    Handler müdür = new ConcreteHandler1(); //Yetkili
    müdür.handleRequest(5);
    müdür.handleRequest(20);
    Handler memur = new ConcreteHandler2(müdür); //Kısmi yetkili
    foreach (int request in requests)
        memur.handleRequest(request);
}
}
/* Program Çıktısı:
Requests:5 26 10 15 9 45
Yetki Devretmeyen ConcreteHandler1
5 ile gelen istek yerine getirildi!
Yetki Devretmeyen ConcreteHandler1
20 ile gelen istek yerine getirildi!
Yetki Devretmeyen ConcreteHandler2
5 ile gelen istek yerine getirildi!
ConcreteHandler2 Yetkisini Devretti!
Yetki Devretmeyen ConcreteHandler1
26 ile gelen istek yerine getirildi!
ConcreteHandler2 Yetkisini Devretti!
Yetki Devretmeyen ConcreteHandler1
10 ile gelen istek yerine getirildi!
ConcreteHandler2 Yetkisini Devretti!
Yetki Devretmeyen ConcreteHandler1
15 ile gelen istek yerine getirildi!
Yetki Devretmeyen ConcreteHandler2
9 ile gelen istek yerine getirildi!
ConcreteHandler2 Yetkisini Devretti!
Yetki Devretmeyen ConcreteHandler1
45 ile gelen istek yerine getirildi!

...Program finished with exit code 0
*/

```

Komut Deseni

Bazı durumlarda bir nesne diğer bir nesneye **ileti** (message) verip bir işlem başlattığında, bu işlem bitmeden başka işlemler yapmak isteyebilir. İşte bu tür durumlarda **komut deseni** (command pattern) kullanılır.



Şekil 48. Komut Deseni UML Diyagramı

Bu desende istemcinin istekleri, **Invoker** nesnesi tarafından kuyruğa sokulur ve kuyruğa sokulan adımların her biri **Command** nesnesi tarafından asıl işi yapan **Receiver** nesnesine yaptırılır. Bu durum bize **geri alma** (undo) işlemi yapmamızı sağlar.

İlk önce asıl işi yapacak **Receiver** sınıfını kodluyoruz;

```
class Receiver
{
    public void Action()
    {
        Console.WriteLine("Receiver.Action() İşlevi Çağrıldı");
    }
    public void Action(string pParam)
    {
        Console.WriteLine("Receiver.Action({0}) İşlevi Çağrıldı",pParam);
    }
    public void ActionA(string pParam)
    {
        Console.WriteLine("Receiver.ActionA({0}) İşlevi Çağrıldı", pParam);
    }
    public void ActionB(string pParam)
    {
        Console.WriteLine("Receiver.ActionB({0}) İşlevi Çağrıldı", pParam);
    }
}
```

Sonrasında her bir komutu tutacak soyut **Command** sınıfı ve somut sınıflarını yazıyoruz;

```
abstract class Command
{
    public abstract void Execute();
}
class ConcreteCommand : Command
{
    private Receiver receiver = new Receiver();
    private string state;

    public ConcreteCommand(string pState)
    {
        state = pState;
    }
    public override void Execute()
    {
        receiver.Action();
    }
}
class SimpleConcreteCommand: Command
{
    private Receiver receiver = new Receiver();
    private string state;
    public SimpleConcreteCommand(string pState)
    {
        state = pState;
    }
    public override void Execute() {
        receiver.Action(state);
    }
}
class ComplexConcreteCommand: Command
{
    private Receiver receiver = new Receiver();
    private string state;
    public ComplexConcreteCommand(string pState)
    {
        state = pState;
    }
    public override void Execute() {
```

```

        receiver.ActionA(state);
        receiver.Action();
        receiver.ActionB(state);
    }
}

```

Daha sonra ihtiyaç olursa birden fazla komut tutan ve çalıştıran **Invoker** sınıfını tanımlıyoruz;

```

class Invoker
{
    private List<Command> commands;
    public Invoker(List<Command> pList)
    {
        commands = pList;
    }
    public void addCommand(Command pCommand)
    {
        this.commands.Add(pCommand);
    }
    public void doCommands()
    {
        foreach (Command command in commands)
            command.Execute();
    }
}

```

Son olarak istemci kodunu yazıyoruz;

```

class Client
{
    static void Main(string[] args)
    {
        Command simpleCommand = new SimpleConcreteCommand("Yaz");
        simpleCommand.Execute();

        Command complexCommand = new ComplexConcreteCommand("Hem Dosyaya Hem Yazıcıya Yaz");

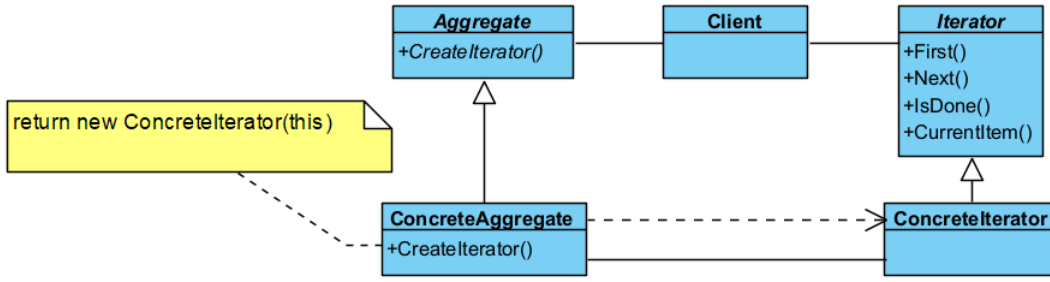
        List< Command> commandList = new List<Command>();
        commandList.Add(simpleCommand);
        commandList.Add(complexCommand);
        Invoker invoker = new Invoker(commandList);
        invoker.doCommands();
    }
}
/* Program çalıştığında;
Receiver.Action(Yaz) İşlevi Çağrıldı
Receiver.Action(Yaz) İşlevi Çağrıldı
Receiver.ActionA(Hem Dosyaya Hem Yazıcıya Yaz) İşlevi Çağrıldı
Receiver.Action() İşlevi Çağrıldı
Receiver.ActionB(Hem Dosyaya Hem Yazıcıya Yaz) İşlevi Çağrıldı

...Program finished with exit code 0
*/

```

Yineleyici Deseni

Yineleyici deseninde (iterator pattern), birden fazla elemandan oluşan küme (aggregate) içindeki her bir elemana sırayla erişim (iteration) sağlar. Erişim işleminde, istemcinin kümenin nasıl yapılandırıldığı konusunda bilgi sahibi olması gerekmez. Bu desende, istemci tarafından veri kümesi soyut (abstract) olarak kullanılmış olur. Yani istemciye, veri yapısından (data structure) bağımsız bir şekilde verilere erişim sağlayan bir ara yüz (interface) sunulur.



Şekil 49. Yineleyici Deseni UML Diyagramı

Burada **Aggregate** ve **Iterator** sınıflarını ilk önce tanımlıyoruz;

```

abstract class Aggregate
{
    public abstract Iterator CreateIterator();
}
class ConcreteAggregate : Aggregate
{
    private ArrayList items = new ArrayList();
    public override Iterator CreateIterator()
    {
        return new ConcreteIterator(this);
    }
    public int Count
    {
        get { return items.Count; }
    }
    public object this[int index]
    {
        get { return items[index]; }
        set { items.Insert(index, value); }
    }
}
abstract class Iterator
{
    public abstract object First();
    public abstract object Next();
    public abstract bool IsDone();
    public abstract object CurrentItem();
}
class ConcreteIterator : Iterator
{
    private ConcreteAggregate aggregate;
    private int current = 0;
    public ConcreteIterator(ConcreteAggregate aggregate)
    {
        this.aggregate = aggregate;
    }
    public override object First()
    {
        return aggregate[0];
    }
    public override object Next()
    {
        object ret = null;
        if (current < aggregate.Count - 1)
        {
            ret = aggregate[++current];
        }
        return ret;
    }
    public override object CurrentItem()
    {

```

```

    {
        return aggregate[current];
    }
    public override bool IsDone()
    {
        return current >= aggregate.Count ? true : false;
    }
}

```

Şimdi de istemci kısmını kodluyoruz;

```

class Client
{
    static void Main(string[] args)
    {
        ConcreteAggregate a = new ConcreteAggregate();
        a[0] = "Item A";
        a[1] = "Item B";
        a[2] = "Item C";
        a[3] = "Item D";
        ConcreteIterator i = new ConcreteIterator(a);
        // Burada Yığın birden çok nesneden oluşan heshangi bir
        // nesneler topluluğudur.
        Console.WriteLine(
            "Yığın (collection) üzerinde yineleme:");
        // Yineleyici, yığın hakkında bilgi sahibi olmadan
        // elemanlar üzerinde işlem yapmamızı sağlar.
        // Bir başka deyişle kullanıcı yığına bağlı kalmaz.
        object item = i.First();
        while (item != null)
        {
            Console.WriteLine(item);
            item = i.Next();
        }
    }
}
/*
Yığın (collection) üzerinde yineleme:
Item A
Item B
Item C
Item D

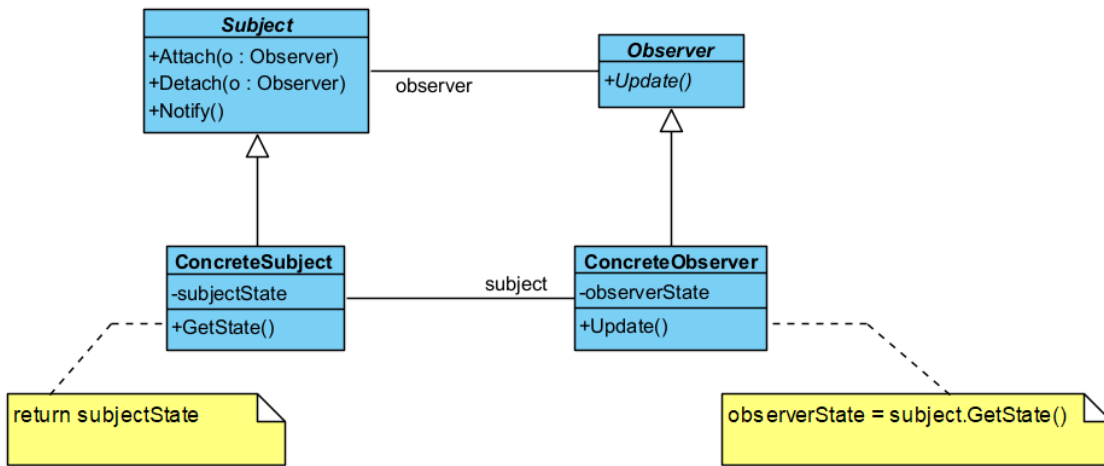
...Program finished with exit code 0
*/

```

Gözlemci Deseni

Gözlemci deseni (**observer pattern**), sistemdeki diğer nesnelerin durum değişiklikleri hakkında bir veya daha fazla nesnenin bilgilendirilmesini sağlar. Bağımlı olunan nesnenin durumu veya işin **konusu** (**subject**) değiştiğinde, bağımlı olan veya o işi izleyen **gözlemcilerin** (**observer**) kendileri de güncellenir. Bu desen aşağıdaki durumlarda kullanılır;

- Bir **soyutlama** (**abstraction**) sonucu ortaya çıkan bağımsız iki **farklı yönün** (**aspect**) ortaya çıktığı durumlarda, bu yönler birbirinden bağımsız olarak geliştirilebilir ve değiştirilebilir.
- Bir değişikliğin başka değişiklikleri gerektirdiği durumlar.
- Bir nesnenin, diğer nesnelerdeki değişiklikleri kendine vazife etmemesini sağlayan durumlar.



Şekil 50. Gözlemci Deseni UML Diyagramı

İlk önce sanal **Observer** sınıfını tanımlayalım;

```

abstract class Observer
{
    public abstract void Update();
}
  
```

Daha sonra sanal **Subject** ve somut **ConcreteSubject** sınıfını kodlayalım;

```

abstract class Subject
{
    private ArrayList observers = new ArrayList();
    public void Attach(Observer observer)
    {
        observers.Add(observer);
    }
    public void Detach(Observer observer)
    {
        observers.Remove(observer);
    }
    public void Notify()
    {
        foreach (Observer o in observers)
        {
            o.Update();
        }
    }
}
// "ConcreteSubject"
class ConcreteSubject : Subject
{
    private string subjectState;
    public string SubjectState
    {
        get { return subjectState; }
        set { subjectState = value; }
    }
}
  
```

Daha sonra somut **ConcreteObserver** Sınıfını tanımlayalım;

```

class ConcreteObserver : Observer
{
    private string name;
    private string observerState;
    private ConcreteSubject subject;
}
  
```

```

public ConcreteObserver(ConcreteSubject subject,
    string name)
{
    this.subject = subject;
    this.name = name;
}
public override void Update()
{
    observerState = subject.SubjectState;
    Console.WriteLine(
        "Observer {0}'s new state is {1}", name,
        observerState);
}
public ConcreteSubject Subject
{
    get { return subject; }
    set { subject = value; }
}
}

```

Son olarak istemci tarafını kodlayalım;

```

class Client
{
    static void Main(string[] args)
    {
        ConcreteSubject s = new ConcreteSubject();
        s.Attach(new ConcreteObserver(s, "X"));
        s.Attach(new ConcreteObserver(s, "Y"));
        s.Attach(new ConcreteObserver(s, "Z"));
        s.SubjectState = "ABC";
        s.Notify();
    }
}
/*Program Çıktısı:
Observer X's new state is ABC
Observer Y's new state is ABC
Observer Z's new state is ABC

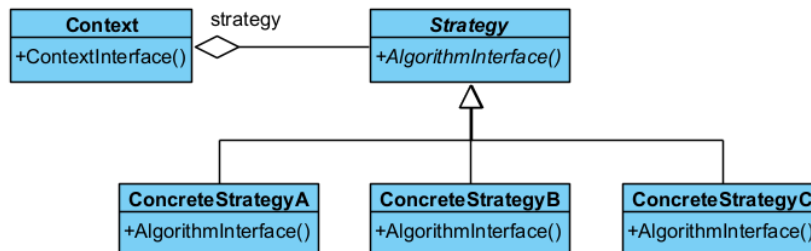
...Program finished with exit code 0
*/

```

Strateji Deseni

Strateji deseni (strategy pattern), belirli bir davranışı gerçekleştirmek için değiştirilebilen sarmalanmış (encapsulated) algoritmalar kümesini tanımlar. Bu desen aşağıdaki durumda kullanılır;

- Aralarında ilişki bulunan birçok sınıfın davranışlara bağlı olarak anlaşmazlığa düşmesini önlemek için,
- Birden çok algoritmanın olması durumunda,
- Algoritma, istemcinin bilmeyeceği bir veri yapısına sahip olduğu durumlarda,
- Bir sınıf kendi yöntemlerinde çok fazla sayıda duruma göre seçim (conditional choice) kullanıyorsa.



Şekil 51. Strateji Deseni UML Diyagramı

Bu desenin **bağlam** (**Context**) nesnesi, diğer tasarım desenlerinin biri ile iç içe kullanılması halinde, **sıfır desen** (**null pattern**) olarak adlandırılır. Sıfır desende, **Context** nesnesi akıllıdır ve her zaman ne yapacağını bilir.

İlk önce **Strategy** somut sınıfı ve alt sınıflarını kodlayalım;

```
abstract class Strategy
{
    public abstract void AlgorithmInterface();
}
class ConcreteStrategyA : Strategy
{
    public override void AlgorithmInterface()
    {
        Console.WriteLine(
            "ConcreteStrategyA.AlgorithmInterface() Çağrıldı.");
    }
}
class ConcreteStrategyB : Strategy
{
    public override void AlgorithmInterface()
    {
        Console.WriteLine(
            "ConcreteStrategyB.AlgorithmInterface() Çağrıldı.");
    }
}
class ConcreteStrategyC : Strategy
{
    public override void AlgorithmInterface()
    {
        Console.WriteLine(
            "ConcreteStrategyC.AlgorithmInterface() Çağrıldı.");
    }
}
```

Şimdi de **Context** sınıfını kodlayalım;

```
class Context
{
    Strategy strategy;
    public Context(Strategy strategy)
    {
        this.strategy = strategy;
    }
    public void ContextInterface()
    {
        strategy.AlgorithmInterface();
    }
}
```

Son olarak istemciyi kodlayalım;

```
class Client
{
    static void Main(string[] args)
    {
        Context context;
        // Three contexts following different strategies
        context = new Context(new ConcreteStrategyA());
        context.ContextInterface();
        context = new Context(new ConcreteStrategyB());
        context.ContextInterface();
        context = new Context(new ConcreteStrategyC());
        context.ContextInterface();
    }
}
```

```

    }
}
/*Program Çıktısı:
ConcreteStrategyA.AlgorithmInterface() Çağrıldı.
ConcreteStrategyB.AlgorithmInterface() Çağrıldı.
ConcreteStrategyC.AlgorithmInterface() Çağrıldı.

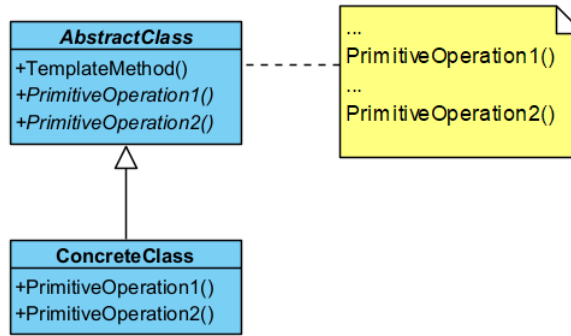
...Program finished with exit code 0
*/

```

Şablon Yöntem Deseni

Şablon yöntem deseninde (template method pattern), bir algoritmanın çerçevesini belirler ve uygulayıcı sınıfların gerçek davranışı tanımlamasına olanak tanır. Kullanılacak algoritmalarından hangilerinin standart hangilerinin somut sınıflara özgü olacağı belirlenmelidir;

- İçinde “Bizi çağırmanın, biz sizi çağıracağız” sloganını taşıyan yöntemler olan bir soyut bir taban sınıf tasarlanmalıdır.
- Taban sınıf içinde algoritmanın kabuğunu oluşturacak standart adımları içeren şablon yöntem tanımlanır.
- Somut sınıflarda detaylandırılacak yöntemlere ilişkin sanal yöntemler taban sınıfta tanımlanır.
- Şablon yöntem içinde algoritmayı oluşturan yöntemler çağrılır.
- Şablon yöntem içinde yer almayan tüm detay kodlamalar somut sınıflar için de yapılır.



Şekil 52. Şablon Yöntem Deseni UML Diyagramı

Şimdi Programı kodlayalım;

```

abstract class AbstractClass
{
    public abstract void PrimitiveOperation1();
    public abstract void PrimitiveOperation2();
    // The "Template method"
    public void TemplateMethod()
    {
        PrimitiveOperation1();
        PrimitiveOperation2();
    }
}
// "ConcreteClass"
class ConcreteClassA : AbstractClass
{
    public override void PrimitiveOperation1()
    {
        Console.WriteLine(
            "ConcreteClassA.PrimitiveOperation1()");
    }
    public override void PrimitiveOperation2()
    {
        Console.WriteLine(
            "ConcreteClassA.PrimitiveOperation2()");
    }
}

```

```

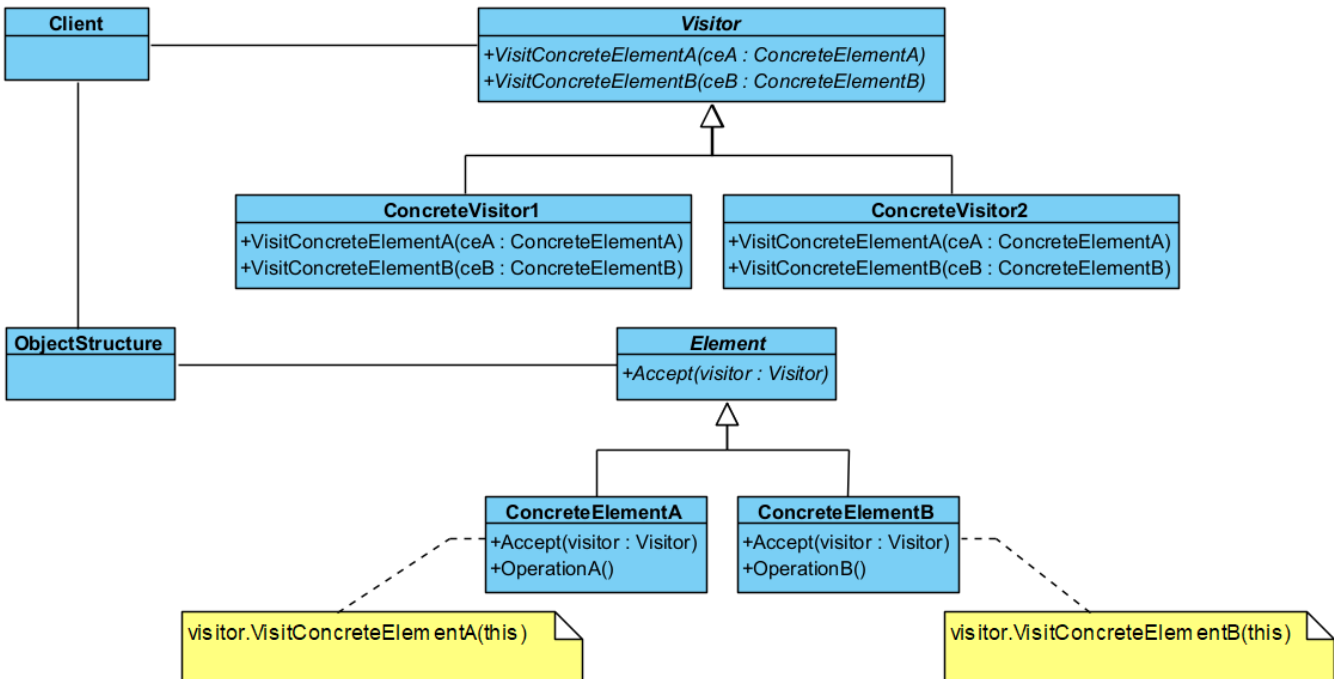
}
class ConcreteClassB : AbstractClass
{
    public override void PrimitiveOperation1()
    {
        Console.WriteLine(
            "ConcreteClassB.PrimitiveOperation1()");
    }
    public override void PrimitiveOperation2()
    {
        Console.WriteLine(
            "ConcreteClassB.PrimitiveOperation2()");
    }
}
}
class Client
{
    static void Main(string[] args)
    {
        AbstractClass c;
        c = new ConcreteClassA();
        c.TemplateMethod();
        c = new ConcreteClassB();
        c.TemplateMethod();
    }
}
}
/*Program Çıktısı:
ConcreteClassA.PrimitiveOperation1()
ConcreteClassA.PrimitiveOperation2()
ConcreteClassB.PrimitiveOperation1()
ConcreteClassB.PrimitiveOperation2()

...Program finished with exit code 0
*/

```

Ziyaretçi Deseni

Ziyaretçi deseni (**visitor pattern**), bir sınıfa ilişkin verileri kullanarak işlem yapan bir başka yeni sınıf tanımlanmak istendiğinde kullanılır. Yani çalışma zamanında bir veya daha fazla işlemin bir nesne kümesine uygulanmasına izin verir ve işlemleri nesne yapısından ayırır.



Şekil 53. Ziyaretçi Deseni UML Diyagramı

Başka amaçla tanımlanan yeni sınıfın (**Visitor**), mevcut sınıfın (**Element**) verilerine ulaşmak istemesi halinde bu desen kullanılır. Bu desen aşağıdaki üstünlükleri sağlar;

- Yeni işlemleri (**operation**) sisteme eklemeyi kolaylaştırır.
- Birbiriyle ilişkili olan işlemleri ilişkili olmayandan ayırmaya yardımcı olur.
- Yeni **ConcreteElement** nesnelerinin eklenmesini zorlaştırır.
- Bütün nesne hiyerarşisi baştanbaşa ziyaret edilebilir.
- Sınıflardaki tüm durumlar (**state**) biriktirilebilir.
- Bazı durumlarda **sarma** (**encapsulation**) işlemini engeller. Yani nesnenin iç durumlarından hareketle herkese açık işlemler tanımlanabilir.

Bu desen aşağıdaki durumlarda kullanılır;

- Nesneler birden fazla olup birçok ara yüze sahip olduğunda, bu nesnelerin somut sınıfları ile bir işlem yapmak gerektiğinde,
- Sınıflar üzerinde yapılacak işlemlerin, sınıflar üzerinde bir iz bırakmamasını sağlamak amacıyla,
- Nadir değişen sınıf yapılarına, sınıf yapısını değiştirmeden, yeni işlemler eklemek gerektiğinde.

İlk önce **Visitor** sınıflarını kodlayalım;

```
abstract class Visitor
{
    public abstract void VisitConcreteElementA(ConcreteElementA concreteElementA);
    public abstract void VisitConcreteElementB(ConcreteElementB concreteElementB);
}
class ConcreteVisitor1 : Visitor
{
    public override void VisitConcreteElementA( ConcreteElementA concreteElementA)
    {
        Console.WriteLine("{0} visited by {1}",
            concreteElementA.GetType().Name,
            this.GetType().Name);
    }
    public override void VisitConcreteElementB(ConcreteElementB concreteElementB)
    {
        Console.WriteLine("{0} visited by {1}",
            concreteElementB.GetType().Name,
            this.GetType().Name);
    }
}
class ConcreteVisitor2 : Visitor
{
    public override void VisitConcreteElementA(ConcreteElementA concreteElementA)
    {
        Console.WriteLine("{0} visited by {1}",
            concreteElementA.GetType().Name,
            this.GetType().Name);
    }
    public override void VisitConcreteElementB(ConcreteElementB concreteElementB)
    {
        Console.WriteLine("{0} visited by {1}",
            concreteElementB.GetType().Name,
            this.GetType().Name);
    }
}
```

Daha sonra **Element** sınıflarını kodlayalım;

```
abstract class Element
{
    public abstract void Accept(Visitor visitor);
}
class ConcreteElementA : Element
{
    }
```



```

    public override void Accept(Visitor visitor)
    {
        visitor.VisitConcreteElementA(this);
    }
    public void OperationA() { }
}
class ConcreteElementB : Element
{
    public override void Accept(Visitor visitor)
    {
        visitor.VisitConcreteElementB(this);
    }
    public void OperationB() { }
}

```

Bir de **ObjectStructure** sınıfını kodlayalım;

```

class ObjectStructure
{
    private ArrayList elements = new ArrayList();
    public void Attach(Element element)
    {
        elements.Add(element);
    }
    public void Detach(Element element)
    {
        elements.Remove(element);
    }
    public void Accept(Visitor visitor)
    {
        foreach (Element e in elements)
        {
            e.Accept(visitor);
        }
    }
}

```

Son olarak istemci kodumuzu kodlayalım;

```

class Client
{
    static void Main(string[] args)
    {
        // Setup structure
        ObjectStructure o = new ObjectStructure();
        o.Attach(new ConcreteElementA());
        o.Attach(new ConcreteElementB());
        // Create visitor objects
        ConcreteVisitor1 v1 = new ConcreteVisitor1();
        ConcreteVisitor2 v2 = new ConcreteVisitor2();
        // Structure accepting visitors
        o.Accept(v1);
        o.Accept(v2);
    }
}
/* Programın Çıktısı:
ConcreteElementA visited by ConcreteVisitor1
ConcreteElementB visited by ConcreteVisitor1
ConcreteElementA visited by ConcreteVisitor2
ConcreteElementB visited by ConcreteVisitor2

...Program finished with exit code 0
*/

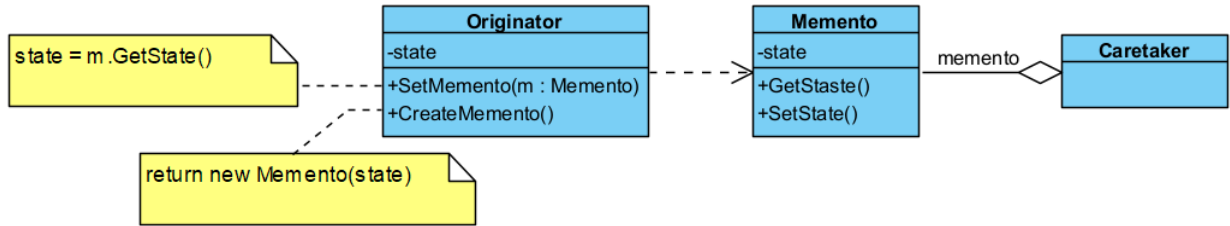
```

Ziyaretçi deseni uygulanırken iki kodlama (**implementation**) sorunu irdelenmelidir;

- **Çifte sevkıyat** (**double dispatch**), Bir sınıfı hiçbir şekilde değiştirmeden, söz konusu sınıfa etkin biçimde yöntem eklenebilmesi işlemidir. Bu çok bilinen bir tekniktir ve **Common Lisp Object System (CLOS)** gibi programlama dilleri tarafından desteklenir ve nesne üzerinde sarma (**encapsulation**) yapılmasına izin verilmez.
- C++, C#, Java gibi birçok nesne yönelimli programlama dili **tek sevkıyat** (**single dispatch**) yöntemi ile çalışır. Bu teknik ziyaretçi deseni için anahtardır. **Accept** yöntemi çifte sevkıyat sağlayan yöntemdir. Yöntemler, statik olarak yalnızca **Element** sınıfına bağlanmaz. **Element** sınıfı yapılacak işlemleri **Visitor** üzerinde birleştirir ve **Accept** yöntemi ile çalıştırma anında icra eder. İcra edilen yöntemler, **Visitor** ve **Element** veri tipi olmak üzere iki veri tipine bağlıdır.
- İşlemler dışarıda tutularak oluşturulan nesne yapısının sorumlusu kimdir? **Visitor**, oluşturulan nesne yapısının her bir elemanını tek tek ziyaret etmek zorundadır. Bunun nasıl ve hangi sırada yapılacağı belirgin değildir. Birçok durumda nesne yapısının kendisi bundan sorumlu tutulur. Nesne yapısı basit bir küme olarak tanımlanır ve **Accept** yöntemi her biri için çağrılır. Bazı durumlarda da **yineleyici deseni** (**iterator pattern**) kullanılır.

Hatıra Deseni

Hatıra deseni (**memento pattern**), nesnelerin bir andaki **durumlarını** (**state**) saklayıp bir başka zaman da bu duruma geri dönmek gerektiğinde kullanılır. Bir nesnenin iç durumunun yakalanmasına ve dışarda saklanmasına olanak tanır, böylece daha sonra geri yüklenebilir fakat tüm bunlar **sarmalamayı** (**encapsulation**) ihlal etmeden yapılır.



Şekil 54. Hatıra Deseni UML Diyagramı

Yukarıdaki diyagram incelendiğinde; **Originator** nesnesi, geçmişteki duruma dönecek nesneyi, **Memento** ise **Originator** nesnesinin bir anlık resmini çeken nesneyi, **Caretaker** ise **Memento** nesnelerini saklayan yardımcı nesneyi ifade etmektedir.

İlk olarak **Memento** sınıfını kodlayalım;

```
class Memento
{
    private string state;
    public Memento(string state)
    {
        this.state = state;
    }
    public string State
    {
        get { return state; }
    }
}
```

Daha sonra **Caretaker** sınıfını kodlayalım;

```
class Caretaker
{
    private Memento memento;
    public Memento Memento
    {
        set { memento = value; }
        get { return memento; }
    }
}
```

```
}
```

Bir de **Originator** sınıfını kodlayalım;

```
class Originator
{
    private string state;
    // Property
    public string State
    {
        get { return state; }
        set
        {
            state = value;
            Console.WriteLine("State = " + state);
        }
    }
    public Memento CreateMemento()
    {
        return (new Memento(state));
    }
    public void SetMemento(Memento memento)
    {
        Console.WriteLine("Restoring state:");
        State = memento.State;
    }
}
```

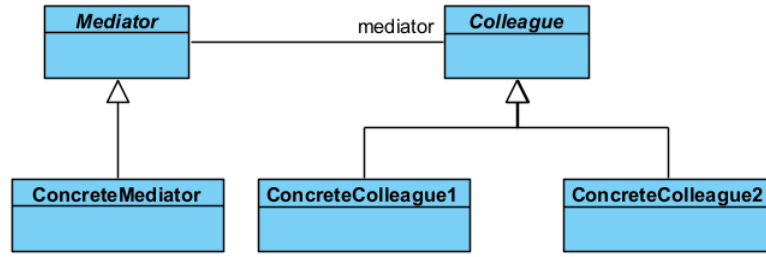
Son olarak istemci kodunu yazalım;

```
class Client
{
    static void Main(string[] args)
    {
        Originator o = new Originator();
        o.State = "On";
        // Store internal state
        Caretaker c = new Caretaker();
        c.Memento = o.CreateMemento();
        // Continue changing originator
        o.State = "Off";
        // Restore saved state
        o.SetMemento(c.Memento);
    }
}
/*Program Çıktısı:
State = On
State = Off
Restoring state:
State = On

...Program finished with exit code 0
*/
```

Arabulucu Deseni

Arabulucu deseni (**mediator pattern**), sınıfların arasındaki iletişimi basitleştirir. Bir iletişim nesnesi tanımlanır, diğer nesneler bu nesne üzerinden birbirleriyle haberleşir. Diğer nesnelerin birbiriyle doğrudan iletişim kurmasına izin verilmez. Amaç, iletişimin kontrol altına alınmasıdır. Bu desende **arabulucunun** (**mediator**) haberi olmadan hiçbir **kişi** (**colleauge**) birbiriyle haberleşmez. Ya da bir başka deyişle arabulucunun haberi olmadan hiç kimse birbiriyle görüşmez.



Şekil 55. Arabulucu Deseni UML Diyagramı

Bu desende, sistemde yalnızca bir adet arabulucu olacaksa soyut **Mediator** nesnesi tanımlanmaz. Ayrıca arabulucu ile kişiler arasındaki iletişim iki türlü yapılır;

- Birincisinde bu iletişim **gözlemci deseni (observer pattern)** ile sağlanır. **Colleague** sınıfı gözlemci deseniindeki **Subject** sınıf gibi davranır.
- İkincisinde ise **Mediator** nesnesine, **kişiler (colleauge)** görüşecekleri kişileri belirterek doğrudan ileti gönderirler.

İlk olarak **Mediator** sınıfını kodlayalım;

```

abstract class Mediator
{
    protected List<Colleague> colleagues = new List<Colleague>();
    public abstract void Attach(Colleague colleague);
    public abstract void Send(string message, Colleague colleague);
}
class ConcreteMediator : Mediator
{
    public override void Attach(Colleague colleague)
    {
        colleagues.Add(colleague);
    }
    public override void Send(string message, Colleague colleague)
    {
        Console.WriteLine("LOG: {0}, {1} Mesajını Gönderdi!", colleague.Name, message);

        foreach (Colleague college in colleagues)
            college.GetMessage(message);
    }
}
  
```

Daha Sonra **Colleague** sınıfını kodlayalım;

```

abstract class Colleague
{
    protected Mediator mediator;
    public string Name { get; set; }
    public Colleague(string name, Mediator mediator)
    {
        this.mediator = mediator;
        Name = name;
        mediator.Attach(this);
    }
    public abstract void SendMessage(string message);
    public abstract void GetMessage(string message);
}
class ConcreteColleague : Colleague
{
    public ConcreteColleague(string name, Mediator mediator) : base(name, mediator)
    {
        Name = name;
    }
}
  
```

```

public override void SendMessage(string message)
{
    mediator.Send(message, this);
}
public override void GetMessage(string message)
{
    Console.WriteLine("{0} gets message: {1}", Name, message);
}
}

```

Son olarak istemci sınıfını kodlayalım;

```

class Client
{
    static void Main(string[] args)
    {
        Mediator arabulucu = new ConcreteMediator();
        Colleague ilhan = new ConcreteColleague("İlhan", arabulucu);
        Colleague mehmet = new ConcreteColleague("Mehmet", arabulucu);
        Colleague abdullah = new ConcreteColleague("Abdullah", arabulucu);

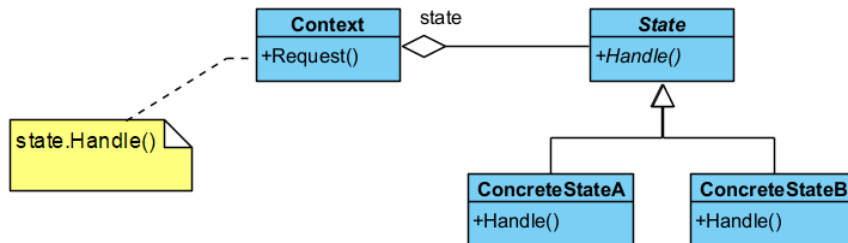
        ilhan.SendMessage("How are you?");
        mehmet.SendMessage("Fine, thanks");
        abdullah.SendMessage("I am good too");
    }
}
/* Program Çıktısı:
LOG: İlhan, How are you? Mesajını Gönderdi!
İlhan gets message: How are you?
Mehmet gets message: How are you?
Abdullah gets message: How are you?
LOG: Mehmet, Fine, thanks Mesajını Gönderdi!
İlhan gets message: Fine, thanks
Mehmet gets message: Fine, thanks
Abdullah gets message: Fine, thanks
LOG: Abdullah, I am good too Mesajını Gönderdi!
İlhan gets message: I am good too
Mehmet gets message: I am good too
Abdullah gets message: I am good too

...Program finished with exit code 0
*/

```

Durum Deseni

Durum deseni (**state pattern**), nesnenin durumunu davranışına bağlar, nesnenin içsel durumuna göre farklı şekillerde davranmasına olanak tanır.



Şekil 56. Durum Deseni UML Diyagramı

Bu desenin bağlam (**Context**) nesnesinin, diğer tasarım desenleri ile iç içe kullanılması halinde, **sıfır desen** (**null pattern**) olarak adlandırılır. Bu desenle ilgili olarak iki önemli şeyden bahsetmek gerekir:

- Sahip olunan duruma göre icra edilecek davranış, tanımlanan **State** sınıfına bağlıdır. Yani ilgili davranış somut sınıflardan biri tarafından icra edilir. Bu nedenle somut **State** sınıfları içinde tanımlanan yöntemler durumu kullanan bağlam (**Context**) sınıf içinde tanımlanmak zorundadır.

- Durumu kullanan bağlam (**Context**) sınıf içinde hangi duruma geçildiğinin anlaşılması için ilgili duruma ilişkin tanımlanan **özellik** (**property**) içinde set ifadesi kullanılmak zorundadır.

Bu desen netice olarak, nesnenin duruma özgü davranış göstermesini sağlar veya farklı durumlara göre davranışları birbirinden ayırır, **durum geçişlerini** (**state transition**) açığa çıkarır. Bu deseni kullanırken aşağıda verilen durumlar özellikle irdelenmelidir;

- Durum geçişleri kim tarafından tanımlanmalıdır? Bu desen, hangi şartlarda durumların ayrıştırılacağını belirtmez. Eğer şartlar belirgin ise bu işlem **Context** nesnesi tarafından yapılır. Bu şartlar belirgin değil ve karmaşık ise bu işlem **State** nesnesi ve **halefi** (**successor**) tarafından yapılır.
- Tablo tabanlı alternatif: Bu durumda durum geçişleri bir tablo aracılığı ile yapılır. Tablo üzerine sağlanacak her bir durum yazılır. Bu durumda, durum geçişlerini sağlayacak değişiklikler yeniden kodlama gerektirmez. Ancak bu yöntemde hız düşer. Durum geçişleri **tekdüze** (**uniform**) tanımlanmak zorundadır.
- **State** nesnelerini yaratma ve yok etme: Bu nesneler çalıştırma anında yaratılıp yok edildiklerinden, bu süre zarfında yapılacak işlemler için bir çözüm bulunmalıdır. İki yaklaşım vardır. Birincisi, **State** nesnesini ihtiyaç olduğunda yaratmak kullanmak ve öldürmektir. İkincisi ise bu nesneleri baştan yaratıp hiç öldürmemektir. Genellikle birinci yöntem tercih edilir.
- **Dinamik kalıtım** (**dynamic inheritance**) kullanma: Bazı durumlarda nesne, ait olduğu sınıfı çalıştırma anında değiştirir. Bu durumda talep edilen isteğin yerine getirilmesi tam olarak başarılabilir. Ancak birçok nesne yönelimli programlama dili bunu desteklemez.

Aşağıda durum nesnelerini değiştiren basit bir örnek verilmiştir;

```
class Context
{
    private State state, state1, state2;

    public Context(State state1, State state2)
    {
        this.state1 = state1;
        this.state2 = state2;
        state = state1;
    }

    private void durumDegistir()
    {
        if (state == state1)
            state = state2;
        else
            state = state1;
        Console.WriteLine("Context: Durum nesnesi değişti!");
    }
    public void Request()
    {
        state.Handle();
        durumDegistir();
    }
}

abstract class State
{
    public abstract void Handle();
}

class ConcreteStateA : State
{
    public override void Handle()
    {
        Console.WriteLine("ConcreteStateA: Handle method called.");
    }
}

class ConcreteStateB : State
```

```

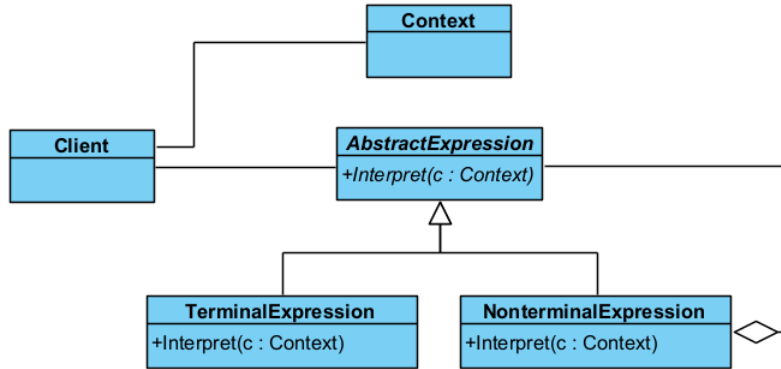
{
    public override void Handle()
    {
        Console.WriteLine("ConcreteStateB: Handle method called.");
    }
}
}
class Client
{
    static void Main(string[] args)
    {
        State durumA = new ConcreteStateA();
        State durumB = new ConcreteStateB();
        Context context = new Context(durumA, durumB);
        context.Request();
        context.Request();
        context.Request();
    }
}
}
/* Program Çıktısı:
ConcreteStateA: Handle method called.
Context: Durum nesnesi değişti!
ConcreteStateB: Handle method called.
Context: Durum nesnesi değişti!
ConcreteStateA: Handle method called.
Context: Durum nesnesi değişti!

...Program finished with exit code 0
*/

```

Yorumlayıcı Deseni

Yorumlayıcı deseni (interpreter pattern), programlama dili yorumlama özelliğini uygulamalarımıza katmanın yolunu gösterir. Yani bir dilbilgisi için bir temsili ve aynı zamanda dilbilgisini anlayıp ona göre hareket etmeyi sağlayacak bir mekanizmayı tanımlar.



Şekil 57. Yorumlayıcı deseni UML Diyagramı

Bu desen, yorumlanacak dilin dilbilgisi (grammar) kurallarının basit olduğu durumlarda kullanılır. Karmaşık dilbilgisi kurallarına sahip diller için yaratılacak sınıfların yönetimi oldukça zorlaşır. Burada, bağlam (Context) nesnesi dilbilgisi kurallarına sahip yorumlanacak nesneyi gösterir. **TerminalExpression**, en ilkel dil öğesini yorumlayacak nesnedir. **NonterminalExpression** ise ilkel dil öğelerinden oluşacak karmaşık yapıyı yorumlayacak nesneyi belirtir.

Bu deseni uygularken aşağıda verilen durumlar irdelenmelidir;

- Soyut sözdizimin oluşturulması: Bu desen yorumlanacak dile ilişkin soyut bir sözdizimin nasıl olması gerektiğini açıklamaz. Tabloya dayalı, ağaç yapısı şeklinde el becerisi ile veya doğrudan istemci tarafından yapılabilir.
- Yorumlayacak işlemin tanımlanması: Eğer işlemler ortak bir yapıdaysa yeni bir **Expression** nesnesi tanımlanarak yapılabilir. Daha karmaşık durumlarda ziyaretçi deseni kullanılır. Programlama

dillerinin çoğu soyut bir sözdizimine sahiptir ve yorumlanacak öğeler ayrı bir **Visitor** nesnesi tarafından yorumlanır.

- Cümle sonlarını bitiren ortak semboller için sineksiklet deseni kullanılabilir: Cümle sonlarındaki semboller genellikle bilgi saklamazlar. Yorumlamaya gerek duyulmazlar. Bu nedenle [sineksiklet deseni](#) ([flyweight pattern](#)) yorumlanacak yerler için bir eleme yapar.

Aşağıda kavramsal olarak bu desenin kolanmış örneği bulunmaktadır;

```
class Context
{
    private string context;
    public Context(string pContext)
    {
        this.context = pContext;
    }
}

abstract class AbstractExpression
{
    public abstract void interpret(Context context);
}

class TerminalExpression : AbstractExpression
{
    public override void interpret(Context context)
    {
        Console.WriteLine("Terminal Expression İşlemi Yorumluyor...");
    }
};

class NonterminalExpression : AbstractExpression
{
    private AbstractExpression abstractexpression;
    public override void interpret(Context context)
    {
        Console.WriteLine("Non Terminal Expression İşlemi Yorumluyor...");
        abstractexpression = new TerminalExpression();
        abstractexpression.interpret(context);
    }
}

class Client
{
    static void Main(string[] args)
    {
        List<AbstractExpression> expressions=new List<AbstractExpression>();
        AbstractExpression nte = new NonterminalExpression();
        AbstractExpression te = new TerminalExpression();
        expressions.Add(te);
        expressions.Add(nte);
        Context context = new Context("987698");
        foreach (AbstractExpression ae in expressions)
            ae.interpret(context);
    }
}

/*Program Çıktısı:
Terminal Expression İşlemi Yorumluyor...
Non Terminal Expression İşlemi Yorumluyor...
Terminal Expression İşlemi Yorumluyor...

...Program finished with exit code 0
*/
```


Model-View-Controller Mimari Deseni

Model-View-Controller (MVC) mimari deseni (*architectural pattern*) ilk kez 1979 yılında Trygve Reenskaug tarafından "Applications Programming in Smalltalk-80: How to use Model-View-Controller" makalesiyle tanımlanmıştır. Günümüze kadar bu desenin "Model View Presenter-MVP" gibi birçok türevi geliştirilmiştir.

Bu mimari deseni kullanmak, iş süreci ile ilgili olan katmanın gösterim katmanından bağımsız tasarlanmasını sağlar. Bunun yanında sunum katmanını bağımsız olarak kontrol eden bir nesneye sahip oluruz. Bu nedenle bu desene uygun bağımsız yapılan katmanların bakımı kolaylaşır, hem de **yeniden kullanımı** (*reusing*) artırılmış olunur.

MVC deseni Borland firması Visual Component Library (VCL) ve Component Library for Cross Platform (CLX) bileşenleri altyapısında yıllarca kullanmıştır. Benzer şekilde Apple firması Cocoa için bu deseni kullanmıştır. Microsoft firması da Document/View mimarisi olarak yıllarca kullanmıştır. Günümüzde .NET Framework Web Uygulamaları bu kapsamda daha kolay ve yönetilebilir olarak geliştirilmektedir.

MVC Deseni Temel Yapısı

MVC deseni adından da anlaşıldığı üzere üç nesneden veya katmandan oluşur. Bunlar aşağıda detaylı olarak açıklanmıştır;

1. **Model** : Sadece işlevsel davranışları yerine getiren kodlar burada toplanır. Kullanıcıya gösterilen ara yüzde kullanıcının gördüğü ve isteklerini ilettiği kısım dışında kalan konular bu katmanda gerçekleştirilir. Bazen kullanıcı tarafından modelin o andaki **durumu** (*state*) hakkında bilgi almasına izin verilir. **Gösterimler** (*views*) bilgiyi modelden aldığından, önceden modele **kayıt** (*subscribe*) olmak zorundadır. Bunun yanında model üzerinde bilgi değişikliği olduğunda bütün gösterimler bundan **haberdar edilir** (*notify*).
2. **Gösterimler** (*view*): MVC deseni birden fazla gösterim olmasına izin verilir. Gösterim, modele ilişkin bir kullanıcı ara yüzüdür (*Graphical User Interface-GUI*). Yani modelden kullanıcıya gösterilecek değerleri alır ve kullanıcıya gösterir.
3. **Denetçiler** (*controller*): Denetçi, kullanıcı tarafından etkileşilen gösterimden gelen isteklere göre modeli günceller. Denetçi modeli güncellemek için, modelin değişik yöntemlerini çağırır ve modelin kendi **durumunu** (*state*) değiştirmesini sağlar. Buna bağlı olarak da model, tüm gösterimleri bundan haberdar eder.

Yukarıda kabaca çalışma prensibi verilen desende, model değiştiğinde gösterimlere haber verilmesi işlemi, Gözlemci Deseni ile yapılır. Bu durumun da ilave edildiği MVC deseni örneği aşağıda verilmiştir.

Gözlemci ile MVC Deseni İçin Örnek Proje

Bu deseni bir bardak ve bu bardağı dolduran garson örneğini ele alarak inceleyelim. MVC deseni için bardağı dolduracak ya da boşaltacak olan garson, denetçi; Doldurulacak ve boşaltılacak olan bardak ise modeldir. Bunun için Visual Studio üzerinde ile bir Windows uygulaması örnek olarak oluşturulacaktır. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

MVCWindowsUygulaması	→ Çözüm (Solution)
└ WinFormsApp1	→ Proje (Project)
├─ IController.cs	→ Interface (Controller)
├─ IModel.cs	→ Interface (Model)
├─ IView.cs	→ Interface (View)
├─ Controllers.cs	→ Class (Controller)
├─ Models.cs	→ Class (Model)
├─ ProgressBarView.cs	→ UserControl (View)
├─ PieView.cs	→ UserControl (View)
├─ Form1.cs	→ Form (Windows Form)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **MVCWindowsUygulaması** adlı çözüm içinde **WinFormsApp1** adlı ilk projemiz *Windows Form App* olarak oluşturulur.
2. Oluşan projede **Form1** adlı bir form ve **Program.cs** dosyasını otomatik olarak oluşur.

Projede MVC Ara Yüzlerinin Oluşturulması

3. Solution Explorer üzerinde projeye sağ tıklanarak *Add→New→New Item...* seçilerek *Interface* seçilir ve dosya adı **IController.cs** oluşturulur ve aşağıdaki şekilde değiştirilir;

```
public interface IController
{
    //Denetçinin (garsonun) Davranışları
    void AlınanDoldurmaİsteği(int pArttırımOranı);
    void AlınanBoşaltmaİsteği(int pEksilmeOranı);
}
```

4. Solution Explorer üzerinde projeye sağ tıklanarak *Add→New→New Item...* seçilerek *Interface* seçilir ve dosya adı **IModel.cs** oluşturulur ve aşağıdaki şekilde değiştirilir;

```
/*
Doluluk oranı, en az, en çok ne kadar doldurulacağı ve adı belirlenecek bir bardağın yani
modelin, denetçiden (garson) gelen istekleri yerine getirir.
*/
public interface IModel
{
    //Özellikler
    string Adı { get; set; }
    int DolulukOranı { get; }
    int EnAz { get; }
    int EnÇok { get; }
    //Davranışlar
    void Doldur(int pArttırımOranı);
    void Boşalt(int pEksilmeOranı);
}
```

5. Solution Explorer üzerinde projeye sağ tıklanarak *Add→New→New Item...* seçilerek *Interface* seçilir ve dosya adı **IView.cs** oluşturulur ve aşağıdaki şekilde değiştirilir;

```
/*
Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
çıkartır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
boşaltma isteğine gösterim izin vermemelidir.
*/
public interface IView
{
    //View Davranışları
    void DoldurmaAktif();
    void DoldurmaPasif();
    void BoşaltmaAktif();
    void BoşaltmaPasif();
}
```

6. **IController** ara yüzümüze hangi model ile çalışacağını söyleyen değişiklikler de yapılır.

```
public interface IController
{
    //Denetçinin (garsonun) Davranışları
    void AlınanDoldurmaİsteği(int pArttırımOranı);
    void AlınanBoşaltmaİsteği(int pEksilmeOranı);
    IModel Model { set; get; }
}
```

Projeye Gözlemci Deseninin Uygulanması

7. Bundan sonraki aşama biraz daha ustalık istemektedir. Çünkü modeldeki değişikliklerden gösterimin haberi olmasını istiyoruz. İşte bunun için *Gözlemci* desenini bu ara yüze uyguluyoruz. Bunun için ilk önce **IModel** ara yüzü aşağıdaki şekilde değiştirilir;

```
/*
Doluluk oranı, en az, en çok ne kadar doldurulacağı ve adı belirlenecek bir bardağın yani
modelin, denetçiden (garson) gelen istekleri yerine getirir.
*/
using System.Collections; //ArrayList İçin
public interface IModel
{
    //Özellikler
    string Adı { get; set; }
    int DolulukOranı { get; }
    int EnAz { get; }
    int EnÇok { get; }
    //Davranışlar
    void Doldur(int pArtırımOranı);
    void Boşalt(int pEksilmeOranı);
    //Gözlemci Desenine İlişkin Davranışlar
    ArrayList Observers { get; }
    void AddObserver(IView pView);
    void RemoveObserver(IView pView);
    void NotifyObservers();
}
```

8. Devamında **IView** ara yüzü de aşağıdaki şekilde değiştirilir;

```
/*
Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
çıkartır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
boşaltma isteğine gösterim izin vermemelidir.
*/
public interface IView
{
    //View Davranışları
    void DoldurmaAktif();
    void DoldurmaPasif();
    void BoşaltmaAktif();
    void BoşaltmaPasif();
    //Gözlemci Deseni Davranışları:
    //Modelden gelen bilgilere göre View güncellenir
    void Update(IModel pModel);
}
```

9. Son olarak gösterim ara yüzünden alınacak kullanıcı isteklerini denetçiye iletmek için **Iview** ara yüzü aşağıdaki gibi değiştirilir;

```
/*
Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
çıkartır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
boşaltma isteğine gösterim izin vermemelidir.
*/
public interface IView
{
    //View Davranışları
    void DoldurmaAktif();
    void DoldurmaPasif();
    void BoşaltmaAktif();
    void BoşaltmaPasif();
}
```

```
//View üzerinden kullanıcı isteklerini Controller'e
//iletme için özellik tanımlama
IController Control { set; }
//Gözlemci Deseni Davranışları:
//Modelden gelen bilgilere göre View güncellenir
void Update(IModel pModel);
}
```

MVC İskeletine Uygun Projenin Tamamlanması

Artık verilen örneği gerçekleştirmek için iskelet ara yüzlerimiz hazır. Bu ara yüzlere bağlı kalarak sırasıyla soyut ve somut bir model (bardak), denetçi (hızlı veya çalışan garson), ve gösterimler (**ProgressBarView**, **PieView**) oluşturabiliriz.

10. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New→New Item...* seçilerek **Models.cs** adında *Class* seçilir ve içeriği aşağıdaki şekilde değiştirilir;

```
using System.Collections; //ArrayList İçin
abstract class SoyutKap: IModel
{
    private ArrayList views = new ArrayList();
    private string adi;
    private int dolulukOranı;
    private int enAz;
    private int enÇok;
    public SoyutKap(string pAdi, int pEnAz, int pEnÇok,
        int pDolulukOranı)
    {
        this.adi = pAdi;
        this.enAz = pEnAz;
        this.enÇok = pEnÇok;
        this.dolulukOranı = pDolulukOranı;
    }
    public string Adı
    {
        get { return adi; }
        set { adi = value; }
    }
    public int DolulukOranı
    {
        get { return dolulukOranı; }
    }
    public int EnAz
    {
        get { return enAz; }
    }
    public int EnÇok
    {
        get { return enÇok; }
    }
    public void Doldur(int pArtırımOranı)
    {
        this.dolulukOranı += pArtırımOranı;
        if (dolulukOranı >= this.enÇok) dolulukOranı = enÇok;

        this.NotifyObservers();
    }
    public void Boşalt(int pEksilmeOranı)
    {
        this.dolulukOranı -= pEksilmeOranı;
        if (dolulukOranı <= this.enAz) dolulukOranı = enAz;

        this.NotifyObservers();
    }
}
```

```

    }
    public ArrayList Observers
    {
        get { return views; }
    }
    public void AddObserver(IView pView)
    {
        views.Add(pView);
    }
    public void RemoveObserver(IView pView)
    {
        views.Remove(pView);
    }
    public void NotifyObservers()
    {
        foreach (IView view in views)
        {
            view.Update(this);
        }
    }
}
class somutBardak : SoyutKap
{
    public somutBardak(string pAdi) : base(pAdi, 0, 100, 0) { }
}

```

11. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New→New Item...* seçilerek **Controllers.cs** adında *Class* seçilir içeriği aşağıdaki şekilde değiştirilir;

```

class HızlıGarson : IController
{
    private IModel model;
    public IModel Model
    {
        set { model = value; }
        get { return model; }
    }
    public void AlınanDoldurmaİsteği(int pArtırımOranı)
    {
        if (model != null)
        {
            model.Doldur(pArtırımOranı * 10);
            DurumKontrol();
        }
    }
    public void AlınanBoşaltmaİsteği(int pEksilmeOranı)
    {
        if (model != null)
        {
            model.Boşalt(pEksilmeOranı * 10);
            DurumKontrol();
        }
    }
    public void DurumKontrol()
    {
        foreach (IView view in model.Observers)
        {
            if (model.DolulukOranı >= model.EnÇok)
                view.DoldurmaPasif();
            else
                view.DoldurmaAktif();

            if (model.DolulukOranı <= model.EnAz)

```

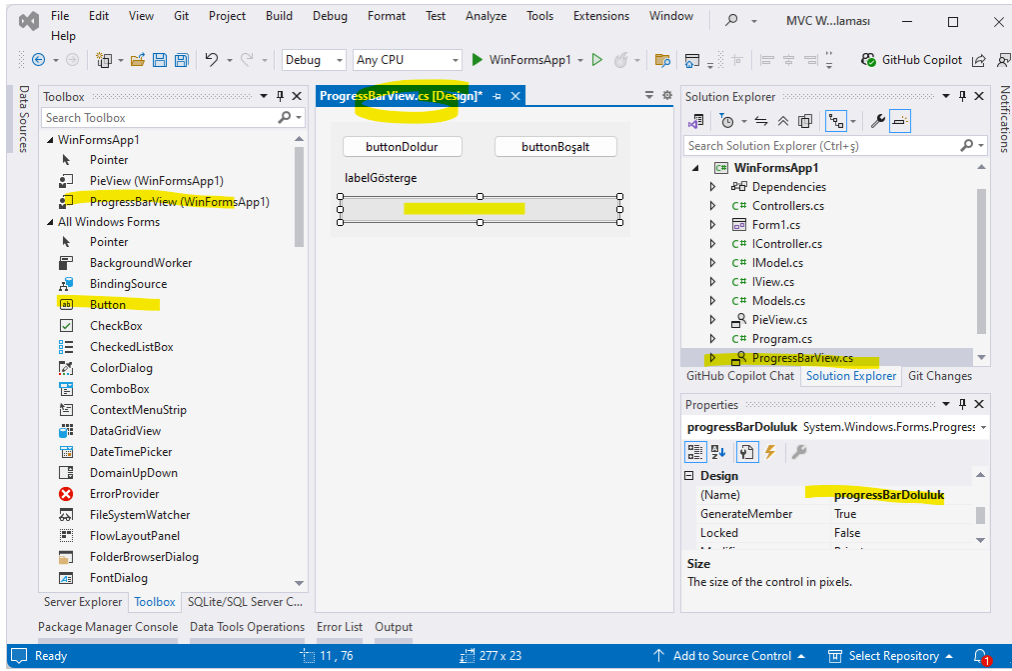
```

        view.BoşaltmaPasif();
    else
        view.BoşaltmaAktif();
    }
}
}
class YavaşGarson : IController
{
    private IModel model;
    public IModel Model
    {
        set { model = value; }
        get { return model; }
    }
    public void AlınanDoldurmaİsteği(int pArtırımOranı)
    {
        if (model != null)
        {
            model.Doldur(pArtırımOranı);
            DurumKontrol();
        }
    }
    public void AlınanBoşaltmaİsteği(int pEksilmeOranı)
    {
        if (model != null)
        {
            model.Boşalt(pEksilmeOranı);
            DurumKontrol();
        }
    }
    public void DurumKontrol()
    {
        foreach (IView view in model.Observers)
        {
            if (model.DolulukOranı >= model.EnÇok)
                view.DoldurmaPasif();
            else
                view.DoldurmaAktif();

            if (model.DolulukOranı <= model.EnAz)
                view.BoşaltmaPasif();
            else
                view.BoşaltmaAktif();
        }
    }
}
}

```

12. Artık gösterimler için örnek olarak iki farklı kullanıcı kontrolü (user control) tasarlanacaktır. Bunlardan biri **ProgressBarView** kontrolüdür.
13. Projemize kullanıcı kontrolü eklemek için Solution Explorer üzerinde projemize sağ tıklanarak *Add→New→New Item...* ile **ProgressBarView.cs** adında *User Control (Windows Forms)* seçilir.
14. Daha sonra *View→ToolBox* tıklanır ve açılan penceren kullanıcı kontrolüne *Toolbox* üzerinden sürükleyip bırak ile iki adet **Button** ve bir adet **ProgressBar** eklenir;



Şekil 58. ProgressBarView Kullanıcı Kontrolü

15. Eklenen kontrollerin **Name** özellikleri **buttonDoldur**, **buttonBoşalt**, **labelGösterge** ve **progressBarDoluluk** olarak değiştirilir ve butonların **onClick()** olayına çift tıklanır. Açılan **ProgressBarView.cs** dosyasında aşağıdaki değişiklikler yapılır;

```
namespace WinFormsApp1
{
    public partial class ProgressBarView : UserControl, IView
    {
        public ProgressBarView()
        {
            InitializeComponent();
        }

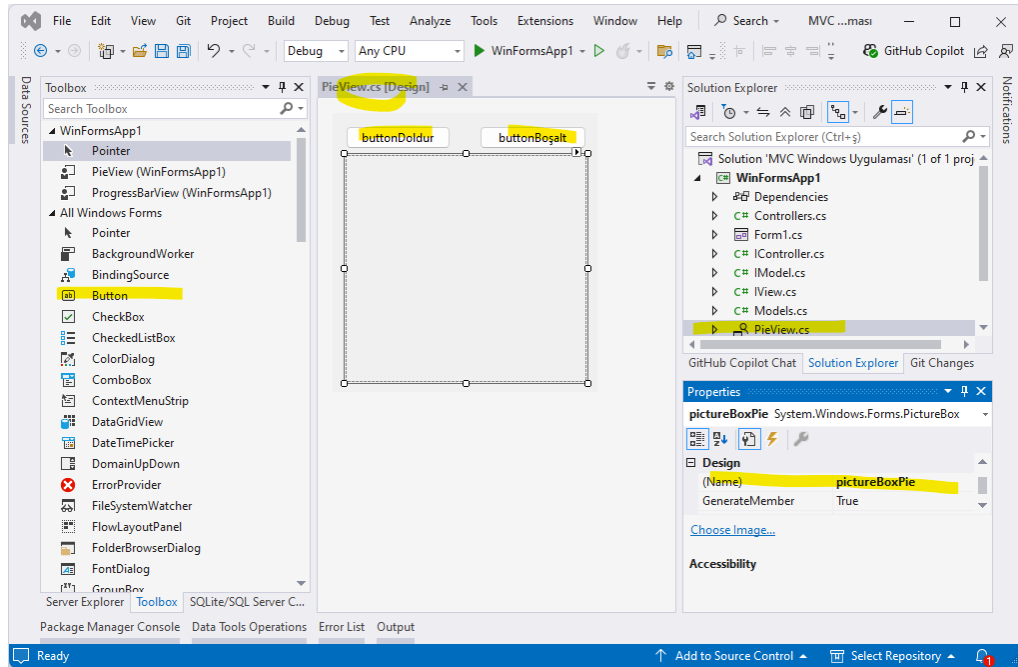
        private IController control;
        [DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        public IController Control
        {
            set { control = value; }
        }
        public void DoldurmaAktif()
        {
            this.buttonDoldur.Enabled = true;
        }
        public void DoldurmaPasif()
        {
            this.buttonDoldur.Enabled = false;
        }
        public void BoşaltmaAktif()
        {
            this.buttonBoşalt.Enabled = true;
        }
        public void BoşaltmaPasif()
        {
            this.buttonBoşalt.Enabled = false;
        }
        public void Update(IModel pModel)
        {
            this.labelGösterge.Text = string.Format(
                "{0} Doluluğu (%):{1}", pModel.Adı,
```

```

        pModel.DolulukOranı);
        this.progressBarDoluluk.Value = pModel.DolulukOranı
            * 100 / pModel.EnÇok;
    }
    private void buttonBoşalt_Click(object sender, EventArgs e)
    {
        control.AlınanBoşaltmaİsteği(1);
    }
    private void buttonDoldur_Click(object sender, EventArgs e)
    {
        control.AlınanDoldurmaİsteği(1);
    }
}
}

```

16. İkinci kullanıcı kontrolü ise **PieView** kontrolüdür. Bunun için mevcut projemize **PieView** adında yeni bir kullanıcı kontrolü eklenir.
17. Kullanıcı kontrolüne *Toolbox* üzerinden sürükleyip bırak ile iki adet **Button**, ve bir adet **PictureBox** eklenir.



Şekil 59. PictureBox Kullanıcı Kontrolü

18. Eklenen kontrollerin **Name** özellikleri **buttonDoldur**, **buttonBoşalt** ve **pictureBoxPie** olarak değiştirilir ve butonların **onClick()** olayına çift tıklanır. Açılan **PieView.cs** dosyasında aşağıdaki değişiklikler yapılır;

```

namespace WinFormsApp1
{
    public partial class PieView : UserControl, IView
    {
        public PieView()
        {
            InitializeComponent();
        }
        private IController control;
        [DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        public IController Control
        {
            set { control = value; }
        }
        public void DoldurmaAktif()
    }
}

```



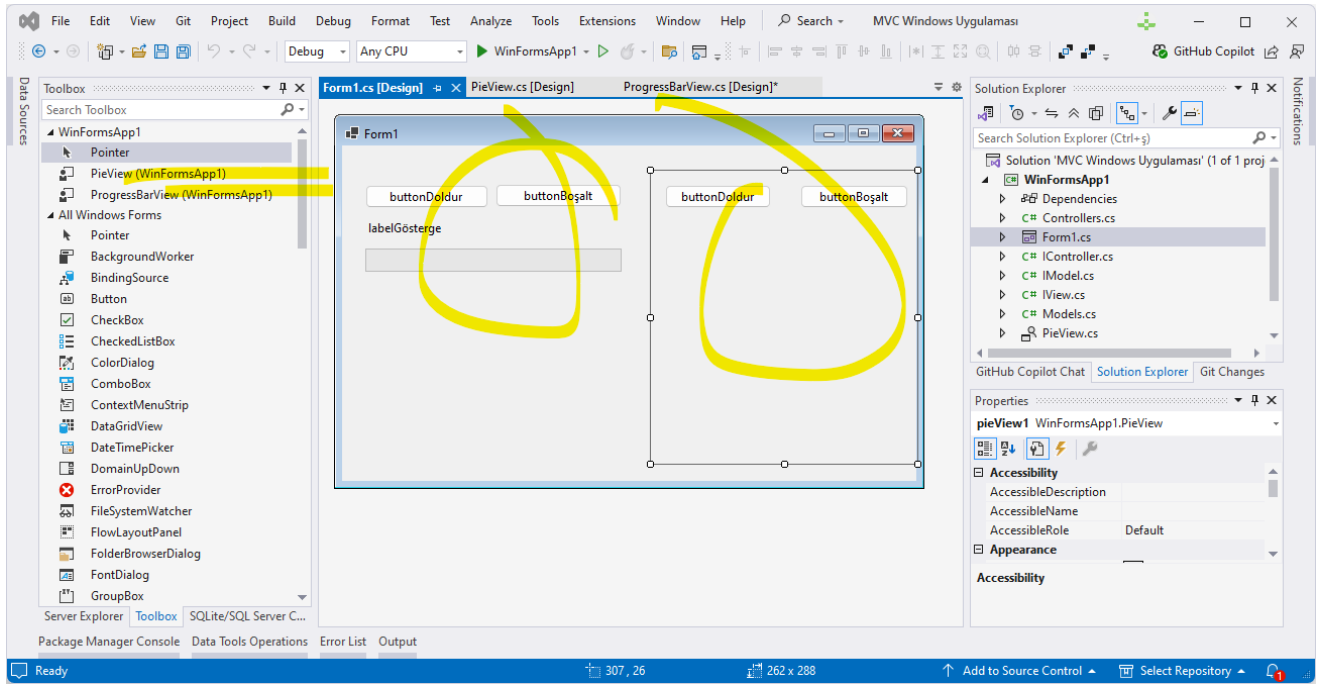
```

{
    this.buttonDoldur.Enabled = true;
}
public void DoldurmaPasif()
{
    this.buttonDoldur.Enabled = false;
}
public void BoşaltmaAktif()
{
    this.buttonBoşalt.Enabled = true;
}
public void BoşaltmaPasif()
{
    this.buttonBoşalt.Enabled = false;
}
public void Update(IModel pModel)
{
    SolidBrush whiteBrush = new SolidBrush(Color.White);
    SolidBrush redBrush = new SolidBrush(Color.Red);
    Graphics graphics = this.pictureBoxPie.CreateGraphics();
    graphics.FillRectangle(whiteBrush, 0, 0,
        this.pictureBoxPie.Width,
        this.pictureBoxPie.Height);
    graphics.FillPie(redBrush, 0, 0,
        this.pictureBoxPie.Width,
        this.pictureBoxPie.Height, 0.0F,
        pModel.DolulukOranı * 360.0F / pModel.EnÇok);
    Font font = new Font(FontFamily.GenericSerif, 12);
    Point point = new Point(this.pictureBoxPie.Width / 2,
        this.pictureBoxPie.Height / 2);
    SolidBrush blackBrush = new SolidBrush(Color.Black);
    graphics.DrawString(string.Format("{0}",
        pModel.DolulukOranı), font, blackBrush, point);
}
private void buttonBoşalt_Click(object sender, EventArgs e)
{
    control.AlınanBoşaltmaİsteği(1);
}
private void buttonDoldur_Click(object sender, EventArgs e)
{
    control.AlınanDoldurmaİstegi(1);
}
}
}

```

19. Kullanıcı Kontrollerimizi tanımladıktan sonra uygulamanın asıl form olan **Form1.cs** formuna dönüp bu kontrolleri ekleyebiliriz.

20. Bu forma kullanıcı kontrollerimizi *Toolbox* üzerinden sürükleyip bırakarak ekleyebiliriz.



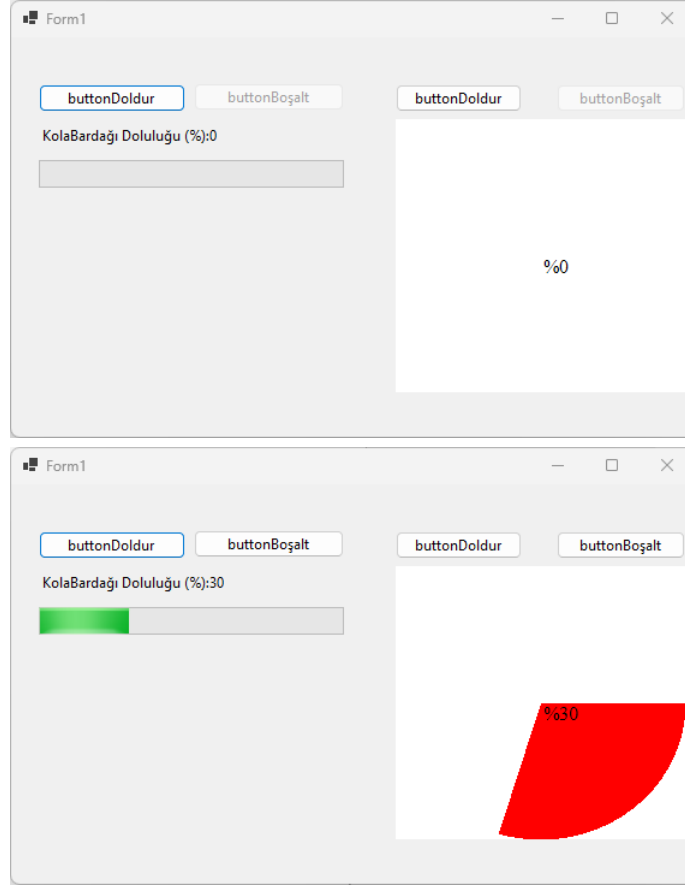
Şekil 60. WinFormApp1 Uygulamasında Form1

21. Formun kod kısmına geçerek **Form1.cs** dosyasında aşağıdaki değişiklikleri yaparak programı son haline getiririz.

```
namespace WinFormsApp1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
            MVCBaşlat();
        }
        public void MVCBaşlat()
        {
            //Controller
            IController garson = new HızlıGarson();
            //Model
            IModel bardak = new somutBardak("KolaBardağı");
            //Denetçinin Modeli
            garson.Model = bardak;
            //Her bir Gösterime Denetçi Ataması
            pieView1.Control = garson;
            progressBarView1.Control = garson;
            //Modele Gösterimlerin Kayıt Olması
            bardak.AddObserver(pieView1);
            bardak.AddObserver(progressBarView1);
        }
    }
}
```

Program çalıştırıldığında, MVC deseni ile **bağlaşımı az olan** (loosely coupled) denetleme ara yüzlerinin yazılımlara kolay eklendiğini görebiliriz. Ayrıca programda yapılacak değişiklik isteklerine MVC deseni daha hızlı cevap verecektir. Bunun yanı sıra hazırlanan ara yüzlerle birlikte, soyut sınıfları da başka projelere taşıyıp **yeniden kullanmak** (reusing) oldukça kolay olacaktır. Böylece ileride olabilecek değişikliklere daha hızlı ayak uydurabilen daha esnek bir projeye sahip olmuş oluruz. Sonuç olarak MVC deseni, kullanıcı ara yüzü işlemleri ile iş mantığının birbirinden ayrılmasını sağlar. Ayrıca Model,

Gösterim ve Denetçi bileşenleri ayrı katmanlar halinde *Katmanlı Mimari* başlığı altında anlatıldığı şekilde tasarlanabilir.



Şekil 61. MVC Uygulanmış Windows Form Uygulamamız.

ASP.NET MVC ve Jakarta EE gibi günümüzde en çok kullanılan teknolojiler MVC ara yüzlerini web uygulamasına entegre edilmiş olarak sunmaktadır. ASP.NET MVC altyapısında gösterim bileşeni ASP.NET web sayfası olacak şekilde hazırlanır. Jakarta EE altyapısında ise sunum bileşeni JSP olacak şekilde hazırlanır. Model ve Denetçi bileşenleri ise genellikle nesneye dayalı sınıflar olarak tasarlanır.

KURUMSAL YAZILIMLAR

Kurumsal Yazılım İçin Gereklilikler

Kurumsal bir yazılım sisteminin inşası için aşağıdakiler sırasıyla yapılır;

- **İşlevsel model** (**functional model**) çıkarılır. Kullanıcılara **ne** (**what**) yapmak istedikleri sorulur ve use-case diyagramları oluşturulur.
- **Yapısal model** (**structural model**) çıkarılır. İşlemlerin **kimler** (**who**) tarafından yapılacağı belirlenir ve bu kapsamda sınıf diyagramları oluşturulur.
- **Davranış modeli** (**behavioral model**) oluşturulur. İşlemlerin kim tarafından **nasıl** (**how**) ve hangi sırada yapılacağı belirlenir ve silsile diyagramları oluşturulur.
- **Mimari modelleme** (**architectural model**) oluşturulur. Bu modelde nesnelerin nasıl ve nerede (**where**) bir araya geleceği üzerine durulur ve yayılma ve paket diyagramları oluşturulur. Bu modelde iki gereklilik detaylı incelenir. Birincisi iş süreçlerini ya da kullanıcı isteklerini yerine getiren işlevsel gerekliliklerdir. İkincisi ise yazılımla birlikte ortaya çıkan performans, **güvenlik** (**security**), **günlük** (**logging**) ve benzeri gerekliliklerdir.

Kurumların iş süreçlerinin de dâhil edildiği büyük yazılımlar **kurumsal uygulamalar** (**enterprise application**) olarak adlandırılır. Adından da anlaşılacağı üzere, kurumsal yazılımlar, kurumun iş süreçlerini elektronik ortama taşır. Kurumsal yazılım geliştirmek için aşağıda belirtilen üç şeye sahip olmamız gerekir:

1. **Takım** (**Team**): Analizci, testçi ve programcı gibi elemanlardan oluşan uygulama geliştirecek olan yazılımcı grubu.
2. **Yazılım Geliştirme Yaşam Döngüsü** (**Software Development Life Cycle-SDLC**): Kurumsal yazılım geliştirme için kullanılan modeller; **Şelale modeli**(**waterfall model**), **Rational Unified Process** (**RUP**), **Çevik model** (**agile model**) gibi modeller ya da kendimize has yazılım geliştirme modelimiz.
3. **Teknoloji** (**Technology**): Yazılımın, sahip olacağı teknoloji, kurumsal yazılım geliştirme için gerekli bileşenleri programcıya sunarlar. Bu teknolojiler, yazılım geliştirme sırasında gerekli olan bileşenleri, bir mağazadan alıp kullanır gibi programcıya sunarlar. Böylece programcının iş süreci üzerine yoğunlaşarak daha hızlı ve doğru bir çözüm üretmesini sağlarlar. Jakarta EE (Java Enterprise Edition) ve .NET Framework en çok kullanıldığı teknolojilerdir.

Yazılım geliştirme sırasında bu üç şeyi çok iyi seçmemiz halinde başarılı bir kurumsal yazılım ortaya çıkar. Burada özellikle teknoloji üzerinde durmak gerekir. Jakarta EE ve .NET gibi iki büyük teknolojinin programcıya sunduğu bileşenler arkasında birçok sunucu çalışır. Bunların bazıları aşağıda verilmiştir;

- Kurumsal yazılım geliştirmede kullanılan modüler bileşenleri sunan sunucular: Jakarta Enterprise Beans, Component Object Model-COM, COM+ gibi teknolojileri sağlayan .NET Bileşenleri
- Kullanıcı, bilgisayar, yazıcı ve benzeri öğeleri içeren kurumsal izin sunucuları: Java Naming and Directory Interface (JNDI), Microsoft Active Directory
- Kurumsal Uygulamada bileşenlerin birbirine **ileti** (**message**) gönderip almalarını sağlayan servisler: Jakarta Messaging API, Microsoft Message Queue (MSMQ)
- Web Sunucuları üzerinde bileşenlerin çalışmasını sağlayan servisler: Jakarta Server Pages (JSP), ASP.NET Core (Active Server Pages for .NET)
- Mevcut kurumsal uygulamaları birbirine bağlayan servisler: Jakarta Connectors, Java Database Connectivity (JDBC), Java XML, Jakarta Transactions, ADO.NET (Active Data Objects for .NET), WCF (Windows Communication Foundation)

JEE ile .NET teknolojileri arasında seçim yapmak için kurumun sahip olduğu donanım altyapısı ve yukarıda belirtilen servisleri sağlayan yazılım kaynakları irdelenmelidir.

Tüm projelerde olduğu gibi kurumlarda geliştirilen uygulamalar da proje olarak ele alınır ve yönetilirler. Proje yönetimi, belirli bir hedefe ulaşmak için zaman, bütçe, kaynaklar ve kalite kısıtları altında yürütülen işleri planlama, yürütme, izleme ve tamamlama sürecidir. Kısaca doğru işi, doğru şekilde, doğru zamanda ve doğru kaynaklarla yapmak demektir.

Proje yönetimi genellikle beş ana aşamada yürütülür:

1. **Başlatma (initiation)**: Projenin amacı, kapsamı ve paydaşları belirlenir. Proje yöneticisi atanır ve gerekirse fizibilite çalışması yapılır. Proje yöneticisi, proje planı hazırlamak ve güncel tutar, takımı yönetir, iletişimi sağlar, riskleri öngörür ve önlem alır, paydaşlarla iletişimi sürdürür ve teslimatları zamanında gerçekleştirir.
2. **Planlama (planning)**: Proje hedefleri, teslimatlar belirlenir. **Zaman çizelgesi (Gantt Chart)** ve alt kırılımlar oluşturulur. Kaynak planlaması, risk yönetimi, bütçe, iletişim planı hazırlanır.
3. **Yürütme (execution)**: Takım kurulur, görevler dağıtılır. İşler gerçekleştirilir, yazılım geliştirme, inşaa gibi ana faaliyetler yapılır. İletişim ve koordinasyon sağlanır.
4. **İzleme ve Kontrol (monitoring and controlling)**: Proje ilerlemesi takip edilir. Gecikmeler, sapmalar analiz edilir. Riskler yönetilir, gerekirse planlar revize edilir.
5. **Kapanış (closing)**: Proje sonuçlandırılır. Teslimatlar yapılır. Geri bildirim alınır, raporlama yapılır, dersler çıkarılır.

Proje yönetimini yapmak için Microsoft Project, Jira, Trello, Asana, ClickUp, Notion, Monday.com, Smartsheet ve Wrike gibi araçlar kullanılır.

Bir yazılım projesini yönetmek için yukarıdaki yönetim aşamalarını içerecek **şekilde yazılım geliştirme yaşam döngüsü (Software Development Life Cycle-SDLC)** ile yapılır. Yazılım geliştirme modeli olarak da adlandırılan SDLC, kaliteli yazılım tasarlamak, geliştirmek ve test etmek için kullanılan yapılandırılmış bir süreçtir. SDLC, bir yazılım mühendisi veya geliştiricisi tarafından çeşitli aşamalarda gerçekleştirilecek görevi/görevleri belirtir. Son ürünün müşterinin beklentilerini karşılayabilmesini ve tanımlanan bütçeye uymasını sağlar³⁷.

Burada **yazılım mühendisliği (software engineering)** devreye girer. Yazılım mühendisliği, yazılım sistemlerinin planlanması, tasarımı, geliştirilmesi, test edilmesi, dağıtımı ve bakımı ile ilgilenir.

Yazılım mühendisi;

- Yazılım geliştirme veya kısaca programlama yapar,
- Yazılım mimarisi belirler,
- Yazılım testini yapar ve kalitesini belirler,
- Yazılım proje yönetimini yapar,
- Yazılım yaşam döngüsü sürecini (SDLC) yürütür,
- **Geliştirme ve işletim (Developer Operations-DevOps)** süreci ile **sürekli entegrasyon/sürekli dağıtım (Continuous Integration/Continuous Delivery-CI/CD)** süreçlerini yürütür.

Yazılım Geliştirme Modelleri

Günümüzde kullanılan birçok model bulunmaktadır. İlk model olan şelale modeli ile günümüzde çok kullanılan çevik modele değinilecektir.

Şelale Modeli

Şelale modeli (waterfall model) 1970 yılında Winston W. Royce tarafından tanımlanmıştır. Bu modelde bir sonraki aşamaya geçmek için, bir önceki aşamanın tamamlanması gerekir. Bir sonraki aşamaya geçmeden, içinde bulunulan aşamanın tüm dokümanları hazırlanır. Bu model, **klasik yaşam döngüsü (classic life cycle)**, **doğru sıralı (linear sequential)** model olarak da adlandırılır.

Şelale modelinin aşamaları;

1. **Gereksinim Analizi (requirement analysis)**: Müşteri ihtiyaçları toplanır ve belgelenir. Projenin kapsamı belirlenir, Projenin hedef ve amaçları belirlenir. Projenin kaynakları planlanır. Fonksiyonel ve teknik gereksinimler belirlenir. Gereksinimler gözden geçirilir ve kabul koşulları belirlenir.
2. **Sistem ve Yazılım Tasarımı (design)**: Yazılım mimarisi ve sistem yapısı planlanır. Veri tabanı, ara yüz, iş mantığı gibi bileşenler tasarlanır. Düşük ve yüksek düzey tasarım yapılır. **Düşük Seviye Tasarım (Low Level Design-LLD)**, yazılım geliştirme sürecinde detaylı sistem bileşenlerinin ve bunların etkileşimlerinin belirtildiği ve bunların uygulandığı bir aşamadır. **Yüksek Seviye Tasarım (High Level**

³⁷ <https://www.geeksforgeeks.org/software-development-life-cycle-sdlc/>

Design-HLD), bir sistemin genel yapısının planlandığı uygulamaların geliştirilmesinde ilk adım olup, esas olarak sistemin farklı bileşenlerinin, kodlama ve uygulama hakkında bilgi sahibi olmadan birlikte nasıl çalıştığına odaklanır. Bu, projede yer alan herkesin hedefleri anlamasına yardımcı olur ve geliştirme sırasında iyi iletişim sağlar.

3. **Uygulama ya da Kodlama (implementation)**: Tasarlanan yapıya uygun olarak yazılım geliştirme yapılır. Kod standardı belirlenir yani programlama dilleri burada belirlenir. Kod ölçeklendirme yapılır. Kod ölçeklendirme, uygulamanın daha fazla kullanıcı, daha fazla veri veya daha yoğun trafik ile etkili şekilde çalışabilmesi için yapılan düzenlemelerdir. Sürüm kontrolü belirlenir. Kodun gözden geçirilmesi için süreç belirlenir.
4. **Test Etme (testing)**: Kodlanan yazılım test edilir. Genel sistem testi yanında manuel test ve otomatik testler yapılabilir. Varsa hatalar düzeltilir, kalite kontrolü sağlanır.
5. **Dağıtım (deployment)**: Yazılımın uygun sürümü son kullanıcıya sunulur. Kurulumlar ve eğitimler yapılabilir.
6. **Bakım (maintenance)**: Kullanım sırasında oluşan hatalar düzeltilir. Geri bildirimler sonrası güncellemeler ve iyileştirmeler yapılır.

Bu modelde yazılıma ilişkin dokümantasyon çok net ve kapsamlıdır. Bir aşama tamamlanmadan sıradakine geçilmez. Süreç kontrollüdür. Gereksinimi değişmeyen büyük projelerde kullanılır. Yönetimi ve izlenebilirliği kolaydır. Ancak geri dönmek zordur. Bir hata sonradan fark edilirse süreç baştan sarar. Değişen gereksinimler varsa uygun değildir. Bu nedenle esnek değildir. Teslimat çok geç ortaya çıkar yani müşteri ürünü en sonunda görür.

Çevik Model

Çevik model (agile model), şelale modelinin aksine temel olarak proje süresince gelen değişen isteklere hızla uyum sağlamak için tasarlanmıştır. Günümüzde çok tercih edilen modellerden biridir. Bu model, yazılım geliştirme süreçlerinde esnekliği, müşteri memnuniyetini ve hızlı teslimatı ön planda tutan **tekrarlamalı (iterative)** bir yaklaşımdır.

Büyük bir projeyi küçük parçalara ayırarak, adım adım geliştirme ve sürekli geri bildirimle iyileştirme sağlar. Bu model, değişen müşteri ihtiyaçlarına hızlıca adapte olabilmek için tasarlanmış, sürekli iletişim, erken teslimat ve kademeli gelişim ilkelerine dayanan bir geliştirme modelidir.

Çevik modelde projeyi yönetmek için proje personelinin bazı **değerleri (values)** benimsemesi gerekir.

1. Bireyler ve etkileşimler, süreçler ve araçlardan daha önemlidir.
2. Çalışan yazılım, kapsamlı dokümantasyondan daha önemlidir.
3. Müşteri ile iş birliği, sözleşme pazarlığından daha önemlidir.
4. Değişime karşılık vermek, bir planı izlemekten daha önemlidir.

Bu değerler ile 12 adet ilkeye uyulur. Bunlar **çevik manifestoda (agile manifesto)** yer alır;

1. Müşteri memnuniyeti, erken ve sürekli yazılım teslimatıyla sağlanır. Müşteriye sık aralıklarla çalışan yazılım sunarak geri bildirim alınır ve güven kazanılır.
2. Değişen gereksinimler, projenin geç aşamalarında bile kabul edilir. Çevik süreçler, değişime yanıt vererek müşterinin rekabet avantajını korumasına yardımcı olur.
3. Çalışan yazılım, birkaç haftada bir, mümkün olan en kısa sürede teslim edilir. Kısa döngülerle (sprintlerle) geliştirme yapılır.
4. İş birliği, iş birimlerinden (müşteriden) ve geliştiricilerden oluşan bir ekip arasında sürekli olmalıdır. Müşteri ve geliştirici ekip sürekli iletişim halinde olur.
5. Motivasyonu yüksek bireyler etrafında projeler inşa edilir. Ekip üyelerine güvenilir, destekleyici bir ortam sunulur ve onlara güvenilir.
6. Yüz yüze iletişim, bilgi aktarımının en etkili yoludur. Özellikle aynı fiziksel ortamda çalışan ekiplerde hızlı karar alınmasını sağlar.
7. Çalışan yazılım, ilerlemenin birincil ölçüsüdür. Raporlar ya da belgeler değil, gerçek çalışan yazılım önemlidir.
8. Çevik süreçler sürdürülebilir geliştirmeyi teşvik eder. Ekip, sabit bir hızda uzun süre çalışabilmelidir. Aşırı mesai değil, dengeli tempo hedeflenir.

9. Teknik mükemmellik ve iyi tasarım, çevikliği artırır. Kaliteli kod ve temiz mimari, değişimlere uyumu kolaylaştırır.
10. Basitlik esastır – yapılmaması gereken işleri yapmaktan kaçınmak. Gereksiz özellik ve aşırı mühendislikten uzak durulur.
11. En iyi mimariler, gereksinimler ve tasarımlar, kendi kendini organize eden ekiplerden çıkar. Ekip içindeki bireyler, en uygun çözümleri birlikte bulur.
12. Ekip düzenli aralıklarla nasıl daha verimli olabileceğini düşünür ve davranışlarını buna göre uyarlar. **Sürekli iyileştirme (retrospective)** ile gelişim sağlanır.

Çevik proje yönetiminde ön tanımlı roller vardır;

1. **Product Owner (Ürün Sahibi)**: Müşteri ihtiyaçlarını belirler. Ürünün ne olacağına karar verir, iş önceliklerini belirler.
2. **Scrum Master**: Takımın çevik kurallarına uymasını sağlar. Engelleri kaldırır, kolaylaştırıcı rolündedir.
3. **Development Team (Geliştirme Takımı)**: Yazılımı analiz eder tasarlar, kodlar, test eder.

Çevik modelde proje, sprint adı verilen kısa geliştirme döngülerine ayrılır. Her sprint sonunda çalışan bir yazılım parçası müşteriye teslim edilir. Bunun için;

- **Product Backlog** yani (**İş Listesi**) hazırlanır.
- Her sprint için **Sprint Planning** toplantısı yapılır.
- Yazılım takımı, 1 ila 4 haftalık sprint süresince belirlenen işleri yapar.
- **Daily Scrum** yani günlük toplantılar ile ilerleme izlenir.
- **Sprint Review** ile müşteriye sunum yapılır.
- **Sprint Retrospective** ile ekip kendi sürecini değerlendirir ve iyileştirir.

Bir kurumda çevik proje yönetimi yapabilmek için kurumun buna hazırlanması gerekir;

- Kurum Kültürü: Çevik proje yönetimi, hiyerarşik ve katı yönetim anlayışı olan kurumlarda zor olur. Bu yüzden üst yönetimin çevik felsefesini benimsemesi gerekir. Takımlara güven, şeffaflık ve sürekli iyileştirme desteklenmelidir. "Komuta-kontrol" yerine "hizmetkar liderlik" yaklaşımı benimsenmelidir.
- Kurum İçi Eğitim: Tüm ilgili ekipler (yazılım geliştiriciler, testçiler, analistler, ürün sahipleri, yöneticiler) çevik eğitimi almalıdır. Scrum, Kanban, Lean gibi çerçeveler tanıtılmalı ve hangi modelin kuruma uygun olduğuna karar verilmelidir.
- Roller: Scrum kullanılıyorsa roller atanmalıdır; Product Owner, Scrum Master, Development Team.
- Takım: Küçük, çapraz fonksiyonel ve **çok disiplinli (multi-disipliner)** takımlar oluşturulmalıdır. Her takım kendi içinde analiz, tasarım, geliştirme ve test yapabilir durumda olmalıdır.
- Sprint Yönetimi: Scrum uygulanıyorsa;
 - o **Sprint Planning (Sprint Planlama)**: İşlerin belirlenmesi ve sprint hedefinin tanımlanması.
 - o **Daily Stand-up (Günlük Toplantı)**: Kısa günlük toplantılarla durum değerlendirmesi.
 - o **Sprint Review**: Ürün sahipleri ve paydaşlarla yapılan toplantı (**demo**).
 - o **Sprint Retrospective**: Ekip içi iyileştirme toplantısı.
- Agile süreçleri destekleyen yazılımlar: Jira, Trello, Azure DevOps, ClickUp, Asana gibi araçlarla iş takibi yapılabilir. CI/CD altyapıları kurulmalıdır.
- Pilot Proje Deneyimi: Öncelikle küçük bir projede çevik yöntem uygulanır. Deneyim kazanılır, süreçler test edilir. Başarılar ve hatalar analiz edilerek diğer ekiplerde yaygınlaştırılır.
- Sürekli iyileştirme (**Kaizen**): Ekipler düzenli olarak süreçlerini gözden geçirip iyileştirme önerileri üretmelidir. Geri bildirim döngüleri (müşteriden ve ekip içinden) önemlidir.
- Başarı ve Performans Ölçütleri: Çevik ekiplerin performansı birey bazında değil, takım bazında ölçülür. Teslim süreleri, müşteri memnuniyeti, iş değeri odaklı metrikler kullanılır.

Çevik modelde müşteri memnuniyeti yüksektir. Hızlı teslimat ve erken geri bildirim alınır. Değişime açık ve uyumlu yazılım geliştirilir. Takım içi iletişim güçlüdür. Kalite artar, çünkü sürekli test edilir. Ancak sürekli müşteri katılımı gerekir. Belirsiz projelerde zaman ve maliyet tahmini zordur. Deneyimsiz ekiplerde karmaşıklık ortaya çıkar. Dokümantasyon önemsiz gibi görülüp ihmal edilebilir.

Modern Yazılımları Özellikleri

Günümüzde geliştirilen modern uygulamaların üç temel özelliği vardır:

1. Katmanlı mimari (**layered architecture**): İstemci/sunucu ya da n katmanlı model. Katman sayısı arttıkça **karmaşıklık** (**complexity**) azalır.
2. Bileşen kullanımı (**component using**): Bileşen kullanımı yazılımda **tekrar kullanılabilirliği** (**reusing**) artırır.
3. Dağıtık programlama (**distributed programming**): **Ölçeklenebilirlik** (**scalability**) artar. Yazılım birden fazla bilgisayar üzerine çeşitli görevleri dağıtarak çalışır.

Modern kurumsal yazılımlarda, kurumun gerçek dünyada yaptığı işi, elektronik ortamda **iş katmanı** (**business layer**) yapar. Buradan hareketle kurumsal bir yazılıma başlamadan önce, süreçlerin incelenerek **detaylı modellenmesi** (**business model**) gerektiği ortaya çıkar. İş modelinin ortaya çıkarılmasına yönelik yapılan çalışmalara **iş analizi** (**business analysis**) adı verilir. Burada dikkat edilmesi gereken, iş analizinin doğru yapılmasıdır. İş analizinin yanlış yapılması, yanlış bir yazılımın ortaya çıkacağı anlamına gelir.

Katmanlı Mimari

Katman kavramı iki türlü düşünülmelidir. Kodların gruplanarak birbirinden ayrıldığı katmanlara **kod katmanı** (**layer**) adı verilir. Fiziksel olarak birbirinden ayrılmış kodlara ise **fiziksel katman** (**tier**) adı verilir. Kısaca derlenmiş olarak birbirinden ayrı olan katmanlar fiziksel katman, diğer tip katman ise kod katmanı olarak adlandırılır. Katman sayısı göz önüne alınarak uygulamalar aşağıdaki şekilde sınıflandırılabilir.

Yekpare Uygulamalar

Genel olarak hiç katmanı olmayan, tek bir çalıştırılabilir dosyadan oluşan uygulamalar **yekpare** (**monolithic**) uygulamalar olarak adlandırılır. Kullanıcı tanımlı sınıf kütüphanesi olmayan tüm konsol ve Windows uygulamaları bu sınıfa girer. Şu ana kadar verilen örneklerin çoğu bu kapsamda değerlendirilebilir. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

KonsolUygulaması	→ Çözüm (Solution)
└─ Yekpare	→ Proje (Console Application)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **KonsolUygulaması** adlı çözüm içinde **Yekpare** adlı projemiz *Console Application* olarak oluşturulur.
2. Oluşan projede **Program.cs** dosyası otomatik olarak oluşur. Bu dosyayı aşağıdaki şekilde değiştiriyoruz;

```
namespace Yekpare
{
    class Server
    {
        public void Sunucuİşlevi()
        {
            Console.WriteLine("Sunucu İşlevi Çağrıldı.");
        }
    }
    class Client
    {
        static void Main(string[] args)
        {
            Server sunucu = new Server();
            sunucu.Sunucuİşlevi();
        }
    }
}
```

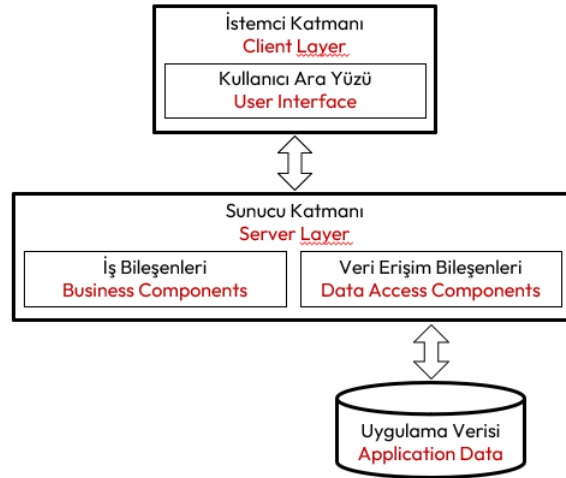

Yekpare uygulamalar, her ne kadar veri tabanı kullansalar da bu tür yazılımlar tek bir proje olarak kodlanır ve derlenir. Proje genellikle az sayıda kod dosyasından oluşur. Seksenli yıllara kadar geliştirilen uygulamalar genellikle bu yapıdadır.

Bilgisayara yaptırılan işlerin kapsamının zamanla daha fazla genişlemesi, kurumların iş süreçlerini daha çok içermesi ve uygulamanın işletme ve bakım maliyetlerinin artmasıyla birlikte, bu yazılımları uygun katmanlara bölünmesi ihtiyacı doğurmuştur.

İki Katmanlı İstemci-Sunucu Uygulamaları

İstemci-Sunucu (client-server) mimarisinde uygulama iki katmanlı olup, genellikle veri katmanı bir bilgisayarda diğer katman olan istemci katmanı ise kullanıcı bilgisayarlarında bulunur.

Bu tür uygulamalarda kullanıcı ara yüzü bileşenleri (User Interface Components-UIC) istemci (client) katmanını oluşturur. Geri kalan iş süreçleriyle ilgili geliştirilen bileşenler ile veri tabanı ile ilgili iletişimi sağlayacak olan bileşenler ise sunucu (server) katmanını oluşturur.



Şekil 62. İstemci-Sunucu Mimarisi

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

İstemciSunucuUygulaması	→ Çözüm (Solution)
└ İstemciBileşenleri	→ Proje (Console Application)
└ Program.cs	→ Ana (Main) Program
└ SunucuBileşenleri	→ Proje (Class Library)
└ İstemci.cs	→ Class

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **İstemciSunucuUygulaması** adlı çözüm içinde **İstemciBileşenleri** adlı projemiz *Console Application* olarak oluşturulur.
2. Oluşan projede **Program.cs** dosyası otomatik olarak oluşur.

```
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
        }
    }
}
```

3. Daha sonra sunucu katmanı yapmak için *Solution Explorer* üzerinde **KonsolUygulaması** çözümüne sağ tıklanarak *Add→New→Project...* seçerek **SunucuBileşenleri** adlı *Class Library* oluşturulur
4. Proje ile oluşan **Class1.cs** dosyasının adı **Sunucu.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace SunucuBileşenleri
{
```

```

public class Server {
    public void Sunucuİşlevi(){
        Console.WriteLine("Sunucu İşlevi Çağrıldı.");
    }
}

```

- İstemci tarafında sunucu bileşenlerini kullanmak için *Solution Explorer* üzerinde **SunucuBileşenleri** projesi sağ tıklanarak *Build* seçilir ve derlenir.
- Ardından **İstemciBileşenleri** projesi seçilir ve *Add→Project Reference→Project...* ile sunucu projemiz istemci projemize dahil edilir.
- Bunu yaptıktan sonra **İstemciBileşenleri** projesinde aşağıdaki değişiklikler yapılır;

```

using Sunucu; //Bunu yapabilmek için:İstemciye Sunucu Referans olarak eklenir.
namespace İstemciBileşenleri
{
    class Client {
        static void Main(string[] args){
            Sunucu.Server sunucu = new Sunucu.Server();
            sunucu.Sunucuİşlevi();
        }
    }
}

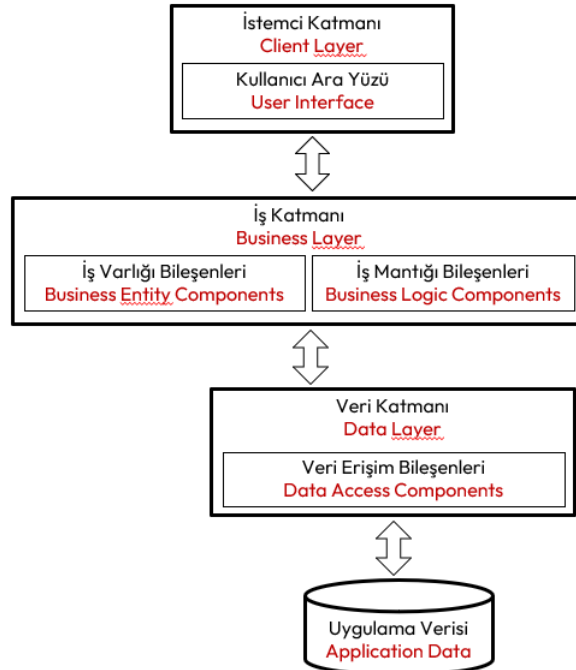
```

Artık programımız çalışır.

Üç Katmanlı Uygulamalar

Nesne yönelimli yazılım geliştirme ile bildikte nesneler arasındaki iletişimin bir başka katman olarak tasarlandığı uygulamalardır. İki katmanlı mimariden farklı olarak kullanıcı ara yüzü ile iş bileşenleri birbirinden ayrılarak, kullanıcılara istemci yükleme işleminin çok kolaylaştığı uygulamalardır.

İstemci-sunucu yazılımlarında zamanla, sunucu tarafında *iş süreçlerini modelleyen bileşenler* (**Business Entity Components-BEC**) ile veri tabanı ile iletişimi sağlayan bileşenler (**Data Access Components-DAC**) birbirinden ayrılarak farklı iki katman olarak geliştirilmiştir.



Şekil 63. Üç Katmanlı Mimari

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

ÜçKatmanlıUygulama	→ Çözüm (Solution)
└─ İstemciBileşenleri	→ Proje (Console Application)

└─ Program.cs	→ Ana (Main) Program
└─ İşBileşenleri	→ Proje (Class Library)
└─ İşSınıfı.cs	→ Class
└─ VeriBileşenleri	→ Proje (Class Library)
└─ Veritabanı.cs	→ Class

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **ÜçKatmanlıUygulama** adlı çözüm içinde **İstemciBileşenleri** adlı projemiz *Console Application* olarak oluşturulur.
2. Oluşan projede **Program.cs** dosyası otomatik olarak oluşur.

```
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
        }
    }
}
```

3. Daha sonra sunucu katmanı yapmak için *Solution Explorer* üzerinde çözüm adına sağ tıklanarak *Add→New→Project...* seçerek **İşBileşenleri** adlı *Class Library* oluşturulur.
4. Proje ile oluşan **Class1.cs** dosyasının adı **İşSınıfı.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace İşBileşenleri
{
    public class İşSınıfı
    {
        public void işYöntemi()
        {
            // İş mantığı burada yer alır
        }
    }
}
```

5. *Solution Explorer* üzerinde çözüm adına sağ tıklanarak *Add→New→Project...* seçilir ve **VeriBileşenleri** adında *Class Library* oluşturulur.
6. Proje ile oluşan **Class1.cs** dosyasının adı **Veritabanı.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace VeriBileşenleri
{
    public class Veritabanı
    {
        public void veriEkle(int veri)
        {
            // Veri ekleme işlemleri burada yer alır
            Console.WriteLine($"Veri tabanına veri ekleme {veri} işlemi çalıştı.");
        }
        public void veriSil(int veri)
        {
            // Veri silme işlemleri burada yer alır
            Console.WriteLine("Veri tabanından veri silme {veri} işlemi çalıştı.");
        }
    }
}
```

7. *Solution Explorer* üzerinde **VeriBileşenleri** projesi sağ tıklanarak *Build* seçilir ve derlenir.

8. Daha sonra *Solution Explorer* üzerinde **İşBileşenleri** projesi seçilir ve *Add→Project Reference→Project...* seçilerek **VeriBileşenleri** projesi seçilir. Bunu yaptıktan sonra **İşBileşenleri** tarafında aşağıdaki değişiklikleri yapabiliriz.

```
using VeriBileşenleri; // İş projesine Veri Projesi Referans olarak eklenir.
namespace İşBileşenleri
{
    public class İşSınıfı
    {
        public void işYöntemi()
        {
            // İş mantığı burada yer alır
            int işlenenVeri = 0;
            VeriTabanı veriTabanı = new VeriTabanı(); //
            veriTabanı.veriEkle(işlenenVeri); //
            //...
            veriTabanı.veriSil(işlenenVeri); //
            Console.WriteLine("İş sınıfı iş yöntemi çalıştı.");
        }
    }
}
```

9. *Solution Explorer* üzerinde **İşBileşenleri** projesi sağ tıklanarak *Build* seçilir ve derlenir.
10. Daha sonra *Solution Explorer* üzerinde **İstemciBileşenleri** projesi seçilir ve *Add→Project Reference→Project...* seçilerek **İşBileşenleri** projesi seçilir. Bunu yaptıktan sonra **İstemciBileşenleri** tarafında aşağıdaki değişiklikleri yapabiliriz.
11. Artık projemiz çalışabilir durumdadır.

```
using İşBileşenleri; // İstemci projesine İş Projesi Referans olarak eklenir.
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
            İşSınıfı işnesnesi = new İşSınıfı(); //
            işnesnesi.işYöntemi(); //
        }
    }
}
```

Çok Katmanlı Uygulamalar

Yukarıda örneği verilen üç katmanlı uygulamalarda ihtiyaca göre katmanlar daha alt katmanlara ayrılabilir.

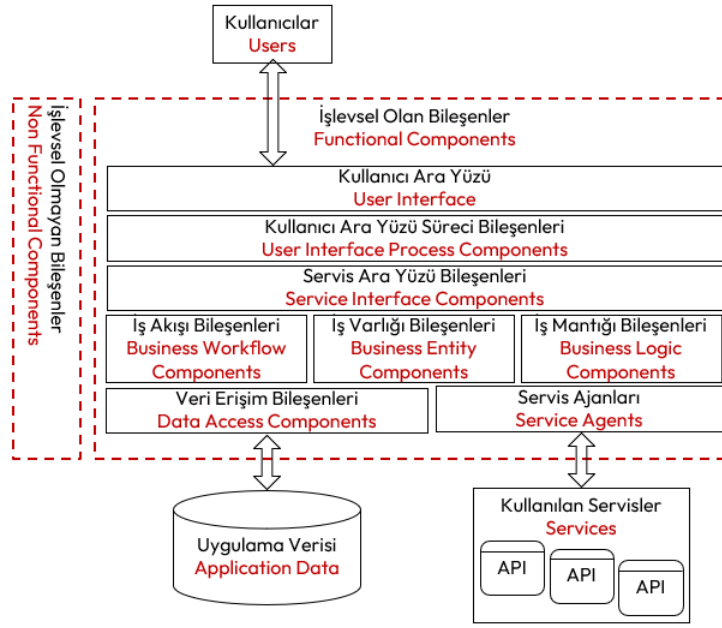
Örneğin kullanıcı ara yüzü katmanı, aşağıdaki bileşenlerden oluşacak şekilde ikiye ayrılabilir;

1. İnsanların yazılıma talimatlarını verdiği **insan talimatları bileşenleri** (**Human Instruction Components-HIC**) ve
2. Kullanıcı talimatlarını, kullanıcı ara yüzünden alıp, iş katmanının anlayacağı şekle çeviren **kullanıcı süreci bileşenleri** (**User Process Components-UPC**).

Benzer şekilde İş katmanı, aşağıdaki bileşenlerden oluşacak şekilde üçe ayrılabilir;

1. **İş akışını sağlayan bileşenler** (**Business Workflow Components-BWC**),
2. **İş varlıkları bileşenleri** (**Business Entity Components-BEC**) ve
3. İş varlıklarının toplu olarak işlenmesi ile ilgili **iş mantığı bileşenleri** (**Business Logic Components-BLC**)
4. Çok katmanlı mimari, günümüz yazılım mimarisi olup mümkün olduğunca katmanların birbirinden bağımsız tasarlandığı uygulamalardır. Üç katmanlı mimariden daha alt katmanlara ayrılarak geliştirilen yazılımlara, n-katmanlı (n-layer/ n-tier) yazılım adı verilir.

Çok katmanlı yazılım geliştirilirken kullanılan bileşenler ve bu bileşenlerin bulunduğu katmanlar detaylı olarak aşağıdaki şekilde verilmiştir.



Şekil 64. Çok Katmanlı Mimari

Yukarıda verilen şekilde, dikine yazılı olan yerler, yazılımı gerçekleştirilen sistemde bulunmayan, ancak yazılım geliştirilirken ihtiyaç duyulan bileşenleri ve bununla ilgili servisleri temsil etmektedir. Bu nedenle bu bileşenler, yazılımın **işlevsel olmayan** (non-functional) bölgeleridir. Yani ortada bir yazılım olmayınca hiçbir önemi olmayan bileşenlerdir. Geri kalan bileşenler ise **işlevsel** (functional) bileşenler olarak adlandırılır. İşlevsel olmayan bileşenler detaylı olarak aşağıda gösterilmiştir.



Şekil 65. İşlevsel Olmayan Bileşenler

Burada bahsedilen **güvenlik** (security) katmanlara **rol bazlı** (role based) veya **kod erişimi** (code access) olmak üzere iki şekilde ilave edilir.

Sonuç olarak yazılımlarda katman sayısı arttıkça **performans** (performance) düşer ama **esneklik** (flexibility) artar, **bakım** (maintainability) kolaylaşır, **yeniden kullanılabilirlik** (reusability) ve **ölçeklenebilirlik** (scalability) sağlanmış olur.

Katmanların Tasarlanması

İş modeli (business model), kurumun yaptığı işi anlatan bir modeldir. Bu modelin çıkarılmasına yönelik yapılan analize ise **iş analizi** (business analysis) adı verilir. İş modelinde yer alan bütün süreçler doğal olarak bilgisayar tarafından yapılamaz. İşte iş modelindeki bilgisayar tarafından süreçleri içeren analize ise **gereksinim analizi** (requirement analysis) adı verilir.

Yapılan gereksinim analizinden sonra katmanlar ayrı ayrı tasarlanır;

1. **Business Logic Components (BLC)**, **sorun alanı bileşenleri (Problem Domain Components-PDC)** olarak da adlandırılır. Uygulamanın, iş modelinde belirtilen, iş kurallarını içeren bileşenlerdir. PDC, gerçek dünyadaki problemin yazılımda modellenmiş haliyle ilgilidir. Sistem çözümünün temel mantığını ve **iş kurallarını (business rules)** kapsar. Gerçek hayat problemini, teknik detaylardan bağımsız şekilde doğru ve tam olarak modellemek amacıyla tanımlanırlar.

```
//Örnek PDC/BLC Bileşenleri
public class Sipariş
{
    public Guid Id { get; private set; }
    public List<SiparişKalemi> SiparişKalemleri { get; private set; }
    public DateTime SiparişZamanı { get; private set; }
    public Sipariş()
    {
        Id = Guid.NewGuid();
        SiparişKalemleri = new List<SiparişKalemi>();
        SiparişZamanı = DateTime.UtcNow;
    }
    public void YeniKalemEkle(SiparişKalemi item)
    {
        if (item.Miktar <= 0)
            throw new ArgumentException("Miktar 0 dan fazla olmalıdır!");

        SiparişKalemleri.Add(item);
    }
    public decimal ToplamSiparişÜcretiGetir()
    {
        return SiparişKalemleri.Sum(i => i.BirimFiyat * i.Miktar);
    }
}
public class SiparişKalemi
{
    public Guid KalemId { get; set; }
    public int Miktar { get; set; }
    public decimal BirimFiyat { get; set; }
}
```

2. **Business Entity Components (BEC)** veri tabanı tabloları ile eşleşen ve iş nesnelerini temsil eden sınıflar içerir. Sınıflar yalnızca özellik barındırır ve veri taşıma, saklama veya ileme süreçlerinde kullanılır, içinde iş mantığı barındırmaz. Bu sınıflar çok nadir olarak **doğrulama (validation)** yöntemleri içerir. Bu sınıflardan imal edilen nesneler **veri transfer nesneleri (Data Transfer Object-DTO)** olarak kullanılır.

```
//Örnek BEC Bileşenleri
public class SiparişDto
{
    public Guid Id { get; set; }
    public List<SiparişKalemiDto> SiparişKalemleri { get; set; }
    public decimal ToplamÜcret { get; set; }
    public DateTime SiparişZamanı { get; set; }
}
public class SiparişKalemiDto
{
    public Guid KalemId { get; set; }
    public int Miktar { get; set; }
    public decimal BirimFiyat { get; set; }
}
```

3. PDC bileşenleri, iş süreçlerini işlerken kullanılabilir; fakat veri tabanına yazılacağı veya servis üzerinden bir istemciye gönderileceği zaman iş varlığına (BEC) dönüştürülür. Dönüştürme için ayrı bileşenler tasarlanır.

```
//PDC den BEC bileşenine dönüştürme
public static class SiparişMapper
{
    public static SiparişDto ToDto(Sipariş sipariş)
    {
        return new SiparişDto
        {
            Id = sipariş.Id,
            SiparişZamanı = sipariş.SiparişZamanı,
            SiparişKalemleri = sipariş.SiparişKalemleri.Select(i => new SiparişKalemiDto
            {
                KalemId = i.KalemId,
                Miktar = i.Miktar,
                BirimFiyat = i.BirimFiyat
            }).ToList(),
            ToplamÜcret = sipariş.ToplamSiparişÜcretiGetir()
        };
    }
}
```

4. **Business Workflow Components (BWC)**, kurumsal yazılım mimarilerinde **iş süreçlerinin (business processes/workflows)** akışını ve yönetimini tanımlayan bileşenlerdir. Genellikle iş akışlarının adım adım yürütülmesi, durumların yönetilmesi ve süreç bazlı kararların verilmesi gibi konuları kapsar.

```
public class SiparişWorkflow
{
    private readonly IÖdemeServisi ödemeServisi;
    private readonly IStokServisi stokServisi;
    private readonly IGönderimServisi gönderimServisi;

    public OrderWorkflow(IÖdemeServisi ödeme, IStokServisi stok, IGönderimServisi gönderim)
    {
        ödemeServisi = ödeme;
        stokServisi = stok;
        gönderimServisi = gönderim;
    }

    public async Task ProcessOrderAsync(Sipariş sipariş)
    {
        if (!await ödemeServisi.Charge(sipariş))
            throw new Exception("Ödeme Alınamadı!");

        if (!await stokServisi.CheckStock(sipariş.SiparişKalemleri))
            throw new Exception("Yetersiz Stok!");

        await gönderimServisi.DağıtımHazırla(sipariş);

        sipariş.Durum = SiparişDurumu.DağıtımaÇıkacak;
    }
}
```

5. **Service Interface Components (SIC)**, uygulamanın dış dünya (kullanıcılar, sistemler, API istemcileri vs.) ile iletişim kurduğu giriş noktalarıdır. Diğer bir deyişle, uygulamanın hizmetlerini dışa açan ara yüzlerdir. RESTful API Başlığını inceleyiniz.
6. **Data Access Components (DAC)**, iş varlıklarının (BEC) tuttuğu verilere ilişkin bir kalıcılık sağlamak için veritabanı ya da diğer kalıcı veri kaynaklarına erişimi soyutlayan ve yöneten bileşenlerdir. Kalıcılığı sağlamak için bu bileşenler bazen detaylı bir mantık içerebilir. Bu mantığa, **kalıcılık mantığı (persistence logic)** adı verilir. Bunlar sayesinde uygulamanın diğer katmanları (özellikle uygulama ve iş mantığı katmanları), veri kaynağının teknik detaylarını bilmeden veri okuma/yazma işlemlerini gerçekleştirebilir. Burada veri işlemek için **depo deseni (repository pattern)** kullanılır. Bu desende CRUD işlevlerini de içine alan **ara yüzler (interface)** üzerinden iş bileşenleri (BEC) ihtiyaçları çözülür.

```
public interface ISiparişReporistory
```

```
{
    Task<Sipariş?> GetByIdAsync(Guid id);
    Task AddAsync(Sipariş sipariş);
    Task SaveChangesAsync();
    Task<List<Sipariş>> GetAllAsync();
    Task DeleteAsync(Guid id);
}

public class SiparişRepository : ISiparişRepository
{
    private readonly string _connectionString;
    public SiparişRepository(string connectionString)
    {
        _connectionString = connectionString;
    }
    public async Task<List<Sipariş>> GetAllAsync()
    {
        var sipariş = new List<Sipariş>();
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "SELECT Id, SiparişZamanı FROM Siparişler",
            conn);
        await conn.OpenAsync();
        using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            sipariş.Add(new Sipariş
            {
                Id = reader.GetGuid(0),
                SiparişZamanı = reader.GetDateTime(2)
            });
        }
        return sipariş;
    }
    public async Task<Sipariş?> GetByIdAsync(Guid id)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "SELECT Id, SiparişZamanı FROM Siparişler WHERE Id = @Id",
            conn);
        cmd.Parameters.AddWithValue("@Id", id);
        await conn.OpenAsync();
        using var reader = await cmd.ExecuteReaderAsync();
        if (await reader.ReadAsync())
        {
            return new Sipariş
            {
                Id = reader.GetGuid(0),
                SiparişZamanı = reader.GetDateTime(2)
            };
        }
        return null;
    }
    public async Task AddAsync(Sipariş order)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "INSERT INTO Siparişler (Id,SiparişZamanı) VALUES (@Id, @SiparişZamanı)",
            conn);
        cmd.Parameters.AddWithValue("@Id", order.Id);
        cmd.Parameters.AddWithValue("@SiparişZamanı", order.SiparişZamanı);
        await conn.OpenAsync();
    }
}
```



```

        await cmd.ExecuteNonQueryAsync();
    }
    public async Task UpdateAsync(Sipariş order)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "UPDATE Siparişler SET SiparişDate = @SiparişZamanı WHERE Id = @Id",
            conn);
        cmd.Parameters.AddWithValue("@Id", order.Id);
        cmd.Parameters.AddWithValue("@CustomerName", order.CustomerName);
        cmd.Parameters.AddWithValue("@SiparişDate", order.SiparişDate);
        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
    public async Task DeleteAsync(Guid id)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "DELETE FROM Siparişler WHERE Id = @Id",
            conn);
        cmd.Parameters.AddWithValue("@Id", id);
        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
}

```

7. Veri erişim bileşenleri (DAC), her BEC ile ilgili veri kaynağında, **yeni kayıt** (**create**), mevcut kaydı **okuma** (**read**), kaydı **güncelleme** (**update**) ve bir kaydı **silme** (**delete**) yöntemlerini yerine getirir. Açıklanan bu işlevleri yerine getiren yöntemlerin tümü kısaca CRUD (**Create-Read-Update-Delete**) olarak adlandırılır. CRUD yöntemleri; ABCD (**Add-Browse-Change-Delete**), ACID (**Add-Change-Inquire-Delete**), BREAD (**Browse-Read-Edit-Add-Delete**) ve VADE (**View-Add-Delete-Edit**) olarak da adlandırılır.

Katmanlar Arası İletişim

Katmanlı mimaride, iş varlığı bileşenleri (BEC) orta katmanda yer aldığından hem kullanıcı katmanında hem de veri erişim katmanında kullanılırlar. Bu nedenle bu katmanlarda iş varlıklarını, verileri, katmanlar arasında taşırlar. Bu taşınma işlemi, genel olarak üç değişik teknoloji ile sağlanır;

- **Sınıf** (**class**): Katmanların birbirine çok yakın olduğu durumda tercih edilir.
- **Veri Seti** (**DataSet**): Katmanlar arası veri iletişimi veri tabanı üzerinden yapılır. Genellikle aynı ağı kullanan katmanlar için tercih edilir.
- **Serileştirme** (**serializing**): Bir nesnenin **durumunu** (**state**) kalıcı veya taşınabilir bir şekle dönüştürme sürecidir. Katmanların birbirinden uzak olduğu durumda tercih edilir.

Buradaki tercihler uygulamanın hızını etkiler. Dolayısıyla iş varlıkları tasarlanırken yukarıdaki maddelerden doğru olanını tercih etmek gerekir. Serileştirme, .Net Framework'te, üç farklı teknoloji ile yapılır;³⁸

1. JSON serileştirmesi, .NET nesnelerini JavaScript Nesne Gösterimi'ne (**JavaScript Object Notation-JSON**) eşler. JSON, web genelinde veri paylaşmak için yaygın olarak kullanılan açık bir standarttır. JSON serileştiricisi varsayılan olarak genel özellikleri serileştirir ve özel ve dahili üyeleri de serileştirmek üzere yapılandırılabilir.
2. XML ve SOAP serileştirmesi yalnızca genel özellikleri ve alanları serileştirir ve veri tipine olan sadakati korumaz. Bu durum, verileri kullanan uygulamayı kısıtlamadan veri göndermek veya tüketmek istediğinizde yararlıdır. XML açık bir standart olduğundan, Web genelinde veri paylaşmak için çekici bir seçimdir. SOAP da açık bir standarttır ve bu da onu çekici bir seçim haline getirir.

³⁸ <https://learn.microsoft.com/en-us/dotnet/standard/serialization/>

3. **İkili serileştirme** (**binary serialization**), veri tipi sadakatini korur, yani nesnenin tam durumu kaydedilir ve **serileştirmeden geri aldığınızda** (**deserialization**) tam bir kopyası oluşturulur. Bu tür serileştirme, bir uygulamanın farklı çağrılar arasında bir nesnenin durumunu korumak için yararlıdır. Örneğin, bir nesneyi katmanlar ortasındaki bir panoya serileştirerek farklı uygulamalar arasında paylaşabilirsiniz. Bir nesneyi bir akışa, bir diske, belleğe, ağ üzerinden vb. serileştirebilirsiniz. Uzaktan erişim, nesneleri bir bilgisayardan veya uygulama alanından diğerine "değere göre" geçirmek için bu serileştirmeyi kullanır. Ancak her uygulama ikili serileştirmeyi aynı şekilde yapmalıdır. Aksi durumda tehlikeli durumlar ortaya çıkar.

Bunların yanında çok katmanlı mimaride, katmanların birinde olan istisna, kullanıcı katmanına kadar **yeniden fırlatılır** (**re-throw**). Aksi takdirde istisna kullanıcı katmanına ulaşmadan program kilitlenir. Kullanıcı istisnanın ne olduğu konusunda bilgi sahibi olamaz. Aşağıda istemci sunucu mimarisinde istisna iletimi örneği verilmiştir.

```
//Sunucu.cs (Class Library)
namespace Sunucu
{
    public class Server {
        public int Bölme(int p1,int p2){
            int ret = 0;
            try {
                ret = p1 / p2;
            } catch (Exception e){
                throw e;
                //hata iletme(rethrow):hata kullanıcı
                //katmanına iletilir.
            }
            return ret;
        }
    }
}
```

Bu sunucu kodunu kullanan istemci uygulaması da aşağıda verilmiştir;

```
//İstemci.cs (Console Application)
using Sunucu; //Bunu yapabilmek için:İstemciye Sunucu Referans olarak eklenir.
namespace İstemci
{
    class Client {
        static void Main(string[] args){
            Sunucu.Server sunucu = new Sunucu.Server();
            sunucu.Bölme(12,0); //Hata burada oluşur.
        }
    }
}
```

Bileşen Kullanımı

Uygulamamızda bir **bileşen** (**component**) kullanacak isek bileşenin çalıştığı altyapıdan bağımsız ve soyut olarak kullanılması gerekir. Burada soyut kullanım, bileşenin ve bileşenin işletim sistemindeki yerinin kesin olarak belirtilmemesi ve her kullanımda derlenmemesi ile sağlanır.

Bütün bunlara göre bileşen kullanımında karşılaşılan beş ana zorluk aşağıda verilmiştir.

1. Platform: **Birlikte çalışabilirlik** (**interoperability**), bileşen kullanımında bir problemdir. Bileşenin çalıştığı işletim sistemi ve geliştirildiği platform, kullanımı etkilemektedir. Bileşeni geliştirildiği platformdan bağımsız kullanabilmeliyiz.
2. **Programlama Dili** (**language**): Her programlama dilinin yetenekleri birbirinden farklıdır. Bileşenin geliştirildiği programlama dili farklılıkları bileşen kullanımında sorun yaratabilir.
3. **Süreç** (**process**): Burada sözü geçen süreç, uygulamanın içinde çalıştığı sanal bir zarftır. Bu zarfın sınırları, işletim sistemi tarafından belirlenir. Bu sürecin, yazılımın modellediği süreçle ilgisi yoktur. **İstemci** (**client**) ile **sunucu** (**server**) **bileşenlerinin** (**component**) içinde çalıştığı süreçler arası

iletişimden (**inter-process communication**) kaynaklanan problemler yaşanabilir. Sunucu bileşenlerinin bulunduğu konuma göre dört değişik süreç iletişimi vardır;

- a. **İç süreç (in-process)**: Sunucu ile istemci bileşenlerinin aynı süreç içinde çalışması durumudur.
 - b. **Dış süreç (out-process)**: Sunucu bileşenleri ile istemci bileşenlerinin aynı bilgisayarda farklı süreçler içinde çalışması durumudur.
 - c. **Ağlar arası süreç (inter-network process)**: Sunucu bileşenleri ile istemci bileşenlerinin aynı ağda olan farklı bilgisayardaki süreçler içinde çalışması durumudur.
 - d. **Uzak süreç (remote process)**: Sunucu bileşenleri ile istemci bileşenlerinin farklı ağda olan farklı bilgisayardaki süreçler içinde çalışması durumudur.
4. **Konumlama (location)**: Bileşeni, konumunun nerede olduğunu bilmeden kullanamayız. Yani bileşeni kullanırken bileşenin bulunduğu yeri "C:\DLLs\MyDll.dll" gibi doğrudan konum belirten kodlamaları (**hard-coding**) yapmamalıyız. Bileşenin dosya sistemindeki yeri değiştiğinde, bileşene bağlı olarak uygulamamızı yeniden derlememiz gerekebilir.
 5. **Uyarlama (version)**: Bileşenin bir uyarlaması bir uygulamada farklı, diğer uyarlaması bir başka uygulamada farklı davranabilir. Bu nedenle uyarlama da bileşen kullanımında bir sorun olarak ele alınır.

Burada bileşen kullanımında bahsedilen zorlukları gideren ve bileşenlerin soyut kullanımını sağlayan teknolojiler vardır. Yani bileşenler, bir mağazadan alıp kullanır gibi **soyut (abstract)** olarak kullanmamızı sağlarlar. Bu teknolojiler aşağıda sıralanmıştır:

- DLL (**Dynamic Linked Library**) teknolojisi,
- COM (**Component Object Model**) teknolojisi,
- COM teknolojisinin gelişmiş hali COM+ teknolojisi,
- DCOM (**Distributed COM**) teknolojisi,
- ActiveX teknolojisi,
- CORBA (**Common Object Request Broker Architecture**) veya RMI (**Remote Method Invocation**) teknolojisi.

Microsoft, bileşen kullanımındaki sıkıntıları kronolojik olarak geliştirerek çözmüştür. DLL teknolojisi ile konumlama sorununu çözmüştür. Ardından COM teknolojisi ile programlama dili, süreç, konumlama ve uyarlama problemlerini çözmüştür. .NET teknolojisi ile platform dâhil tüm problemleri çözmüştür.

Dağıtık Programlama

Katmanların birbirinden uzak bulunduğu yazılımlara **dağıtık uygulama (distributed application)** adı verilir. Dağıtık yazılımlarda, bütün işler tek bir kod, hatta tek bir bilgisayar tarafından yürütülmez. Her bir katman ayrı bir bilgisayar ya da süreç tarafından koşturulur. İstemci ile sunucunun birbirinden ayrıldığı dağıtık yazılımlara birçok sebepten dolayı ihtiyaç duyarız. Bunlardan birkaçı aşağıda sıralanmıştır;

- Bazı karmaşık programlar dağıtık bilgisayarlar üzerinde çalışır.
- Birçok ucuz bilgisayar üzerinde bulunan işlemcileri kullanarak çoklu işlemci ile çalışmanın birçok üstünlüğü vardır.
- Dağıtık programlama aynı zamanda **ölçeklenebilirliği (scalability)** de artırır.
- Bazı yazılımlar vardır ki sadece belli donanımlara sahip bilgisayarlarda çalışırlar.
- Güvenlik gerekçesi ile bazı servisler ayrı bilgisayarlarda olmak zorundadır.
- Servislerin sadece kaynak olarak kullanıldığı durumlar.

Dağıtık Programlama Teknolojileri

Dağıtık uygulamalarda, sunucu bileşenlerinin istemci bileşenleri ile iletişim sağlayabilmesi için bizi günümüze kadar getiren birçok teknoloji bulunmaktadır;

- **İç süreç (in-process)** için bir gereklilik bulunmaz.
- **Dış süreç (out-process)** için iletişim, süreçler arası iletişim olan **Inter-Process Communication (IPC)** ile sağlanır. Clipboard, Message Queue and Send Message, Shared Memory, **Lightweight Procedure Call (LRPC)**, **Remote Procedure Call (RPC)**, COM, DCOM, CORBA ve RMI teknolojileri bunu sağlar.

- Ağlar arası süreç (inter-network process) için Jakarta XML Binding (JAXB) ve .NET Remoting teknolojileri kullanılmaktadır.
- Uzak süreç (remote process) için iletişimi için Windows Communication Foundation (WCF) ve Java Architecture for XML Web Service (JAX-WS)

Günümüzde bağlantı hızlarının artması ve bilgisayarların işlem kapasitesinin artması nedeniyle eskimiş kabul edilmektedir. Yeni teknolojiler aşağıda sıralanmıştır;

1. Web Servisleri: ASP.NET Web API veya ASP.NET Core Web API ile RESTful servisler, oluşturarak veri alışverişi yapılabilir.
2. gRPC: Google tarafından geliştirilen hızlı ve verimli bir Remote Procedure Call (RPC) protokolüdür. Grpc.AspNetCore ile kolayca kullanılabilir. Performans açısından REST API'lere göre genelde daha hızlıdır.
3. Message Queue: Uygulamalar arasında mesaj geçişi ile asenkron haberleşme sağlanır. RabbitMQ, Apache Kafka, ActiveMQ ve Azure Service tercih edilebilir.
4. SignalR: Sohbet gibi gerçek zamanlı iletişim için kullanılır. Sunucudan istemcilere anlık veri gönderebilir.

Bir ağ söz konusu olduğunda aşağıdaki kavramların üzerinden geçmek gerekir;

Her bir bilgisayarlar ağ üzerinde bir Internet Protocol (IP) adresine sahiptir. Bu adresleri ezberlememek için bilgisayarın IP adresi yerine Uniform Resource Locator (URL) adı verilen bir dizgi (string) metni kullanılır. Bu URL ile IP eşleştirmesini alan adı sunucuları (name server) yapar.

```
http://www.ornek.org/
```

Bilgisayarlardaki her bir kaynak da URL olarak gösterilebilir.

```
http://www.ornek.org/index.html
http://www.ornek.org/CSS/index.css
http://www.ornek.org/Images/Logo.jpg
```

Portlar uygulamaların bilgisayarda çalıştığını gösteren sanal kapılardır. Bilgisayarlar genellikle tek bir bağlantı (single connection) ile ağa bağlanırlar. Buradaki tek bağlantıya sahip bilgisayara ağ üzerinden gelen paketlerin, hangi uygulamaya ait olduğunun belirlenmesi ve söz konusu paketin ilgili uygulamaya ulaştırılmasının sağlanması amacıyla portlar kullanılır.

```
http://www.ornek.org:80/
http://www.ornek.org:8080/
ftp://ftp.ornek.com:21/
```

Portlar 0 ile 65535 arasında olup, ilk 1024 tanesi özel uygulamalar için ayrılmıştır. Örneğin 80 numaralı port, Hyper Text Transmission Protocol (HTTP) sunucusunu, 25 numaralı port Simple Mail Transport Protocol (SMTP) sunucusunu ve 21 numaralı port ise File Transfer Protocol (FTP) sunucusunu temsil eder. URL'nin başındaki http ve ftp ise bilgisayara ulaşılan port üzerinde çalışan protokolü göstermektedir.

Web Servisler

Web servis, internet üzerinden uygulamalar arasında veri alışverişi yapmayı sağlayan bir yapıdır. C# ile web servis oluşturmanın en yaygın yolu ASP.NET Web API veya ASP.NET Core Web API kullanmaktır. Günümüzde ASP.NET Core Web API, modern, platformlar arası (cross-platform) ve performanslı olduğu için tercih edilir.

Representational State Transfer (REST), 2000 yılında Roy Fielding'in doktora tezinde tanımlanan bir mimari stildir. Web sunucuları ile tarayıcılar arasında kullanılan HTTP yöntemlerinin http için belirlenen görevleri dışında veri değişiminde kullanılması esasına dayanır.

REST kurallarına uygun bir şekilde tasarlanmış web servisine RESTful API denir. Bu API, kaynaklara (veri nesneleri) HTTP yöntemleri (GET, POST, PUT, DELETE vb.) üzerinden erişim sağlar. RESTful servisindeki her şey bir kaynak (resource) olarak görülür. Kaynaklar genelde isimlendirilmiş Uniform Resource Identifier (URI) ile tanımlanır. URI, web adresine benzer şekilde bir kaynağı eşsiz ve açık bir biçimde tanımlayan standart biçime sahip bir dizgidir (string).

<https://api.ornek.com/ogrenciler/3>

RESTful servisleri http yöntemlerini yeni görev vererek anlamlı kullanır;

Yöntem	Görevi	Örnek URI	Açıklama
GET	Veri getir	/ogrenciler	Tüm öğrencileri getir
GET	Belirli veriyi getir	/ogrenciler/5	ID'si 5 olan öğrenciyi getir
POST	Yeni veri oluştur	/ogrenciler	Yeni öğrenci ekle
PUT	Veriyi tamamen güncelle	/ogrenciler/5	Öğrenci güncelle
PATCH	Veriyi kısmen güncelle	/ogrenciler/5	Sadece postasını değiştir
DELETE	Veriyi sil	/ogrenciler/5	Öğrenciyi sil

Tablo 22. http Yöntemlerinin RESTful Serviste Kullanılması

RESTful servislerde her HTTP isteği bağımsızdır yani **durumsuzdur** (stateless);

- Sunucu önceki istekleri hatırlamaz. Durum bilgisi (örneğin, oturum bilgisi) her istekte genellikle JWT veya **jeton** (token) ile yeniden iletilmelidir.
- RESTful API isteklerinde taşınan veriler ön belleğe alınabilir. Web sunucusuna, verinin ne kadar süreyle geçerli olduğunu belirtmelidir.
- İstemci, ara katman olup olmadığını bilmez bu nedenle katmanlı mimari için uygundur. API istekleri Load Balancer, API Gateway gibi yönlendirici sunuculardan geçebilir.
- Veri alışverişi genellikle **JavaScript Object Notation** (JSON) tercih edilmekte birlikte XML gibi formatlarla da kullanılır.

Aşağıda JSON formatında örnek bir veri verilmiştir;

```
{
  "id": 1,
  "adi": "İlhan ÖZKAN",
  "posta": "ilhanozkan@outlook.com"
}
```

RESTful Web API Örneği

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

WebAPI	→ Çözüm (Solution)
└─ WebApplication1	→ Proje (ASP.NET Core Web AP)
└─ Controllers	→ Denetçilerin Olacağı Klasör
└─ WeatherForecastController.cs	→ Class (WeatherForecastController-Controller)
└─ KitapKontroler.cs	→ Class (KitapController-Controller)
└─ WeatherForecas.cs	→ Class (WeatherForecast-Model)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde **File**→**New**→**Project...** seçilerek **WebAPI** adlı çözüm içinde **WebApplication1** adlı projemiz **ASP.NET Core Web API** olarak oluşturulur.
2. Oluşan projede **Program.cs** ve **WeatherForecast.cs** ile **Controllers** klasöründe bulunan **WeatherForecastController.cs** dosyaları otomatik olarak oluşur.
3. İlk önce **WeatherForecast.cs** dosyasını inceleyelim;

```
namespace WebApplication1
{
    public class WeatherForecast
    {
        public DateOnly Date { get; set; }

        public int TemperatureC { get; set; }

        public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);

        public string? Summary { get; set; }
    }
}
```

```
}
```

4. Burada sözü geçen **Contollers** dizini altındaki **WeatherController.cs** dosyası MVC mimari desenindeki denetçidir. **WeatherForecastController.cs** dosyası incelendiğinde URL **rotası** (route) üzerinden GET isteği ile gelindiğinde 5 adet rastgele hava tahmini üreteceği görülmektedir.

```
using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class WeatherForecastController : ControllerBase
    {
        private static readonly string[] Summaries = new[]
        {
            "Freezing", "Bracing", "Chilly", "Cool", "Mild",
            "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
        };

        private readonly ILogger<WeatherForecastController> _logger;

        public WeatherForecastController(ILogger<WeatherForecastController> logger)
        {
            _logger = logger;
        }

        [HttpGet(Name = "GetWeatherForecast")]
        public IEnumerable<WeatherForecast> Get()
        {
            return Enumerable.Range(1, 5).Select(index =>
                new WeatherForecast
                {
                    Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
                    TemperatureC = Random.Shared.Next(-20, 55),
                    Summary = Summaries[Random.Shared.Next(Summaries.Length)]
                }).ToArray();
        }
    }
}
```

5. Son olarak **Program.cs** dosyasını inceleyelim. **builder.Services.AddControllers();** talimatı ile MVC deseninde biz nasıl **MVCBaşlat()** yöntemi var ise burada da bir web uygulamasının denetçileri web uygulamamıza **Controllers** klasöründen eklenir. Geri kalan tüm işlemler web sunucusunu http isteklerini ayarlamak için ek ayarlardır.

```
var builder = WebApplication.CreateBuilder(args);
/*
Controller tabanlı API oluşturmak için gerekli servisi aşağıdaki talimat kaydeder.
*/
builder.Services.AddControllers();
/*
Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
*/
builder.Services.AddOpenApi();

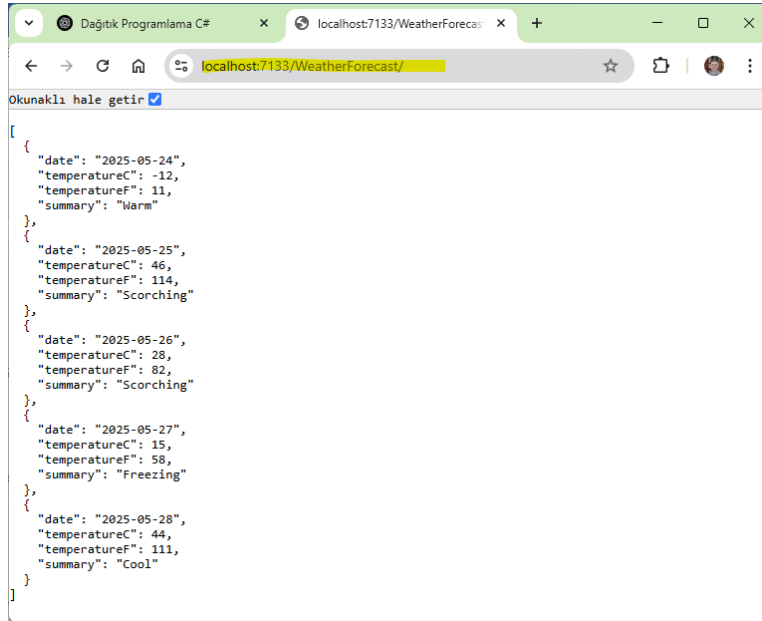
var app = builder.Build();
// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}
```

```
/*
HTTP isteklerini otomatik olarak HTTPS'e yönlendirmek için aşağıdaki talimat yazılır.
*/
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

6. Programı çalıştırdığımızda API uygulamamız http web sunucu üzerinden istekleri kabul etmeye başlar.

```
info: Microsoft.Hosting.Lifetime[14]
  Now listening on: https://localhost:7133
info: Microsoft.Hosting.Lifetime[14]
  Now listening on: http://localhost:5085
info: Microsoft.Hosting.Lifetime[0]
  Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
  Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
  Content root path: D:\CSharp\Samples\services\Web API\WebApplication1
```

7. Çalışan Web API üzerinden veri almak için bir web tarayıcısı açıp kullanabiliriz;



Şekil 66. WebApplication1 API Uygulamamızdan GET yöntemi ile Veri Çekme

8. Şimdi uygulamayı durdurup yeni bir denetçi ekleyelim. Burada model kullanmayacağız; Bunun için Solution Explorer üzerinde **Controllers** sağ tıklanarak *Add → Controller...* seçilir ve **KitapController.cs** adında *MVC Controller -Empty* denetçi oluşturulur.

```
using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    public class KitapController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}
```


9. Burada model ve gösterim ihtiyacımız olmadığından **Controllers/KitapController.cs** içinde aşağıdaki değişiklikleri yapalım;

```
using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class KitapController : Controller
    {
        [HttpGet]
        public IActionResult TumKitaplariGetir()
        {
            var kitaplar = new[]
            {
                new { Id = 1, Ad = "Sefiller", Yazar = "Victor Hugo" },
                new { Id = 2, Ad = "Suç ve Ceza", Yazar = "Dostoyevski" }
            };

            return Ok(kitaplar);
        }

        [HttpGet("{id}")]
        public IActionResult KitapGetir(int id)
        {
            if (id == 1)
                return Ok(new { Id = 1, Ad = "Sefiller", Yazar = "Victor Hugo" });
            else
                return NotFound("Kitap bulunamadı");
        }
    }
}
```

10. Uygulamayı tekrar çalıştırıp tarayıcıda tüm kitapları getirmek için <https://localhost:7133/api/kitap> veya 1 numaralı kitabı getirmek için <https://localhost:7133/api/kitap/1> adresi yazılır.

API'mizden veri almak için örnekteki gibi tarayıcı kullanabiliriz ama bu her zaman yeterli olmaz. API'mizi test etmek için en yaygın HTTP test aracı olan Postman, komut satırından istek gönderme için cURL veya otomatik API dokümantasyonu da çıkaran Swagger UI kullanılabilir.

Örnek Kurumsal Uygulama

Kurumlar, içinde oldukları zamana bağlı olarak, iş yapma şekillerini ya da süreçlerini değiştirirler. Buna kullandıkları girdilerdeki değişim, istenilen çıktılardaki değişim ya da yasal mevzuattaki değişim sebep olmaktadır. Buna paralel olarak organizasyonu modelleyen ya da benzetim yapan yazılımların bu değişiklikleri karşılayabilir olması gerekir.

Yazılımlar, değişim olmadığı takdirde, sonsuza kadar çalışabilirler. Ancak yazılımda yapılacak her değişimin, canlı bir varlığa atılan bir bıçak darbesi olacağı akıldan çıkarılmamalıdır. Bu nedenle yazılımlar geliştirilirken, değişim yönetiminin ya da hangi sürecin ne kadar değişebileceğinin önceden belirlenmesi gerekir. Değişim yönetimi ne kadar başarılı olur ise yazılım da o kadar başarılı olacaktır.

Değişim yönetiminin yanında, yazılımda daha önceden yapılmış ve başarı sağlamış olan modellerin kullanılması da oldukça önemlidir. Yani yazılım tasarımı ve analizi yapılırken başarılı tecrübeleri içeren **desenlerin (pattern)** kullanılması gerekir. Bir yazılım, ne kadar fazla desenlere uygun yazılır ise, o kadar değişime açık olacaktır.

Bütün bunların yanında yazılımların katmanlara ayrılması işletim ve bakım açısından çok önemlidir. Çok katmanlı mimaride yazılan yazılımlar, ölçeklenebilirliği arttırdığı gibi birden fazla donanıma yükü paylaştırdığı için, küçük fiyatlı donanımlarla büyük işler başarılabilmektedir.

Örnek uygulama olarak müşteriden sipariş alan basit bir uygulama ele alınacaktır. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

ÇokKatmanlıUygulama	→ Çözüm (Solution)
└─ Entities	→ Proje (Class Library)
└─ Sipariş.cs	→ Class
└─ DataAccess	→ Proje (Class Library)
└─ SiparişRepository.cs	→ Class
└─ Business	→ Proje (Class Library)
└─ SiparişYöneticisi.cs	→ Class
└─ UI	→ Proje (Console Application)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **ÇokKatmanlıUygulama** adlı çözüm içinde **Entities** adlı ilk projemiz *Class Library* olarak oluşturulur.
2. Proje ile oluşan **Class1.cs** dosyasını **Sipariş.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştiriyoruz;

```
//İş Varlığı Bileşenleri (BEC)
//Katmanlar arası taşınacak veri yapısını temsil eder
namespace Entities
{
    public class Sipariş
    {
        public Guid Id { get; set; }
        public string MüşteriAdı { get; set; } = string.Empty;
        public DateTime SiparişZamanı { get; set; }
    }
}
```

3. Daha sonra Solution Explorer üzerinde çözüme sağ tıklanarak *Add→New Project...* seçilerek **DataAccess** adlı ikinci projemiz *Class Library* olarak oluşturulur.
4. Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır ve **Microsoft.Data.Sqlite Sqlite** paketi yüklenir.
5. Solution Explorer üzerinde **Entities** projesi sağ tıklanarak *Build* seçilir ve derlenir.
6. Solution Explorer üzerinde bu proje sağ tıklanarak *Add→Project Reference* ile **Entities** projesi dahil edilir.
7. Verileri veri tabanında işleyeceğimizden ilk önce **Siparişler** tablosunu oluşturmamız gerekir. Bunun için veri tabanı yöneticisinde aşağıdaki SQL komutu çalıştırılabilir;

```
CREATE TABLE Orders (
    Id TEXT PRIMARY KEY, -- GUID stringolarak
    CustomerName TEXT NOT NULL,
    OrderDate TEXT NOT NULL -- ISO 8601 tarih formatında saklanabilir
);
```

8. Daha sonra bu tabloyu veri eklemek ve verileri okumak için kullanabiliriz. Proje ile oluşan **Class1.cs** dosyasını **SiparişRepository.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştirilir;

```
//Veri Erişim Bileşenleri (DAC)
//Veri kaynağında CRUD işlemlerini yapar
using Entities;
using Microsoft.Data.Sqlite;

namespace DataAccess
{
    public class SiparişRepository
    {
        private readonly string connectionString;
        public SiparişRepository(string connectionString)
        {
```

```

        this.connectionString = connectionString;
    }
    public List<Sipariş> GetAll()
    {
        var orders = new List<Sipariş>();
        using var conn = new SqlConnection(connectionString);
        conn.Open();
        using var cmd = conn.CreateCommand();
        cmd.CommandText="SELECT Id, MüşteriAdı, SiparişZamanı FROM Siparişler";

        using var reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            orders.Add(new Sipariş
            {
                Id = reader.GetGuid(0),
                MüşteriAdı = reader.GetString(1),
                SiparişZamanı = reader.GetDateTime(2)
            });
        }
        conn.Close();
        return orders;
    }
    public void Add(Sipariş sipariş)
    {
        using var conn = new SqlConnection(connectionString);
        conn.Open();
        using var cmd = conn.CreateCommand();
        cmd.CommandText = "INSERT INTO Siparişler (Id, MüşteriAdı, SiparişZamanı)" +
            "VALUES(@Id, @Ad, @Zaman)";
        cmd.Parameters.AddWithValue("@Id", sipariş.Id);
        cmd.Parameters.AddWithValue("@Ad", sipariş.MüşteriAdı);
        cmd.Parameters.AddWithValue("@Zaman", sipariş.SiparişZamanı);
        cmd.ExecuteNonQuery();
        conn.Close();
    }
}

```

9. Ardından Solution Explorer üzerinde çözüme sağ tıklanarak *Add→New Project...* seçilerek **Business** adlı üçüncü projemizi *Class Library* olarak oluşturuyoruz.
10. Bu projeye de sağ tıklanarak *Add→Project Reference* ile **Entities** projesi dahil edilir.
11. Solution Explorer üzerinde **DataAccess** projesi sağ tıklanarak *Build* seçilir ve derlenir.
12. **Business** projesi sağ tıklanarak *Add→Project Reference* ile **DataAccess** projesi dahil edilir.
13. Proje ile oluşan **Class1.cs** dosyasını **SiparişYöneticisi.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştirilir;

```

//İş Mantığı Bileşenleri (BLC)
//İş kurallarını uygular, doğrulama (validaton) yapar
using Entities;
using DataAccess;
namespace Business
{
    public class SiparişYöneticisi
    {
        private readonly SiparişRepository depo;
        public SiparişYöneticisi(SiparişRepository repo)
        {
            depo = repo;
        }
        public List<Sipariş> GetSiparişler()
    }
}

```

```

    {
        return depo.GetAll();
    }
    public void SiparişOluştur(string müşteriAdı)
    {
        if (string.IsNullOrEmpty(müşteriAdı))
            throw new ArgumentException("Müşteri adı gereklidir");
        var yeniSipariş = new Sipariş
        {
            Id = Guid.NewGuid(),
            MüşteriAdı = müşteriAdı,
            SiparişZamanı = DateTime.UtcNow
        };
        depo.Add(yeniSipariş);
    }
}

```

14. Son olarak Solution Explorer üzerinde çözüme sağ tıklanarak *Add→New Project...* seçilerek **UI** adlı dördüncü projemizi *Console Application* olarak oluşturuyoruz.

15. Bu proje sağ tıklanarak *Add→Project Reference...* ile **DataAccess** projesi dahil edilir.

16. Solution Explorer üzerinde **Business** projesi sağ tıklanarak *Build* seçilir ve derlenir.

17. **UI** projesine sağ tıklanarak *Add→Project Reference...* ile **Business** projesi dahil edilir.

18. Proje ile oluşan **Program.cs** aşağıdaki şekilde değiştirilir;

```

// See https://aka.ms/new-console-template for more information
//Kullanıcı Arayücü (UI-Console Application)
//Kullanıcı ile etkileşim sağlar

using Business;
using DataAccess;

var connectionString = @"Data Source=D:\CSharp.sqlite";
var depo = new SiparişRepository(connectionString);
var yönetici = new SiparişYöneticisi(depo);

// Yeni sipariş ekleme
Console.Write("Müşteri adı girin: ");
string ad = Console.ReadLine() ?? "";
yönetici.SiparişOluştur(ad);
Console.WriteLine("Sipariş eklendi!");

// Siparişleri listeleme
var siparişler = yönetici.GetSiparişler();
foreach (var sipariş in siparişler)
{
    Console.WriteLine($"{sipariş.Id} - {sipariş.MüşteriAdı} - {sipariş.SiparişZamanı}");
}

```

19. Uygulamayı çalıştırmak için Solution Explorer üzerinde **UI** projesi sağ tıklanarak *Set As Startup Project* seçilir. Artık programımız çalışır.

VARLIK ÇERÇEVESİ

Nesne-İlişkili Eşleme

Katmanlı yazılım mimarisinin işlendiği *Katmanların Tasarlanması* başlığında iş katmanındaki veri taşıyan sınıflar (BEC) ışığında veri erişim katmanı tasarlandığı ve iş varlıklarının soyut olarak veri tabanında nasıl saklanacağına veri erişim bileşenleri (DAC) ile yapıldığı anlatılmıştı.

Veri erişim katmanında yer alacak her veri erişim bileşenlerinin (DAC) her biri, **yaratma** (create), veri tabanında **kayıt** (record), **okuma** (read), **güncelleme** (update) ve **silme** (delete) işlemini gerçekleştirecek şekilde yöntemlere sahiptir.

Günümüzde iş varlığı bileşenlerini (BEC) için bu CRUD yöntemlerini otomatik sağlayan ve en az kodlama ile eşleştiren teknolojiler ortaya çıkmıştır. Bu teknolojinin adı **nesne-ilişkili eşlemedir** (Object-Relational Mapping-ORM). Kısaca iş varlıklarımızı veri tabanındaki tablolardaki kayıtlarla eşler. Yani projemizde CRUD işlevleri için gerekli SQL komutları ORM tarafından otomatik olarak oluşturulur ve bu teknoloji sayesinde daha hızlı ve hatasız yazılım geliştirilir.

ORM, çok fazla programlama yapmadan, otomatik bir şekilde iş nesnelerinden ilişkisel veri tabanına veri depolamak için bir araçtır. Üç ana bölümden oluşur:

1. İş bileşenlerini içeren etki alanı sınıf nesneleri
2. İlişkisel veri tabanı nesneleri
3. Etki alanı nesnelerinin ilişkisel veri tabanı nesnelerine nasıl eşlendiğine dair eşleme bilgileri.

ORM, veri tabanı tasarımıyı iş bileşeni sınıf tasarımıymızdan ayrı tutmamızı sağlar. Bu, uygulamayı sürdürülebilir ve genişletilebilir hale getirir. Ayrıca standart CRUD işlemini (Oluşturma, Okuma, Güncelleme ve Silme) otomatikleştirir, böylece geliştiricinin bunu manuel olarak yazmasına gerek kalmaz.

Veri erişimi için veri erişim katmanında ADO.Net kodunu yazmak ve yönetmek sıkıcı ve monoton bir iştir. Microsoft, uygulamanız için veri tabanı ile ilgili faaliyetleri otomatikleştirmek amacıyla **varlık çerçevesini** (Entity Framework-kısaca EF) geliştirmiştir.

Örnek Kurumsal Uygulama başlığında verilen örnekte ilk önce veri tabanında tablo oluşturulmuş, daha sonrasında bu tabloya veri ekleme ve veri çekme işlemi kodlanmıştı. Bu yaklaşıma **önce veri tabanı** (database first) adı verilir.

Önce veri tabanı yaklaşımında veri tabanı ile iş varlıklar (BEC) eşleşmesi için ilk önce veri tabanı tabloları tanımlanır ve sonrasında bu tablolar ile varlıklar **eşleştirilir** (mapping). Bu da yazılımın devreye alınacağı zaman veri tabanı yöneticisine bir iş çıkarır. Yazılımın devreye alınmasında ve sonraki uygulama sürümlerinde kodlamayı yapan yazılımcının veri tabanı yöneticisi ile çalışmasını gerekir.

Önce veri tabanı yaklaşımında genel olarak **Entity Data Model XML** (EDMX) dosyaları kullanılır. Bu dosyalar temel olarak mevcut bir veri tabanı üzerinden görsel olarak .NET nesneleri arasındaki ilişkiyi tanımlayan bir XML belgesidir. Visual Studio içinde **.edmx** dosyası açıldığında genellikle bir grafik **tasarım** (designer) ekranı ile gösterilir. Burada tablolar, ilişkiler ve **varlıklar** (entity) sürükle-bırak ile düzenlenebilir. Yeni projelerde genellikle Entity Framework Core tercih edilir ve EF Core, **.edmx** dosyasını desteklemez.

Projemizde Entity Framework'ü projemize dahil etmek için *Tools → NuGet Package Manager → Package Manager Console* üzerinden komut vererek bileşenleri projemize dahil ederiz

SQLite veritabanı için;

```
Install-Package Microsoft.EntityFrameworkCore
Install-Package Microsoft.EntityFrameworkCore.Sqlite
```

DbContext, kullanmayı öğrenmemiz gereken ilk ve en önemli EF veri tipidir. Veri tabanı sorguları için bir ara yüz görevi görür ve nesnelerde yaptığınız değişiklikleri takip eder, böylece veriler veri tabanında kalıcı hale getirilebilir.

Nesnelerini kullanarak verileri sorgulamak, eklemek, güncellemek ve silmek için EF kullanmak amacıyla öncelikle modelinizde tanımlanan varlıkları (BEC) ve ilişkileri bir veri tabanındaki tablolara eşleyen bir model (PDC ve BEC) oluşturmanız gerekir. Aşağıdaki **Öğrenci** varlığını ele alalım;

```
public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public int Yaş { get; set; }
}
```

Bir modeliniz olduğunda, iş varlığınızın uygulamanızın etkileşime girdiği ilk sınıf **System.Data.Entity.DbContext** sınıfıdır. Bu sınıf genellikle bağlam sınıfı olarak adlandırılır. Bağlam sınıfını şu amaçlarla kullanabilirsiniz:

- Sorguları yazıp çalıştırabilirsiniz.
- Sorgu sonuçlarını varlık nesneleri olarak somutlaştırabilirsiniz
- Varlık nesnelerinde yapılan değişiklikleri izleyebilirsiniz.
- Nesne değişikliklerini veri tabanında kalıcı hale getirebilirsiniz.
- Nesneleri bellekte UI denetimlerine bağlayabilirsiniz.

Bağlamla çalışmanın önerilen yolu, **DbContext** sınıfından türetilen bağlamda belirtilen varlıkların koleksiyonlarını temsil eden **DbSet** özelliklerini açığa çıkaran bir sınıf tanımlamaktır. EF Designer (EDMX) ile çalışıyorsanız, bağlam sizin için üretilecektir. Önce kod yöntemine bağlamı programcı yazar;

```
public class OkulContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }
    public DbSet<Bölüm> Bölümler { get; set; }
}
```

Bir bağlamanız olduğunda, bu özellikler (**DbSet**) aracılığıyla bağlamdaki varlıkları sorgulayabilir, ekleyebilir veya silebilirsiniz. Bu bağlam nesnesindeki **DbSet** özelliğine eriştiğinizde belirtilen tipteki tüm varlıkları döndüren bir başlangıç sorgusunu çalışır. Bu sorgu sonucu;

- Numaralandırılan nesne listesi **foreach** ile gezilebilir.
- Numaralandırılan nesne listesi **ToArray()**, **ToDictionary()** ve **ToList()** gibi yöntemlerle koleksiyonlara çevrilebilir.
- **First** veya **Any** gibi LINQ genişleme yöntemleri ile sorgulanabilir.
- Belirtilen anahtara sahip bir varlık bağlamda yüklü değilse **DbEntityEntry.Reload**, **Database.ExecuteSqlCommand** ve **DbSet<T>.Find()** ile yeniden yüklenebilir.

Bağlam nesnesinin ömrü, örneğin oluşturulmasıyla başlar ve örneğin atılması veya çöp toplanmasıyla sona erer. Bunun için **using** anahtar kelimesi kullanılır;

```
using (var context = new AppDbContext())
{
    var öğrenciler = context.Öğrenciler;
    foreach (var o in öğrenciler)
    {
        Console.WriteLine($"Ad: {o.Ad}, Yaş: {o.Yaş}");
    }
}
```

Önce Kod

Önce kod (code first) yaklaşımında ise geliştirici, modeli (PDC ve BEC) belirtmek için kod yazar. EF, geliştirici tarafından kodlanan varlık sınıflarına ve ek model yapılandırmasına bağlı olarak ver modellerini ve veri tabanı eşlemelerini **çalışma zamanında** (run time) üretir. Kısaca kodlanan varlık sınıflarından EF, veri tabanını otomatik olarak oluşturulur. Yani **eşleme** (mapping) otomatik olarak yapılır.

Bu yaklaşımda veri tabanının tasarlandığı **.edmx** dosyası kullanmadan eşleme yapmamıza olanak tanır. Ayrıca mevcut veri tabanı tablolarını da kullanabilirsiniz, buna **mevcut veri tabanı ile önce kod (code first with existing database)** denir.

Aşağıdaki gibi bir BEC bileşenimiz olsun;

```
public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int Yaş { get; set; }
}
```

Şimdi varlık sınıfımız ve veri tabanınız arasındaki köprü olan bağlamınız olan **DbContext** sınıfı tasarlayacağız. Bu sınıfa da varlık sınıfına ait bir **DbSet<>** özelliği eklenir;

```
public class AppDbContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }
}
```

Artık veri tabanına ilk bağlandığımız anda **Öğrenci** tablosu veri tabanında oluşacaktır; Bunu yapmak için aşağıdaki program yazılabilir,

```
using (var context = new AppDbContext())
{
    var yeniÖğrenciler = new[]
    {
        new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
        new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
        new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
    };
    context.Öğrenciler.Add(yeniÖğrenciler[0]);
    context.Öğrenciler.Add(yeniÖğrenciler[1]);
    context.Öğrenciler.Add(yeniÖğrenciler[2]);
    // ya da context.Öğrenciler.AddRange(yeniÖğrenciler);
    context.SaveChanges();
}
```

Bu program bu haliyle çalışmaz. Çünkü veri tabanı bağlantı kısmı eksiktir. Ama genel eşleşme iskeleti bu şekildedir. SQLite veri tabanı kullanarak buna bir örnek verelim. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

EFSQLiteUygulama	→ Çözüm (Solution)
└─ CodeFirst1	→ Proje (Console Application)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **EFSQLiteUygulama** adlı çözüm içinde **CodeFirst1** adlı ilk projemizi *Console Application* olarak oluşturulur.
2. Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır **Microsoft.EntityFrameworkCore** ve **Microsoft.EntityFrameworkCore.Sqlite** paketleri yüklenir.
3. Proje ile oluşan **Program.cs** dosyasını aşağıdaki şekilde değiştirilir;

```
// See https://aka.ms/new-console-template for more information
using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.Sqlite; // Install Nuget package

using (var context = new AppDbContext())
{
    context.Database.EnsureCreated();
    Console.WriteLine("SQLite Veri Tabanı Üzerinde Tablolar Oluşturuldu!");
    var yeniÖğrenciler = new[]
```

```

{
    new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
    new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
    new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
};
context.Öğrenciler.AddRange(yeniÖğrenciler);
context.SaveChanges();

var öğrenciler = context.Öğrenciler;
Console.WriteLine("Veritabanındaki Öğrenciler:");
foreach (var o in öğrenciler)
{
    Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
}

var öğrenci = context.Öğrenciler.FirstOrDefault(u => u.Ad == "İlhan");
if (öğrenci != null)
{
    context.Öğrenciler.Remove(öğrenci);
    context.SaveChanges();
    Console.WriteLine($"'{öğrenci.Ad}' adlı öğrenci silindi.");
}
context.Database.CloseConnection();
//context.Database.EnsureDeleted();
}

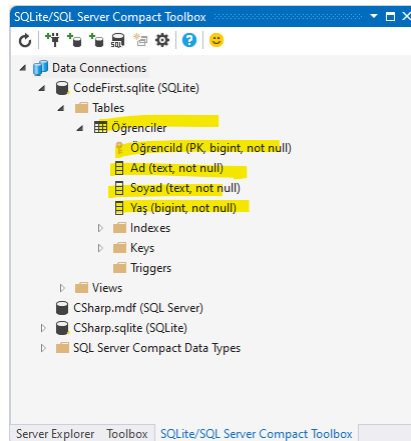
public class AppDbContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlite(@"Data Source=..\..\..\CodeFirst.sqlite");
    }
}

public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int Yaş { get; set; }
}

```

4. Program çalışıp `context.Database.EnsureCreated();` icra edildiği anda SQLite veri tabanında **Öğrenciler** tablosu oluşur;



Şekil 67. SQLite Veri Tabanı Önce Kod Tabloları

5. Aynı programa bir başka sınıf olan **User** sınıfı eklenebilir. Yeni programda **AppDbContext** nesnesi **DbContextOptionsBuilder** parametresi ile başka şekilde başlatılmıştır.

```
// See https://aka.ms/new-console-template for more information
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Sqlite;

const string databaseFile = @"..\..\..\CSharp.sqlite";
var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlite($"Data Source={databaseFile}")
    .Options;
using (var context = new AppDbContext(options))
{
    context.Database.EnsureCreated();
    Console.WriteLine("SQLite Veri Tabanı Üzerinde Tablolar Oluşturuldu!");

    var newUser = new User { Ad = "İlhan", Yaş = 30 };
    context.Users.AddRange(newUser);
    context.SaveChanges();

    var users = context.Users;
    Console.WriteLine("Veritabanındaki Kullanıcılar:");
    foreach (var u in users)
    {
        Console.WriteLine($"Id: {u.Id}, Adı: {u.Ad}, Yaşı: {u.Yaş}");
    }
    var user = context.Users.SingleOrDefault(u => u.Ad == "İlhan");
    if (user != null)
    {
        context.Users.Remove(user);
        context.SaveChanges();
        Console.WriteLine($"'{user.Ad}' adlı kullanıcı silindi.");
    }

    var yeniÖğrenciler = new[]
    {
        new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
        new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
        new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
    };
    context.Öğrenciler.AddRange(yeniÖğrenciler);
    context.SaveChanges();
    var öğrenciler = context.Öğrenciler;
    Console.WriteLine("Veritabanındaki Öğrenciler:");
    foreach (var o in öğrenciler)
    {
        Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
    }
    var öğrenci = context.Öğrenciler.FirstOrDefault(u => u.Ad == "İlhan");
    if (öğrenci != null)
    {
        context.Öğrenciler.Remove(öğrenci);
        context.SaveChanges();
        Console.WriteLine($"'{öğrenci.Ad}' adlı öğrenci silindi.");
    }
    Console.WriteLine("Sildikten Sonra Veritabanındaki Öğrenciler:");
    foreach (var o in öğrenciler)
    {
        Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
    }
    context.Database.CloseConnection();
}
```



```

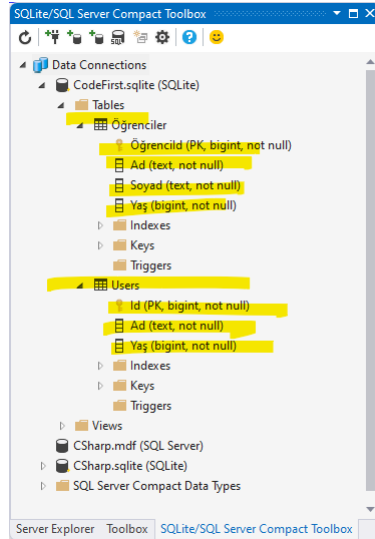
}
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<User> Users { get; set; }

    public DbSet<Öğrenci> Öğrenciler { get; set; }
}
public class User
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public int Yaş { get; set; }
}
public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int Yaş { get; set; }
}

```

6. Program çalışıp `context.Database.EnsureCreated();` icra edildiği anda SQLite veri tabanında `Öğrenciler` ve `Users` tablosu oluşur;



Şekil 68. SQLite Veri Tabanında Entity Framework Tabloları

Burada oluşan tabloların isimleri `AppDbContext` sınıfında tanımlanan `DbSet<>` özelliklerinden gelmiştir. Ayrıca veri tabanı incelendiğinde `ÖğrenciId` ve `Id` kolonlarının **birincil anahtar** (**primary key**) olduğu da görülmektedir. Bu alanlar `User` sınıfındaki `Id` ve `Öğrenci` sınıfındaki `ÖğrenciId` özelliklerinin “Id” kelimesi içeriyor olmasından kaynaklanmaktadır. Sınıfları tasarlarlarken göz önüne alınan bunun gibi birçok **kural** (**convention**) vardır.

Önce Kod Kuralları

Var Olan Bir Kuralı Geçersiz Kılma

`System.Data.Entity.ModelConfiguration.Conventions` ile tanımlanmış herhangi bir kuralı, `OnModelCreating` yöntemini geçersiz kılarak kaldırabilirsiniz.

```

public class DersContext : DbContext
{
    public DbSet<Saat> Saatler { set; get; }
    protected override void OnModelCreating(DbModelBuilder modelBuilder)
    {

```

```

        modelBuilder.Conventions.Remove<PluralizingTableNameConvention>();
        // PluralizingTableNameConvention adlı kural kaldırılmıştır.
    }
}

```

Birincil Anahtar Kuralı

Yukarıdaki örnekte olduğu gibi varsayılan olarak, bir sınıftaki bir özellik "ID" (büyük/küçük harfe duyarlı değil) olarak adlandırılmışsa veya sınıf adı "ID" ile takip edilmişse, bu özellik birincil anahtardır. Birincil anahtar özelliğinin türü sayısal veya GUID olabilir;

```

public class Okul
{
    public int okulId { get; set; } // Primary key
    //...
}

```

Veri Tipi Keşfi

Bağlam (context) sınıfında **DbSet** özelliği olarak tanımlanan tipler, varlık tiplerinin (BEC) içerdiği referans tipler ve yalnızca temel sınıf **DbSet** özelliği olarak tanımlanmış olsa bile türetilmiş sınıflar için veri tipini otomatik olarak tanır. Buna ilişkin bir örnek SQL Server üzerinden verilecektir. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

EFSQLUygulama	→ Çözüm (Solution)
└─ CodeFirst2	→ Proje (Console Application)
└─ DataBase1.mdf	→ MSSQL Veritabanı
└─ DataBase1.ldf	→ MSSQL Veritabanı (Log)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **EFSQLUygulama** adlı çözüm içinde **CodeFirst2** adlı ilk projemizi *Console Application* olarak oluşturulur.
2. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New Item...* seçilir ve **Database1.mdf** adlı *Service-based Database* adlı boş veri tabanını projemize dahil edilir.
3. Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır ve **Microsoft.EntityFrameworkCore** ile **Microsoft.EntityFrameworkCore.SqlServer** paketleri yüklenir.
4. Proje ile oluşan **Program.cs** dosyasını aşağıdaki şekilde değiştirilir;

```

// See https://aka.ms/new-console-template for more information
using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.SqlServer; // Install Nuget package

string connectionString = @"Server=(LocalDB)\MSSQLLocalDB;";
connectionString += @"AttachDbFilename=D:\CSharp\EntityFramework\CodeFirst2\Database1.mdf;";
connectionString += "Integrated Security=True;Connect Timeout=30;";

var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlServer(connectionString)
    .Options;

using (var context = new AppDbContext(options))
{
    context.Database.EnsureCreated();
    Console.WriteLine("SQL Veri Tabanı Üzerinde Tablolar Oluşturuldu!");

    var yerleşkeler = context.Yerleşkeler;
    Console.WriteLine("Yerleşkeler:");
    foreach (var y in yerleşkeler)
    {
        Console.WriteLine($"Id: {y.Id}, Yerleşke Adı: {y.YerleşkeAdı}");
        foreach (var b in y.Birimler)

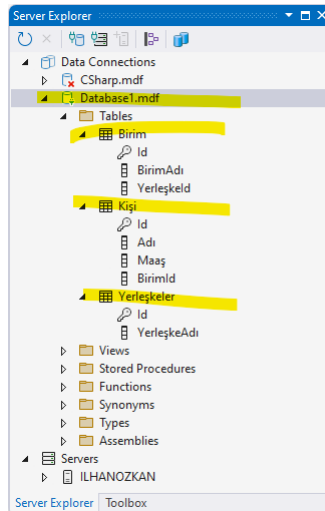
```

```

    {
        Console.WriteLine($"Id: {b.Id}, Birim Adı: {b.BirimAdı:}");
    }
}
context.Database.CloseConnection();
}
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
    public DbSet<Yerleşke> Yerleşkeler { set; get; }
}
public class Yerleşke
{
    public int Id { set; get; }
    public string YerleşkeAdı { set; get; }
    public virtual ICollection<Birim> Birimler { set; get; }
}
public class Birim
{
    public int Id { set; get; }
    public string BirimAdı { set; get; }
    public virtual ICollection<Kişi> Personel { set; get; }
}
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    public decimal Maaş { set; get; }
}
public class Müdür : Kişi
{
    public string MüdürÖzelliği { set; get; }
}
public class Şef : Kişi
{
    public string ŞefÖzelliği { set; get; }
}
public class Muhasebeci : Kişi
{
    public string MuhasebeciÖzelliği { set; get; }
}

```

5. Program çalıştığında oluşacak veri tabanı içeriği aşağıda verilmiştir;



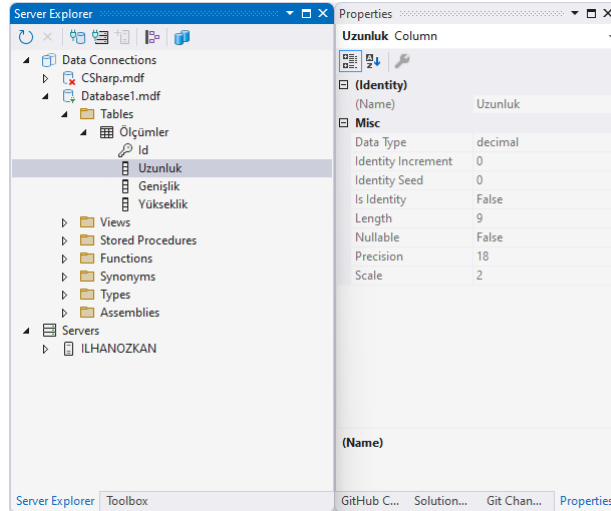
Şekil 69. SQL Veri Tabanında Entity Framework Tabloları

Ondalık Kuralı

Varsayılan olarak Entity Framework, veri tabanı tablolarındaki ondalık özellikleri tam kısmını 18 basamak, ondalık kısmı 2 basamak olacak şekilde sütunlarına eşler.

```
public class Ölçüm
{
    public int Id { set; get; }
    public decimal Uzunluk { set; get; }
    public decimal Genişlik { set; get; }
    public decimal Yükseklik { set; get; }
}
```

Yukarıdaki sınıfın veri tabanındaki karşılığı aşağıda verilmiştir;



Şekil 70. Entity Framework Ondalık Kuralı

Ondalık özelliklerin hassasiyetini aşağıdaki şekilde değiştirebiliriz:

```
protected override void OnModelCreating(DbModelBuilder modelBuilder)
{
    modelBuilder.Entity<Ölçüm>().Property(b => b.Uzunluk).HasPrecision(5, 3);
}
//Ya da
protected override void OnModelCreating(DbModelBuilder modelBuilder)
{
    modelBuilder.Conventions.Remove<DecimalPropertyConvention>();
    modelBuilder.Conventions.Add(new DecimalPropertyConvention(5, 3));
}
```

İlişki Kuralı

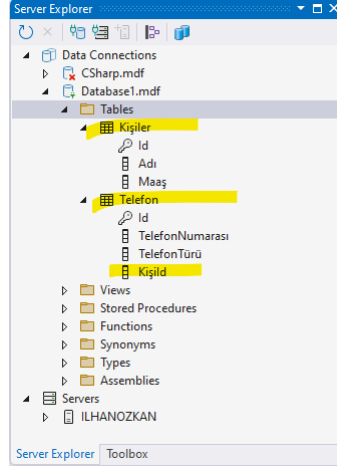
İki varlık arasındaki ilişki, **gezinme** (navigation) kuralını kullanarak otomatik olarak yapılır. Bu gezinme özelliği basit bir referans tipi veya koleksiyon tipi olabilir.

```
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    public decimal Maaş { set; get; }

    //Telefon için gezinme(navigability) özelliği
    public ICollection<Telefon> Telefonlar { set; get; }
}
public class Telefon
{
    public int Id { set; get; }
    public string TelefonNumarası { set; get; }
```

```
public string TelefonTürü { set; get; }
}
```

Yukarıdaki sınıfların veri tabanındaki karşılığı aşağıda verilmiştir;



Şekil 71. Entity Framework Gezinme Kuralı

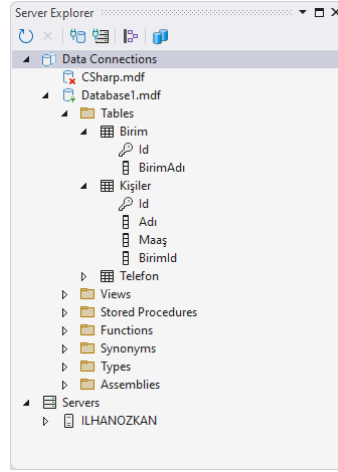
Yabancı Anahtar Kuralı

Eğer A sınıfı B sınıfıyla ilişkiliyse ve B sınıfı, A'nın birincil anahtarıyla aynı ada ve tipe sahip bir özelliğe sahipse, Entity Framework otomatik olarak bu özelliğin **yabancı anahtar** (foreign key) olduğunu varsayar.

```
public class Birim
{
    public int Id { set; get; }
    public string BirimAdı { set; get; }
    public virtual ICollection<Kişi> Personel { set; get; }
}
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    public decimal Maaş { set; get; }
    public ICollection<Telefon> Telefonlar { set; get; }
    public int BirimId { set; get; }
    public virtual Birim Birim { set; get; }
}
public class Telefon
{
    public int Id { set; get; }
    public string TelefonNumarası { set; get; }
    public string TelefonTürü { set; get; }
}
```

Yukarıdaki sınıfların veri tabanındaki karşılığı aşağıdaki şekilde verilmiştir. Server Explorer üzerinde Kişiler tablosu sağ tıklanıp *Open Table Definition* seçildiğinde tablonun SQL kodu aşağıdaki gibi olduğu görülecektir;

```
CREATE TABLE [dbo].[Kişiler] (
    [Id] INT IDENTITY (1, 1) NOT NULL,
    [Adı] NVARCHAR (MAX) NOT NULL,
    [Maaş] DECIMAL (18, 2) NOT NULL,
    [BirimId] INT NOT NULL,
    CONSTRAINT [PK_Kişiler] PRIMARY KEY CLUSTERED ([Id] ASC),
    CONSTRAINT [FK_Kişiler_Birim_BirimId] FOREIGN KEY ([BirimId]) REFERENCES [dbo].[Birim]
([Id]) ON DELETE CASCADE
);
```



Şekil 72. Entity Framework Yabancı Anahtar Kuralı

Önce Kod Veri Açıklamaları

Veri açıklamaları (**data annotations**), Entity Framework'ü veri tabanında tabloları ve kolonları oluştururken ve işlerken yönlendirir.

[Column] Niteliği

Entity Framework'e, özelliğin adını kullanmak yerine belirli bir sütun adını kullanmasını söyler.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
    public int Id { set; get; }
    [Column("Adı ve Soyadı")] //Kolon adı Adı veyine "Adı ve Soyadı" olsun
    public string Adı { set; get; }
}
```

[Required] Niteliği

Bir etki alanı sınıfının (PDC) bir özelliğine uygulandığında, veri tabanı sütunu **NOT NULL** olarak oluşturacaktır.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
    public int Id { set; get; }
    [Required] //Adı Girilmeden Kişiye kayıt atılmaz!.
    public string Adı { set; get; }
}
```

[MaxLength] ve [MinLength] Nitelikleri

[MaxLength(int)] veya [MinLength(int)] niteliği bir etki alanı sınıfının (PDC) **dizgi (string)** veya dizi türü özelliğine uygulanabilir. Entity Framework bir sütunun boyutunu belirtilen değere ayarlayacaktır.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
    public int Id { set; get; }
    [MinLength(3), MaxLength(100)] //Adı enz 3 krakter en fazla 100 karakter olabilir!.
    public string Adı { set; get; }
}
```

[ForeignKey(string)] Niteliği

Aşağıdaki Örneği ele alalım;

```
using System.ComponentModel.DataAnnotations.Schema;
```

```

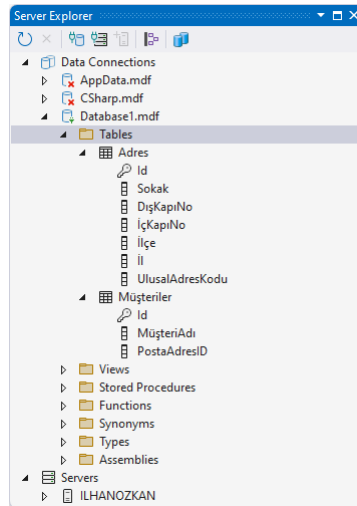
public class Müşteri
{
    public int Id { get; set; }
    [Required, MaxLength(100)]
    public string MüşteriAdı { get; set; }
    public int PostaAdresID { get; set; }
    [ForeignKey("PostaAdresID")]
    public virtual Adres PostaAdresi { get; set; }
}

public class Adres
{
    public int Id { get; set; }
    public string Sokak { get; set; }
    public string DışKapıNo { get; set; }
    public string İçKapıNo { get; set; }
    public string İlçe { get; set; }
    public string İl { get; set; }
    public long UlusalAdresKodu { get; set; }
}

```

ForeignKey nitelikleri olmadan, EF bunları karıştırabilir ve **PostaAdresi** alırken başka ID içeren değerini kullanabilir. Bunu önlemek için bu nitelik kullanılır.

Yukarıdaki örneğe ilişkin veri tabanı aşağıda verilmiştir;



Şekil 73. Entity Framework ForeignKey Niteliği

[InverseProperty(string)] Niteliği

InverseProperty, iki varlık arasında birden fazla iki yönlü ilişki olduğunda iki yönlü ilişkileri tanımlamak için kullanılabilir. Entity Framework'e diğer taraftaki özelliklerle hangi **gezinme** (**navigation**) özelliklerini eşleştirmesi gerektiğini söyler. EF, iki varlık arasında birden fazla çift yönlü ilişki olduğunda diğer taraftaki hangi gezinme özelliğinin hangi özelliklerle eşleştiğini bilmez. İlgili sınıftaki karşılık gelen gezinme özelliğinin adını parametresi olarak alması gerekir.

```

using System.ComponentModel.DataAnnotations; //gerekli
public class Birim
{
    public int Id { set; get; }
    public string BirimAdı { set; get; }
    public virtual ICollection<Kişi> AsılPersonel { set; get; }
    public virtual ICollection<Kişi> GörevliPersonel { set; get; }
}

public class Kişi
{

```

```

public int Id { set; get; }
public string Adı { set; get; }
[InverseProperty("AsılPersonel")]
public virtual Birim AnaBirim { get; set; }
[InverseProperty("GörevliPersonel")]
public virtual Birim GörevliOlduğuBirim { get; set; }
}

```

Ağaç yapısında nesneler için bu nitelik kullanılabilir;

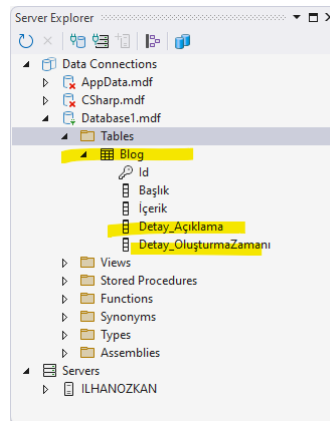
```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
public class AğaçDüğümü
{
    public int ID { get; set; }
    public int ParentID { get; set; }
    [ForeignKey("ParentID")]
    public AğaçDüğümü EbeveynDüğüm { get; set; }
    [InverseProperty("ParentNode")]
    public virtual ICollection<AğaçDüğümü> ÇocukDüğümler { get; set; }
}

```

[ComplexType] Niteliği

Alan odaklı tasarımda (Domain Driven Design-DDD) değer nesneleri (value object) gibi karmaşık tipler kendi başlarına izlenemezler ancak bir varlığın parçası olarak izlenirler. Böyle durumda bu nitelik kullanılır.



Şekil 74. Entity Framework ComplexType Niteliği

Veri tabanını oluşturan kod içeriği aşağıda verilmiştir;

```

[ComplexType]
public class BlogDetay
{
    public DateTime? OluşturmaZamanı { get; set; }
    [MaxLength(250)]
    public string Açıklama { get; set; }
}
public class Blog
{
    public int Id { get; set; }
    public string Başlık { get; set; }
    public string İçerik { get; set; }
    public BlogDetay Detay { get; set; }
}

```

[Range(min,max)] Niteliği

İş varlığına ilişkin sayısal alanlarda sınır belirlemek için kullanılır;


```

public class MezuniyetDerecesi
{
    public int DereceID { get; set; }
    [Range(0, 4)]
    public Nullable<decimal> Derece { get; set; }
}

public class Öğrenci
{
    public int Id { get; set; }
    [Required, MaxLength(50)]
    public string Adı { get; set; }
    [Required, MaxLength(50)]
    public string Soyadı { get; set; }
    [EmailAddress, MaxLength(150)]
    public string EPosta { get; set; }
    public MezuniyetDerecesi MezuniyetDerecesi { get; set; }
}

```

[NotMapped] Niteliği

Önce kod yaklaşımında, Entity Framework, desteklenen bir veri türünde olan her genel özellik için bir sütun oluşturur. [NotMapped] niteliği, bir veri tabanı tablosunda sütun istemediğimiz tüm özelliklere uygulanmalıdır.

```

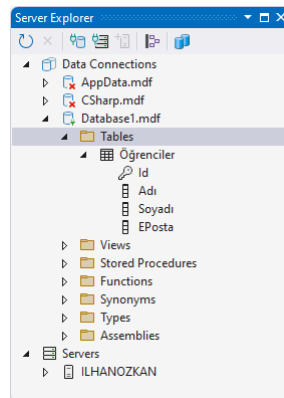
public class Öğrenci
{
    public int Id { get; set; }
    [Required, MaxLength(50)]
    public string Adı { get; set; }
    [Required, MaxLength(50)]
    public string Soyadı { get; set; }

    public string TamAdı => string.Format("{0} {1}", Adı, Soyadı);

    [Required, MaxLength(150)]
    public string EPosta { get; set; }
    [NotMapped]
    public float Ortalama { set; get; }
}

```

Oluşan veri tabanı aşağıda verilmiştir;



Şekil 75. Entity Framework NotMapped Niteliği

[Table] Niteliği

İş varlığına ilişkin veri tabanında tablo adı belirlemek için kullanılır.

```

[Table("Personel")]
public class Kişi

```

```
{
    public int Id { set; get; }
    public string Adı { set; get; }
}
```

[Index] Niteliği

Veri tabanında oluşan kolonlar için **dizin** (index) oluşturmak için kullanılır;

```
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }

    [Index]
    public int Yaş { set; get; }
}
```

İstenirse dizine ad da verilebilir;

```
[Index("IX_Kişi_Yaş1")]
public int Yaş { set; get; }
```

İstenirse **tekil** (unique) dizin de oluşturulabilir;

```
[Index(IsUnique = true)]
public int Yaş { set; get; }
```

Ayrıca birden fazla kolon kullanılarak da dizin oluşturulabilir;

```
[Index("IX_Kişi_AdveYaş",1)]
public string Adı { set; get; }
[Index("IX_Kişi_AdveYaş ",2)]
public int Yaş { set; get; }
```

[Key] Niteliği

Anahtar (key), bir veri tabanı tablosundaki her satırı/kaydı benzersiz bir şekilde tanımlayan bir alandır. Varlık sınıfında "ID" içeren bir özellik yoksa bu nitelik kullanılabilir;

```
public class Personel
{
    [Key]
    public int SicilNo { set; get; }
    public string Adı { set; get; }
}
```

Aslında bu niteliği kullanmadan aşağıdaki gibi yapılabilirdi;

```
public class Personel
{
    public int PersonelId { set; get; } //Personel ID anahtar olarak görülür
    public string Adı { set; get; }
}
```

[StringLength(int)] Niteliği

Varlık nesnesinde metinlere karakter sınırlaması getirmek için kullanılır;

```
public class Personel
{
    public int PersonelId { set; get; }

    [StringLength(100)]
    public string Adı { set; get; }
}
```

[DatabaseGenerated] Niteliği

Bazı durumlarda anahtar alanın otomatik artan bir kolon olması istenmez. Bunun gibi durumlarda bu nitelik kullanılabilir;

```
public class İl
{
    public int İlId { get; set; } // İlId veritabanında otomatik artıtılan (IDENTITY)
    public string İlAdı { get; set; }
}
```

Eğer il trafik kodlarını **anahtar** (key) olarak otomatik artmayan bir sütun istiyorsak varlığı aşağıdaki şekilde tanımlamalıyız;

```
public class İl
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.None)]
    public int TrafikKodu { get; set; }
    public string İlAdı { get; set; }
}
```

Veri tabanında hesaplanan bir kolon varsa da bu nitelik kullanılabilir. Aşağıdaki gibi bir tablo oluşturmak istiyorsak;

```
CREATE TABLE [Personel] (
    [Adı] varchar(100) PRIMARY KEY,
    [DoğumTarihi] Date NOT NULL,
    [Yaş] AS DATEDIFF(year, [DoğumTarihi], GETDATE())
)
```

Bu varlığı aşağıdaki gibi kodlamalıyız;

```
[Table("Personel")]
public class Kişi
{
    [Key, StringLength(100)]
    public string Adı { get; set; }
    public DateTime DoğumTarihi { get; set; }

    [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
    public int Yaş { get; set; }
}
```

Burada çok kullanılan niteliklere yer verilmiştir³⁹.

Mevcut Veri Tabanı Tablolarıyla Çalışma

Önce kod (code first) yaklaşımıyla hali hazırda var olan tablolar içeren veri tabanı ile çalışılabilir. Bunun için içerisinde il listesi olan bir veri tabanını (Database2.mdf) örnek alalım. Veri tabanında aşağıdaki tablo var olsun;

```
CREATE TABLE [dbo].[İller] (
    [İlTrafikKodu] INT NOT NULL,
    [İlAdı] NVARCHAR (MAX) NOT NULL,
    CONSTRAINT [PK_İller] PRIMARY KEY CLUSTERED ([İlTrafikKodu] ASC)
);
```

Bu tablonun içinde de il listemiz işlemiş olsun. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```
EFSQLUygulama          → Çözüm (Solution)
├── CodeFirst3          → Proje (Console Application)
│   └── DataBase2.mdf   → MSSQL Veritabanı
```

³⁹ <https://learn.microsoft.com/en-us/ef/ef6/modeling/code-first/data-annotations>

└─ DataBase2.ldf	→ MSSQL Veritabanı (Log)
└─ Program.cs	→ Ana (Main) Program

Bu durumda aşağıdaki kod ile veri tabanını sorgulayabiliriz;

```
// See https://aka.ms/new-console-template for more information using
Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

string connectionString = @"Server=(LocalDB)\MSSQLLocalDB;";
connectionString += @"AttachDbFilename=D:\CSharp\EntityFramework\CodeFirst3\Database2.mdf;";
connectionString += "Integrated Security=True;Connect Timeout=30;";

var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlServer(connectionString)
    .Options;

using (var context = new AppDbContext(options))
{
    var il=context.İller.FirstOrDefault();
    if (il != null)
    {
        Console.WriteLine($"İl: {il.İlAdı}, Trafik Kodu: {il.İlTrafikKodu}");
        //İl: Adana, Trafik Kodu: 1
    }
    else
    {
        Console.WriteLine("İl bulunamadı.");
    }
    il.İlAdı="Adana 1 Numara";
    context.SaveChanges();
    context.Database.CloseConnection();
}

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
    public DbSet<İl> İller { set; get; }
}

public class İl
{
    [Key]
    public int İlTrafikKodu { get; set; }
    public string İlAdı { get; set; }
}
}
```

Veri Göçü

Gerçek olarak uygulanan projelerde, özellikler uygulandıkça veri modelleri değişir: yeni varlıklar veya özellikler eklenir ve kaldırılır ve veri tabanı şemalarının uygulama ile senkronize tutulabilmesi için buna göre değiştirilmesi gerekir.

EF Core'daki göç (migration) özelliği, veri tabanındaki verileri korurken veri tabanı şemasını uygulamanın veri modeliyle senkronize tutmak için artımlı olarak güncellemenin bir yolunu sağlar.

EF Core'daki göç özelliğini kullanabilmek için; EF tarafından yönetilmiş veri tabanı nesnelerinizi içeren geçerli bir **DbContext** içeren bir uygulamanız olması gerekir. Bunun için bir proje oluşturalım. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

EFSQLUygulama	→ Çözüm (Solution)
---------------	--------------------

└─ CodeFirst4	→ Proje (Console Application)
└─ DataBaseEF.mdf	→ MSSQL Veritabanı
└─ DataBaseEF.ldf	→ MSSQL Veritabanı (Log)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **EFSQLUygulama** adlı çözüm içinde **CodeFirst4** adlı ilk projemizi *Console Application* olarak oluşturulur.
2. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New Item...* seçilir ve **DatabaseEF.mdf** adlı *Service-based Database* adlı boş veri tabanını projemize dahil edilir.
3. Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır **Microsoft.EntityFrameworkCore** ve **Microsoft.EntityFrameworkCore.SqlServer** paketleri yüklenir.
4. Oluşan projede **Program.cs** aşağıdaki gibi değiştirilir ve bir kez çalıştırılarak veri tabanında ilgili tablolar ve verilerin oluşması sağlanır.
5. Burada dikkat edilmesi gereken **AppDbContext** sınıfının yapıcısının parametre almamasını sağlamaktır. Bunu yapmak için **onConfiguring()** yöntemi bu sınıfa eklenir. Ayrıca il listesini de göç sırasında oluşturmak istiyorsak **OnModelCreating()** yöntemini de ekleyip yeni verileri göç edilen veri tabanına kaydedebiliriz.

```
// See https://aka.ms/new-console-template for more information
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

using (var context = new AppDbContext())
{
    context.Database.EnsureCreated();
    var iller = context.İller.ToList();
    Console.WriteLine("İller:");
    foreach (var il in iller)
    {
        Console.WriteLine($"Trafik Kodu: {il.İlTrafikKodu}, İl Adı: {il.İlAdı}");
    }
    context.Database.CloseConnection();
}

public class AppDbContext : DbContext
{
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        string connectionString = @"Server=(LocalDB)\MSSQLLocalDB;";
        connectionString += @"AttachDbFilename=D:\CSharp\EntityFramework\CodeFirst4\";
        connectionString += "DatabaseModel.mdf;";
        connectionString += "Integrated Security=True;";
        connectionString += "Connect Timeout=30;";
        optionsBuilder.UseSqlServer(connectionString);
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Personel>()
            .HasOne(p => p.Adres)
            .WithMany()
            .HasForeignKey(p => p.PostaAdresID);
        modelBuilder.Entity<Adres>()
            .HasOne(a => a.İl)
            .WithMany()
            .HasForeignKey(a => a.İlTrafikKodu);
        modelBuilder.Entity<İl>().HasData(
            new İl { İlTrafikKodu = 1, İlAdı = "Adana" },
            new İl { İlTrafikKodu = 2, İlAdı = "Adıyaman" },
            new İl { İlTrafikKodu = 3, İlAdı = "Afonkarahisar" },

```

```
new İl { İlTrafikKodu = 4, İlAdı = "Ağrı" },
new İl { İlTrafikKodu = 5, İlAdı = "Amasya" },
new İl { İlTrafikKodu = 6, İlAdı = "Ankara" },
new İl { İlTrafikKodu = 7, İlAdı = "Antalya" },
new İl { İlTrafikKodu = 8, İlAdı = "Artvin" },
new İl { İlTrafikKodu = 9, İlAdı = "Aydın" },
new İl { İlTrafikKodu = 10, İlAdı = "Balıkesir" },
new İl { İlTrafikKodu = 11, İlAdı = "Bilecik" },
new İl { İlTrafikKodu = 12, İlAdı = "Bingöl" },
new İl { İlTrafikKodu = 13, İlAdı = "Bitlis" },
new İl { İlTrafikKodu = 14, İlAdı = "Bolu" },
new İl { İlTrafikKodu = 15, İlAdı = "Burdur" },
new İl { İlTrafikKodu = 16, İlAdı = "Bursa" },
new İl { İlTrafikKodu = 17, İlAdı = "Çanakkale" },
new İl { İlTrafikKodu = 18, İlAdı = "Çankırı" },
new İl { İlTrafikKodu = 19, İlAdı = "Çorum" },
new İl { İlTrafikKodu = 20, İlAdı = "Denizli" },
new İl { İlTrafikKodu = 21, İlAdı = "Diyarbakır" },
new İl { İlTrafikKodu = 22, İlAdı = "Edirne" },
new İl { İlTrafikKodu = 23, İlAdı = "Elazığ" },
new İl { İlTrafikKodu = 24, İlAdı = "Erzincan" },
new İl { İlTrafikKodu = 25, İlAdı = "Erzurum" },
new İl { İlTrafikKodu = 26, İlAdı = "Eskişehir" },
new İl { İlTrafikKodu = 27, İlAdı = "Gaziantep" },
new İl { İlTrafikKodu = 28, İlAdı = "Giresun" },
new İl { İlTrafikKodu = 29, İlAdı = "Gümüşhane" },
new İl { İlTrafikKodu = 30, İlAdı = "Hakkari" },
new İl { İlTrafikKodu = 31, İlAdı = "Hatay" },
new İl { İlTrafikKodu = 32, İlAdı = "Isparta" },
new İl { İlTrafikKodu = 33, İlAdı = "Mersin" },
new İl { İlTrafikKodu = 34, İlAdı = "İstanbul" },
new İl { İlTrafikKodu = 35, İlAdı = "İzmir" },
new İl { İlTrafikKodu = 36, İlAdı = "Kars" },
new İl { İlTrafikKodu = 37, İlAdı = "Kastamonu" },
new İl { İlTrafikKodu = 38, İlAdı = "Kayseri" },
new İl { İlTrafikKodu = 39, İlAdı = "Kırklareli" },
new İl { İlTrafikKodu = 40, İlAdı = "Kırşehir" },
new İl { İlTrafikKodu = 41, İlAdı = "Kocaeli" },
new İl { İlTrafikKodu = 42, İlAdı = "Konya" },
new İl { İlTrafikKodu = 43, İlAdı = "Kütahya" },
new İl { İlTrafikKodu = 44, İlAdı = "Malatya" },
new İl { İlTrafikKodu = 45, İlAdı = "Manisa" },
new İl { İlTrafikKodu = 46, İlAdı = "Kahramanmaraş" },
new İl { İlTrafikKodu = 47, İlAdı = "Mardin" },
new İl { İlTrafikKodu = 48, İlAdı = "Muğla" },
new İl { İlTrafikKodu = 49, İlAdı = "Muş" },
new İl { İlTrafikKodu = 50, İlAdı = "Nevşehir" },
new İl { İlTrafikKodu = 51, İlAdı = "Niğde" },
new İl { İlTrafikKodu = 52, İlAdı = "Ordu" },
new İl { İlTrafikKodu = 53, İlAdı = "Rize" },
new İl { İlTrafikKodu = 54, İlAdı = "Sakarya" },
new İl { İlTrafikKodu = 55, İlAdı = "Samsun" },
new İl { İlTrafikKodu = 56, İlAdı = "Siirt" },
new İl { İlTrafikKodu = 57, İlAdı = "Sinop" },
new İl { İlTrafikKodu = 58, İlAdı = "Sivas" },
new İl { İlTrafikKodu = 59, İlAdı = "Tekirdağ" },
new İl { İlTrafikKodu = 60, İlAdı = "Tokat" },
new İl { İlTrafikKodu = 61, İlAdı = "Trabzon" },
new İl { İlTrafikKodu = 62, İlAdı = "Tunceli" },
new İl { İlTrafikKodu = 63, İlAdı = "Şanlıurfa" },
new İl { İlTrafikKodu = 64, İlAdı = "Uşak" },
new İl { İlTrafikKodu = 65, İlAdı = "Van" },
```

```

        new İl { İlTrafikKodu = 66, İlAdı = "Yozgat" },
        new İl { İlTrafikKodu = 67, İlAdı = "Zonguldak" },
        new İl { İlTrafikKodu = 68, İlAdı = "Aksaray" },
        new İl { İlTrafikKodu = 69, İlAdı = "Bayburt" },
        new İl { İlTrafikKodu = 70, İlAdı = "Karaman" },
        new İl { İlTrafikKodu = 71, İlAdı = "Kırıkkale" },
        new İl { İlTrafikKodu = 72, İlAdı = "Batman" },
        new İl { İlTrafikKodu = 73, İlAdı = "Şırnak" },
        new İl { İlTrafikKodu = 74, İlAdı = "Bartın" },
        new İl { İlTrafikKodu = 75, İlAdı = "Ardahan" },
        new İl { İlTrafikKodu = 76, İlAdı = "Iğdır" },
        new İl { İlTrafikKodu = 77, İlAdı = "Yalova" },
        new İl { İlTrafikKodu = 78, İlAdı = "Karabük" },
        new İl { İlTrafikKodu = 79, İlAdı = "Kilis" },
        new İl { İlTrafikKodu = 80, İlAdı = "Osmaniye" },
        new İl { İlTrafikKodu = 81, İlAdı = "Düzce" }
    );
}

public DbSet<Personel> Personel { set; get; }
public DbSet<İl> İller { set; get; }
}

public class Personel
{
    public int PersonelId { get; set; }
    [StringLength(100)]
    public string Adı { get; set; }
    public string Soyadı { get; set; }
    public int PostaAdresID { get; set; }
    [ForeignKey("PostaAdresID")]
    public virtual Adres Adres { get; set; }
}

public class Adres {
    public int AdresId { get; set; }
    [StringLength(200)] //Cadde-Sokak-Bulvar-Meydan
    public string CSBM { get; set; }
    [StringLength(10)]
    public string DışKapıNo { get; set; }
    [StringLength(10)]
    public string İçKapıNo { get; set; }
    [StringLength(100)]
    public string İlçe { get; set; }
    [StringLength(50)]
    public int İlTrafikKodu { get; set; }
    [ForeignKey("İlTrafikKodu")]
    public virtual İl İl { get; set; }
    public long PostaKodu { get; set; }
}

public class İl
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.None)]
    public int İlTrafikKodu { get; set; }
    public string İlAdı { get; set; }
}

```

Bir EF Core projesinde göç (migration) işlemleri şu şekilde kullanılır;

- Bir veri modeli değişikliği yapıldığında, geliştirici veri tabanı şemasını senkronize tutmak için gerekli güncellemeleri açıklayan karşılık gelen bir göç eklemek için **EF Core Tools** kullanır. EF Core, farklılıkları belirlemek için geçerli modeli eski modelin bir anlık görüntüsüyle karşılaştırır ve göç

kaynak dosyaları oluşturur; dosyalar projenizin kaynak denetiminde diğer kaynak dosyaları gibi izlenebilir.

- Yeni bir göç oluşturulduktan sonra, çeşitli şekillerde bir veri tabanına uygulanabilir. EF Core uygulanan tüm göçleri özel bir geçmiş tablosunda kaydeder ve hangi göçlerin uygulandığını ve hangilerinin uygulanmadığını bilmesini sağlar.

Projemizde göç yapabilmek için Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır **Microsoft.EntityFrameworkCore.Tools** paketi yüklenir. Ardından rojemizde veri tabanının ve modelin ilk tapısını saklamak için; Visual Studio üzerinde *Tools→NuGet Package Manager→Package Manager Console* tıklanır ve açılan pencerede **Add-Migration Başlangıç** komutu yazılır ve çalıştırılır. Burada dikkat edilmesi gereken “Default Project” ayarının mevcut projemizi göstermesidir.

```
PM> Add-Migration Başlangıç
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

Komut sonrasında projemizde aşağıdaki klasör ve dosyalar oluşur;

EFSQLUygulama	→ Çözüm (Solution)
└─ CodeFirst4	→ Proje (Console Application)
└─ Migrations	→ Klasör
└─ 20250530122325_Başlangıç.cs	→ Klasör
└─ AppDbContextModelSnapshot.cs	→ Klasör
─ DataBaseEF.mdf	→ MSSQL Veritabanı
─ DataBaseEF.ldf	→ MSSQL Veritabanı (Log)
─ Program.cs	→ Ana (Main) Program

Oluşan dosyanın adı (**20250530083726_Başlangıç.cs**), **zaman damgası** (time stamp) ve göç adını içerir. Diğerisi ise varlıklarımızın o anki bir fotoğrafı olan **AppDbContextModelSnapshot.cs** dosyasıdır. Şimdi göç dosyasının içeriğini inceleyelim;

```
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable
#pragma warning disable CA1814 // Prefer jagged arrays over multidimensional

namespace CodeFirst4.Migrations
{
    /// <inheritdoc />
    public partial class Başlangıç : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "İller",
                columns: table => new
                {
                    İlTrafikKodu = table.Column<int>(type: "int", nullable: false),
                    İlAdı = table.Column<string>(type: "nvarchar(max)", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_İller", x => x.İlTrafikKodu);
                });
            migrationBuilder.CreateTable(
                name: "Adres",
                columns: table => new
                {
                    AdresId = table.Column<int>(type: "int", nullable: false)
```



```

        .Annotation("SqlServer:Identity", "1, 1"),
        CSBM = table.Column<string>(type: "nvarchar(200)",
            maxLength: 200, nullable: false),
        DışKapıNo = table.Column<string>(type: "nvarchar(10)",
            maxLength: 10, nullable: false),
        İçKapıNo = table.Column<string>(type: "nvarchar(10)",
            maxLength: 10, nullable: false),
        İlçe = table.Column<string>(type: "nvarchar(100)",
            maxLength: 100, nullable: false),
        İlTrafikKodu = table.Column<int>(type: "int",
            maxLength: 50, nullable: false),
        PostaKodu = table.Column<long>(type: "bigint", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Adres", x => x.AdresId);
        table.ForeignKey(
            name: "FK_Adres_İller_İlTrafikKodu",
            column: x => x.İlTrafikKodu,
            principalTable: "İller",
            principalColumn: "İlTrafikKodu",
            onDelete: ReferentialAction.Cascade);
    });
migrationBuilder.CreateTable(
    name: "Personel",
    columns: table => new
    {
        PersonelId = table.Column<int>(type: "int", nullable: false)
            .Annotation("SqlServer:Identity", "1, 1"),
        Adı = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
            nullable: false),
        Soyadı = table.Column<string>(type: "nvarchar(max)", nullable: false),
        PostaAdresID = table.Column<int>(type: "int", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Personel", x => x.PersonelId);
        table.ForeignKey(
            name: "FK_Personel_Adres_PostaAdresID",
            column: x => x.PostaAdresID,
            principalTable: "Adres",
            principalColumn: "AdresId",
            onDelete: ReferentialAction.Cascade);
    });

migrationBuilder.InsertData(
    table: "İller",
    columns: new[] { "İlTrafikKodu", "İlAdı" },
    values: new object[,]
    {
        { 1, "Adana" },
        { 2, "Adıyaman" },
        { 3, "Afyonkarahisar" },
        { 4, "Ağrı" },
        { 5, "Amasya" },
        { 6, "Ankara" },
        { 7, "Antalya" },
        { 8, "Artvin" },
        { 9, "Aydın" },
        { 10, "Balıkesir" },
        { 11, "Bilecik" },
        { 12, "Bingöl" },
    }
);

```

```
{ 13, "Bitlis" },
{ 14, "Bolu" },
{ 15, "Burdur" },
{ 16, "Bursa" },
{ 17, "Çanakkale" },
{ 18, "Çankırı" },
{ 19, "Çorum" },
{ 20, "Denizli" },
{ 21, "Diyarbakır" },
{ 22, "Edirne" },
{ 23, "Elazığ" },
{ 24, "Erzincan" },
{ 25, "Erzurum" },
{ 26, "Eskişehir" },
{ 27, "Gaziantep" },
{ 28, "Giresun" },
{ 29, "Gümüşhane" },
{ 30, "Hakkari" },
{ 31, "Hatay" },
{ 32, "Isparta" },
{ 33, "Mersin" },
{ 34, "İstanbul" },
{ 35, "İzmir" },
{ 36, "Kars" },
{ 37, "Kastamonu" },
{ 38, "Kayseri" },
{ 39, "Kırklareli" },
{ 40, "Kırşehir" },
{ 41, "Kocaeli" },
{ 42, "Konya" },
{ 43, "Kütahya" },
{ 44, "Malatya" },
{ 45, "Manisa" },
{ 46, "Kahramanmaraş" },
{ 47, "Mardin" },
{ 48, "Muğla" },
{ 49, "Muş" },
{ 50, "Nevşehir" },
{ 51, "Niğde" },
{ 52, "Ordu" },
{ 53, "Rize" },
{ 54, "Sakarya" },
{ 55, "Samsun" },
{ 56, "Siirt" },
{ 57, "Sinop" },
{ 58, "Sivas" },
{ 59, "Tekirdağ" },
{ 60, "Tokat" },
{ 61, "Trabzon" },
{ 62, "Tunceli" },
{ 63, "Şanlıurfa" },
{ 64, "Uşak" },
{ 65, "Van" },
{ 66, "Yozgat" },
{ 67, "Zonguldak" },
{ 68, "Aksaray" },
{ 69, "Bayburt" },
{ 70, "Karaman" },
{ 71, "Kırıkkale" },
{ 72, "Batman" },
{ 73, "Şırnak" },
{ 74, "Bartın" },
```

```

        { 75, "Ardahan" },
        { 76, "Iğdır" },
        { 77, "Yalova" },
        { 78, "Karabük" },
        { 79, "Kilis" },
        { 80, "Osmaniye" },
        { 81, "Düzce" }
    });
    migrationBuilder.CreateIndex(
        name: "IX_Adres_İlTrafikKodu",
        table: "Adres",
        column: "İlTrafikKodu");
    migrationBuilder.CreateIndex(
        name: "IX_Personel_PostaAdresID",
        table: "Personel",
        column: "PostaAdresID");
}
/// <inheritdoc />
protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropTable(
        name: "Personel");

    migrationBuilder.DropTable(
        name: "Adres");

    migrationBuilder.DropTable(
        name: "İller");
}
}
}

```

Gördüğünüz gibi, **Up()** yöntemi **İller**, **Adres** ve **Personel** adlı üç tablo ve alanları oluşturulur. Ek olarak, tablolar arasındaki ilişkiler ve **dizinler** (index) de oluşturulur. Diğer tarafta **Down()** yöntemi göç aktivitelerini (**Up** yöntemi ile yapılanları) tersine çevirmeye çalışır. Göçünüz konusunda kendinize güveniyorsanız, göçü istediğiniz veri tabanına uygulayabilirsiniz. Bunun için mevcut projemizde kullandığımız veri tabanını değiştireceğiz.

1. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New Item...* seçilir ve **DatabaseModel.mdf** adlı *Service-based Database* adlı boş veri tabanını projemize dahil edilir.
2. **Program.cs** içinde yeni veri tabanı bağlantımızı sağlayacak değişikliği **AppDbContext** sınıfının **onConfiguring()** yönteminde yapıyoruz.

```

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    string connectionString = @"Server=(LocalDB)\MSSQLLocalDB;";
    connectionString += @"AttachDbFilename=D:\CSharp\EntityFramework\CodeFirst4\";
    connectionString += "DatabaseModel.mdf;";
    connectionString += "Integrated Security=True;";
    connectionString += "Connect Timeout=30;";
    optionsBuilder.UseSqlServer(connectionString);
}

```

3. Visual Studio üzerinde *Tools→NuGet Package Manager→Package Manager Console* tıklanır ve açılan pencerede **Add-Migration Başlangıç** komutu yazılır ve çalıştırılır;

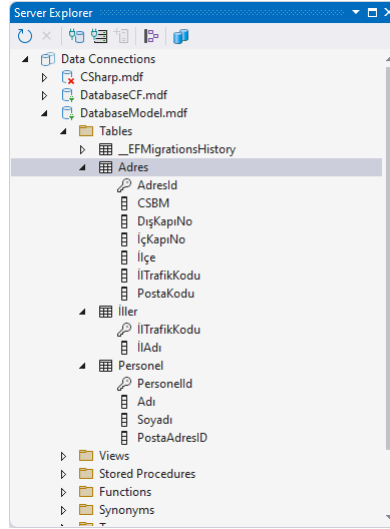
```

PM> Update-Database
Build started...
Build succeeded.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-
migrations-lock for more information if this takes too long.
Applying migration '20250530122325_Başlangıç'.
Done.

```

PM>

Artık projemiz yeni veri tabanında tabloları oluşturmuştur;



Şekil 76. Göç Sonrası Yeni Veri Tabanı

Add-Migration ve **Update-Database** komutları için aktiviteleri ayarlamak için kullanılabilecek çeşitli seçenekler mevcuttur. Bunları görmek ve incelemek için **get-help Add-Migration** ve **get-help Update-Database** komutları kullanılabilir.

Geliştirmenin yapıldığı ortamın dışında çalışan uygulamalar genellikle veri tabanı **güncellemeleri** (**update**) gerektirir. Veri tabanı **yamalarını** (**patch**) oluşturmak için **AddMigration** komutunu kullandıktan sonra, güncellemeleri test ve diğer ortamlarda çalıştırmanız gerekir. Bunun yanında uygulamanın müşteri ortamına bağlantı izni yok ya da bu ortamda Visual Studio kurulmamış ise bu komutu çalıştıramayız.

Bunun çözümü örneğimizde olduğu gibi **DbContext** sınıfına **OnModelCreating** yöntemini eklemektir. Göç sırasında yeni veri tabanında verileri oluşturma işlemine **veri ile besleme** (**seeding data**) adı verilir.

ŞEKİL LİSTESİ

Şekil 1.	Motorola 6802, Intel 8086 işlemciler ile 2732 EPROM Belleği	11
Şekil 2.	Frekansı 1 olan Kare Dalga İşareti.....	12
Şekil 3.	Basit bir İşlemcinin Yapısı.....	12
Şekil 4.	Ortak Tip Sistemi (CTS).....	24
Şekil 5.	Klasik Derleme Süreci.....	25
Şekil 6.	.Net Framework Derleme Süreci	25
Şekil 7.	.Net Altyapısı Derleme ve Çalıştırma Süreci	25
Şekil 8.	C# Kodu Temel Yapısı.....	31
Şekil 9.	Akış Diyagramları Genel Şekilleri	57
Şekil 10.	Örnek Akış Diyagramı	57
Şekil 11.	If Talimatı İcra Akışı.....	57
Şekil 12.	If-Else Talimatı Sözde Kodu ve İcra Akışı.....	59
Şekil 13.	Programın If Talimatlarıyla Yazılması Halinde İcra Akışı.....	60
Şekil 14.	Programın If-else Talimatlarıyla Yazılması Halinde İcra Akışı.....	61
Şekil 15.	Switch Talimatı Sözde Kodu ve İcra Akışı.....	62
Şekil 16.	Sayaç Kontrollü Döngü İcra Akışı	65
Şekil 17.	While Talimatı İcra Akışı.....	66
Şekil 18.	Do-While Talimatı ve İcra Akışı	66
Şekil 19.	For Talimatı İcra Akışı.....	67
Şekil 20.	While ve Do-While Döngü Kodunda break ve continue Talimatları İcra Akışı.....	71
Şekil 21.	For Döngü Kodunda break ve continue Talimatları İcra Akışı.....	71
Şekil 22.	Fonksiyon çağırma sürecinde yığın bellek.....	88
Şekil 23.	Öğrenci, Öğretmen ve Müdür UML Sınıf Diyagramları	99
Şekil 24.	Kişi Taban Sınıfı Eklenmiş Kalıtım UML Sınıf Diyagramı	100
Şekil 25.	Örnek Bir Windows Forms Uygulaması.....	123
Şekil 26.	Örnek Bir Sınıf Diyagramı.....	130
Şekil 27.	Sınıf Diyagramlarında Sınırlama ve Çokluk Örneği.....	130
Şekil 28.	Sınıf Diyagramlarında Ara yüz Gerçekleştirme	131
Şekil 29.	Genelleştirme İlişkisi UML Diyagramı.....	131
Şekil 30.	Bağımlılık İlişkisi UML Diyagramı.....	132
Şekil 31.	Açık ve Gizli İş Birliği UML Diyagramı.....	133
Şekil 32.	Bütünleşme UML Diyagramı.....	134
Şekil 33.	Güvenli Olmayan Kod Derlenmesi.....	138
Şekil 34.	MSSQL Veritabanı Oluşturma	185
Şekil 35.	Soyut Fabrika Deseni UML Diyagramı.....	189
Şekil 36.	Fabrika Yöntemi Deseni UML Diyagramı	191
Şekil 37.	Tekil Deseni UML Diyagramı.....	192
Şekil 38.	Kurucu Deseni UML Diyagramı	193
Şekil 39.	Prototip Deseni UML Diyagramı	195
Şekil 40.	Vitrin Deseni UML Diyagramı	197
Şekil 41.	Adaptör Deseni UML Diyagramı.....	198
Şekil 42.	Bileşik Deseni UML Diyagramı.....	200
Şekil 43.	Dekorator Deseni UML Diyagramı.....	202
Şekil 44.	Sineksiklet Deseni UML Diyagramı	203
Şekil 45.	Vekil Deseni UML Diyagramı	205
Şekil 46.	Köprü Deseni UML Diyagramı.....	206
Şekil 47.	Sorumluluk Zinciri Deseni UML Diyagramı.....	208
Şekil 48.	Komut Deseni UML Diyagramı	209
Şekil 49.	Yineleyici Deseni UML Diyagramı	212
Şekil 50.	Gözlemci Deseni UML Diyagramı	214
Şekil 51.	Strateji Deseni UML Diyagramı.....	215

Şekil 52.	Şablon Yöntem Deseni UML Diyagramı.....	217
Şekil 53.	Ziyaretçi Deseni UML Diyagramı	218
Şekil 54.	Hatıra Deseni UML Diyagramı.....	221
Şekil 55.	Arabulucu Deseni UML Diyagramı.....	223
Şekil 56.	Durum Deseni UML Diyagramı	224
Şekil 57.	Yorumlayıcı deseni UML Diyagramı.....	226
Şekil 58.	ProgressBarView Kullanıcı Kontrolü	234
Şekil 59.	PictureBox Kullanıcı Kontrolü	235
Şekil 60.	WinFormApp1 Uygulamasında Form1	237
Şekil 61.	MVC Uygulanmış Windows Form Uygulamamız	238
Şekil 62.	İstemci-Sunucu Mimarisi.....	244
Şekil 63.	Üç Katmanlı Mimari.....	245
Şekil 64.	Çok Katmanlı Mimari.....	248
Şekil 65.	İşlevsel Olmayan Bileşenler	248
Şekil 66.	WebApplication1 API Uygulamamızdan GET yöntemi ile Veri Çekme.....	258
Şekil 67.	SQLite Veri Tabanı Önce Kod Tabloları.....	266
Şekil 68.	SQLite Veri Tabanında Entity Framework Tabloları.....	268
Şekil 69.	SQL Veri Tabanında Entity Framework Tabloları.....	270
Şekil 70.	Entity Framework Ondalık Kuralı	271
Şekil 71.	Entity Framework Gezinme Kuralı.....	272
Şekil 72.	Entity Framework Yabancı Anahtar Kuralı.....	273
Şekil 73.	Entity Framework ForeignKey Niteliği.....	274
Şekil 74.	Entity Framework ComplexType Niteliği	275
Şekil 75.	Entity Framework NotMapped Niteliği	276
Şekil 76.	Göç Sonrası Yeni Veri Tabanı	287

TABLO LİSTESİ

Tablo 1.	Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri.....	11
Tablo 2.	6802 Emir Seti Örneği ve Sembolik İsim Listesi.....	13
Tablo 3.	Tamsayı Değişkenler ve Sayı Sınırları	18
Tablo 4.	Kayan Noktalı Sayılar ve Sınırları.....	19
Tablo 5.	İki ile çarpma fonksiyonu.....	19
Tablo 6.	Anahtar Kelime Listesi	35
Tablo 7.	Bağlam Olarak Kullanılan Anahtar Kelime Listesi	36
Tablo 8.	Tamsayı Değer Tipleri.....	37
Tablo 9.	Kayan Noktalı Değer Tipler.....	37
Tablo 10.	Örnek bir C# Programının İcra Sırası	42
Tablo 11.	Aritmetik İşleçler.....	43
Tablo 12.	Mantıksal İşleçler	44
Tablo 13.	İlişkisel İşleçler.....	44
Tablo 14.	Bit Düzeyi İşleçler	45
Tablo 15.	Atama İşleçleri.....	46
Tablo 16.	İşleçlerin İşlem Öncelikleri	46
Tablo 17.	Kaçış Dizisi Karakterleri	51
Tablo 18.	If Talimatı İçeren Bir Programın İcra Sırası	58
Tablo 19.	If-Else Talimatı İçeren Bir Programın İcra Sırası	59
Tablo 20.	Aşırı Yüklenebilecek İşleçler	105
Tablo 21.	İstisna (Özel Durum) Tipleri	145
Tablo 22.	http Yöntemlerinin RESTful Serviste Kullanılması	256

DİZİN

.Net Class Library, 23, *Bakın* Base Class

Library

abstract, 94, 110

abstract class, 94, 115, 129

abstract factory pattern, 188

abstract method, 115, 129

abstraction, 49, 94, 133, 212

access, 38

access modifier, 84, 97, 129

accumulator, 13

actual parameter.*Bakın* argument

adapter pattern, 197

aggregation, 129, 132

agile manifesto, 240

agile model, 238, 240

ALU.*Bakın* Aritmetic Logic Unit

analysis pattern, 187

anonymous method, 121

anonymous type, 40

anti-pattern, 187

application, 31

architectural model, 238

architectural pattern, 187, 227

argument, 20

Aritmetic Logic Unit, 13

aritmetic operator, 43

array, 20, 74, 107, 156

array size, 74

aspect, 212

aspect oriented programming, 138

assembly, 14, 25

assignment operator, 45

association, 129, 131

attribute, 129, 139

base, 19

base class, 99, 115, 130

Base Class Library, 23

behavior, 21, 22, 31, 83, 97, 121

behavioral model, 238

behavioral patterns, 187, 207

binary, 18, 19

binary digit, 11

binary serialization, 252

bit.*Bakın* binary digit

bitwise copy.*Bakın* shallow copy

bitwise operators, 45

boxing, 151, 156

bridge pattern, 205

builder pattern, 192

bus, 10

business analysis, 247

Business Entity Components, 244

call, 20

call function, 87

calling convention, 92

calling overhead, 49

cast.*Bakın* explicit conversion

catch an exception, 143

Central Processing Unit, 10

chain of responsibility pattern, 207

change management, 21, 49, 133

child class, 99, 115

CI/CD.*Bakın* Continious

Integration/Continious Delivery

CISC.*Bakın* Complex Instruction Set

Computing

class, 31, 35, 83, 85, 97, 107, 129

class diagram, 99, 129

class library, 31, 49

classic life cycle.*Bakın* waterfall model

client-server, 243

CLOS.*Bakın* Common Lisp Object System

code first, 263, 277

code first with existing database, 264

code segment, 26

collection, 74, 156

comma operator, 68

command, 181

command pattern, 208

command prompt.*Bakın* console

comment, 36

Common Intermediate Language, 23, 24

Common Language Runtime, 23

Common Language Spesification, 24

Common Lisp Object System, 220

Common Object Request Broker Architecture,
253

Common Type System, 23, 24

compile time error, 29

compiler, 14, 23

Complex Instruction Set Computing, 13

component, 97

Component Object Model, 253

composite pattern, 198

composition, 129, *Bakın* private association

computer program, 13

concrete, 94

concrete class, 94, 115

condition, 58

conditional choice, 56, 57

conditional code, 57, 58, 59, 60
configuration settings, 116
connected, 184
connection, 181
connection string, 180
console, 14, 28
const method, 107
const qualifier, 39
constant pattern, 62
constructor, 93, 136
containment.*Bakin* private association
context, 268
Continuous Integration/Continuous Delivery, 239
control abstraction, 94, 96
control operation, 56, 58
control structure, 20, 30, 36
controller, 227
convention, 267
copy constructor, 104
counter, 65
CPU.*Bakin* Central Processing Unit
Create-Read-Update-Delete, 251
creational patterns, 187, 188
CRUD.*Bakin* Create-Read-Update-Delete
dangling else, 62
data, 10, 97
data abstraction, 94, 96
Data Access Components, 244
data annotations, 272
data pattern, 187
data segment, 26
data structure, 20, 30, 36, 37, 74, 156, 210
Data Transfer Object, 248
data type, 19, 20, 38, 74
database, 180
database first, 262
DataSet, 184, 251
debug, 33
debugger, 28
declaration, 37, 38
decode, 13
decorator pattern, 200
deep copy, 195
default argument, 89, 105
default constructor, 94
definition, 38
delegate, 118
dependency, 130
dependency as a parameter, 130
dependency as an instantiation, 130
Dependency Inversion Principle, 134
de-reference operator, 152, 154
derived, 99
derived class, 99, 115, 130
deserialization, 252
design pattern, 148, 187
destructor, 101, 112
Developer Operations, 239
DevOps.*Bakin* Developer Operations
dictionary, 159
disconnected, 184
distributed application, 253
Distributed COM, 253
dot operator, 84, 135
double dispatch, 220
duplicate code, 21, 133
dynamic, 41
dynamic binding, 114
dynamic inheritance, 224
Dynamic Linked Library, 253
EF.*Bakin* Entity Framework
EF Core Tools, 281
else code, 59, 60
encapsulation, 96, 97, 133, 220
encoding, 177
enterprise application, 238
entity, 262
Entity Framework, 262
enum, 37
enumeration, 41
error handling, 21
escape sequence, 51
event, 121, 124
exception, 142
execute, 13
explicit, 154
explicit conversion, 149
expression, 19, 56
extension method, 125, 164
facade pattern, 195
factory method pattern, 190
fail, 21
fetch, 12
field, 31, 35, 37, 83, 93, 107, 108, 195
file, 175
File Transfer Protocol, 254
finalizer, 112
flag, 41
floating point number, 18
fluent, 163
flyweight pattern, 202, 226
foreach loop, 82
foreign key, 271
fraction, 19
free format, 35
function, 19
function pointer, 118

functional, 247
functional model, 238
garbage collector, 112
general class, 99, 115
generalization, 130
generic, 156
generic programming, 156
handler, 121, 124
hashset, 159
heap segment, 26
high cohesion, 133
high level programming language, 16
Hyper Text Transmission Protocol, 254
IDE.Bakın Integrated Development Environment
Integrated Development Environment, 28
Interface Segregation Principle, 134
Internet Protocol, 254
identifier, 38
identifier definition, 38, 56
imperative programming, 21
implementation, 220
implementation pattern, 187
implicit, 154
implicit conversion, 149
index, 74, 276, 285
indexer, 161
information, 10
information hiding, 94
inheritance, 99, 100, 104, 130, 133
initial value, 39
initialization, 39
input output operation, 56
instance, 83, 86, 93, 112, 129
instance member, 107
instantiation, 84, 108
instruction, 12
instruction code, 13
instruction set, 13
interface, 85, 101, 111, 130
interface implementation.Bakın interface realization
interface realization, 111, 130, 136
interoperability, 26
interpreter pattern, 225
invoke function, 87
iteration, 71, 75, 89
iterator, 158
iterator pattern, 210, 220
jagged array, 74, 82
JavaScript Object Notation, 255
JIT compiler.Bakın Just-in-time compiler
join, 171
jump, 56, 70
Just-in-time compiler, 26
key, 276, 277
keyword, 35
lambda expression, 118, 119
lambda tatement, 119
Language Integrated Query, 163
last in first out, 26
late binding, 114
layer, 242
lazy copy, 195
leaf class, 116
LIFO.Bakın last in first out
LINQ.Bakın Language Integrated Query
library, 97
linkedlist, 161
Liskov Substitution Principle, 134
list, 160
literal, 18, 39
local variable, 38
logical error, 30
logical sequence, 56, 65, 66, 67, 68, 72
logical sequences, 17, 35, 119
loop, 89
loop code, 65, 66, 68, 71
loosely copling, 236
loosely coupling, 133, 188
low level programming language, 16
machine language, 13
main function, 20, 30
managed, 152
managed code, 25
mantissa, 19
mapping, 262, 263
mediator pattern, 221
memberwise copy, 195
memento pattern, 220
memory, 10
message-passing, 21, 22, 31, 84, 100
method, 31, 35, 37
migration, 278
mnemonic, 14
model, 227
Model-View-Controller, 227
modulus, 43
monolithic, 242
multiple inheritance, 101
multiplicity, 129
mutable, 109
name server, 254
namespace, 126
NAOT.Bakın native ahaed-of-time
native ahaed-of-time, 26
native code, 26
navigation, 270, 273

nested generalization, 205
non-functional, 247
NoSQL database, 180, 185
null coalescing operator, 64
null colescing assingment operator, 64
null pattern, 215, 223
object, 21, 83, 93
object oriented programming, 21, 22, 133
Object-Relational Mapping, 262
objects, 22
observer, 212
observer pattern, 212, 222
opcode, 13, *Bakın* instruction code
Open Closed Principle, 134
operand, 43
operating system, 14
operator, 30, 43
operator precedence, 46
overloading, 102
override, 116
overriding, 110
package manager, 140
padding, 51
parameterized constructor, 103, 104
parent class, 99
pass by reference, 88
pass by value, 88
pattern, 187, 258
pattern oriented programming, 187
physical sequence, 56, 65, 66, 67, 68, 72
pinning, 153
plug-in, 28
pointer, 21, 74, 118, 152
polymorphism, 113, 133, 150
post-decrement, 43
post-increment, 43
pre-decrement, 43
pre-increment, 43
primary key, 267
private, 95, 116
private association, 131
private class, 99
Problem Domain Components, 248
programming, 13
property, 31
protected, 101
prototype pattern, 194
proxy pattern, 204
public association, 131, 132
publish, 121
queue, 161
range based loop.*Bakın* foreach loop
readonly, 108
recursion, 89
recursive, 89, 90
Reduced Instruction Set Computing, 13
reference type, 24
reflection, 21
register, 12
relational loop, 56, 64
relational operator, 44
relational pattern, 62
relationship, 129
release, 33
Remote Method Invocation, 253
Remote Procedure Call, 254
repository pattern, 249
Representational State Transfer, 254
requirement analysis, 239, 247
reserved word.*Bakın* keyword
RESTful API, 254
re-throw, 147, 148, 252
return, 19
reusing, 49, 133, 236
RISC.*Bakın* Reduced Instruction Set Computing
role.*Bakın* interface
root class, 116
route, 256
run time, 83
run time error, 29
safe, 152
scalability, 253
sealed class, 116
sealed method, 116
seeding data, 286
segment, 26
sentinel value, 69
seperation of concern, 133
sequential operation, 56, 58, 63
serializing, 251
shallow copy, 195
signature, 111
Simple Mail Transport Protocol, 254
single dispatch, 220
Single Responsibility Principle, 133
singleton class, 116
singleton pattern, 191
Software Development Life Cycle, 238, 239
software engineering, 128, 239
SOLID Principles, 133
solution pattern, 187
sortedset, 160
spaghetti code, 18
special class, 115
stack, 161
stack segment, 88
state, 21, 22, 31, 83, 93, 107, 227

- state pattern, 223
- state transition, 224
- stateless, 255
- statement, 17, 36
- static, 116
- static binding, 114
- static member, 107
- strategy pattern, 214
- stream, 175, 177
- string, 50, 107
- string interpolation, 52
- struct, 37, 74, 85, 107, 135
- structural model, 238
- structural patterns, 187, 195
- structural programming, 20
- structure, 20
- Structured Query Language, 180
- subject, 212
- subscribe, 121, 124
- template method pattern, 216
- tenary operator, 63
- terminal, 28, *Bakın* console
- test pattern, 187
- text segment. *Bakın* code segment
- this pointer, 106
- throw an exception, 143
- tier, 242
- time stamp, 282
- trace method, 116
- transaction, 183
- UML. *Bakın* Unified Modelling Language
- unary operator, 44
- Unified Modeling Language, 128
- Unified Modelling Language, 99
- Uniform Resource Identifier, 254
- Uniform Resource Locator, 254
- union, 138
- unique, 76
- unmanaged, 152
- unmanaged code, 25
- unsafe, 152
- user defined class, 83
- User Interface Components, 243
- using, 130
- utility class, 116
- value object, 274
- value type, 24, 36
- variable, 18, 20, 36, 37
- variable declaration, 38
- variable scope, 38
- verbatim string, 52
- version control, 28
- view, 227
- virtual, 110
- virtual constructor, 190
- virtual method), 113
- visitor pattern, 217
- void, 37
- waterfall model, 238, 239
- Windows Communication Foundation, 254